



UNIVERSITÀ DI PISA

DEPARTMENT OF COMPUTER SCIENCE

Ph.D. Thesis

Combining over and under-approximation for program analysis

Candidate

Flavio Ascari

Advisors

Roberto Bruni

Roberta Gori

External reviewers

Patrick Cousot

Peter W. O'Hearn

Academic year 2024/2025

Abstract

Static analysis techniques and toolsets have always been mainly designed as over-approximation methods to prove program correctness. Recently, the dual approach based on under-approximation has gained attention as a formal basis to prove incorrectness. While these two paradigms can be applied separately, it has been shown that they are also able to cooperate to decide more effectively both correctness and incorrectness properties. In this thesis, we study analogies and differences between over and under-approximation methods to understand what can and cannot be ported from the well studied over-approximation theory to the less studied under-approximation one, and how the two can be combined synergistically. We first focus on abstract interpretation theory, finding limitations in approaches based on under-approximation abstract domains. We then turn our attention to program logics, classifying known results as over or under-approximation and according to the direction of the semantics. In particular, we define a new backward-oriented proof system for under-approximation that exposes sources of errors. Then we present novel extensions that combine over and under-approximations in non-trivial ways, namely Local Completeness Logic and Property Directed Reachability, showing how they improve on the current state of the art for simultaneous correctness and incorrectness analyses.

Contents

1	Introduction	3
2	Background	7
2.1	Order structures	7
2.2	Propositional logic	8
2.3	Regular commands	8
2.4	Hoare logic	10
2.5	Incorrectness Logic	12
2.6	Necessary Conditions	13
2.7	Separation logic	14
2.8	Kleene algebras	15
2.9	Abstract interpretation	18
3	Related work	23
3.1	Unifying formalism	23
3.2	Local completeness	25
3.3	IC3/PDR	28
3.4	Outcome Logic	33
3.5	Summary	35
4	Under-approximation abstract domains	37
4.1	Overview	37
4.2	Comparison with over-approximation	40
4.3	Non-emptying functions	42
4.4	Integer domains	44
4.5	General infinite concrete domains	46
4.6	General finite concrete domains	52
4.7	Summary	55
5	Sufficient incorrectness logic	57
5.1	Taxonomy	57
5.2	Sufficient Incorrectness Logic	59
5.3	Relations among logics	62
5.4	Separation Sufficient Incorrectness Logic	68
5.5	Summary	78
6	Local Completeness Logic	79
6.1	Motivation	79
6.2	Extensional soundness	81
6.3	Locally complete refinement	82

6.4	Locally complete simplification	91
6.5	Exploiting convexity	94
6.6	Backward analysis	96
6.7	Summary	98
7	AdjointPDR	101
7.1	Overview	101
7.2	Adjoint PDR	103
7.3	Properties of AdjointPDR	105
7.4	AdjointPDR [↓]	112
7.5	Instantiating AdjointPDR [↓] for MDPs	116
7.6	AdjointPDR ^{AI}	121
7.7	Implementation	124
7.8	Summary	127
8	Conclusions	129
	Bibliography	140
A	Under-approximation abstract domains supplementary materials	141
B	Logics comparison supplementary materials	147
B.1	Proofs about Separation SIL	151
C	LCL_A supplementary materials	161
D	AdjointPDR supplementary materials	175
D.1	Proofs of Section 7.4	181

Chapter 1

Introduction

Static program analyses are techniques used to infer properties of programs directly from their source code, without executing them. They have been studied and successfully applied for over 50 years to produce effective methods and tools to support the development of correct software. For instance, some of these tools include the Astrée static analyzer [Bla+03], the SLAM model checker [BR01], the certified C compiler CompCert [Ler09], and VCC using the calculus of weakest precondition to verify safety properties [Coh+09]. For all these years, the main focus of static analysis was to prove *correctness* property of software such as, for instance, the absence of bugs or security requirements. To this end, static analyses compute *over-approximations*, ie. supersets of all possible behaviours, of the semantics of programs: the absence of unwanted behaviour in the over-approximation guarantees the correctness of the program. However, over-approximation cannot be used to disprove correctness, eg. by exposing real bugs, since any alert raised by the analyser may be caused by the over-approximation rather than by the program: it can be a so called false alarm.

Hoare logic [Hoa69] is perhaps the first example of formal static analysis whose goal is proving the absence of errors, and indeed it is an over-approximation one. It is possible that early works on over-approximation, together with influential opinions such as Dijkstra’s renown quote “*Program testing can be used to show the presence of bugs, but never to show their absence!*” [Dij70], focused the theoretical interest on over-approximation. However, from the point of view of a software developer, false alarms are undesirable because they undermine the credibility and usefulness of the analysis. Therefore, in practice static analysis technique are deployed to *find true bugs* rather than proving their absence [BKY05]. This idea gained mainstream interest with O’Hearn’s introduction of Incorrectness Logic [OHe20]. In his paper, he argues for the relevance of formal methods for bug catching, and more in general for disproving correctness – in other words, proving *incorrectness*. He proposes Incorrectness Logic, a dual version of Hoare logic thought from the ground up for scalable bug-finding, which serves as the theoretical basis for Pulse, a tool in production at Meta which catches thousands of bugs on programs with millions of lines of code [DFLO19]. Incorrectness Logic computes an *under-approximation*, ie. a subset of all possible behaviours of a program. Dually to over-approximation, under-approximation can then expose defects in the code, but it is unable to show their absence. Given the recent interest in this idea (eg. [Raa+20; RBDO22; Le+22; Car+22; Cou24] and the first Workshop on “Formal Methods for Incorrectness” affiliated with POPL’24), we believe the field poses many interesting research questions that can improve the software development process in many ways.

The problem. Even accounting for the rising interest in under-approximation techniques in the last years, “*there is a rich variety of problems [...] to bring the foundations of reasoning about program incorrectness onto a par with the extensively developed foundations for correctness*” [OHe20]. To fill this gap, instead of starting from scratch, it seems reasonable to start from the vast literature on over-approximation techniques and try to recast it to the new under-approximation context. However, this process is not straightforward.

Question Q1: which of these techniques can be effectively transferred from the current over-approximation literature to under-approximation methodologies?

Other than a “static” transfer of techniques to derive new under-approximation methods from over-approximation ones, we envisage a more “dynamic” cooperation: an interesting research avenue lies in the possibility of combining over and under-approximation together, exploiting a synergic information transfer to improve on both. However, the details of this interaction are non-trivial.

Question Q2: how can we effectively combine the two paradigms?

Question Q3: how can the information produced by over-approximation help the precision and speed of an under-approximation analysis, and vice versa?

Contributions. The goal of the thesis is to study the effect of over/under-approximation interaction. The fundamental insight is that even partial information about correctness can help the search for an incorrectness proof and vice versa. This means that a technique employing both over and under-approximation at the same time can, in principle, be more effective than a disjoint application of the two for deciding correctness and incorrectness of a program. However, designing an effective cooperation of over and under-approximation techniques is not a trivial task. It requires an instantiation for the over and under-approximation analysis, and insightful understanding of both to identify the kind of information that can be taken from one to help the other. To this end, we first consider over and under-approximation separately to understand how they can effectively communicate: we do so by drawing analogies and differences between the two. Then, we consider some known approaches that combine over and under-approximation and try to extend them using the insight we gained from the previous study.

We first try to address Q1. We consider the possibility of using **abstract interpretation** for under-approximation. While in principle this should be an easy task thanks to duality, it turns out the apparent symmetry between over and under-approximation is not so sharp. Thus, defining an effective under-approximation abstract domain is a challenging task. We explore intuitive and technical reasons behind this, pointing out non-trivial asymmetries between over and under-approximation. Given the difficulties in using abstract interpretation for under-approximation, we then study **logical frameworks** instead, such as Incorrectness Logic. Particularly, we follow *duality* considerations along two main axes—direction of the approximation and of the computation—to draw connections between known program logics. Moreover, this leads us to define the new **Sufficient Incorrectness Logic**, a *backward under-approximation* logic intended to start from some output errors and track back their origins in the input. We study its properties and its extension to handle pointers and dynamic memory allocation. Our investigation shows that some over-approximation techniques can be adapted easily to under-approximation, while other are less effective.

After identifying the formalisms we want to use for under-approximation, we focus on approaches that combines the two paradigms.

Addressing Q2, we extend **LCL_A** [BGGR23], a technique that combines the under-

approximating Incorrectness Logic and over-approximating abstract interpretation to ensure that the analysis, if it terminates, can decide *both correctness and incorrectness* of a program. Particularly, we extend it in different ways to relax the hypothesis of the original formulation. First, we allow the use of *different abstract domains* for different parts of the program, which let the logic prove all the properties that are true of the semantics of the program, *irrespective of its syntax*. Second, we propose a framework to apply it even when there is *no best abstraction* in the abstract domain (a feature that some practical domains lack). Third, we combine it with Sufficient Incorrectness Logic to obtain an LCL_A that is suitable for *backward analysis*.

For Q3, we introduce a new **PDR-like algorithm** [Bra11], which combines an over-approximation of the initial Kleene chain with an under-approximation of a set of possible counterexamples to the safety of the chain. The novelty of our algorithm lies in the thorough use of *adjoints*, both in the forward/backward duality that in the form of abstraction/concretization functions. Our approach yields a framework to understand and compare heuristics as a trade-off between precision and performances. Moreover, we propose an extension which relaxes some of the hypothesis, and we compare our algorithm with state-of-the-art tools.

These two results about LCL_A and PDR provide examples of how over and under-approximation analysis can effectively share information, and shows that this interaction is non-trivial and requires thorough understanding of both approaches.

Structure of the thesis. After giving some background (to be used as a reference) in Chapter 2, we examine other works that combine over and under-approximations in Chapter 3 to get a baseline. Chapter 4 shows some difficulties in using abstract interpretation for under-approximation for program analysis. Chapter 5 proposes a duality-driven taxonomy of program logics and introduce Sufficient Incorrectness Logic and its extension to handle memory errors. Chapter 6 proposes various approaches to extend LCL_A capabilities based on domain refinement and simplification. Chapter 7 details our new PDR-like algorithm and its experimentation. We draw conclusions in Chapter 8. For presentation purposes not all proofs and technical details are included in the main text: we omit those whose details are tedious and not enlightening. However, all these proofs are included in full details in Appendices A–D.

Origins of material. The work on under-approximation abstract domains was first published at the FoSSaCS conference [ABG22], and its extended version appeared in the TOPLAS journal [ABG24]. The taxonomy of program logics and Sufficient Incorrectness Logic is under review at a conference, and is available on ArXiv [ABGL24]. The chapter on LCL_A extensions includes results from a paper presented at the ESOP conference [ABG23] and from its extended version currently submitted to a journal. Our new PDR-like algorithms were first presented at the CAV conference [Kor+23], where the paper was selected as one of the distinguished papers of the conference. An extended version was invited to the conference special issue.

Chapter 2

Background

In this chapter, we lay the background for the thesis. We fix the notation and briefly recall the main concepts of well known theories for program analysis as the basis of our research.

2.1 Order structures

Given a set S , we write $\mathcal{P}(S)$ for the powerset of S and $\text{id}_S : S \rightarrow S$ for the identity function on S . We omit subscripts when obvious from the context. If $f : S \rightarrow T$ is a function, we overload the symbol f to denote also its lifting $f : \mathcal{P}(S) \rightarrow \mathcal{P}(T)$ defined as $f(X) = \{f(x) \mid x \in X\}$ for any $X \subseteq S$. Given two functions $f : S \rightarrow T$ and $g : T \rightarrow V$ we denote their composition as $g \circ f$ or simply gf . For a function $f : S \rightarrow S$, we denote $f^n : S \rightarrow S$ the composition of f with itself n times, i.e., $f^0 = \text{id}_S$ and $f^{n+1} = f \circ f^n$.

In ordered structures (such as posets and lattices) with carrier set C , we denote the order relation by $\leq_C$, least upper bounds (lubs) by $\vee_C$, greatest lower bounds (glbs) by $\wedge_C$, least element by $\perp_C$ and greatest element by $\top_C$. We recall that any powerset is a complete lattice with ordering given by inclusion. In this case, we use standard symbols $\subseteq$, $\cup$, etc. If C is a poset, we denote by C^{op} the opposite (also called dual) poset with the same carrier set but reverse order: $a \leq_{C^{\text{op}}} b$ if and only if $b \leq_C a$. We lift poset operators and relations to functions pointwise: given a poset T and two functions $f, g : S \rightarrow T$, the notation $f \leq g$ means that, for all $s \in S$, $f(s) \leq_T g(s)$. Analogously, $f \wedge g$ denotes the function $(f \wedge g)(s) = f(s) \wedge g(s)$ for all $s \in S$. A function f between complete lattices is additive (resp. co-additive) whenever it preserves lubs (resp. glbs) of any set of elements (including the empty set). Given a function $f : C \rightarrow C$ on a poset C , we call a point $x \in C$ a fixed point (or fixpoint) if $f(x) = x$, and write both $\text{lfp}(f)$ and $\mu(f)$ to denote its least fixpoint, if it exists. We recall two standard results guaranteeing the existence of (least) fixpoints (see, e.g., [DP02]):

Theorem 2.1 (Tarski). *Let L be a complete lattice and let $f : L \rightarrow L$ be a monotone function. Then the set of fixed points of f is a complete lattice.*

Corollary 2.2. *Since a complete lattice cannot be empty, $\text{lfp}(f)$ exists.*

Theorem 2.3 (Tarski-Scott). *Let C be a complete partial order and $f : C \rightarrow C$ a Scott-continuous function. Then $\text{lfp}(f)$ is the least upper bound of the chain*

$$\perp \leq f(\perp) \leq f^2(\perp) \leq f^3(\perp) \leq \dots$$

The above chain is also called the *initial chain* of f . If instead of C we consider C^{op} , by duality we get that whenever f is Scott-co-continuous the greatest fixpoint of f , denoted

by $\text{gfp}(f)$, is the greatest lower bound of the *final chain* of f

$$\dots \leq f^3(\top) \leq f^2(\top) \leq f(\top) \leq \top.$$

Let us introduce the following notation. Given an element $x \in C$, we sometimes overload the symbol x also to denote the constant function $x : C \rightarrow C$, ie. $x(y) = x$ for all $y \in C$. For instance, using this notation, given a function $f : C \rightarrow C$ and an element $x \in C$, we write $(f \vee x)$ to denote the function $(f \vee x)(y) = f(y) \vee x$. If we assume f is additive, then for any point $x \in C$ we have $(f \vee x)^n(\perp) = \bigvee_{j < n} f^j(x)$: it can be proved by induction on n recalling that, since f is additive, it preserves the lub of the empty set, that is $\perp$. By Tarski-Scott, this means $\text{lfp}(f \vee x) = \bigvee_{n \geq 0} f^n(x)$. Dually, if f is co-additive, $\text{gfp}(f \wedge x) = \bigwedge_{n \geq 0} f^n(x)$.

Given $f : S \rightarrow T$ and $g : T \rightarrow S$, we say f and g are *adjoints*, notation $f \dashv g$, if

$$\forall s \in S, t \in T. \quad f(s) \leq_T t \iff s \leq_S g(t)$$

If $f \dashv g$, then by Knaster-Tarski

$$\text{lfp}(f \vee x) \leq y \iff x \leq \text{gfp}(g \wedge y). \quad (2.1)$$

Lastly, from Knaster-Tarski follows the following two (co)induction proof principles:

$$\frac{\exists x. f(x) \leq x \leq p}{\text{lfp}(f) \leq p} \qquad \frac{\exists x. p \leq x \leq f(x)}{p \leq \text{gfp}(f)} \quad (2.2)$$

2.2 Propositional logic

When talking about propositional logical formulas, we group variables $x_1, x_2, \dots, x_n$ in a vector denoted by $\bar{x}$. Moreover, we often omit the variables a formula depends on. For instance, if the formula $F(\bar{x})$ depends on variables $\bar{x}$, we shall write just F . We use the special shorthand F' to denote $F[\bar{x}'/\bar{x}]$, where we primed all variables appearing in F . An assignment s of all variables $\bar{x}$ appearing in a formula $F(\bar{x})$ either satisfies it, written $s \models F$, or falsifies it, written $s \not\models F$. Given an assignment s for variables $\bar{x}$, we shall write s' for the corresponding assignment on primed variables $\bar{x}'$, that is the value of variable x' in s' is the same as x in s . Moreover, we use the comma to “merge” assignments when they refer to disjoint sets of variables: for instance, given the two assignments s and t , we write s, t' for the assignment that uses s for $\bar{x}$ and t' for $\bar{x}'$ (we assume the set of primed variables $\bar{x}'$ to be disjoint from the original set $\bar{x}$). A *literal* is either a variable or its negation. A *clause* c is a disjunction of literals. Any assignment s of variables $\bar{x}$ has a corresponding clause that is satisfied exactly by that assignment of the same variables, and we overload the symbol s to denote both. A *subclause* $d \subseteq c$ is a clause whose literals are a subset of literals of c . A formula F in conjunctive normal form (CNF) is a conjunction of clauses, and we write $\text{clause}(F)$ to denote the set of clauses that appear in F .

2.3 Regular commands

It is common to study static analysis by considering a simple while-language, which contains basic constructs needed to write imperative programs [Win93]. However, following e.g. [OHe20; BGGR21; MOH21; Raa+20; ZDS23], we consider a different system that is able to encode the standard while-language. Namely, we focus on regular commands:

$$\text{Reg } \ni r ::= e \mid r; r \mid r \oplus r \mid r^* \quad (2.3)$$

$$\begin{aligned}
\llbracket e \rrbracket &\triangleq \langle e \rangle c \\
\llbracket r_1; r_2 \rrbracket c &\triangleq \llbracket r_2 \rrbracket \llbracket r_1 \rrbracket (c) \\
\llbracket r_1 \oplus r_2 \rrbracket c &\triangleq \llbracket r_1 \rrbracket c \vee \llbracket r_2 \rrbracket c \\
\llbracket r^* \rrbracket c &\triangleq \bigvee_{n \geq 0} \llbracket r \rrbracket^n c
\end{aligned}$$

Figure 2.1: Generic semantics of regular commands.

Regular commands can be instantiated differently by changing the set Exp of atomic commands e . These determines the kind of basic operations allowed in the language. For instance, when Exp contains deterministic assignments and Boolean guards, it can encode a standard while-language (see (2.5)). Another example is Kleene algebras with tests [Koz97] (see Section 2.8).

The command $r; r$ represent the usual sequential composition. The command $r \oplus r$ is nondeterministic choice. The Kleene star r^* denotes a nondeterministic iteration where r can be executed any number of times (possibly 0) before exiting. It can be thought of as the least solution to the recursive equation $r^* \equiv \text{skip} \oplus (r; r^*)$.¹ We write r^n to denote sequential composition of r with itself n times, analogously to the notation f^n for function repetition.

The semantics of regular commands is defined over a concrete set of values C that forms a complete lattice. We also assume the semantics function for atomic commands $\langle \cdot \rangle : \text{Exp} \rightarrow C \rightarrow C$ to be given. The semantics function $\llbracket \cdot \rrbracket : \text{Reg} \rightarrow C \rightarrow C$ is defined inductively in Figure 2.1. Intuitively, this defines the collecting semantics of a program: if C is the powerset of the set of states, $\llbracket r \rrbracket c$ is the set of output states reachable from the set of input states c .

To recover the standard while-language, we first consider standard integer arithmetic expressions $a \in \text{AExp}$ and Boolean expressions $b \in \text{BExp}$:

$$\text{AExp} \ni a ::= n \mid x \mid a \diamond a \quad \text{BExp} \ni b ::= \text{false} \mid \neg b \mid b \wedge b \mid a \asymp a$$

where $n \in \mathbb{Z}$ is a natural number, x is a (integer) variable, $\diamond \in \{+, -, \cdot, \dots\}$ encodes standard arithmetic operations, and $\asymp \in \{=, \neq, \leq, <, \dots\}$ standard comparison operators. We then let

$$\text{Exp} \ni e ::= \text{skip} \mid b? \mid x := a \tag{2.4}$$

The command **skip** does nothing. The command $x := a$ is a standard deterministic assignment. The command $b?$ is an “assume” statement: it filters out inputs that falsify b . With this set of expressions, regular commands are very similar to Dijkstra’s guarded commands [Dij75]. We can define **if** and **while** statements as syntactic sugar:

$$\begin{aligned}
\text{if } (b) \text{ then } c_1 \text{ else } c_2 &\triangleq (b?; c_1) \oplus ((\neg b)?; c_2) \\
\text{while } (b) \text{ do } c &\triangleq (b?; c)^*; (\neg b)?
\end{aligned} \tag{2.5}$$

For the semantics of basic expressions in (2.4), we consider a finite set of variables Var , then the set of stores $\Sigma \triangleq \text{Var} \rightarrow \mathbb{Z}$ that are (total) functions σ from Var to integers.

¹Note that, by Tarski-Scott, $\llbracket r^* \rrbracket \triangleq \bigvee_{n \geq 0} \llbracket r \rrbracket^n = \text{lfp}(\lambda f. \text{id} \vee f \circ \llbracket r \rrbracket)$, the least solution of this equation.

The complete lattice C is defined as $\mathcal{P}(\Sigma)$ with the usual order structure given by set inclusion. Given a store $\sigma \in \Sigma$, $\sigma[x \mapsto v]$ denotes function update as usual for $x \in \text{Var}$ and $v \in \mathbb{Z}$. We consider standard, inductively defined semantics $\llbracket \cdot \rrbracket : \text{AExp} \rightarrow \Sigma \rightarrow \mathbb{Z}$ for arithmetic expressions and $\llbracket \cdot \rrbracket : \text{BExp} \rightarrow \Sigma \rightarrow \{\text{tt}, \text{ff}\}$ for Boolean expressions. The semantics $\llbracket \cdot \rrbracket : \text{Exp} \rightarrow \mathcal{P}(\Sigma) \rightarrow \mathcal{P}(\Sigma)$ for atomic commands $e \in \text{Exp}$ is then defined as

$$\begin{aligned} \llbracket \text{skip} \rrbracket S &\triangleq S \\ \llbracket x := a \rrbracket S &\triangleq \{\sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \sigma \in S\} \\ \llbracket b? \rrbracket S &\triangleq \{\sigma \in S \mid \llbracket b \rrbracket \sigma = \text{tt}\} \end{aligned}$$

This defines the collecting denotational semantics $\llbracket \cdot \rrbracket : \text{Reg} \rightarrow \mathcal{P}(\Sigma) \rightarrow \mathcal{P}(\Sigma)$ of programs. In fact, it is easy to check that the semantics of **if** and **while** is the standard one for while-language. We also remark that this semantics $\llbracket \cdot \rrbracket$ is additive and therefore monotone, and these properties are lifted to the semantics of regular commands $\llbracket \cdot \rrbracket$:

Proposition 2.4. *If $\llbracket \cdot \rrbracket$ is monotone (resp. additive), then the semantics $\llbracket \cdot \rrbracket$ defined as in Figure 2.1 is monotone (resp. additive) as well.*

As notational shorthand, we shall write $\llbracket r \rrbracket \sigma$ instead of $\llbracket r \rrbracket \{\sigma\}$.

2.4 Hoare logic

Hoare logic (HL for short) [Hoa69] is a triple-based logic to prove correctness properties about programs. Given two assertions P, Q and a regular command r ,² the HL triple

$$\{P\} r \{Q\}$$

means that, whenever the execution of r begins in a state σ satisfying P and it ends in a state σ' , then σ' satisfies Q . When Q is a correctness specification, any HL triple $\{P\} r \{Q\}$ provides a *sufficient* condition P for the so called *partial correctness* of the program r . Formally, given the semantics $\llbracket \cdot \rrbracket$ of regular commands and abusing notation by writing P instead of the set of states satisfying that formula, the validity of an HL triple is given by over-approximation property of postconditions

$$\llbracket r \rrbracket P \subseteq Q. \tag{HL}$$

A HL triple $\{P\} r \{Q\}$ is *valid*, written $\models \{P\} r \{Q\}$, if condition (HL) holds.

In its original formulation, HL was proposed for a deterministic while-language and assuming P and Q were first-order logic formulae. Subsequent work generalized it to many other settings, such as nondeterminism [Apt84] and regular commands [MOH21]. For the latter, the rules in Figure 2.2 define a minimal set of sound rules for HL, but we remark that there are many other valid rules other than those in the figure. We also point out that rule $\{\text{iter}\}$ is based on *invariants*, i.e., properties whose validity is preserved by the loop body. From this, if an invariant is true at the beginning, it is true also after any number of iterations. Invariants are a simple, yet sound and complete proof technique for loop over-approximation. We use the standard notation $\vdash \{P\} r \{Q\}$ to express that a triple is *provable* in the HL proof system.

²The original formulation uses the while-language. However, since regular commands are the primary syntax used in this thesis, we focus on them instead.

$\frac{}{\{P\} e \llbracket e \rrbracket P} \{\text{atom}\}$	$\frac{P \Rightarrow P' \quad \{P'\} r \{Q'\} \quad Q' \Rightarrow Q}{\{P\} r \{Q\}} \{\text{cons}\}$
$\frac{\{P\} r_1 \{R\} \quad \{R\} r_2 \{Q\}}{\{P\} r_1; r_2 \{Q\}} \{\text{seq}\}$	$\frac{\{P\} r_1 \{Q\} \quad \{P\} r_2 \{Q\}}{\{P\} r_1 \oplus r_2 \{Q\}} \{\text{choice}\}$
$\frac{\{P\} r \{P\}}{\{P\} r^* \{P\}} \{\text{iter}\}$	

Figure 2.2: Hoare logic for regular commands

Theorem 2.5 (HL is sound [HL74]). *All provable triples in HL are valid:*

$$\vdash \{P\} r \{Q\} \implies \models \{P\} r \{Q\}.$$

The reverse implication is called *completeness* of the logic,³ and it may not hold for HL depending on the set of assertions used. Particularly, when we use first-order logic formulae as assertions, HL is not complete because first-order logic is not able to represent all the properties needed to prove completeness, notably loop invariants [Apt81, §2.7]. Moreover, the undecidability of implications in first-order logic makes completeness an undecidable problem even when we are able to express all the needed properties. Cook [Coo78] overcame these limitations for the first time by adding an oracle to decide implications and requiring that all strongest post (including loop-invariants) are expressible in the assertion language, proving the first completeness result for HL.⁴ Another approach (see, e.g., [CCLB12; OHe20; Zil24]) is to assume P and Q to be *set* of states instead of formulae in some assertion language. This allows us to dodge at once both the issue of expressibility of the assertion language (since we can account for any subset of states) and of decidability of the implication (since it reduces to subset inclusion). This latter approach is sometimes called “semantics assertions”, as opposed to the “syntactic assertions” approach of using formulae in some language. In this thesis, we will consider semantics assertions where P and Q are sets of states, with the only exception of Separation SIL (Section 5.4). Therefore, we write $\sigma \in P$ to mean that state σ satisfies the assertion P , and use standard set-theoretic symbols such as $\subseteq$ and $\cup$ instead of the logical $\implies$ and $\vee$ to remark the distinction.

When P and Q are sets of states, the HL proof system in Figure 2.2 is complete.

Theorem 2.6 (HL is complete [MOH21]). *All valid triples in HL are provable:*

$$\models \{P\} r \{Q\} \implies \vdash \{P\} r \{Q\}.$$

Lastly, we remark that HL is tightly connected to Dijkstra’s weakest liberal precondition [Dij75]. Given a program r and a predicate Q on final states, the weakest liberal precondition $\mathbf{wlp}[r](Q)$ is a predicate on initial states such that a state σ satisfies $\mathbf{wlp}[r](Q)$ if and only if the execution of r starting from σ either does not terminate or terminates in a state satisfying Q . This definition is reminiscent of the validity condition for HL

³For the sake of readability, we must warn the reader that this thesis deals with three different notions of completeness. (Logical) completeness, used here, refers to the ability of a proof system to derive all valid triples. Global and local completeness are instead properties of abstract interpretation, that will be defined later. We will make sure to disambiguate the term whenever it is not clear from the context.

⁴This was later called completeness *in the sense of Cook* [Apt81, §2.8].

$$\boxed{
\begin{array}{c}
\frac{}{[P] \text{ e } [\llbracket e \rrbracket P]} \text{ [atom]} \qquad \frac{P \supseteq P' \quad [P'] \text{ r } [Q'] \quad Q' \supseteq Q}{[P] \text{ r } [Q]} \text{ [cons]} \\
\frac{[P] \text{ r}_1 [R] \quad [R] \text{ r}_2 [Q]}{[P] \text{ r}_1; \text{r}_2 [Q]} \text{ [seq]} \qquad \frac{[P] \text{ r}_1 [Q_1] \quad [P] \text{ r}_2 [Q_2]}{[P] \text{ r}_1 \oplus \text{r}_2 [Q_1 \cup Q_2]} \text{ [choice]} \\
\frac{\forall n \geq 0. [P_n] \text{ r } [P_{n+1}]}{[P_0] \text{ r}^* [\bigcup_{n \geq 0} P_n]} \text{ [iter]}
\end{array}
}$$

Figure 2.3: Simplified Incorrectness Logic, adapted from [MOH21].

triples, but the “if and only if” requirement makes it stronger: $\mathbf{wlp}[r](Q)$ is the *weakest* precondition such that $\models \{\mathbf{wlp}[r](Q)\} \text{ r } \{Q\}$. In other words, $\models \{P\} \text{ r } \{Q\}$ if and only if $P \implies \mathbf{wlp}[r](Q)$. While Dijkstra’s work focused on giving an inductive definition for $\mathbf{wlp}$, we focus on it as a sort of inverse of $\llbracket \cdot \rrbracket$.⁵ As discussed above, we also consider $\mathbf{wlp}[r](Q)$ to be a set of states instead of a formula.

2.5 Incorrectness Logic

Incorrectness Logic (IL) [OHe20] was introduced as a formalism for under-approximation with the idea of finding true bugs in the code. The IL triple

$$[P] \text{ r } [Q]$$

means that all the states in Q are reachable from states in P . Therefore, any error state in Q is a true error of the program and the analysis can report it safely to developers. Formally, the validity of an IL triple is given by the following formula

$$\forall \sigma' \in Q. \exists \sigma \in P. \sigma' \in \llbracket r \rrbracket \sigma, \quad (\text{IL}_{\text{FOL}})$$

which can be compactly rewritten as the under-approximation condition

$$\llbracket r \rrbracket P \supseteq Q. \quad (\text{IL})$$

This property ensures that any error in Q reported by the analysis is in fact a true error of the program, reachable in some concrete execution. Note that a consequence of this “concrete reachability” means that nontermination is never a bug, since no concrete execution can show nontermination.

The validity condition of IL was first proposed in [VK11]. There, the authors considered the same condition to specify correctness of randomized programs, and defined the “reverse Hoare logic” proof system to prove it. The novelty of [OHe20] is to distinguish correct and erroneous termination and, most importantly, to focus on finding true bugs rather than proving correctness. We consider the proof system for IL in Figure 2.3, adapting the one in [MOH21], simplified to not separate correct and erroneous termination states (see also Remark 2.9). Rule [cons] looks similar, but uses reversed implication: in IL we can expand the precondition ($P' \subseteq P$) and shrink the post ($Q \subseteq Q'$), while in HL we do the opposite (cf. {cons} in Figure 2.2). This fits the under/over-approximation duality: if

⁵More precisely, $\mathbf{wlp}[r]$ and $\llbracket r \rrbracket$ are adjoint functions for every r , see Section 5.3.3.

we know Q' is an under-approximation of the result, also $Q \subseteq Q'$ is such, and dually in HL. Another important difference is in rule [iter] for Kleene iteration. HL rule {iter} is based on loop invariants, which are sound but not complete for under-approximation [OHe20, §4]. Intuitively, under-approximate invariants cannot account for states that are reached after one or more loop iterations.

The proof system in Figure 2.3 is sound and complete for IL.

Theorem 2.7 (IL is sound and complete [OHe20]). *An IL triple is provable if and only if it is valid:*

$$\vdash [P] \text{ r } [Q] \iff \models [P] \text{ r } [Q].$$

The following example compares HL and IL on a simple program to give an intuition on how they work and differs.

Example 2.8. HL and IL aim at addressing different properties. To show their differences, we use the simple, nondeterministic, terminating program `r42`:

```
x := nondet();
if (even(x) && odd(y)) {
  z := 42;
}
// assert(z != 42)
```

where we assume that $Q_{42} \triangleq (z = 42)$ denotes the set of erroneous states, ie. the incorrectness specification. The valid HL triple $\{\text{odd}(y)\} \text{ r42 } \{Q_{42}\}$ identifies input states that will certainly end up in error states, while the triple $\{\neg Q_{42} \wedge \neg \text{odd}(y)\} \text{ r42 } \{\neg Q_{42}\}$ characterize input states that will not produce any error.

On the other hand, the valid IL triple $[z = 11] \text{ r42 } [Q_{42} \wedge \text{odd}(y) \wedge \text{even}(x)]$ expresses the fact that error states in Q_{42} are reachable by safe initial state. Similarly, also the IL triple $[\text{true}] \text{ r42 } [\neg Q_{42} \wedge \neg(\text{odd}(y) \wedge \text{even}(x))]$ is valid since the postcondition $\neg Q_{42}$ can be reached only when the path conditions to reach the assignment are not satisfied. ■

Remark 2.9 (ok/er flags). One distinguishing feature of proof systems for incorrectness (e.g., [OHe20; Raa+20; RBDO22; RVBO23]) is to tag postconditions, but not preconditions, with either the flag *ok* or *er* to separate successful computations from those leading to errors. An immediate consequence is that the number of proof rules is increased by the necessity to deal with such tags and that the definitions of both the semantics and the proof system become longer and more complex. Striving for simplicity, in this thesis we mostly follow the alternative from [BGGR23], where the whole concrete domain is extended with such flags, e.g., we use $C = \mathcal{P}(\{\text{ok}, \text{er}\} \times \Sigma)$ instead of $\mathcal{P}(\Sigma)$, and we assume that error states are preserved by all commands, ie. that $\llbracket r \rrbracket(\text{er} : \sigma) = \text{er} : \sigma$, for any $r \in \text{Reg}$ and $\sigma \in \Sigma$. This way, we tag both pre and post, but the treatment of tags is transparent to the rules of the logic. Therefore, in this extended setting, when we write P and Q we leave implicit that they may contain both kinds of tagged states. The distinction between successful and erroneous outputs will be explicitated in the analysis presented in Section 5.4.6.

2.6 Necessary Conditions

The notion of Necessary Conditions (NC) was introduced in [CCL11; CCFL13] for contract inference. The goal was to relax the burden on programmers when presenting function summaries: while sufficient conditions require the caller of a function to supply parameters

that will never cause an error, NC only prevents the invocation of the function with arguments that will inevitably cause an error. Intuitively, given a correctness specification Q , the NC triple

$$(P) \text{ r } (Q) \quad (\text{NC})$$

means that any state σ that admits at least one non-erroneous execution of the program r is in P .

Following the original formulation [CCFL13], we can partition the traces of a nondeterministic execution starting from a memory σ in three different sets: $\mathcal{T}(\sigma)$, those without errors, $\mathcal{E}(\sigma)$, those with an error, and $\mathcal{I}(\sigma)$, those which do not terminate. A sufficient precondition $\overline{P}$ is such that $(\sigma \in \overline{P}) \implies (\mathcal{E}(\sigma) = \emptyset)$, that is, $\overline{P}$ excludes all error traces. Instead, a necessary precondition $\underline{P}$ is a formula such that $(\mathcal{T}(\sigma) \neq \emptyset \vee \mathcal{I}(\sigma) \neq \emptyset) \implies (\sigma \in \underline{P})$, which is equivalent to

$$(\sigma \notin \underline{P}) \implies (\mathcal{T}(\sigma) = \mathcal{I}(\sigma) = \emptyset).$$

In other words, a necessary precondition *rules out no good run*: when it is violated by the input state, the program has only erroneous executions. Note that we consider infinite traces as good traces. We do this by analogy with sufficient preconditions, where bad traces are only those which end in a bad state.

Example 2.10. Consider again the program `r42` from Example 2.8. As in the previous example, error states are those satisfying $(z = 42)$. Therefore, we consider the *correctness* specification $(z \neq 42)$. Then

	$z = 42$	$z \neq 42$	$z \neq 42 \wedge \text{even}(y)$
$\mathcal{T}(\sigma)$	$\emptyset$	$\neq \emptyset$	$\neq \emptyset$
$\mathcal{E}(\sigma)$	$\neq \emptyset$	$\neq \emptyset$	$\emptyset$

The weakest sufficient precondition for this program is $\overline{P} = (z \neq 42 \wedge \text{even}(y))$ because no input state σ that violates $\overline{P}$ is such that $\mathcal{E}(\sigma) = \emptyset$. On the contrary, we have, e.g., that $\underline{P} = (z \neq 42)$ is a necessary precondition, while $(z > 42)$ is not, because it excludes some good runs. ■

2.7 Separation logic

Separation Logic (SL) [Rey02; ORY01] is an extension of HL introduced to handle pointers and dynamic memory management. Its ingenuity lies in the assertion language used to specify heaps in pre and postconditions: the key insight is that logical conjunction is not apt to describe pointers because of aliasing, therefore the need for a new kind of conjunction. Consider for instance the simple program

$$r \triangleq *x := 1; *y := 2; *z := 3$$

where x , y and z are pointers. The HL triple $\{\text{true}\} \text{ r } \{x \mapsto 1 \wedge y \mapsto 2 \wedge z \mapsto 3\}$ is not valid because some pointers may be aliased at the entry point of the program. To avoid aliasing, the precondition should explicitly state that all the addresses are pairwise different, adding the condition $x \neq y \wedge x \neq z \wedge y \neq z$, which becomes unfeasible as the number of addresses grows. This aliasing problem makes also hard to reuse specifications, since variables modified internally by some function calls may be implicitly exposed to the outside through aliasing. To address this problem, SL introduces the separating conjunction $*$ (read "and separately"). To understand how it works, consider the simple formula

$$\begin{array}{ll}
\llbracket a_1 \prec a_2 \rrbracket \triangleq \{(s, h) \mid \llbracket a_1 \rrbracket s \prec \llbracket a_2 \rrbracket s\} & \llbracket \mathbf{true} \rrbracket \triangleq \Sigma \\
\llbracket \exists x. p \rrbracket \triangleq \{(s, h) \mid \exists v \in \text{Val}. (s[x \mapsto v], h) \in \llbracket p \rrbracket\} & \llbracket p \wedge q \rrbracket \triangleq \llbracket p \rrbracket \cap \llbracket q \rrbracket \\
\llbracket x \mapsto a \rrbracket \triangleq \{(s, [s(x) \mapsto \llbracket a \rrbracket s])\} & \llbracket \neg p \rrbracket \triangleq \Sigma \setminus \llbracket p \rrbracket \\
\llbracket p * q \rrbracket \triangleq \{(s, h_p \bullet h_q) \mid (s, h_p) \in \llbracket p \rrbracket, (s, h_q) \in \llbracket q \rrbracket, h_p \perp h_q\} & \llbracket \mathbf{emp} \rrbracket \triangleq \{(s, [])\}
\end{array}$$

Figure 2.4: Semantics of the assertion language.

$x \mapsto 1 * y \mapsto 2$. A heap satisfies this formula if it can be split in two disjoint sub-heaps such that one satisfies $x \mapsto 1$ and the other $y \mapsto 2$. Therefore, this implicitly means that x and y cannot be aliased: if they were, the unique memory location could not be at the same time in both disjoint sub-heaps for the two assertions. Intuitively, whenever there is a separating conjunction, every heap location is “owned” by exactly one of the two subformulae involved. Note that SL does not prevent aliasing: it is always possible to write $x \mapsto 1 * x = y$. The key difference is that here the aliasing is *explicit* in the formula instead of being implicit in the model.

Formally, SL is a triple-based program logic analogous to HL, which uses as assertion language formulae built from the following grammar:

$$p, q ::= \mathbf{true} \mid p \wedge q \mid \neg p \mid \exists x. p \mid \mathbf{a} \prec \mathbf{a} \mid \mathbf{emp} \mid x \mapsto \mathbf{a} \mid p * q.$$

The first five constructs describe standard first-order logic. The last three describes heaps, and are called spatial constructs: **emp** is satisfied only by the empty heap, $x \mapsto \mathbf{a}$ is satisfied by heaps with a single location, corresponding to the location x and containing the value $\mathbf{a}$, and $*$ is the separating conjunction described above.

To be more formal, consider a set of locations Loc , stores as total functions $s : \text{Var} \rightarrow \mathbb{Z} \uplus \text{Loc}$, heaps as partial functions $h : \text{Loc} \rightarrow \mathbb{Z} \uplus \text{Loc}$, and states as pairs $(s, h) \in \Sigma$ of a heap and a store. We use $s[x \mapsto v]$ for function update, $[]$ for the empty heap and $[l \mapsto v]$ for the heap defined only on l and associating value v to it. When two heaps are *disjoint*, ie. $\text{dom}(h_1) \cap \text{dom}(h_2) = \emptyset$, we define the $\bullet$ operation as the merge of the two: $h_1 \bullet h_2$ coincides with h_1 on $\text{dom}(h_1)$, with h_2 on $\text{dom}(h_2)$ and it is undefined everywhere else. We define inductively the set of states that satisfy a Separation Logic assertion in Figure 2.4. The semantics of the first five constructs is standard for first-order logic. The semantics of spatial constructs is more interesting. The formula **emp** is satisfied only by states whose heap is empty. More importantly, $x \mapsto a$ is satisfied only by states whose heap has a single location defined: this is crucial when considering the separating conjunction operator. The formula $p * q$ is satisfied by a state whose heap can be split in two *disjoint* heaps h_1, h_2 such that one satisfied p and the other q . This means that each location defined in the heap $h_1 \bullet h_2$ of the whole state must be in exactly one of h_1 or h_2 : if it is in h_1 (resp. h_2) then that location is “owned” by p (resp. q). Particularly, this means that p and q must specify only the heap locations they want to take ownership of.

In the thesis, we will use (an extension of) Separation Logic as the assertion language for Separation SIL (Section 5.4).

2.8 Kleene algebras

Kleene algebras are an algebraic formulation that can describe programs [Koz97; Har79].

Definition 2.11 (Idempotent semiring). An idempotent semiring is an algebraic structure $(A, +, \cdot, 0, 1)$ satisfying

- $(A, +, 0)$ is a commutative and idempotent monoid
- $(A, \cdot, 1)$ is a monoid
- multiplication distributes over sum
- 0 is an annihilator for sum, that is $a \cdot 0 = 0 \cdot a = 0$

Intuitively, elements of the idempotent semiring are programs, and the operations define possible ways to compose them: sum $+$ is nondeterministic choice ($\oplus$ in the syntax of regular commands), product $\cdot$ is sequential composition ($;$), the multiplicative unit 1 is a no-op (**skip**) and the additive unit 0 is divergence (the test **false?**, which is equivalent to the regular command **while (true) do skip = (true?; skip)*; false?**).

In any idempotent semiring we define the natural partial order $a \leq b$ iff $a + b = b$. With this definition, $+$ defines the lub of two elements: $a \vee b = a + b$. Moreover, sum and product are monotone in both arguments, and 0 is the bottom element.

To get a Kleene algebra, we add the Kleene star operator to model iteration [Koz94]:

Definition 2.12 (Kleene algebra). A Kleene algebra is an idempotent semiring with an additional operator $\star$ satisfying the following axioms:

$$\begin{aligned} 1 + a \cdot a^\star &= a^\star & 1 + a^\star \cdot a &= a^\star & (\text{Star unfold}) \\ b + a \cdot c \leq c &\implies a^\star \cdot b \leq c & b + c \cdot a \leq c &\implies b \cdot a^\star \leq c & (\text{Star induction}) \end{aligned}$$

(Star unfold) axioms describe that $\star$ behaves as nondeterministic iteration, given it satisfies the same recursive equation. *(Star induction)* axioms define the induction principle for Kleene star. Consider b as the base case, a as the “increment” operation, c as the inductive thesis and $a \cdot c$ as the inductive step (since it is the “increment” applied to the inductive hypothesis). The axiom says that if we prove that both the base case b and the inductive step $a \cdot c$ are below (ie. satisfy) the inductive thesis c (since we prove their lub, that is their sum, is below c), we proved it for any number of iterations of the increment a starting from the base case b . $\star$ can also be seen as a least fixpoint. *(Star unfold)* axioms state it behaves as the expected fixpoint, while *(Star induction)* require it to be minimal.

Kleene algebras are a versatile formalism that has been applied as is. However, the main algebraic structure we consider is Kleene algebra with tests (KAT for short), since this addition allows to encode programs [Koz97].

Definition 2.13 (Test). A test p in an idempotent semiring is an element with a unique complement $\neg p$ satisfying $p + \neg p = 1$ and $p \cdot \neg p = \neg p \cdot p = 0$. We denote by $\text{test}(A)$ the set of tests of an idempotent semiring A .

With this definition, $(\text{test}(A), +, \cdot, \neg, 0, 1)$ is a Boolean algebra contained within A . This means that sum $+$ represent logical disjunction, product $\cdot$ is conjunction, 0 is false and 1 is true.

An interesting example of KAT are so-called “relational KAT”.

Example 2.14 (Relational KAT). A relational KAT is a KAT with carrier $\mathcal{P}(C \times C)$, the binary relations on a given set C . $+$ is defined as union and $\cdot$ as sequential composition of relations:

$$a \cdot b = \{(x, z) \mid \exists y. (x, y) \in a, (y, z) \in b\}$$

$$\begin{array}{c}
\frac{}{\{p\} 0 \{1\}} \{\text{zero}\} \qquad \frac{}{\{p\} 1 \{p\}} \{\text{one}\} \\
\frac{p \cdot a = p \cdot a \cdot q}{\{p\} a \{q\}} \{\text{atom}\} \qquad \frac{p \leq p' \quad \{p'\} a \{q'\} \quad q' \leq q}{\{p\} a \{q\}} \{\text{cons}\} \\
\frac{\{p\} a \{r\} \quad \{r\} b \{q\}}{\{p\} a \cdot b \{q\}} \{\text{seq}\} \qquad \frac{\{p\} a_1 \{q\} \quad \{p\} a_2 \{q\}}{\{p\} a_1 + a_2 \{q\}} \{\text{choice}\} \\
\frac{\{p\} a \{p\}}{\{p\} a^* \{p\}} \{\text{iter}\}
\end{array}$$

Figure 2.5: KAT encoding of Hoare logic

0 is the empty relation, 1 is the identity relation, $\star$ is reflexive and transitive closure. A test in this KAT is a subset of 1, that is for any subset $P \subseteq C$ we have the test $\{(x, x) \mid x \in P\}$. Intuitively, this test represent the property P . Nonetheless, when interpreted as an element of the KAT (ie. a command) it behaves as an “assume(P)” statement, or $P?$, which “filters” states satisfying P and diverges on all other. ■

If C is the set of program states, the corresponding relational KAT can encode programs just like regular commands. Basic expressions are elements of (a subset of) $\mathcal{P}(C \times C)$. Encoding of imperative constructs such as **if** and **while** is done as for regular commands.

Since programs can be encoded in KATs (possibly with the addition of extra structure to manage program features such as, for instance, exceptions or dynamic memory management), it is natural to ask ourselves whether we can talk about program properties, too. It turns out this is the case, as we can formulate HL in a KAT [Koz00]. Given that we are able to encode program properties (as tests) and programs (as general elements of the algebra), the remaining question is how do we encode the validity of an HL triple. In relational KATs, where we can talk about strongest postcondition, $\mathbf{sp}(a, p) \leq q$ is equivalent to the equation $p \cdot a \cdot (\neg q) = 0$. Intuitively, this says that if we start from p , apply a and test for the negation of q , we do not get any result, meeting the intuition that $\mathbf{sp}(a, p)$ is contained in q . A provably equivalent (in any KAT) formulation is the equation $p \cdot a = p \cdot a \cdot q$, that intuitively means that testing for q after applying a on p is redundant. Given the equivalence in relational KATs, it seems reasonable to take this equation as validity for an HL triple in general KATs.

The HL proof system embedded in KATs is shown in Figure 2.5. It handles triples of shape $\{p\} a \{q\}$, with $p, q \in \text{test}(A)$ and $a \in A$. It is very similar to Figure 2.2 thanks to regular commands and KATs being syntactically very close. The first two rules are subsumed by $\{\text{atom}\}$, but we prefer to show them explicitly since 0 and 1 are part of the syntax of KATs. Rule $\{\text{atom}\}$ is analogous to the homonym HL rule. It requires explicitly to check the validity condition since there is no equivalent of the semantics in a KAT. All the remaining rules are the same as HL, just using the syntax of KAT instead of regular commands one.

2.9 Abstract interpretation

2.9.1 Abstract domains

Abstract interpretation [CC77; CC79] is a general framework to define static analyses sound by construction, with the main idea of approximating the program semantics on some abstract domain A instead of working on the concrete domain C . The main tool used to study abstract interpretation are Galois connections.⁶

Definition 2.15 (Galois connection). Given two posets C and A , a pair of monotone functions $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ define a Galois connection when $\alpha \dashv \gamma$. As notation, we write $\langle C \xrightarrow[\alpha]{\gamma} A \rangle$.

We call C and A the concrete and the abstract domain respectively, α the abstraction function and γ the concretization function. We recall some properties of Galois connections, where a function is (co-)additive when it preserves lubs (resp. glbs) of arbitrary subsets, including the empty set:

Proposition 2.16. *Let $\langle C \xrightarrow[\alpha]{\gamma} A \rangle$ be a Galois connection. Then*

1. $id_C \leq \gamma\alpha$
2. $\alpha\gamma \leq id_A$
3. α is additive and γ is co-additive

A concrete value $c \in C$ is called *expressible* in A if $\gamma\alpha(c) = c$. We mostly consider Galois connections in which $\alpha\gamma = id_A$, called Galois insertions. In a Galois insertion α is onto and γ is injective. A Galois insertion is said to be trivial if A is isomorphic to the concrete domain or if it is the singleton $\{\top_A\}$.

To give an intuition of the role of Galois connections in program analysis, we present the following example.

Example 2.17 (Intervals). Consider as C the set of possible values of a variable, for instance i . Since this is an integer value, elements of C are subsets of $\mathbb{Z}$, so $C = \mathcal{P}(\mathbb{Z})$, with the ordering given by set inclusion. A is the set of abstract properties we track in our analysis, and in this example we consider the set of intervals to which i may belong. This means

$$A = \text{Int} = \{[n, m] \mid n \in \mathbb{Z} \cup \{-\infty\}, m \in \mathbb{Z} \cup \{+\infty\}, n \leq m\} \cup \{[+\infty, -\infty]\}$$

This defines the well known abstract domain of intervals [CC77]. The map α is the function that abstracts a set S of possible values of i to the best abstract property, ie. the smallest (most precise) interval that comprises all possible values of i :

$$\alpha(S) = [\min(S); \max(S)]$$

with the usual conventions for $\min$ and $\max$ of empty/unbound set. Since no smaller interval can describe the set S , and this is a superset of S , $\alpha(S)$ is exactly the best abstraction of S .

⁶Galois connections is the name usually given to adjoints in the context of abstract interpretation. We stick to this name when talking about abstract interpretation, and resort to the name adjoint in other contexts.

The concretization map γ is the function that does the inverse operation: given an interval $[n, m]$, thought as formal writing or machine representation, gives back its “meaning”, that is the largest subset of $\mathbb{Z}$ that matches that property:

$$\gamma([n, m]) = \{x \in \mathbb{Z} \mid n \leq x \leq m\}$$

that is exactly what is commonly represented with $[n; m]$: γ is simply translating the formal writing (or, in our context, an abstract property) to a semantic set of values. We omit for simplicity the cases for infinite ends, as they are as expected.

With these definition, it is straightforward to check that these two functions define a Galois connection (actually, a Galois insertion). $\blacksquare$

We overload the symbol A to denote also the function $\gamma\alpha : C \rightarrow C$: this is always an *upper closure operator*, that is a monotone, increasing (ie. $c \leq A(c)$ for all c) and idempotent function. In the following, we use closure operators as much as possible to simplify the notation. Particularly, they are useful to denote domain refinements, as exemplified in the next paragraph. We remark that upper closure operators are as expressive as Galois insertions, as the two formulation are equivalent [CC79]. Particularly, since γ is injective, $A(c) = A(c')$ if and only if $\alpha(c) = \alpha(c')$. Therefore, the use of closure operators is only a matter of notation and it is always possible to rewrite them using abstraction and concretization functions.

The image of an upper closure operator is a Moore family, that is a subset of C containing the top element $\top_C$ and that is closed under meet. Given a subset $S \subseteq C$, we define its Moore-closure (or meet-closure) as

$$\mathcal{M}(S) = \{\bigwedge X \mid X \subseteq S\}$$

It is well known that any Moore family M is the image of a suitable upper closure operator ρ_M , defined as

$$\rho_M(x) = \bigwedge \{y \in M \mid x \leq y\}$$

and that this defines a Galois connection $\langle C \xrightleftharpoons[\rho_M]{\text{id}_M} M \rangle$.

We use $\text{Abs}(C)$ to denote the set of abstract domains over C , and we write $A_{\alpha, \gamma} \in \text{Abs}(C)$ when we need to make the two maps α and γ explicit (we omit them when not needed). Given two abstract domains $A_{\alpha, \gamma}, A'_{\alpha', \gamma'} \in \text{Abs}(C)$ over C , we say A' is a *refinement* of A , written $A' \preceq A$, when $\gamma(A) \subseteq \gamma'(A')$. When this happens, the abstract domain A' is more expressive than A , and in particular for all concrete elements $c \in C$ the inequality $A'(c) \leq_C A(c)$ holds. This define a partial order on $\text{Abs}(C)$, and when C is a complete lattice the resulting structure is known to be a complete lattice too [CC79].

A particular kind of domain refinements are pointed refinement [BGGR22]. They are defined adding a single point to the abstract domain, and then performing Moore closure to recover an abstract domain.

Definition 2.18 (Pointed refinement [BGGR22]). Let $A_{\alpha, \gamma} \in \text{Abs}(C)$ be an abstract domain, and let $z \in C$ be a concrete point. Then the pointed refinement A_z is defined as

$$A_z = \mathcal{M}(\gamma(A) \cup z)$$

We remark that, for any z , $A_z \preceq A$ and, in particular, the two closures are related by

$$A_z(c) = \begin{cases} A(c) \wedge z & \text{if } c \leq z \\ A(c) & \text{otherwise} \end{cases}$$

2.9.2 Abstracting functions

The goal of abstract interpretation is to approximate the computation of functions.

Definition 2.19. Given a monotone function $f : C \rightarrow C$ and an abstract domain $A_{\alpha, \gamma} \in \text{Abs}(C)$, a function $f^\sharp : A \rightarrow A$ is a *sound approximation* (or *abstraction*) of f if

$$\alpha f \leq f^\sharp \alpha$$

The *best correct approximation* (bca for short) of f is $f^A = \alpha f \gamma$.

The bca of f is the most precise of all the sound approximations of f : a function $f^\sharp$ is a sound approximation of f if and only if $f^A \leq f^\sharp$.

Through abstraction, it may very well happens that we lose precision, as shown by the following example.

Example 2.20. Consider the interval domain Int of Example 2.17 and the function f given by (the lifting of) the absolute value:

$$f(S) = \{|x| \mid x \in S\}$$

Its bca is $f^A = \alpha f \gamma$. However, even though this is the most precise abstraction of f we can consider in Int , on some concrete points it still loses precision: fixing as input $S = \{-1, 1\}$ we have

$$\begin{aligned} \alpha(f(\{-1, 1\})) &= \alpha(\{1\}) = [1; 1] \\ f^A \alpha(\{-1, 1\}) &= f^A([-1; 1]) = [0; 1] \end{aligned}$$

■

This issue is well known in abstract interpretation, and for this reason the definition of (global) *completeness* was given [CC79; GRS00].

Definition 2.21 (Complete abstraction). A sound abstraction $f^\sharp$ of f is *complete* when

$$\alpha f = f^\sharp \alpha$$

Intuitively, completeness means that the abstract function $f^\sharp$ is as precise as possible in the given abstract domain A . In program analysis this allows to have greater confidence in the alarms raised because false alarms are only due to the imprecision of the abstraction and not of the computation. It turns out that there exists a complete abstraction of f if and only the bca f^A is complete, and if this happens we say that A is complete for f and write $\mathbb{C}^A(f)$. Moreover, since A is complete for f if and only if $\alpha f = f^A \alpha = \alpha f \gamma \alpha$, and since γ is injective (we always assume a Galois insertion), this is true if and only if $\gamma \alpha f = \gamma \alpha f \gamma \alpha$. Recalling that $A = \gamma \alpha$ we define the completeness property $\mathbb{C}^A(f)$ by the equation

$$\mathbb{C}^A(f) \iff Af = AfA. \quad (2.6)$$

2.9.3 Abstract semantics of regular commands

Consider again regular commands introduced in Section 2.3. While any command r has a bca $\llbracket r \rrbracket^A$, in general an abstract analyser does not know how to compute it. Instead, structural analysers work inductively on the syntax of r to compute a sound abstraction of the concrete semantics $\llbracket r \rrbracket$. This abstract semantics is given in Figure 2.6, and defines

$$\begin{aligned}
\llbracket e \rrbracket_A^\sharp a &\triangleq \llbracket e \rrbracket^A a = \alpha(\llbracket e \rrbracket) \gamma(a) \\
\llbracket r_1; r_2 \rrbracket_A^\sharp a &\triangleq \llbracket r_2 \rrbracket_A^\sharp \llbracket r_1 \rrbracket_A^\sharp(a) \\
\llbracket r_1 \oplus r_2 \rrbracket_A^\sharp a &\triangleq \llbracket r_1 \rrbracket_A^\sharp a \vee A \llbracket r_2 \rrbracket_A^\sharp a \\
\llbracket r^* \rrbracket_A^\sharp a &\triangleq \bigvee_{n \geq 0} (\llbracket r \rrbracket_A^\sharp)^n a
\end{aligned}$$

Figure 2.6: Abstract semantics of regular commands.

the *abstract interpreter* $\llbracket \cdot \rrbracket_A^\sharp : \text{Reg} \rightarrow A \rightarrow A$. This formulation tells a constructive way to compute an abstract semantics of any regular command r provided that the analyser knows the bca of all atomic commands $e \in \text{Exp}$ (or at least those appearing in r). This condition could be further relaxed replacing $\llbracket e \rrbracket^A$ with any sound abstraction $\llbracket e \rrbracket^\sharp$ of $\llbracket e \rrbracket$, but for the sake of simplicity we assume the analyser is able to compute all bcas of basic commands. All other rules are completely analogous to those of the concrete semantics but for the use of the abstract interpreter of syntactic components instead of the concrete semantics. A straightforward proof by structural induction shows that this semantics is monotone and sound:

Proposition 2.22. *The abstract interpreter of Figure 2.6 is monotone and sound w.r.t. the concrete semantics of Figure 2.1; namely for all $r \in \text{Reg}$*

$$\alpha \llbracket r \rrbracket \leq \llbracket r \rrbracket_A^\sharp \alpha$$

Even though the abstract interpreter is sound, in general $\llbracket r \rrbracket_A^\sharp \neq \llbracket r \rrbracket^A$, that is the compositional abstraction is less precise than the bca. This is a well-known issue in abstract interpretation, and its main cause is sequential composition. In fact, while in the concrete domain $\llbracket r_1; r_2 \rrbracket = \llbracket r_2 \rrbracket \llbracket r_1 \rrbracket$, in the abstract domain

$$\llbracket r_1; r_2 \rrbracket^A = \alpha \llbracket r_2 \rrbracket \llbracket r_1 \rrbracket \gamma \leq \alpha \llbracket r_2 \rrbracket \gamma \alpha \llbracket r_1 \rrbracket \gamma = \llbracket r_2 \rrbracket^A \llbracket r_1 \rrbracket^A.$$

This is caused by the fact the abstract interpreter operates on abstract properties only (hence the need for the abstraction between r_1 and r_2).

2.9.4 Under-approximation abstract domains

The definition of Galois connection is not symmetric, in the sense that it puts γ above and α below. This favours over-approximation. A way to see this is the fact that A is an *upper* closure operator, that is $c \leq_C A(c)$. For this reason, to talk about under-approximation domains, we consider “reversed” Galois connections:

Definition 2.23 (Under-approximation Galois connection). Given two posets C and A , a pair of monotone functions $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ define an under-approximation Galois connection when

$$\forall c \in C, a \in A. \quad a \leq_A \alpha(c) \iff \gamma(a) \leq_C c$$

It is important to remark that an under-approximation Galois connection is just a standard Galois connection between the opposite (also called dual) posets C^{op} and A^{op} . Alternatively, it can be seen as a Galois connection between A and C in the reversed

order: for this reason, we denote it with $\langle A \stackrel{\gamma}{\underset{\alpha}{\rightharpoonup}} C \rangle$, where we write A on the left and C on the right. The first point of view allows us to reuse all the machinery for Galois connection, just dualizing results: $A = \gamma\alpha : C \rightarrow C$ is a *lower* closure operator (that is monotone, idempotent and reductive, ie. $A(c) \leq_C c$), α is co-additive and γ is additive, the image of A is a dual Moore family (that is a family of sets containing $\perp_C$ and closed under join). Again, we only consider under-approximation Galois insertions (UGI), that are Galois connections in which $\alpha\gamma = \text{id}_A$. We write $A_{\alpha,\gamma} \in \text{Abs}^{\text{op}}(C)$ to say that A is an under-approximation abstract domain on C with adjoints α and γ , possibly omitting these if superfluous.

As an example of under-approximation abstract domain, we propose the following variation of intervals:

Example 2.24. Consider $C = \mathcal{P}(\mathbb{Z})$, while the abstract domain is the set of all intervals (Example 2.17) containing 0, plus the empty interval:

$$\text{Int}_0 = \{I \in \text{Int} \mid 0 \in I\} \cup \{\perp\}$$

where we used $\perp$ to represent the empty interval $[+\infty, -\infty]$. The map γ is the identity since we want an (under-approximation) Galois insertion, and $\alpha(S)$ is the greatest interval fully contained in S that includes 0. Formally,

$$\alpha(S) = \bigcup \{I \in \text{Int}_0 \mid I \subseteq S\}$$

The result of α is always in Int_0 . If there is no interval containing 0 and contained in S , $\alpha(S)$ is $\perp$. On the contrary, if there is any such interval, than $\alpha(S)$ does contain 0, since is the union of intervals in Int_0 that contains 0 themselves. Moreover, it is an interval because union of overlapping intervals is an interval, too. ■

Chapter 3

Related work

This chapter surveys related work about combining over and under-approximations. Section 3.1 analyses KATs as a possible formalism to describe together over and under-approximation, represented respectively by HL and IL. Section 3.2 present LCL_A , a logic that exploits under-approximation to ensure precision of an (over-approximating) abstraction and is able to prove either program correctness or incorrectness with any triple. Section 3.3 discusses $ic3$, a model-checking algorithm that combines over and under-approximations in a non-trivial way, and its generalizations. Section 3.4 introduces Outcome Logic, a triple based program logic that can express both over and under-approximation properties.

3.1 Unifying formalism

HL and IL operate on dual principles. Because of this, some properties are shared among the two, others are dualized and others still are just different. In order to highlight these similarities and differences, it is interesting to embed HL and IL in a common formalism. As recalled in Section 2.8, KATs are able to encode regular commands [Koz97] as well as HL [Koz00] making it a purely equational theory. It would be interesting to do the same for IL, and in principle it seems reasonable to do so given the duality with HL. However, the symmetry is at the level of images of functions (precisely, strongest postconditions), something that is not explicit in the syntax of KAT. Therefore, it is interesting to study which additional algebraic properties are needed to encode IL in KATs.

In [MOH21], the authors propose modal KATs as the diamond modality can be used to represent strongest postconditions (our collecting semantics $\llbracket \cdot \rrbracket$) [DMS06]. This way, they are able to encode all IL rules but the infinitary [iter], that is needed for completeness of the proof system¹ but requires the existence of countable joins of tests. The authors add this requirement explicitly to the algebraic structure, defining countably-test complete (CTC for short) modal KATs.

Definition 3.1 (Backward diamond). A backward diamond modality on a KAT A is an operator $\langle \cdot | : A \rightarrow \text{test}(A) \rightarrow \text{test}(A)$ satisfying, for all $a, b \in A$, $p, q \in \text{test}(A)$

$$\begin{aligned} \langle a | p \leq q &\iff p \cdot a \leq a \cdot q \\ \langle a \cdot b | p &= \langle b | (\langle a | p) \end{aligned}$$

¹Kleene star can be handled using finite unrolling, as we will show for SIL and IL in Figure 5.5. While this approach is not complete, it is sufficient to prove many triples. We refer the reader to [MOH21, Figure 2] for finitary rules dealing with Kleene star in KATs.

$\frac{}{[p] \ 0 \ [0]} \text{[divergence]}$	$\frac{}{[p] \ 1 \ [p]} \text{[skip]}$
$\frac{}{[p] \ a \ [\langle a p]} \text{[atom]}$	$\frac{p \geq p' \quad [p'] \ a \ [q'] \quad q' \geq q}{[p] \ a \ [q]} \text{[cons]}$
$\frac{[p] \ a \ [r] \quad [r] \ b \ [q]}{[p] \ a \cdot b \ [q]} \text{[seq]}$	$\frac{\exists i \in \{1, 2\} \quad [p] \ a_i \ [q]}{[p] \ a_1 + a_2 \ [q]} \text{[choice]}$
$\frac{[p_1] \ a \ [q_1] \quad [p_2] \ a \ [q_2]}{[p_1 \vee p_2] \ a \ [q_1 \vee q_2]} \text{[disj]}$	$\frac{\forall n \geq 0. [p_n] \ a \ [p_{n+1}]}{[p_0] \ a^* \ [\bigvee_{n \geq 0} p_n]} \text{[iter]}$

Figure 3.1: CTC modal KAT encoding of IL, from [MOH21]

Intuitively, $\langle a|p$ is the strongest postcondition of a on input p . This correspondence is exact in relational KATs, and so it is natural to use it in all modal KATs. Moreover, the standard soundness of HL in KATs is defined by the equation $p \cdot a = p \cdot a \cdot q$, that can be proved equivalent to $\langle a|p \leq q$ in a modal KAT. This further justifies the usage of backward diamond as strongest postcondition, given that $\mathbf{sp}(a, P) \leq Q$ is the classical soundness condition of HL.

Since a CTC modal KAT satisfies all the KAT axioms, it has for free the standard encoding of HL. IL embedding in CTC modal KAT is detailed in Figure 3.1, taken from [MOH21]. The rules are close to standard IL (Figure 2.3), replacing the semantics $\llbracket e \rrbracket$ with the backward modality $\langle a|$, sequential composition with $\cdot$ and choice with $+$. This is thanks to closeness of regular command to the syntax of KATs. The two additional rules for 0 and 1 are, just as for HL, subsumed by [atom], but we add them explicitly since 0 and 1 are part of the syntax of KATs.

In this algebraic setting, the author derive soundness and completeness theorems for both IL and HL, that we summarise below. We remark that soundness and completeness are obtained with respect to a validity notion based on backward diamond, precisely $\langle a|p \leq q$ for HL and $\langle a|p \geq q$ for IL. However, these notions coincide with the usual one when our expected interpretation of $\langle \cdot |$ as the semantics holds. What is really interesting about those are the hypotheses required by each:

Theorem 3.2 (cf. [MOH21]).

1. *HL is sound and complete in any modal KAT.*
2. *IL is sound in any CTC modal KAT without the (Star induction) axiom.*
3. *IL is complete in any CTC modal KAT.*

This theorem is interesting because it shows the symmetry between over and under-approximation is less sharp than it appears at first glance. On the one hand, over-approximation requires the KAT axioms for both soundness and completeness (modality is used to define the validity of a triple, but we can do without changing the notion of validity). The fact that the latter has the same requirements as the former is essentially because loop invariants are both sound *and* complete for over-approximation. On the other hand, for under-approximation no such tool is known, so completeness has stronger hypotheses than soundness. Particularly, soundness of IL does not require *(Star induction)*, the KAT rephrasing of loop invariants. Intuitively, to obtain an under-approximation there is no

need for the loop fixpoint but it is enough to consider any (finite) number of iterations. This is no longer the case for completeness, since we do need the loop semantics to prove that it is a sound under-approximation of itself. This motivates the need for all the KAT axioms (including *(Star induction)*) in point (3) above. In other words, we can prove soundness of finite iterations for any fixpoint, not necessarily the minimal one (ie. requiring *(Star unfold)* only), but only the minimal one is reached by the limit of such iteration, thus we can only prove completeness when we force the semantics to be the least fixpoint (ie. adding *(Star induction)*, too). In terms of fixpoint, the symmetry is broken because we use least fixpoints for the semantics both in over and under-approximation, while a fully dual theory would also use greatest fixpoints.

In CTC modal KAT the authors are also able to formally prove an intuitive connection between HL and IL.

Theorem 3.3 (cf. [MOH21]). *In any CTC modal KAT*

$$\not\models \{p\} a \{q\} \iff \exists p', q'. p' \leq p, q' \not\leq q, \models [p'] a [q']$$

On the one hand, this theorem means that whenever a specification (ie. an Hoare triple) is not met there is a valid incorrectness triple showing a violation of that specification. In the theorem, q' is the violation since $q' \not\leq q$, and is in the post of p because we can prove $[p'] a [q']$ with $p' \leq p$. On the other hand, when the specification is true, we don't need incorrectness reasoning since we already know all valid triples (hence all those we can prove) starting from $p' \leq p$ are bound to have a postcondition q' satisfying the specification q (ie. $q' \leq q$). This theorem states a connection between over- and under-approximation that is obvious in the concrete interpretation with programs and states, but that it is not in a general algebraic model. Being able to prove it means the encoding meets the desired intuition, rising confidence in its correctness. Moreover, this theorem is a first (trivial) example of how over- and under-approximation can positively interact.

To conclude this section, we mention the concurrent work [ZAG22], that addresses the same problem. In this work, the authors show that KAT alone are not enough to express IL (cf. Theorem 1 of their paper). To enrich the algebraic structure, they follow the idea of expressing (to some extent) the codomain, too, and propose to add a top element $\top$ to the KAT, defining the so-called TopKAT. With this addition, given any element a of the TopKAT we have that $\top a$ represents the “codomain” of a . This is formal in relational TopKATs:

Proposition 3.4 (cf. [ZAG22]). *In any relational TopKAT $\mathcal{R}$, for any two elements $p, q \in \mathcal{R}$ and letting cod extract the codomain of a relation,*

$$\begin{aligned} \top p = \top q &\iff \text{cod}(p) = \text{cod}(q) \\ \top p \leq \top q &\iff \text{cod}(p) \subseteq \text{cod}(q) \end{aligned}$$

The authors then use this definition to embed IL in a generic TopKAT. We remark that in this article the focus is not on completeness of the logic, so there is not the CTC assumption on the algebraic structure, but only the those join needed to apply $[\text{iter}]$ are required to exist. Even with this focus shift, the work already shows that a top element is a viable alternative to modality for encoding IL in Kleene algebras.

3.2 Local completeness

Local Completeness Logic on an abstract domain A (LCL_A for short) [BGGR21] is a first example of non-trivial over and under-approximation interaction, with the former embodied by abstract interpretation and the latter by IL.

$\frac{\mathbb{C}_P^A(\llbracket e \rrbracket)}{\vdash_A [P] e \llbracket \llbracket e \rrbracket P \rrbracket} \text{ (transfer) } \quad \frac{P' \leq P \leq A(P') \quad \vdash_A [P'] r [Q'] \quad Q \leq Q' \leq A(Q)}{\vdash_A [P] r [Q]} \text{ (relax)}$	
$\frac{\vdash_A [P] r_1 [R] \quad \vdash_A [R] r_2 [Q]}{\vdash_A [P] r_1; r_2 [Q]} \text{ (seq) } \quad \frac{\vdash_A [P] r_1 [Q_1] \quad \vdash_A [P] r_2 [Q_2]}{\vdash_A [P] r_1 \oplus r_2 [Q_1 \vee Q_2]} \text{ (join)}$	
$\frac{\vdash_A [P] r [R] \quad \vdash_A [P \vee R] r^* [Q]}{\vdash_A [P] r^* [Q]} \text{ (rec) } \quad \frac{\vdash_A [P] r [Q] \quad Q \leq A(P)}{\vdash_A [P] r^* [P \vee Q]} \text{ (iterate)}$	

Figure 3.2: The proof system LCL_A , from [BGGR21].

As described in Section 2.9, abstract interpretation is always sound, but in general it is not complete: composition of best correct abstractions (bcas) is not the bca of the composition. Also widening and narrowing operators introduce incompleteness, and the former are required to ensure termination of the analysis on infinite abstract domains. While in theory completeness ensures no precision is lost, it is a very uncommon situation in general. One of the causes is its requirement to hold *for all inputs*. To weaken this condition, in [BGGR21] the authors propose a notion of *local* completeness, that depends on a specific input.

Definition 3.5 (Local completeness, cf.[BGGR21]). Let $f : C \rightarrow C$ be a concrete function, $c \in C$ a concrete point and $A \in \text{Abs}(C)$ and abstract domain for C . Then A is *locally complete* for f on c , written $\mathbb{C}_c^A(f)$ iff

$$Af(c) = AfA(c).$$

A remarkable difference between global and local completeness is that, while the former can be proved compositionally on the command only [GLR15], the latter needs the input to each fragment of the program. Consequently, to carry on a compositional proof of local completeness, information on the input to each subpart of the program is also required, i.e., all traversed states are important. However, local completeness enjoys an “abstract convexity” property: if f is locally complete on a point c , then it is locally complete on any point d between c and $A(c)$:

Lemma 3.6 (Abstract convexity, cf. [BGGR21]). *If $\mathbb{C}_c^A(f)$ and $c \leq d \leq A(c)$, then $\mathbb{C}_d^A(f)$.*

This observation has been crucial in the design of the proof system LCL_A . The proof system depends on an abstract domain A , and is able to prove triples $\vdash_A [P] r [Q]$ ensuring that:

1. Q is an under-approximation of the concrete semantics $\llbracket r \rrbracket P$,
2. Q and $\llbracket r \rrbracket P$ have the same over-approximation in A ,
3. A is locally complete for $\llbracket r \rrbracket$ on input P .

Point (2) means that, given a specification Spec expressible in A , any provable triple $\vdash_A [P] r [Q]$ either proves correctness of r with respect to Spec or expose an alert in $Q \setminus \text{Spec}$. This in turns correspond to a true one because, by point (1), Q is an under-approximation of the concrete semantics $\llbracket r \rrbracket P$, as pointed out by Corollary 3.8 below.

The proof system is defined in Figure 3.2. It is a logic of under-approximations, much like IL (in fact, IL is an instance of LCL_A using the trivial abstract domain A containing

only the $\top$ element), but with one additional constraint: the under approximation Q must have the same abstraction of the concrete semantics $\llbracket r \rrbracket P$, as for instance explicitly required in rule (relax). This, by the abstract convexity property mentioned above, means that local completeness of $\llbracket r \rrbracket$ on the *under-approximation* P of the concrete store is enough to prove local completeness. This way, LCL_A exploits the interaction of over- and under-approximation. The latter is used to ensure the abstraction is locally complete, ie. guarantees precision of the over-approximation. Conversely, the presence of the abstraction in rule (iterate) speeds up the computation as it allows to stop as soon as $A(P)$ is an abstract loop invariant, not having to deal with (possible infinitary) concrete invariants. Note that this doesn't guarantee termination, since an infinite number of iterations may be needed to reach the abstract invariant.

Formally, the three key properties (1–3) above are subsumed by the following result:

Theorem 3.7 (Soundness, cf. [BGGR21]). *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$. If $\vdash_A [P] \text{ r } [Q]$ then:*

1. $Q \leq \llbracket r \rrbracket P$,
2. $\alpha(\llbracket r \rrbracket P) = \alpha(Q)$,
3. $\llbracket r \rrbracket_A^\# \alpha(P) = \alpha(Q)$.

We say that a triple satisfying these three conditions is *valid*, written $\models_A [P] \text{ r } [Q]$. As a consequence of this theorem, given a specification expressible in the abstract domain A , a provable triple $\vdash_A [P] \text{ r } [Q]$ can determine both correctness and incorrectness of the program r :

Corollary 3.8 (Proofs of Verification, cf. [BGGR21]). *Let $A_{\alpha, \gamma} \in \text{Abs}(C)$ and $a \in A$. If $\vdash_A [P] \text{ r } [Q]$ then*

$$\llbracket r \rrbracket P \leq \gamma(a) \iff Q \leq \gamma(a).$$

The corollary is useful in program analysis and verification because, given a specification expressible in A and a provable triple $\vdash_A [P] \text{ r } [Q]$, it allows to distinguish two cases.

- If $Q \subseteq \gamma(a)$, then we have also $\llbracket r \rrbracket P \subseteq \gamma(a)$, so that the program is correct with respect to the specification.
- If $Q \not\subseteq \gamma(a)$, then also $\llbracket r \rrbracket P \not\subseteq \gamma(a)$, that means $\llbracket r \rrbracket P \setminus \gamma(a)$ is not empty and thus contains a true alert of the program. Moreover, since $Q \subseteq \llbracket r \rrbracket P$ we have that $Q \setminus \gamma(a) \subseteq \llbracket r \rrbracket P \setminus \gamma(a)$, so that already Q is able to pinpoint some issues.

To better show how this work, we briefly introduce the following example (discussed also in [BGGR21] where it is possible to find all details of the derivation).

Example 3.9. Consider the concrete domain $C = \mathcal{P}(\mathbb{Z})$, the abstract domain Int of intervals, the precondition $P = \{1; 999\} \in C$ and the command $r \triangleq (r_1 \oplus r_2)^*$, where

$$\begin{aligned} r_1 &\triangleq (x > 0)?; x := x - 1 \\ r_2 &\triangleq (x < 1000)?; x := x + 1 \end{aligned}$$

In LCL_A it is possible to prove the triple $\vdash_{\text{Int}} [P] \text{ r } [Q]$, whose postcondition is $Q = \{0; 2; 1000\} \in C$. Consider the two specification $\text{Spec} = (x \leq 1000)$ and $\text{Spec}' = (x \geq 100)$. The triple is then able to prove correctness of Spec and incorrectness of Spec' . For the former, observe that $Q \subseteq \text{Spec}$. By Corollary 3.8 we then know $\llbracket r \rrbracket P \subseteq \text{Spec}$, that is correctness. For the latter, Q exhibits two witnesses to the violation of Spec' , that are $0, 2 \in Q \setminus \text{Spec}'$. By point (1) of soundness we then know that $0, 2 \in Q \subseteq \llbracket r \rrbracket P$ are true alerts. ■

In LCL_A , being able to prove any triple $\vdash_A [P] \text{ r } [Q]$ allows to show both correctness and incorrectness of r . However, if r is not locally complete on P , or more in general any of the local completeness proof obligations introduced by rule (transfer) (the only axiom of the logic) fails, the proof cannot be completed. To handle this issue, [BGGR22] proposes the idea of changing the abstract domain in which the derivation is performed. Following what had been done for completeness [GRS00], they propose to minimally (in the lattice of abstract interpretations) refine the abstract domain. Unluckily, such a minimal refinement in general does not exist, so that the authors propose a different notion of “best” refinement. They consider pointed refinements, that are defined by the addition of a single point to the abstract domain (followed by a Moore closure operation). Then they compare these pointed refinements not in the lattice of abstract interpretations but by the precision of the additional point. When there exists a most abstract point whose pointed refinement is locally complete, they call this domain *pointed (locally complete) shell*:

Definition 3.10 (Pointed shell, cfr. [BGGR22]). Let $f : C \rightarrow C$ be a monotone concrete function, $A \in \text{Abs}(C)$ be an abstract domain and $c \in C$ a concrete point. The pointed shell of A on c w.r.t. f exists when the maximum of the set

$$\{x \in C \mid \mathbb{C}_c^{A_x}(f)\}$$

exists and, letting u be such maximum, the pointed shell is $A_u \in \text{Abs}(C)$.

Other than characterizing the existence of pointed shell, they propose two strategies to repair the abstract domain using pointed shells. One of the two, the so-called backward repair, mostly operates on the abstract and so does not fit our goal of combining over and under-approximation. The other, forward repair, instead operates on under-approximation of concrete points. It processes local completeness proof obligations in order, starting from the input and following the control flow. Thanks to abstract convexity of local completeness, this strategy works even on under-approximations of concrete stores, so that it integrates well with LCL_A . The strategy computes local completeness proof obligations in order, either reaching the end of the program (thus completing the analysis) or finding a failed one. In this case, it repairs the abstract domain to the pointed shell (using the under-approximation) and then restart the analysis in the refined domain.

To conclude this section, we point out there exists an algebraic formulation of LCL_A . In [MR22], the authors take inspiration from the works discussed in the previous Section 3.1 and embed LCL_A in (a suitable extension of) KAT. Their first contribution is the definition of an abstract interpretation of KAT, a problem not studied before. Exploiting this, they embed LCL_A in both modal KATs and TopKATs. The technical development of these embeddings is similar to that of [MOH21] and [ZAG22], but it shows that such an embedding is effective as it preserves all property of LCL_A . Lastly, we remark that just as LCL_A generalizes IL, in [MR22] they recover the embedding of IL in modal KATs/TopKATs as a special case of LCL_A ’s.

3.3 IC3/PDR

ic3 (“Incremental Construction of Inductive Clauses for Indubitable Correctness”), also called PDR (“Property directed reachability”), was first proposed by Bradley as a model checking algorithm [Bra11]. Given a safety property P and a finite transition system, it either proves the property or outputs a counterexample. In this sense, ic3 operates both as prover and a bug finder [Bra12]. Its ingenuity consists in using an over-approximation to guide the search for counterexamples, that in turn are used to guide the progressive

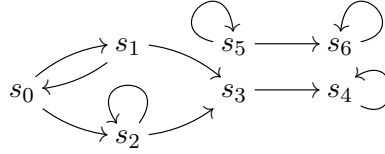


Figure 3.3: The transition system of Example 3.11, with $S = \{s_0, \dots, s_6\}$ and $I = \{s_0\}$.

refinement of the over-approximation. This means that the core of **ic3** is a combination of over- and under-approximations. Thanks to this, it quickly became one of the best hardware model checker. Moreover, the algorithm was later applied to other settings, such as probabilistic transition systems, software model checking or generic complete lattices.

In its original formulation, **ic3** operates on a finite transition system. Let $\mathcal{S}$ be the (finite) set of states, $I \subseteq \mathcal{S}$ the set of initial states and $T : \mathcal{P}(\mathcal{S}) \rightarrow \mathcal{P}(\mathcal{S})$ the transition function: given a set of states S , it returns the sets of states reachable from S in one step. If $\rightarrow \subseteq \mathcal{P}(\mathcal{S} \times \mathcal{S})$ is the transition relation, $T(S) \triangleq \{t \mid \exists s \in S. s \rightarrow t\}$. In this case, $\text{lfp}(T \cup I)$ is the set of all states reachable from I . Actually, I and T are represented by propositional formulas and a SAT solver is employed to prove implications and satisfiability queries.

Example 3.11 (Safety problem for transition systems). Consider the transition system in Figure 3.3. Hereafter we write S_j for the set of states $\{s_0, s_1, \dots, s_j\}$ and we fix the set of safe states to be $P = S_5$. We know that the transition system is safe if and only if $\text{lfp}(T \cup I) \subseteq P$, and by Kleene Theorem 2.3 this correspond to the limit of the initial chain

$$\emptyset \subseteq I \subseteq S_2 \subseteq S_3 \subseteq S_4 \subseteq S_4 \subseteq \dots$$

which stabilizes at S_4 and is therefore below P . Note that the $(j+1)$ -th element of the initial chain contains all the states that can be reached from I in at most j transitions.

It is worth to remark that T has a right adjoint $G : \mathcal{P}(\mathcal{S}) \rightarrow \mathcal{P}(\mathcal{S})$ defined for all $X \in \mathcal{P}(\mathcal{S})$ as $G(X) \triangleq \{s \mid \forall t. s \rightarrow t \implies t \in X\}$ (the right adjoint is also called $\widetilde{\text{pre}}$). Thus by (2.1), $\text{lfp}(T \cup I) \subseteq P$ iff $I \subseteq \text{gfp}(G \cap P)$. We can check this by computing the final chain

$$\dots \subseteq S_4 \subseteq S_4 \subseteq P \subseteq S$$

which again stabilizes at S_4 and is therefore below P . Note that the $(j+1)$ -th element of the final chain contains all the states that in at most j transitions reach safe states only. ■

ic3 fundamental data structures is a sequence $(X_i)_{0 \leq i \leq k+1}$ of over-approximations of states reachable in at most i steps. k is the number of so-called *major iterations* the algorithm has performed. In **ic3**, X_i are logical formulas in CNF. Moreover, the following invariants are kept: (1) $\text{clause}(X_{i+1}) \subseteq \text{clause}(X_i)$, (2) $T(X_i) \Rightarrow X_{i+1}$, (3) $X_k \Rightarrow P$. As a consequence, (1) implies $X_i \Rightarrow X_{i+1}$, and together with (3) this means all elements of the sequence (but possibly X_{k+1}) are strengthening of P . The over-approximation X_k is the “frontier” of the analysis. k is increased by major iterations. Being at major iteration k means the algorithm has proved no violation of P is reachable within k steps, and is working on $k+1$. Once the algorithm can prove $T(X_k) \Rightarrow P$, it refines X_{k+1} by conjoining it with P , increases k and sets $X_{k+2} = \mathbf{true}$ (the empty set of clauses).

At this point, the algorithm also performs what is called clauses propagation. Basically, it considers any clause c in any of the X_i and checks whether $T(X_i) \Rightarrow c$. If this is the case, c is conjoined to X_{i+1} , as this preserves all the invariants. In this sense, the clause c is “propagated” from X_i to X_{i+1} , and the algorithm goes on with other clauses in X_i , then

with X_{i+1} and so on. If during this step, at any point $X_i = X_{i+1}$, the algorithm proved the specification P because $T(X_i) \Rightarrow X_{i+1} = X_i$, so X_i is an invariant, and $X_i \Rightarrow P$.

However, in general the implication $T(X_k) \Rightarrow P$ will not be satisfied. In this case, the SAT solver produces a counterexample s , that is a state in X_k such that one of its successors does not satisfy P . This means that not only X_k is not an invariant, but that it contains some states which violate (after one step) the specification. The idea of **ic3** is then to see whether this state is introduced by the over-approximation, so that it can be ruled out, or is really reachable, so that a true counterexample is found. So the algorithm looks for a clause c to rule out s from X_k . It takes c whose literals are a subset of those in $\neg s$ (so that $c \Rightarrow \neg s$) and checks whether $I \Rightarrow c$ and $T(X_k \wedge c) \Rightarrow c$. If it can find any such c , it conjoins it to all X_i up to X_{k+1} (it's easy to check this preserves all three invariants). Doing so, the algorithm removes s from X_k , so it tries again the implication $T(X_k) \Rightarrow P$, that will either be satisfied or find a different counterexample. If instead no c satisfies this implication, the algorithm does the same for X_{k-1} , and then for X_{k-2} . Suppose it finds it at this last check (it can't go further because s is not in $T(X_{k-2})$, as it has a successor violating P while $T(X_{k-2}) \Rightarrow X_{k-1}$ and $T(X_{k-1}) \Rightarrow P$, hence $T(X_{k-2}) \Rightarrow \neg s$). So the algorithm found a clause c such that $c \Rightarrow \neg s$, $I \Rightarrow c$ and $T(X_{k-2} \wedge c) \Rightarrow c$: it can conjoin it to all X_i up to X_{k-1} . This rules s out of X_{k-1} but does not solve the issue of s being in X_k . However, here's the ingenuity of the algorithm: now the query $T(X_{k-1}) \Rightarrow \neg s$ is either satisfied, so we can conjoin $\neg s$ to X_k , or its failure pinpoints a predecessor t of s in X_{k-1} . That is, it either shows s is a spurious counterexample introduced by the over-approximation of X_k or it traces it back to a possible counterexample in X_{k-1} .

The procedure restarts from t and X_{k-1} . This recursive call will either prove t spurious, so that we can turn back to $T(X_{k-1}) \Rightarrow \neg s$, or find a predecessor u of t in X_{k-2} . This going up and down the sequence of over-approximation, refining it along the way, in the end will either prove $T(X_k) \Rightarrow P$ or find a chain of true counterexample starting from I , disproving P . In this exploration, going up the chain means the over-approximation allows to discard a counterexample, while going down means the counterexample (that is an under-approximation) guides the refinement of the over-approximation.

Our description is high level and leaves out many details. We are not interested in discussing them here, and refer the reader to [Bra11], as we will detail the more general algorithm LT-PDR later in this Section. The one thing we want to point out is that the choice of the clause c used to rule out states violating the specification is minimal, ie. no strict subclause (made of a subset of the literals in c) satisfies the properties required. This in turn means that c includes less states (removing a literal from a disjunction makes it smaller), that is it removes from F_i as much states as possible. This choice is done in order to remove spurious counterexamples as quickly as possible. We will discuss this issue later.

ic3 was developed for hardware model checking, that means variables are boolean and the transition system is finite. However, its core ideas are deep and not tied to the specific domain, which lead to a host of derived work in other fields. One of the main challenges to generalize it is moving to infinite states space, since termination of the algorithm relies on finiteness of the domain. To cope with this issue, the crucial step is “generalization”, that in **ic3** is the choice of $c \Rightarrow \neg s$ to remove s . Taking c as general as possible removes many (unreachable) states at once. Clearly this does not impact termination if the state space is finite, but is crucial whenever it is infinite. Hence, while generalization is not needed for soundness, it becomes fundamental for termination of infinite generalizations. This notwithstanding, **ic3** has been successfully generalized. One example is **PrIC3** (“Probabilistic **ic3**”) [Bat+20], for model checking Markov decision processes (MDPs). These

LT-PDR (F, α)

```

<INITIALISATION>
   $(\vec{x} \parallel \vec{c})_{n,k} := (\perp, F \perp \parallel \varepsilon)_{2,2}$ 
<ITERATION>                                     %  $\vec{X}, \vec{C}$  not conclusive
  case :
     $k \geq 2$  :                                     % (Induction)
      choose  $j \geq 2$  and  $z \in L$  such that  $x_j \not\leq z$  and  $F(x_{j-1} \wedge z) \leq z$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \wedge_j z \parallel \vec{c})_{n,k}$ 
       $\vec{c} = \varepsilon$  and  $x_{n-1} \leq \alpha$  :               % (Unfold)
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x}, \top \parallel \varepsilon)_{n+1,n+1}$ 
       $\vec{c} = \varepsilon$  and  $x_{n-1} \not\leq \alpha$  :           % (Candidate)
      choose  $z \in L$  such that  $z \leq x_{n-1}$  and  $z \not\leq \alpha$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \parallel z)_{n,n-1}$ 
       $\vec{c} \neq \varepsilon$  and  $c_k \leq F(x_{k-1})$  :         % (Decide)
      choose  $z \in L$  such that  $z \leq x_{k-1}$  and  $c_k \leq F(z)$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \parallel z, \vec{c})_{n,k-1}$ 
       $\vec{c} \neq \varepsilon$  and  $c_k \not\leq F(x_{k-1})$  :       % (Conflict)
      choose  $z \in L$  such that  $c_k \not\leq z$  and  $F(x_{k-1} \wedge z) \leq z$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \wedge_k z \parallel \text{tail}(\vec{c}))_{n,k+1}$ 
    endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. x_{j+1} \leq x_j$  then return true %  $\vec{X}$  conclusive
  if  $k = 1$  then return false %  $\vec{C}$  conclusive

```

Figure 3.4: LT-PDR, from [Kor+22]

are basically transition systems where, fixed the action, the state to which the system transitions is not determined but is chosen with a given probability distribution. Without entering in too much details, the key difference is that a system configuration is not a single state but a probability distribution, meaning the state space is infinite. The authors propose a first algorithm which depends on an heuristic for generalization and always terminates, but gives up correctness when the system detects a counterexample: it may very well return a false alarm. Then, to recover correctness, they propose an effective way to find a “good” heuristics, that basically amount to run their algorithm repeatedly, using the false counterexample to refine the heuristic every step until either safety is proved or a true counterexample is found. Another generalization is software `ic3`, that uses the same core algorithm to model check software systems [CG12; LNNK20]. Given their state spaces are infinite and variables are not just boolean, software `ic3` relies on SMT instead of SAT solvers. Moreover, they exploit explicitly the control-flow structure of programs, as doing so implicitly has been shown far less effective [CG12]. This makes the algorithms more involved, but shows that `ic3` has indeed the potential to scale to infinite state spaces.

Cousot: dual narrowing?

3.3.1 LT-PDR

An interesting point of view is taken by [Kor+22]. In this article, the authors propose a generalization of `ic3` whose only constraint is that the space state is a complete lattice, that they call LT-PDR (“Lattice theoretic property directed reachability”). While LT-PDR is extremely generic and needs heuristics to be instantiated for particular domains, it captures the essence of the algorithm, showing in particular which properties are needed for soundness and termination. In fact, the paper highlights how `ic3` is based on Knaster-

Tarski (for proving safety) and Kleene (for finding counterexamples) fixed-point theorems (see Section 2.1). LT-PDR is presented in Figure 3.4. Its notation differs a little from **ic3**. Given a complete lattice L , a monotone function $F : L \rightarrow L$ and a property $\alpha \in L$, the goal of the algorithm is to either prove or find a counterexample to $\text{lfp}(F) \leq \alpha$. To encode for instance the original **ic3** problem in this settings, it is sufficient to take $L = \mathcal{P}(\mathcal{S})$, $F(S) = I \cup T(S)$ and $\alpha = P$; it is not hard to verify that this gives exactly the model checking problem. With this in mind, F broadly correspond to the transition relation T ; the sequence of over-approximations is $(x_i)_{0 \leq i \leq n}$, while $\vec{c}$ is the so-called *negative sequence*, and is more or less the call stack for the recursion on predecessors of error states. Intuitively, the algorithm is building an over-approximating sequence $\vec{x}$ of the fixpoint iterates, hoping to reach a safe abstract fixpoint. However, when the next over-approximating iterate is not safe, it must contain a counterexample, that is then either traced back toward initial states or identified as spurious and hence removed.

The algorithm keeps the following invariants on $\vec{x}$, analogously to **ic3**: (1) $x_i \leq x_{i+1}$, (2) $F(x_i) \leq x_{i+1}$, (3) $x_k \leq \alpha$. **Unfold** increases n , advancing the major iteration. It does not perform clause propagation, though: this is the duty of **Induction**. The algorithm is nevertheless sound, as it basically already propagates clauses in **Conflict**. **Candidate** broadly correspond to the case when $T(X_k) \Rightarrow P$ is not satisfied, with a slight difference. Here z is a counterexample to $x_{n-1} \leq \alpha$, not an element of x_{n-2} that is predecessor of a bad state. This difference is insubstantial, though, as the algorithm just recur on this element going to its predecessors. **Decide** and **Conflict** are, respectively, a new recursive call and the conclusion of a previous one. On the one hand, when c_k , the current predecessor of a bad state, is contained in $F(x_{k-1})$, it means it has a predecessor in the over-approximation. This predecessor is z , and the algorithm goes on recursing on it. On the other hand, if c_k is not in $F(x_{k-1})$ it's a spurious counterexample, introduced by the over-approximation: it is then removed refining x_{k-1} by conjoining with a suitable z that excludes c_k , just like the clause c is conjoined to X_i in **ic3**. The two termination conditions correspond to safety and unsafety respectively: the first one checks whether it reached an invariant (that is $X_{i+1} = X_i$); the second one verifies whether the predecessors reached an initial state (as $F(\perp) = I$ with the given definition of L and F).

The choice of z in **Induction**, **Candidate**, **Decide** and **Conflict** is left unspecified by the algorithm. Save for induction (that is anyway not necessary for neither soundness nor termination), there are canonical choices for z in the rules, but better solutions can be provided by heuristics. This allows LT-PDR to accommodate for other known instances of the algorithm, such as **ic3** and **PrIC3**, just fixing the right lattice and heuristics. While in general LT-PDR does not help with this choice, it is able to highlight pros and cons behind them. As briefly remarked above, in general we can look for either a larger or smaller z in each of the rules. There are two different kind of choices here: **Induction** and **Conflict**, which pick a z to *refine* the over-approximation, and **Candidate** and **Decide**, whose goal is identify a counterexample. For the latter, a larger z means that we are possibly examining more counterexamples at once. However, if any of these counterexample is spurious we have to apply **Conflict** to remove that z , and possibly restart with a smaller one containing all the counterexamples we did not discard with the refinement (that are all the true ones, but possibly also some of the false ones). Moreover, this requires to work on many states (e.g., application of F , that must be exact for the algorithm to work) at once, that may be costly. On the other hand, a smaller z means considering just a few counterexamples at a time. While a lucky choice of such an z may lead very quickly to a true counterexample, if the safety property is satisfied this may cause the removal of counterexample one by one, possibly taking a lot of time or preventing termination. We

note that `ic3` follows the second path, only examining the single counterexample returned by the SAT solver. Considering instead the z used to refine the over-approximation, there are two conflicting, driving forces guiding the choice. On the one hand, a smaller z removes more counterexamples (this is the path chosen by `ic3` with its generalization to a minimal subclause). On the other hand, a larger z ensures more abstract over-approximations, yielding less expensive computations and, when there are no counterexamples, a faster fixpoint convergence.

The authors discuss the termination of LT-PDR. As many choices are left unspecified, the best they were able to prove unconditionally is the *existence* of a sequence of choices that make the algorithm end. In addition, the authors propose a set of sufficient but very restrictive conditions that guarantee termination for all possible choices of the algorithm:

- the complete lattice L is well-founded, and
- either $\text{lfp}(F) \not\leq \alpha$ (i.e., there is a counterexample) or $\text{lfp}(F) \leq \alpha$ (the property is satisfied) and there are no strictly increasing infinite chains bounded by α .

Intuitively, the first condition means that eventually the algorithm proceeds to the next major iteration (i.e., increases n). The second instead limit the number of major iterations to a finite number: either it finds a counterexample at some point, or it does not but then it cannot increase n arbitrarily while staying below α . While these conditions are not necessary for termination, nor they are satisfied by many domains, they give a possible starting direction to investigate termination strategies for LT-PDR.

3.4 Outcome Logic

Outcome Logic (OL) [ZDS23] is a recent triple-based program logic, inspired by IL and its ability to search for true bugs. However, it is based on the key insight that under-approximation (of the program behaviour) and reachability (of true error states) are distinct concepts. To handle them separately, OL starts back from HL and generalizes it to use as assertions not properties of states but properties on an *outcome monoid*, which for instance can be sets of states or probability distributions. This allows OL to unify safety and reachability, as well as over and under-approximation in the same logic.

An OL triple has the familiar shape $\langle P \rangle r \langle Q \rangle$. However, P and Q are not assertions on states in Σ . Rather, they are properties over an outcome monoid $M\Sigma$. Particularly, this allows the assertion language to include a new connective $\oplus$, called *outcome conjunction*, that correspond to the monoidal operation on M . To understand the differences, consider the three formulae $x = 0 \wedge y = 1$, $x = 0 \vee y = 1$ and $x = 0 \oplus y = 1$ and the powerset monoid. In this instance, $S \in M\Sigma = \mathcal{P}(\Sigma)$ is a *set of states* rather than just a state $\sigma \in \Sigma$. A set of states S is a model of $x = 0 \wedge y = 1$ if, for all states $\sigma \in S$, $\sigma(x) = 0$ and $\sigma(y) = 1$. This is in line with the intuition we have from HL of this assertion. Things are already a bit different for $x = 0 \vee y = 1$: a set of states S satisfy this formula if either $\sigma(x) = 0$ for all states $\sigma \in S$ or $\sigma(y) = 1$ for all states $\sigma \in S$. Therefore, for instance the set $[x \mapsto 0, y \mapsto 0], [x \mapsto 1, y \mapsto 1]$ does *not* satisfy $x = 0 \vee y = 1$ because it is not the case that neither all states have $x = 0$ nor that all states have $y = 1$. Note that this notion of having all states satisfying either one or the other disjunct has no correspondence in HL because properties are satisfied only by single states, not sets of states. Lastly, the outcome conjunction $x = 0 \oplus y = 1$ is satisfied by a set of states S if it possible to partition $S = S_1 \cup S_2$ in two *non-empty* sets S_1, S_2 such that S_1 satisfies $x = 0$ and S_2 satisfies $y = 1$. For instance, $[x \mapsto 0, y \mapsto 0], [x \mapsto 1, y \mapsto 1]$ does satisfy

$x = 0 \oplus y = 1$ because we can split it as $[x \mapsto 0, y \mapsto 0] \uplus [x \mapsto 1, y \mapsto 1]$ where the first set trivially satisfies $x = 0$ and the second $y = 1$. Differently than disjunction though, the set $[x \mapsto 0, y \mapsto 0], [x \mapsto 0, y \mapsto 7], [x \mapsto 0, y \mapsto 42]$ does not satisfy $x = 0 \oplus y = 1$ because we cannot split it in two non-empty subsets such that one satisfies $y = 1$: this non-emptiness requirement ensures that all outcomes separated by $\oplus$ are reachable in S .

Given this assertion language, an outcome triple has a validity condition similar to HL. Given a lifting $\llbracket r \rrbracket^\dagger$ of the semantics of r to the monoid $M\Sigma$ (this is, for instance, the Kleisli lifting of $\llbracket r \rrbracket$ when M is a monad) the triple $\langle P \rangle r \langle Q \rangle$ is valid if and only, for all elements of the monoid $m \in M\Sigma$ that satisfy the precondition ($m \models P$) then the final outcomes satisfy Q : $\llbracket r \rrbracket^\dagger m \models Q$. For instance, when M is the powerset monad, the lifted semantics $\llbracket r \rrbracket^\dagger$ is simply the collecting semantics from Figure 2.1. This gives a natural way to encode over-approximation (i.e., HL triples), with the addition of the outcome conjunction to prescribe that all outcomes mentioned in the postcondition are truly reachable.

However, the authors want to describe under-approximation as well, that is, be able to specify only a subset of the program behaviours. To achieve this, they use a very specific kind of over-approximation: they introduce a special assertion $\top$ that is satisfied by any element $m \in M\Sigma$. Since all outcomes in the post must be reachable, to specify only some program behaviour instead of all it is enough to over-approximate with $\top$ all the outcomes you do not want to describe. This is a sound over-approximation, but it only specifies that some of the outcomes are truly reachable, thus ignoring some program behaviours and performing under-approximation.

Example 3.12. Consider the motivating example from [ZDS23, § 2.1].

$$r \triangleq x := \text{malloc}(); *x := 1$$

The above program, written in C syntax, allocates a pointer with `malloc` and then tries to dereference it, forgetting that `malloc` can fail and return `null`. Therefore, the HL triple $\{\text{true}\} r \{x \mapsto 1\}$ is not valid: if `malloc` fails, the program ends in an error state.² The correct HL triple is then $\{\text{true}\} r \{(x \mapsto 1) \vee \text{err}\}$, where `err` describes that some error occurred. However, this HL triple does not tell that both states where $(x \mapsto 1)$ and `err` are reachable: they could have been added by the over-approximation.

On the other hand, IL can show reachability of these states: $[\text{true}] r [(x \mapsto 1) \vee \text{err}]$ is a valid IL triple. Moreover, we may not be interested in the $x \mapsto 1$ outcomes since we already found an error, and for efficiency reasons a tool may want to drop it. IL accounts for this with its consequence rule, that allows to derive the triple $[\text{true}] r [\text{err}]$.

OL is able to do both, via the outcome conjunction $\oplus$ and the trivial outcome $\top$. The OL triple $\langle \text{true} \rangle r \langle (x \mapsto 1) \oplus \text{err} \rangle$ is valid, stating the safety property that all reachable outcomes satisfy $(x \mapsto 1) \oplus \text{err}$. However, it also tells a reachability property, namely that both $(x \mapsto 1)$ and `err` are reachable outcomes of the program. To account for under-approximation, OL can use its consequence rule to weaken the postcondition according to the implication $(x \mapsto 1) \oplus \text{err} \implies \top \oplus \text{err}$: therefore, the OL triple $\langle \text{true} \rangle r \langle \top \oplus \text{err} \rangle$ is valid. Since $\top$ is a trivial assertion, satisfied by any outcome, this triple means that `err` is a reachable outcome, and other reachable outcomes (if any) are unconstrained because they only have to satisfy $\top$. ■

The proof system for OL [ZDS23, Figure 4] is parametric in the chosen outcome monoid. The authors also show how to enrich this proof system with rules specific for probabilistic programs [ZDS23, Figure 7] and heap-manipulating programs [ZDS23, Figure 6]. This last instance is further explored in subsequent work [ZSS24], which propose

²This is actually undefined behaviour in C. We won't deal with that here and just assume this actually causes a recognizable error, such as a segmentation fault.

Outcome Separation Logic, a logic where the heap manipulation is baked into the logic itself and can be further composed with the outcome monoid to get, for instance, a probabilistic separation OL. This allows to explore a new tri-abduction algorithm and to prove a more general frame rule that is valid for any outcome monoid.³

Another result proved for OL is the ability to disprove triples within the logic itself. As show for instance in Theorem 3.3, IL can disprove HL triples, i.e., prove that a given HL triple is not valid. However, neither HL nor IL can disprove triples of the same logic: no (set of) valid HL triple can show that an HL triple is not valid. Instead, OL is able to do just that. If we consider sets of elements of $M\Sigma$ instead of syntactic assertions (see Section 2.4), any OL triple is not valid if and only if some other OL triple is valid. Thus, encoding HL triples as OL ones, it is possible to disprove them within the logic. Moreover, this result is made syntactic (i.e., using formulae in an assertion language) for the nondeterministic and probabilistic instance of OL presented in the paper. Therefore, OL can use under-approximation to disprove over-approximation and vice versa.

We conclude pointing out that [Zil24] propose yet another generalization of OL (using weighted computations) for which they provide a complete proof system, at the cost of using semantics assertions (in that case, arbitrary weighted collections of elements of Σ) instead of syntactic ones.

3.5 Summary

In this chapter, we showed some works that exploits both over and under-approximation. They pinpoint symmetries as well as fundamental differences between the two, and combines them so that they help each other, in order to get the best out of both. We discussed an algebraic formulation that incorporate both, then three techniques - namely LCL_A , $ic3/PDR$ and OL - which are able to exploit this combination in different and non trivial ways. In the next chapters, we first deepen our understanding of the relations between over and under-approximation, and then try to combine them in more effective ways.

³The OL instance already admitted a frame rule but it was limited to deterministic computations.

Chapter 4

Under-approximation abstract domains

In this chapter, we try to use abstract interpretation for under-approximation analysis. In principle, the over-approximation theory can be dualized in an order-theoretic sense to obtain results for under-approximation. However, in this chapter we show that it is not so simple. Particularly, the semantics of basic constructs of the language are not dualized: therefore, the dual of an abstract domain that “behaves well” with respect to basic transfer functions may not enjoy the same property.

We first point out some intuitive reasons that break the symmetry between over and under-approximation. Then, building on these observations, we formally derive some negative results showing that it is not possible to define Galois connection-based under-approximation abstract domains in a large class of instances. More in details, we assume that (i) abstract analyses should return non-trivial results for large classes of programs and (ii) to justify the convenience of the abstract analysis, the abstract domain should be significantly “smaller” than the concrete powerset. Under these assumptions, we prove that there is no under-approximation abstract domain able to analyse programs encoding certain classes of basic transfer functions.

The content of this chapter is based on [ABG22; ABG24].

4.1 Overview

In their first work on Abstract Interpretation [CC77], Patrick and Radhia Cousot introduced the formal theory that could be used to study both over and under-approximations. However, while the former has been extensively studied, there are only sparse studies on under-approximation abstract domain. For instance, Lev-Ami et al. [LSRG07] proposed to use complements of over-approximation domains to infer sufficient preconditions for program correctness. However, such an approach is severely limited in proving incorrectness, as we show in Example 4.1. For the same goal, Miné [Min14] uses directly over-approximation domains, giving up the best abstraction and handling the choice of a maximal one with heuristics. To infer necessary preconditions, Cousot et al. [CCL11; CCFL13] use abstract interpretation techniques but on Boolean formulas, hence bypassing the issue of defining an under-approximation abstract domain. Schmidt [Sch07] uses higher-order domains, defining abstract states with the meaning “there exists a value satisfying this over-approximation property”, hence giving rise to an under-approximation of over-approximations. All the above approaches design under-approximation domains starting from over-approximation ones, and, to the extent of our knowledge, there are no

abstract domains thought from the ground up for under-approximation program analysis.

We consider the problem of defining meaningful under-approximation abstract domains for program analysis over powerset concrete domains under the hypotheses (i) and (ii) above. From a purely mathematical point of view, this seems a trivial task because the theories of over and under-approximation are dual. For instance, as done by Lev-Ami et al. [LSRG07], we can transform any over-approximation domain into an under-approximation by reversing the order of its elements and complementing their interpretation. We call this construction *complement domain*. As an example, consider the (over-approximation) interval domain, where, e.g., the interval $[-1, 1]$ is a correct abstraction for any subset of $\{-1, 0, 1\}$ and is the best abstraction of $\{-1, 1\}$ and $\{-1, 0, 1\}$. Instead, in the complement domain, the interval $[-1, 1]$ is a correct abstraction of any set containing all values strictly smaller than -1 and all values strictly greater than 1 and is the best abstraction of $\{\dots, -3, -2, 0, 2, 3, \dots\}$ and $\{\dots, -3, -2, 2, 3, \dots\}$. Note that, being an under-approximation, $[-1, 1]$ represents correctly any set *larger* than its concretization $\{\dots, -3, -2, 2, 3, \dots\}$. However, we argue that complement domains are not useful for incorrectness analysis: in the above complement domain of intervals, initializations such as $i := 0$ or $i := 1000$ are abstracted to the interval $[-\infty, \infty]$, which is the best abstraction of any finite set but loses any information about the initial value of i . We give more details on complement domains in Example 4.1.

Another important asymmetry we point out is the handling of divergence. While it is not the only solution, it is common to represent divergence with the bottom element $\perp$ of the abstract domain. When doing so, this is case in both over and under-approximation. However, $\perp$ as an under-approximation also represents the absence of information; dually, in over-approximation this is described by $\top$. This is a problem since many concrete functions are strict, that is, when applied to a non-terminating expression, they also fail to terminate (they return $\perp$ if one argument is $\perp$), and, to be a correct under-approximation, also the corresponding abstract function needs to be strict:

$$f^\flat(\perp) = f^\flat(\alpha(\emptyset)) \preceq \alpha(f(\emptyset)) = \alpha(\emptyset) = \perp$$

This implies that whenever the analysis cannot determine any meaningful information at some program point, it has to propagate the absence of information along all program paths, at least until a join in the control flow is found. So “recovery” from $\perp$, that is, producing a result different from $\perp$, once we start with it, is very hard in an under-approximation. On the contrary, “recovery” from $\top$ in over-approximation is easier: for example, this can happen whenever the code contains a constant assignment.

A last asymmetry we remark is that over-approximation abstract domains are closed under intersection, while under-approximation abstract domains are closed under union. In the case of assignments this asymmetry has serious consequences. While the result of an assignment can be over-approximated by any larger set of values, with different degrees of precision, the only admissible under-approximations are either the singleton or the empty set. If not enough singletons are represented, then the under-approximation analysis is likely to give a trivial result. Conversely, if too many singletons are represented, closure under union will make the size of the under-approximation abstract domain grow exponentially, violating assumption (ii).

We further strengthen the asymmetry by using the concept of under-approximation Galois insertion (Definition 2.23) to show how straightforward adaptations of some known over-approximation techniques do not work for under-approximation. Then, we establish some negative results. The general theme is to fix some reasonable hypotheses over the common functions encoded by program fragments and then show that any under-approximate abstract domain with size not exponentially larger than the set of values

Result	Concrete domain	Tight hypotheses	Generalizes
Proposition 4.10	$\mathcal{P}(\mathbb{Z})$	–	–
Theorem 4.15	$\mathcal{P}(C)$	High surjectivity	Proposition 4.10
Theorem 4.19	$\mathcal{P}(C)$	High surjectivity	Proposition 4.10

(a) Tabular comparison of our results for infinite domains.

Result	Concrete domain	Tight hypotheses	Generalizes	Inspired by
Proposition 4.13	$\mathcal{P}([-N, N])$	–	–	–
Theorem 4.28	$\mathcal{P}(C)$	None	Proposition 4.13	Theorem 4.19

(b) Tabular comparison of our results for finite domains.

Figure 4.1: Summary of the results in this chapter. The tables acts as a quick reference, showing which concrete domain they are applicable to, which of the hypotheses are (known to be) tight, and the relations between the results.

(i.e., satisfying assumption (i)) will return no useful information for such program fragments. Since the analysis is unable to recover from this lack of information, the result of the analysis of the entire program will be trivial (i.e., violating assumption (ii)). Therefore, while any abstract domain for under-approximate reasoning may be effective for some carefully crafted programs, it will return trivial results on the majority of programs.

Formally, we first introduce the new definition of *non-emptying function* (Definition 4.4), describing functions that do not tamper the analysis. Roughly speaking, a function on the concrete domain is non-emptying if it admits an under-approximation whose consecutive applications would not waste the analysis result by returning $\perp$. Our first result proves that no abstract domain for integers can be constructed that makes all increments non-emptying. In other words, we prove that an analysis based on under-approximation domains would often report trivial information for programs that involve repeated increments. We do so both for the infinite domain $\mathcal{P}(\mathbb{Z})$ and a finite integer domain $\mathcal{P}([-N, N])$ for large N .

We then study how we can generalize these results to different concrete domains and function families. We first focus on the case where the concrete domain is the powerset of an *infinite* set of values, and we prove two results, one local and one global, for infinite concrete domains. To do so, we introduce the notion of *highly surjective function family* (Definition 4.14), of which sums are an instance. The local condition applies to each function in the family, while the global condition is a property of the whole family. Contrary to the definition of non-emptying functions, the notion of highly surjective function family is independent of the abstract domain. Once again the main consequence of our results is that abstract analyses of programs involving the application of functions in the family will often report trivial information. As in the case of increments, highly surjective function families are commonly coded in programs. Finally, we show that the hypothesis of high surjectivity is tight by presenting mathematical constructions of abstract domains making all functions in a family non-emptying. Our results for infinite domains are summarized in Table 4.1a: both results for an arbitrary infinite set C generalize the result for integers. The hypothesis of high surjectivity is tight, but we do not know about all the others.

Lastly, we focus on the powerset of a *finite* set of values. We discuss why a straightforward adaptation of the two results for the infinite case to the finite one was not possible, most notably a difference with the very definition of highly surjective function family. We then propose a general result for this case, inspired by the global condition for the infinite case, but whose details are different. Our results for finite domains are summarized in

Table 4.1b: our single result generalizes again the result for integers, but we were not able to prove any of our hypotheses tight.

4.2 Comparison with over-approximation

We revisit some known over-approximation analyses and techniques, showing how specific characteristics of under-approximation makes this a challenging task.

Complement domain

The first attempt, already briefly discussed in the introduction, is the use of the complement of an over-approximation abstract domain. This is an application of the order theoretic duality, but it turns out the resulting domains are not often useful for analysis. For instance, for domains already closed under complementation (eg. the Sign domain) this construction yields the same domain. However, this is not the case in general.

Example 4.1 (Complement domain). Whenever the concrete domain is a powerset $\mathcal{P}(C)$, we can exploit the complement $\neg : \mathcal{P}(C) \rightarrow \mathcal{P}(C)$ to define a UGC from any given GC by taking complements of the concretization. Formally, given a GC $\langle \mathcal{P}(C) \xrightarrow[\alpha]{\gamma} A \rangle$, then $\langle \mathcal{P}(C) \xrightarrow[\neg \circ \gamma]{\alpha \circ \neg} A^{\text{op}} \rangle$ is a UGC (this stems from $\neg : \mathcal{P}(C) \rightarrow \mathcal{P}(C)^{\text{op}}$ being an isomorphism of posets). For instance, given the interval domain, we can define its complement by

$$\gamma_{\neg}([n, m]) = \neg \gamma([n, m]) = \mathbb{Z} \setminus \{x \in \mathbb{Z} \mid n \leq x \leq m\} = \{x \in \mathbb{Z} \mid x < n \vee m < x\}$$

The set of abstract elements is the same as Int , but the ordering is reversed, so we call this domain Int^{op} . The (under-approximation) Galois Connection with $\mathcal{P}(\mathbb{Z})$ is given by $\gamma_{\neg} = \neg \circ \gamma$ and $\alpha_{\neg} = \alpha \circ \neg$. Note that, thanks to the ordering in Int^{op} being the opposite, both $\alpha_{\neg}$ and $\gamma_{\neg}$ are monotone.

While Int^{op} is a sound under-approximation abstract domain, we argue that it is not useful for incorrectness analysis. Consider a command as simple as the initialization $i := 0$ that may happen at the beginning of a loop. This requires the analysis to abstract the concrete element $\{0\}$, the set of values i may assume at the beginning of the loop. According to the above definition, we have

$$\alpha_{\neg}(\{0\}) = \alpha(\neg\{0\}) = \alpha(\mathbb{Z} \setminus \{0\}) = [-\infty, +\infty]$$

that is the bottom element of Int^{op} since the ordering is reversed. We can better understand the reason for getting bottom by recalling that the meaning of an interval in Int^{op} is the set of elements that are *not* in the interval:

$$\gamma_{\neg}([-\infty, +\infty]) = \neg \gamma([-\infty, +\infty]) = \neg \mathbb{Z} = \emptyset$$

Other than the intuition that we lost all the information about the initialization, since $[-\infty, +\infty]$ is $\perp$ the analysis incurs in the issue described in the Overview about “recover” from $\perp$, effectively making the analysis unable to infer anything.

This line of reasoning can be generalized to any finite concrete set X : $\mathbb{Z} \setminus X$ is not bounded, as it contains all integers greater than $\max(X)$ and smaller than $\min(X)$ (which are both finite), so its abstraction through α is $[-\infty, +\infty]$. ■

Compositionality

An important property of Abstract Interpretation analysis is compositionality. However, citing O’Hearn [OHe20, §8], “for incorrectness reasoning, you must remember information as you go along a path [...]”. This means that a compositional under-approximation analysis must be precise enough to have locally all the informations which should be carried over to the next piece of code. Over-approximation instead is way less restrictive in this sense, since “for correctness reasoning, you get to forget information as you go along a path” (O’Hearn [OHe20, §8]). As an example, we present dependency analysis, which is compositional for over-approximation but it is not for under-approximation.

Example 4.2 (Dependency analysis). A dependency analysis (e.g., [Ass+17, §6]) aims to compute, for each variable at every program point, the set of values it depends on. They are used for instance to check security properties such as information flow constraints.

The result of the analysis is usually expressed with a collection of atomic dependencies $x \rightsquigarrow y$, meaning that the *current* value of y depends on the *initial* value of x . For instance, consider the code

```
if (x == pwd) { y = y + 100 }; x := 0
```

The analysis of this fragment returns the dependencies $x \rightsquigarrow y$, $y \rightsquigarrow y$, as the final value of y depends on the initial values of both x and y . Note that the analysis does not compute any dependency with x on the right as the final value of x is always 0, hence it carries no dependency.

Over-approximation dependency analysis heavily exploits *transitivity* (as it enables compositional reasoning): if C_1 exhibit the dependency $x \rightsquigarrow y$ and C_2 exhibits $y \rightsquigarrow w$, then $C_1; C_2$ may induce $x \rightsquigarrow w$. As a trivial example, $y := x; w := y$ yields the dependency $x \rightsquigarrow w$. However, transitivity is not well-behaved for under-approximation, whose goal is to find true dependencies only. The issue is that dependencies may cancel each other when composed. As a simple example, consider the code

$$\begin{aligned} C_1 &= y := x; z := -x \\ C_2 &= w := y + z \end{aligned}$$

An under-approximation dependency analysis for C_1 may return $x \rightsquigarrow y$ and $x \rightsquigarrow z$ because they are true dependencies. Similarly, for C_2 it could deduce $y \rightsquigarrow w$. However, for the composition $C_1; C_2$ the transitive inference $x \rightsquigarrow y \rightsquigarrow w$ isn’t sound because w is always 0, so it doesn’t depend on anything. ■

Closure under union

Lastly, we consider non-relational analyses. Intuitively, a non-relational analysis cannot capture relationships between different variables. However, under-approximation cannot forget information along a path, including precise relations between variables. This means that non-relational domains cannot be used for under-approximation.

Example 4.3 (Non-relational domain). Informally, a non-relational abstract domain is a tuple of elements, one for each variable x , and describes the set of concrete states where each variable belongs to the values in its abstract coordinate. The abstraction is performed on each variable independently, projecting all states in S on that variable and then abstracting the resulting set. The concretization is performed on each variable independently, and then the results are combined in all possible ways to get concrete values.

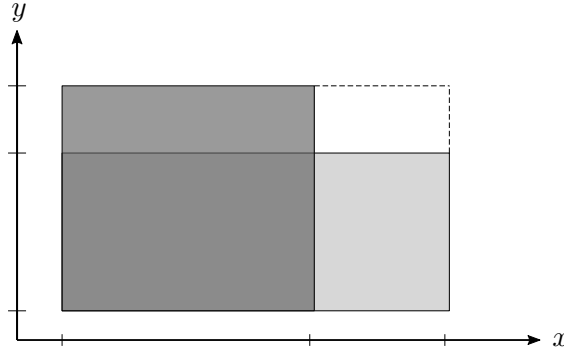


Figure 4.2: Non-relational domains are not closed under union

As an example, consider the product of one interval domains for each variable. For instance, take the code

$$y := 5 - x; z := x + y$$

and assume at the beginning the variable x assumes values in the interval $[0, 3]$. An interval analysis on this fragment would find that y takes values in the interval $5 - [0, 3] = [2, 5]$, and then z is in the result of $[0, 3] + [2, 5] = [2, 8]$. However at the end of the program z is always 5, so this is a sound over-approximation but is not as precise as it can be. The issue here is that the values of x and y are not independent, so an operation between these two variables cannot actually receive all possible inputs with x in $[0, 3]$ and y in $[2, 5]$, but just those that also satisfy the *relationship* $y = 5 - x$. However, the interval domain knows nothing about this relationship since it abstracts each variable independently. More precisely, the possible pair of values for x, y are $\{(0, 5), (1, 4), (2, 3), (3, 2)\}$, but the abstraction is computed projecting on one variable (for instance x) and then computing the interval over-approximating that set (so that the abstraction for x is $[0, 3]$). Then, the concretization contains all the pairs in the product $[0, 3] \times [2, 5]$, which are much more.

In general, this projection is not sound for under-approximation: the concretization is not able to recover which of the original pairs were in the concrete set and which were not. On a more abstract level, such a domain is not closed under union: elements of the abstract domain are “rectangles” in the Cartesian plane with variables on the axes (since they are concretized to the Cartesian product of the abstractions for each variable) and the union of rectangles is not a rectangle. This is shown in Figure 4.2: the union of the light and dark rectangles is not a rectangle as it misses the top-left “corner”. ■

4.3 Non-emptying functions

A central concept in our development is the following definition of *non-emptying function*. To understand the rationale behind it, recall that $\perp$ means no information in the under-approximation setting, while any other abstract value is “something” interesting.

Definition 4.4 (Non-emptying function). Let $\langle C \xrightarrow[\gamma]{\alpha} A \rangle$ be a UGC, $f : C \rightarrow C$ a monotone function and $f^A = \alpha \circ f \circ \gamma$ its bca. We say that f is *non-emptying* (in A) if, for any concrete value c , $\alpha(c) \neq \perp$ and $\alpha(f(c)) \neq \perp$ imply $f^A(\alpha(c)) \neq \perp$.

The idea behind this definition is that if the analysis starts from something meaningful ($\alpha(c) \neq \perp$) and it can find something significant ($\alpha(f(c)) \neq \perp$) then it will find at least one of the possible results ($f^A(\alpha(c)) \neq \perp$), thus not degrading the result of the analysis

to $\perp$ and avoiding the issues discussed above. Intuitively, we want basic transfer functions to be non-emptying so that their analysis doesn't return $\perp$. Note that this idea is tailored to forward analysis. The following toy example illustrates the meaning of the definition.

Example 4.5. Consider the simple imperative fragment

`if (x  $\neq$  0) then { while (x < 10) { y := 7 / x; x := x + 1; } }`

where a careless programmer used the condition $x \neq 0$ instead of the expected $x > 0$: on any initial state where x is negative the program incurs a division by 0 error.

For the analysis, suppose x is an integer value and consider the domain $\text{Int}_{01} = \{I \in \text{Int} \mid 0 \in I \vee 1 \in I\} \cup \{\perp\}$, a variation of Int_0 from Example 2.24 such that each interval in Int_{01} must contain at least one of 0 and 1. By an argument similar to that for Int_0 it can be shown that Int_{01} is closed under union (since 0 and 1 are consecutive values in the integer domain), and thus is an under-approximation domain, whose abstraction function maps each set of integers to the largest interval that is included in the set and that contains 0 or 1.

In this domain, the semantics f of the increment $x := x + 1$ is not non-emptying: for instance, on the concrete value $c = \{-1, 1, 2, \dots, 10\}$ we have $\alpha(c) = [1, 10] \neq \perp$ and $\alpha(f(c)) = \alpha(\{0, 2, 3, \dots, 11\}) = [0] \neq \perp$ but $f^A(\alpha(c)) = f^A([1, 10]) = \alpha(f(\gamma([1, 10]))) = \alpha(\{2, 3, \dots, 11\}) = \perp$. We show in the following the effect of this on the analysis.

Assume to start the analysis in this domain with the initial condition $[-1, 10]$ for variable x : remember that this being an under-approximation analysis, the abstract state $[-1, 10]$ means that x may assume all the values in that interval at the beginning of the code fragment. In the concrete execution, the filter $x \neq 0$ then produces the concrete set of values $c = \{-1, 1, 2, \dots, 10\}$, but the abstract interpreter must abstract this to its largest subset that is an interval containing 0 or 1, that is $[1, 10]$. The abstract analysis of the cycle then proceeds straightforwardly, finding $\perp$ after one iteration of the loop body (since after the increment the set of values for x is $\{2, 3, \dots, 11\}$ that is abstracted to $\perp$ because it contains neither 0 nor 1) and so the abstract fixpoint of the loop is the interval $[1, 10]$. This yields no error, even though the concrete execution starting at $x = -1$ does indeed fail after one iteration. ■

For the remainder of the paper, we assume to be given a set of *values* C and a UGI $\langle \mathcal{P}(C) \stackrel{\alpha}{\hookrightarrow} A \rangle$ whose concrete domain is $\mathcal{P}(C)$. In the following, we present the basic technical tools we use to prove our theorems (Propositions 4.10, 4.13 and Theorems 4.15, 4.19, 4.28), which all follows the same pattern. First we make the assumption that the abstract domain is not *too large* to hinder the analysis, but then we show that if all the functions in a given family were non-emptying, the abstract domain would grow too large. Particularly, we start from a representable element (which is assumed to exist by hypothesis), then use Lemma 4.7 to build other representable elements, up until the size of the abstract domain blows up. To get this, we exploit an exponential increase in the size of A caused by “closure under union”, that is the fact that if two concrete elements are representable in the abstract domain, their union is representable as well.

Definition 4.6. Let $S \subseteq C$ be a subset of C , and $\langle \mathcal{P}(C) \stackrel{\alpha}{\hookrightarrow} A \rangle$ is an a UGI. We say that $d \in C$ is *representable with S* if $S \cup \{d\}$ is representable in A , ie. $\alpha(S \cup \{d\}) = S \cup \{d\}$. We call $R_A(S)$ the set of elements of C representable with S , i.e.

$$R_A(S) = \{d \in C \mid \alpha(\{d\} \cup S) = \{d\} \cup S\}$$

For the sake of brevity, we omit the subscript A whenever it is clear from the context, we write R for $R(\emptyset)$, the set of representable values of C , and use the shorthand $R(c)$

Say forward analysis at the beginning

for $R(\{c\})$ when $c \in C$ is a concrete value. The following lemma, valid for non-emptying functions, explains the role played by Definition 4.4 in proving all our negative results. Roughly speaking, it states that for f to be non-emptying, some concrete values must be representable in the abstract domain.

Lemma 4.7. *Let $f : C \rightarrow C$ be non-emptying, $c \in R$ and $\tilde{c} \notin R(c)$. If $f(\tilde{c}) \in R$, then $f(c) \in R$.*

While the broad outline of all of our theorems is always the same, they differ in how they solve two key issues: first, it must be possible to apply Lemma 4.7; second, all the new representable elements obtained by applying it must be different from one another. In the following, we present various sets of conditions that can guarantee these two points, hence getting hypotheses for non existence of under-approximation abstract domains.

4.4 Integer domains

In this section, we apply the idea from the previous section on the concrete domain of integers and prove that any under-approximation abstract domain will likely return trivial analyses for programs that include sums inside arithmetic expressions.

4.4.1 Infinite integer domain

As a first example, we consider the infinite domain $\mathcal{P}(\mathbb{Z})$ over all integers.

Assumption 4.8. We assume that an abstract domain A , to be feasible for analyses, must be at most countable.

We make this assumption because we want the analysis to have a complexity comparable to that of a single concrete execution: if the analysis could be as complex as an arbitrary set of concrete executions, we could use those instead of the abstract domain. Therefore, we require the abstract domain to have the same size of the set $\mathbb{Z}$ of values handled by the program, and not the concrete domain $\mathcal{P}(\mathbb{Z})$. Many abstract domains, such as intervals, octagons [Min06] and polyhedrons [CH78] with at most n edges, satisfy it; some, such as general polyhedrons, do not, but indeed they also exhibit a worst-case exponential cost.

Based on Assumption 4.8, we prove a simple cardinality estimate that is exploited to prove that there are few representable elements.

Lemma 4.9. *For any fixed subset $S \subseteq \mathbb{Z}$, $R(S)$ is finite.*

The result for integers now shows that no under-approximation abstract domain makes all sums non-emptying. The idea of the proof is to define an infinite sequence of representable elements, which is in contradiction with the previous lemma that says that $R = R(\emptyset)$ is finite. To define such a sequence, we want to use Lemma 4.7: we start from an initial representable n_0 and from a value $\bar{n}$ not representable with it, then find a non-emptying f that maps $\bar{n}$ into n_0 , so that $f(\bar{n})$ is representable and we can then apply the lemma to get the new representable element $f(n_0)$. We then iterate this procedure, changing f , to build the infinite sequence. We believe the hypothesis that there exists an initial representable value is not very restrictive since initializations like $\mathbf{x} = 0$ must be abstracted to $\perp$ if 0 is not representable.

Proposition 4.10. *Let $\langle \mathcal{P}(\mathbb{Z}) \stackrel{\alpha}{=} A \rangle$ be a UGI, and assume that there is an integer n_0 that is representable. Then it cannot be the case that all the functions of the form $f_n(x) = x + n$ are non-emptying in A .*

Proof. Towards a contradiction, let us assume that all f_n are non-emptying in A . By hypothesis, $n_0 \in R$ and $R(n_0)$ is at most finite by Lemma 4.9. Since $\mathbb{Z}$ is infinite, there exists an $\bar{n} \in \mathbb{Z} \setminus (R(n_0) \cup \{n_0\})$, that is $\bar{n}$ is an element such that the pair $\{n_0, \bar{n}\}$ is not representable. Let $d = n_0 - \bar{n}$, and let us prove by induction on t that $n_0 + td$ is representable for all t . The base case $t = 0$ is the hypothesis that n_0 is representable. For the inductive case, assume $n_0 + td$ is representable, and consider the non-emptying function $f_{(t+1)d}$. We get:

$$f_{(t+1)d}(\bar{n}) = \bar{n} + (t+1)d = \bar{n} + n_0 - \bar{n} + td = n_0 + td$$

that is representable by inductive hypothesis. By the instance of Lemma 4.7 for the pair $\{n_0, \bar{n}\}$ and the function $f_{(t+1)d}$, we get that $f_{(t+1)d}(n_0) = n_0 + (t+1)d$ is representable too, that is exactly the inductive step.

Since $\bar{n} \neq n_0$ also $d \neq 0$, hence $\{n_0 + td \mid t \in \mathbb{N}\}$ is infinite. Moreover $\{n_0 + td \mid t \in \mathbb{N}\} \subseteq R$ by the induction above, but this is impossible since R must be finite by Lemma 4.9. $\square$

The meaning of this proposition for program analysis is the fact that a domain small enough (by Assumption 4.8) is probably unable to deduce meaningful information on an integer domain: if it does not contain representable singletons it must abstract to $\perp$ any variable initialization, and otherwise, it cannot be non-emptying for all sums, hence getting $\perp$ when values are manipulated using this operation. In both cases, because of strictness, the abstract $\perp$ is propagated along program paths, yielding it as the final result of the analysis, which means exactly it cannot determine any information. This issue is not bound to manifest for all programs, but for any domain there exist programs for which it does.

Note that the only hypothesis on the abstract domain is related to its size, by Assumption 4.8. This would include all classical numerical domains such as intervals, octagons or congruences [Gra91]. Of course, these are over-approximation domains, so this result does not apply to them, but it shows that our hypotheses are very general and would include many known abstract domains.

4.4.2 Finite integer domain

An analogous result can be obtained for a finite integer domain $\mathcal{P}([-N, N])$, where N is some large integer. This concrete domain models machine integers, that are constrained within an interval, so we assume that operations are performed in machine arithmetic, that is wrapping around in case of overflows. This is modelled working modulo $2N + 1$, the length of the interval, and taking the unique representative of each congruence class in the interval $[-N, N]$ of interest. It is worth noting that we take an interval that is symmetric around 0 to simplify notation, but there is no conceptual difference in using an asymmetric one instead (eg. $[-N - 1; N]$ to be closed under two's complement).

Assumption 4.11. We assume that an abstract domain A , to be feasible for the concrete domain $\mathcal{P}([-N, N])$, must have a cardinality that is polynomial in N .

This assumption, just like Assumption 4.8, guarantees that for any set of input values the cost of the analysis is always polynomial in that of a single concrete execution, while the concrete analysis could require an exponential cost.

In the following, we use asymptotic notation for some quantities. Therefore, for each N we consider the concrete set of values $C_N = [-N, N]$ and define an abstract domains A_N for the concrete domain $\mathcal{P}([-N, N])$. Intuitively, each A_N represents the same abstraction, just instantiated for different values of N . With this notation, Assumption 4.11 becomes $|A_N| = O(\text{poly}(N))$.

The next lemma is analogous to Lemma 4.9 in proving that some sets are small under Assumption 4.11 on the cardinality of A_N . Here, the “exponential increase” we mentioned is explicit, as we are dealing with finite quantities. For brevity, we write R_N instead of R_{A_N} .

Lemma 4.12. *Let $S \subseteq [-N_0, N_0]$ for some N_0 . Then, $|R_N(S)| = O(\log(N))$.*

The following proposition uses the same proof line as Proposition 4.10 above: we define a sequence of representable elements and prove that they are too many since, by the previous lemma, R_N is quite small.

Proposition 4.13. *Let $\langle \mathcal{P}([-N, N]) \stackrel{\alpha}{\approx} A_N \rangle$ be a UGI for all N , and assume that there is an integer n_0 that is representable in all the A_N . Then there exists an N_0 such that, for all $N > N_0$, it cannot be the case that all the functions of the form $f_n(x) = x + n$ (modulo $2N + 1$) are non-emptying in A_N .*

Proof. Let $r_N = |R_N(n_0)|$. By the previous Lemma 4.12 we know that $r_N = O(\log(N))$. For all N , fix an element $\bar{n}_N \notin R_N(n_0)$ not representable with n_0 in A_N such that $d_N = n_0 - \bar{n}_N \leq r_N + 1$. This element exists because otherwise all elements in the interval $[n_0 - r_N - 1, n_0 - 1]$ (modulo $2N + 1$) would be representable with n_0 , that is impossible since that interval contains $r_N + 1 = |R_N(n_0)| + 1$ elements.

Consider an N such that all the functions of the form $f_n(x) = x + n$ (modulo $2N + 1$) are non-emptying in A_N . Following the proof of Proposition 4.10, we can show by induction on $t \geq 1$ that the value $f_{td_N}(\bar{n}) = n_0 + (t - 1)d_N$ is representable in A_N . All these values are different from one another for

$$1 \leq t < \frac{2N + 1}{d_N}$$

so that

$$|R_N| \geq \frac{2N + 1}{d_N}$$

However, we know that $|R_N| = O(\log(N))$ and $(2N + 1)/d_N = \omega(\log(N))$, therefore there exists some N_0 such that for any $N > N_0$ this inequality does not hold. This in turn implies that for all $N > N_0$ it is not possible that all the functions of the form $f_n(x) = x + n$ (modulo $2N + 1$) are non-emptying in A_N . $\square$

Again, the only assumption on A_N is about its size, so our result applies to many kinds of domains. For instance, a predicate abstraction [GS97] with $\Omega(\log(N))$ predicates exceeds this bound, as its size is doubly exponential in the number n of predicates.

4.5 General infinite concrete domains

In this section, we try to extend the results of the previous Section 4.4.1 on the infinite concrete domain of integers to other infinite concrete domains. More precisely, we deal with an infinite set C of concrete values, and a UGI $\langle \mathcal{P}(C) \stackrel{\alpha}{\approx} A \rangle$. Again, we take the

Assumption 4.8 on the size of A . Under this assumption, we can prove again Lemma 4.9, that does not depend on the specific integer domain considered in the previous section.

All conditions we propose in this section are mainly on the family of functions considered and not on the abstract domain. The reason for this is that first we fix a function family, corresponding to a program, and then we look for a domain well suited to analyse the specific family at hand. In other words, the family is given by the applicative context, while the domain can be adapted to it.

Definition 4.14 (Highly surjective function family). Given a family F of functions from C to itself and an element $c \in C$, let

$$P_F(c) = \{d \in C \mid \exists f \in F. f(d) = c\}$$

be the set of *preimages* of c , elements of C that can be mapped to c by a function in F . We say that the family F is *highly surjective* if $P_F(c)$ is infinite for any possible choice of $c \in C$.

This property is needed together with Lemma 4.9 to apply Lemma 4.7 and get a new representable element: since there are infinitely many preimages of c but $R(c)$ is finite, there are elements $\tilde{c} \in P_F(c)$ not in $R(c)$; then by definition of $P_F(c)$ there is an f such that $f(\tilde{c}) = c \in R$, so we can apply the lemma to get $f(c) \in R$. The reason for requiring $f(\tilde{c}) = c$ instead of just in R is that, at the beginning of the proof, we assume R to contain only one element, hence the two conditions are equivalent. Starting from this basic idea, we present two sets of sufficient conditions to prove the non existence of any under-approximation abstract domain.

4.5.1 Local requirements for impossibility

The first set of conditions we propose is in a sense more “local”, in that it requires conditions on each function in the family F , independently of the others.

The proof of this result proceeds as follows: it starts from a representable $c_0 \in R$ and iteratively creates an infinite sequence $\{c_n\}_{n \in \mathbb{N}}$ of representable elements. This yields a contradiction since R should be finite by Lemma 4.9.

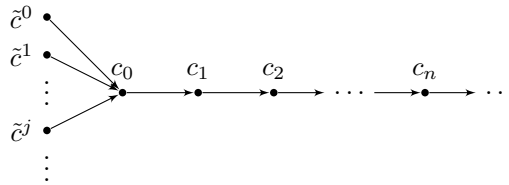


Figure 4.3: Graphical representation of the “final” f

The main idea is to pick a suitable f in F and define the sequence as the iterates $c_{n+1} = f(c_n)$. This function is sketched in Figure 4.3. The initial set of elements $\tilde{c}^j$ mapped to c_0 is required to apply Lemma 4.7 to all pairs $\{\tilde{c}^j, c_n\}$ and get that $c_{n+1} = f(c_n)$ is representable, since $f(\tilde{c}^j) = c_0 \in R$. The difficulty in realizing this idea is that we do not have enough information on the sequence to pick the right function f at the beginning, so we bring along a list of candidate functions that all coincide on a prefix of the sequence. At step n , we pick a new element of the sequence among the possible images of c_{n-1} through all candidate f we have at that point and discard all those functions that cannot match the choice. Actually, instead of directly considering functions, we represent them with

elements $\tilde{c}$ of C . Each one represents a function $f_{\tilde{c}}$ that satisfies $f_{\tilde{c}}(\tilde{c}) = c_0$. Note that this can be done for “enough” (that is, infinitely many) $\tilde{c}$ because of the high surjectivity hypothesis. We call E_n the set of element $\tilde{c}$ that represent functions that are “valid” for the prefix up to n , i.e., they map c_i to c_{i+1} for $0 \leq i \leq n-1$. The core of the proof is an induction that proves that E_n always contains infinitely many elements and that the newly chosen c_n is different from all the previous ones.

Theorem 4.15. *Let F be a highly surjective function family from C to itself such that all functions $f \in F$ are either injective or acyclic. Assume also that $R \neq \emptyset$. Then there is at least one function $f \in F$ that is not non-emptying in A .*

Remark 4.16. In the previous section, we developed an ad hoc proof for the family of sums over integers in Proposition 4.10, but the same result can also be obtained as an application of Theorem 4.15: if $C = \mathbb{Z}$ and $F = \{\lambda x.x + n \mid n \in \mathbb{Z}\}$, the family is highly surjective (actually $P_F(c) = \mathbb{Z}$ for all c) and all these functions are injective, so it meets the hypotheses of the theorem. However, it is interesting to note that the proof of Theorem 4.15 is not a generalization of the proof of Proposition 4.10. Here we iterate a single f to build the entire sequence, while in the previous one we change the function every time, mapping the non representable $\tilde{n}$ to the newly found representable $n_0 + td$ to get that the image of n_0 through that function is representable too, as sketched in Figure 4.4.

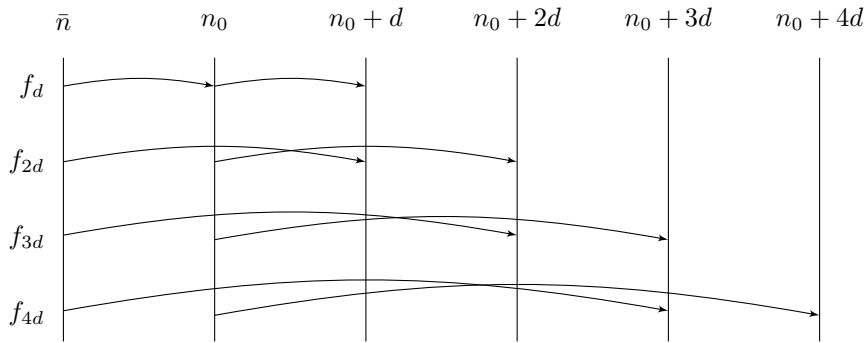


Figure 4.4: Graphical representation of the proof of Proposition 4.10

Another example are rational or real numbers, with sums or products.

Example 4.17. Take $C = \mathbb{Q} \setminus \{0\}$ and $F = \{\lambda x.x \cdot q \mid q \in \mathbb{Q} \setminus \{0\}\}$. The family is highly surjective since $P_F(c) = \mathbb{Q} \setminus \{0\}$ for all c , and all these functions are invertible, hence injective. ■

A possibly more interesting example of application are floating-point numbers as described by the IEEE Standard.

Example 4.18. Take $C = \mathcal{F} \setminus \{0\}$ the set of non-zero floating-point numbers that can be represented with a fixed number of significant digits, say t bits, but with an arbitrary precision exponent. We choose infinite precision exponents but a finite number of significant digits to have an infinite domain, as required by the theorem, but also to preserve characteristics of floating-point arithmetic.

Let $\cdot$ and $\odot$ denote respectively real product and its floating-point approximation, and consider the function family $F = \{\lambda x.x \odot y \mid y \in C\}$. The function family is highly surjective, e.g., considering that all numbers with the same significant digits as a floating-point x but different exponent can be mapped into x by multiplying them by 2 to the power of the difference of exponents. For the second condition, if $y = \pm 1$ we have that

the function $\lambda x.x \odot y$ is invertible, hence injective. Otherwise, assume without loss of generality that $y > 1$ (other cases are analogous), and by contradiction assume it has a cycle $f^n(x_0) = x_0$. By monotonicity of $\odot$ we have $f(x) = x \odot y \geq x \odot 1 = x$, hence $x_0 \leq f(x_0) \leq f^2(x_0) \leq \dots \leq f^n(x_0) = x_0$ so all the elements of the cycle are equal, and in particular $f(x_0) = x_0$. However, if $y \neq 1$, the product $x \odot y$ is never equal to x , that is a contradiction. Hence the function is acyclic. This means F meets the hypotheses of Theorem 4.15, hence no abstract domain on floating-point numbers can be non-emptying for all multiplications. ■

4.5.2 Global requirements for impossibility

The second set of conditions we propose is “global”, in the sense that it requires the family F to satisfy a property as a whole.

The proof of this theorem starts from the infinite set $P_F(c_0)$ and, using the hypotheses that some sets are finite, propagates its infiniteness down to R , yielding a contradiction with Lemma 4.9, stating that R is finite.

Theorem 4.19. *Let F be a highly surjective family of functions from C to itself such that*

1. *for all pair of elements $c, d \in C$, the set $\{f \in F \mid f(d) = c\}$ is finite;*
2. *for all pair of an element $c \in C$ and a function $f \in F$, the set $\{d \in C \mid f(d) = c\}$ is finite.*

Assume also that R is not empty. Then there is at least one function $f \in F$ that is not non-emptying in A .

Remark 4.20. Again, Theorem 4.19 can be used to prove Proposition 4.10, but the proofs are different. The former starts from the infinite set $P_F(c_0) \setminus R(c_0)$ of preimages $\tilde{c}$ of c_0 that are not representable with it. This yields an infinite list of pairs $\{\tilde{c}, c_0\}$ to apply Lemma 4.7: for any such pair, since $\tilde{c}$ is in $P_F(c_0)$, we get a function $f_{\tilde{c}}$ such that $f_{\tilde{c}}(\tilde{c}) = c_0 \in R$, so that $f_{\tilde{c}}(c_0) \in R$, too. The proof then exploits the remaining hypotheses to prove that there are infinitely many distinct $f_{\tilde{c}}(c_0)$. The proof of the latter relies on multiple functions to apply Lemma 4.7, but it always uses the same pair $\{\bar{n}, n_0\}$: at every step, it finds a function that maps $\bar{n}$ to the representable element found at the previous step.

Similarly to Theorem 4.15, this result can be used to prove the impossibility of building an under-approximation abstract domain for floating-point numbers.

Example 4.21. Take $C = \mathcal{F} \setminus \{0\}$ the set of non-zero floating-point numbers with t bits significands and arbitrary precision exponents, and $F = \{\lambda x.x \odot z \mid z \in \mathcal{F} \setminus \{0\}\}$. As observed in Example 4.18 this family is highly surjective. Fixed now two floating-point numbers x, y , and letting $\mathbf{u}$ be the machine precision of floating-point arithmetic, we have that $y = x \odot z$ only if

$$\left| \frac{y - (x \cdot z)}{x \cdot z} \right| < \mathbf{u}$$

that is

$$\left| \frac{y}{x} \right| \frac{1}{1 + \mathbf{u}} < |z| < \left| \frac{y}{x} \right| \frac{1}{1 - \mathbf{u}}$$

This is a bounded interval since $x \neq 0$, and hence contains only a finite amount of floating-point numbers. This in turn means that, fixed x and y , there is only a finite amount of functions of the form $\lambda x.x \odot z$ such that $f(x) = y$. Analogously, fixed a floating-point y and a function $f(x) = x \odot z$, we have that $y = f(x)$ only if $|x|$ belong to a bounded

interval, that contains a finite amount of floating-point numbers. So, fixed y and a function $f = \lambda x.x \odot z$, only a finite number of x satisfies $f(x) = y$. So, through Theorem 4.19 above, we proved again that no abstract domain on floating-point numbers can be non-emptying for all multiplications. ■

We point out once more that the only hypothesis on the abstract domain is about its size for both theorems in this section. For instance, our result applies to any under-approximation predicate abstraction domain [GS97]. Such a domain is defined by a (finite) list of predicates, each one representing the set of states satisfying the predicate, and an abstract state is a subset of predicates. By duality with respect to over-approximation, the concretization of such a set is the *union* of the states described by each predicate. Analogously, the abstraction of a set of concrete states S is the set of predicates which are *entirely contained* in S . Note that this interpretation is consistent with the use of under-approximations in Incorrectness Logic: the difference is that the logic can use any under-approximation formula, while predicate abstraction is constrained to use just a finite set of predicates, fixed beforehand.

4.5.3 On the necessity of high surjectivity

Both sets of conditions we proposed in this section require the function family to be highly surjective. This turns out to be necessary to prove that no under-approximation abstract domain exists, as we show in this section.

Proposition 4.22. *For any fixed family F of functions from C to itself that is not highly surjective, there exists an abstract domain A_F for $\mathcal{P}(C)$ such that:*

- A_F is finite, and
- all functions $f \in F$ are non-emptying in A_F .

Notably, the above proof is constructive. We present an example of such domain construction below.

Example 4.23. Fix the pair of functions $f(x) = x - 1$ and $g(x) = x - 2$ on $\mathbb{Z}$. The family $F = \{f, g\}$ is not highly surjective, so we build an under-approximation abstract domain for which these functions are non-emptying. First, take an integer n_0 such that $P_F(n_0)$ (computed with respect to F) is finite. With this F , any integer is a suitable candidate, so let us fix $n_0 = 0$.

The set of preimages of 0 is $P_F(0) = \{1, 2\}$. We define the abstract domain A_F as

$$A_F = \{\emptyset\} \cup \{X \cup \{0\} \mid X \subseteq P_F(0)\} = \{\emptyset, \{0\}, \{0, 1\}, \{0, 2\}, \{0, 1, 2\}\}.$$

In this abstract domain, a set is abstracted to $\emptyset$ if and only if it does not contain 0 since all elements of A_F but $\emptyset$ contains 0 and the abstraction of a set S must be a subset of S .

To check that f is non-emptying in A_F , fix a set $S \subseteq \mathbb{Z}$. If $\alpha(S) = \emptyset$ the non-emptying condition is vacuously true, so assume this is not the case, or, equivalently, that $0 \in S$. Analogously, if $\alpha(f(S)) = \emptyset$ the condition is true, so assume $0 \in f(S)$ or, equivalently, $1 \in S$. Using these two assumptions we get

$$\begin{aligned} f^A(\alpha(S)) &= \alpha(f(\alpha(S))) && [\text{def. of } f^A] \\ &\supseteq \alpha(f(\alpha(\{0, 1\}))) && [\alpha, f \text{ monotone}, S \supseteq \{0, 1\}] \\ &= \alpha(f(\{0, 1\})) && [\alpha(\{0, 1\}) = \{0, 1\}] \\ &= \alpha(\{-1, 0\}) = \{0\} && [\text{def. of } f \text{ and } \alpha] \end{aligned}$$

The check for g is analogous. ■

Even though this proposition defines an under-approximation abstract domain, it should not be interpreted as a positive result since the resulting domain is almost a power set and hence too large to be feasible in practice. Instead, the proposition should be regarded as a way to show that one of the hypotheses required in the previous theorems is tight and cannot be weakened. Particularly, since these kinds of results require high surjectivity, they are ill-suited when the focus is on a single function.

This proposition can be generalized to consider sets $S \subseteq C$ whose preimages are finite, but a little care is needed when lifting the definition of preimages to sets of values: a preimage is a set for which there exists a function that maps it to S , not the union of the preimages of elements in S . Formally, we let:

$$P_F(S) = \{T \subseteq C \mid \exists f \in F. f(T) = S\}.$$

Using this definition, we can now easily generalize Proposition 4.22:

Proposition 4.24. *Let F be a family of functions from C in itself, and assume there is a set $S_0 \subseteq C$ such that $P_F(S_0)$ is finite. Then there exists a finite abstract domain A_F for $\mathcal{P}(C)$ such that all functions $f \in F$ are non-emptying in A_F .*

This proposition may also be applied to the concrete domain of finite lists to show that a natural function family to consider cannot be used to prove non existence of under-approximation domains using non-emptying functions.

Example 4.25. Fix the concrete domain C as the set of all lists of finite length over a finite, non-empty alphabet Γ , i.e. $C = \Gamma^*$. For $\alpha \in \Gamma^*$ a finite string, let

$$\text{concat}_\alpha(\beta) = \alpha\beta$$

be the function that prefixes its argument by the string α . The family

$$F = \{\text{concat}_\alpha \mid \alpha \in \Gamma^*\}$$

is not highly surjective, because fixed a string γ only its suffixes can be mapped into γ by a function in F , and they are a finite amount. Hence we can define an under-approximation abstract domain for which all these functions are non-emptying by means of Proposition 4.24. Such domains are defined with a construction similar to that of Example 4.23, and in particular, if ϵ is the empty list, considering the set $S_0 = \{\epsilon\}$ whose preimage is only S_0 itself, the construction yields

$$A_F = \{\emptyset, \{\epsilon\}\}.$$

It is easy to check that all functions concat_α are non-emptying in this abstract domain. ■

The previous proposition focuses on preimages, stating that if there is a concrete element that has a finite amount of preimages then it is possible to define an under-approximation domain. A natural dual of this proposition can be formulated in terms of images. For a subset $S \subseteq C$, the set of its images is defined as follows:

$$I_F(S) = \{f(S) \mid f \in F\}.$$

This definition is exactly dual to that of preimages and can be used to formulate a similar result.

Proposition 4.26. *Let F be a family of total functions (ie. if $S \neq \emptyset$ then $f(S) \neq \emptyset$) from $\mathcal{P}(C)$ in itself, and assume there is a non-empty set $S_0 \subseteq C$ such that $I_F(S_0)$ is finite. Then there exists a finite abstract domain A_F such that all functions $f \in F$ are non-emptying in A_F .*

Even though Proposition 4.26 introduces the technical hypothesis that all $f \in F$ are total, we don't believe this condition to be very restrictive in practice. Our results are applicable when F is a family of basic transfer functions; in programming languages, non-termination is often due to control-flow constructs, while assignments and guards are usually total. As an immediate application of the proposition that exploits images instead of pre-images, we consider lists again, and rule out another natural function family.

Example 4.27. Fix again $C = \Gamma^*$, and consider all functions of the form $\text{drop}_n : \Gamma^* \rightarrow \Gamma^*$ that, taken a list, drop its first n elements and return the resulting list. If the input list is shorter than n , the output of drop_n is the empty list ϵ . The function family

$$F = \{\text{drop}_n \mid n \in \mathbb{N}\}$$

is highly surjective since, for any fixed list $\alpha \in \Gamma^*$ and any n , we can prefix α by n arbitrary characters, and map this extended list back to α via drop_n . However, images through this function family are finite:

$$I_F(\alpha) = \{\text{drop}_n(\alpha) \mid n \in \mathbb{N}\}$$

is finite because $I_F(\alpha)$ coincides with the set of all tails of α . By Proposition 4.26 we can define an under-approximation abstract domain such that all functions drop_n are non-emptying. Again, these domains are constructed from sets S_0 with a finite amount of images, and considering $S_0 = \{\epsilon\}$, that satisfies $I_F(S_0) = \{\epsilon\}$, we get

$$A_F = \{\emptyset, \{\epsilon\}\}.$$

It can be easily checked that all functions drop_n are non-emptying in A_F . ■

These last two propositions consider opposite situations in which it is possible to define an under-approximation domain: the former requires to be able to go backward using F in infinitely many ways, while the latter to go forward. This often is not the case in the presence of “boundaries” in the concrete domain, which are points with respect to which functions tend to walk either up or away. For instance, ϵ is such a point with finite lists because concat functions go away from it while drop functions move towards it. Another example of such a boundary is 0 in the domain of integers $\mathbb{Z}$ for multiplications and (rounded) divisions: the former increase absolute value, moving away from 0 (even though 0 itself is never a preimage), while the latter decrease it. Also considering a function family made of both kinds of functions does not work: a slight adaptation of the constructions for the two propositions above shows that, if F can be partitioned into two subfamilies, each satisfying the hypothesis of one of the two propositions, then there exists an under-approximation abstract domain. An example of this is in the set of finite lists, taking as F both concat and drop functions. The construction then yields exactly $A_F = \{\emptyset, \{\epsilon\}\}$, for which all these functions are non-emptying, as shown in Examples 4.25 and 4.27. In light of these observations, to apply the definition of non-emptying function in an effective way for proving the non existence of abstract domains, for all possible boundaries there is the need for a function that can both enter and exit it. This happens for integers since there is no boundary, but does not for finite lists, with $\{\epsilon\}$ being often either a sink or a source for many functions on lists.

4.6 General finite concrete domains

The discussion of Section 4.5 requires C to be infinite. While this is a common simplification, since concrete domains have usually a very big size, an interesting and important

question is whether our findings can be extended to the case of finite concrete domains. However, we believe that the two Theorems 4.15 and 4.19 cannot be straightforwardly adapted to the finite setting.

For both results, the proof showed the existence of infinitely many representable elements, contradicting Lemma 4.9. For a finite C , if $N = |C|$, one possibility would be to resort to Lemma 4.12 to show that $|R| = O(\log(N))$ and get the contradiction by proving the existence of $\omega(\log(N))$ representable elements (we recall that $f(n) = \omega(g(n))$ if and only if $\lim_{n \rightarrow +\infty} (f(n)/g(n)) = +\infty$, which intuitively means that f asymptotically dominates g). Unfortunately, similar constructions do not yield the desired bound: when C is infinite we exploited the fact that finite combinations of finite numbers are also finite, while for finite C we should require that arbitrary combinations of logarithmic factors are $O(N)$, which is not true.

The definition of highly surjective family itself is not easy to translate. The construction of Proposition 4.22 on a finite C yields an abstract domain with the needed features already when $|P_F(c)| = O(\log(N))$. However, to carry out proofs along the lines of Theorems 4.15 and 4.19 we would need stronger hypothesis than just $|P_F(c)| = \omega(\log(N))$, possibly up to $|P_F(c)| = \Theta(N)$.

Even with these theoretical considerations against finite counterparts of Theorems 4.15 and 4.19, we have been able to carry out the proof for the special case of the finite domain $C = [-N, N]$ of integers (Proposition 4.13). Our result exploits the precise structure of both the given concrete domain and function family, particularly two key points. First, functions produce elements that are not “too far away” with respect to the size of the domain (i.e., $n_{i+1} - n_i = O(\log(N))$), and this allows to prove that there are enough distinct representable elements. Second, the domain is circular and hence has no boundaries. If the domain had them, we could have applied results such as Example 4.25 or Proposition 4.26. This was not the case for integers because additions overflow, so the domain has no boundaries near $-N$ and N .

Building on the above discussion, we present a version of the global condition for finite concrete domains. To overcome the limitation of boundaries, we explicitly constrain the initial representable element c_0 , writing hypotheses around it. Such hypotheses imply that c_0 is “far enough” from the boundaries of the domain (if any). Conditions (1-2) of the next theorem correspond to those of the infinite version (Theorem 4.19) rewritten with c_0 in mind. The relation between the two bound functions k_1 and k_2 and the number of preimages of c_0 corresponds to the high surjectivity hypothesis. It only constrains the value c_0 because this is the representable value from which the proof begins.

As we did in the specific case of the integer domain $[-N; N]$, we assume the size of A to be polynomial in N (Assumption 4.11). To say this formally, as we did for finite integers (cfr. Section 4.4.2), we consider a sequence C_N of sets of concrete values with size $O(N)$. In the specific instance of integers, such sets were the intervals $[-N; N]$; in general, we require that $C_{N-1} \subseteq C_N$ for all N . This inclusion formalizes the intuition that all the C_N are the “same” concrete domain instantiated for different sizes. We also assume to have an abstract domain A_N for each concrete domain $\mathcal{P}(C_N)$ such that $|A_N| = O(\text{poly}(N))$. As before, we write R_N for R_{A_N} , and we remark that Lemma 4.12 holds.

Theorem 4.28. *Assume $|C_N| = O(N)$, and let $\langle \mathcal{P}(C_N) \xrightarrow{\alpha_N} A_N \rangle$ be a UGI for all N . Assume there exists a number N_1 and a value $c_0 \in C_{N_1}$ such that, for all $N > N_1$, c_0 is representable in A_N (i.e., $c_0 \in R_N$). Given two functions $k_1, k_2 : \mathbb{N} \rightarrow \mathbb{N}_{>0}$, for any N let F_N be a family of functions from C_N to itself such that:*

1. *for all elements $d \in C_N$, $|\{f \in F_N \mid f(c_0) = d\}| \leq k_1(N)$;*

2. for all functions $f \in F_N$, $|\{d \in C_N \mid f(d) = c_0\}| \leq k_2(N)$.

Lastly, assume that $|P_{F_N}(c_0)| = \omega(\log(N) \cdot k_1(N) \cdot k_2(N))$.

Then, there exists N_0 such that, for all $N > N_0$, it is not possible that all $f \in F_N$ are non-emptying in A_N .

A straightforward corollary is that, whenever we can verify the hypotheses for all values in C , there is no under-approximation domain with a representable element. This is for instance the case for integers and sums, so we recover Proposition 4.13:

Example 4.29. Let $C_N = [-N, N]$ and

$$F_N = \{\lambda x.x + n(\text{modulo } 2N + 1) \mid n \in [-N, N]\}.$$

Fixed any $n_0 \in [-N, N]$, it is easy to check that $P_F(n_0) = [-N, N]$ and $k_1(N) = k_2(N) = 1$. The condition

$$|P_F(n_0)| = \omega(\log(2N + 1) \cdot k_1(2N + 1) \cdot k_2(2N + 1))$$

thus reduces to

$$2N + 1 = \omega(\log(N))$$

that is true. Hence there is no under-approximation abstract domain for integers with at least one integer n_0 representable and that makes all sums non-emptying. ■

The example of integers has the nice property of not having boundaries, but this is seldom the case. For instance, consider floating point numbers: they overflow and underflow to special values, hence they do have boundaries. Nevertheless, if we pick a suitable initial element c_0 we can still apply the theorem above.

Example 4.30. Consider the finite set of non-zero floating-point numbers $C = \mathcal{F} \setminus \{0\}$ with t bits significand, one bit sign and e bit exponents. Consider the function family $F = \{\lambda x.x \odot z \mid z \in \mathcal{F} \setminus \{0\}\}$ of floating-point multiplications, where $\odot$ denotes the floating-point approximation of real product.

As shown in Example 4.21, fixed two floating-point numbers x, y , we have that $y = f(x) = x \odot z$ only if

$$|z| \in \left[\left| \frac{y}{x} \right| \frac{1}{1 + \mathbf{u}}, \left| \frac{y}{x} \right| \frac{1}{1 - \mathbf{u}} \right].$$

If $\mathbf{u} \leq 1/2$ this is entirely contained in the interval

$$\left[\left| \frac{y}{x} \right| (1 - 2\mathbf{u}), \left| \frac{y}{x} \right| (1 + 2\mathbf{u}) \right].$$

It can be shown that, if $\mathbf{u} < 1/2$, the quantity of floating point numbers in that interval is bounded by a constant c that does not depend on x and y .¹ Analogously, fixed y and z there are at most c floating point numbers x such that $y = f(x) = x \odot z$.

With these two bounds, if x_0 has enough preimages we can verify the last hypothesis of the theorem: letting $N = |C|$

$$|P_F(x_0)| = \omega(\log(N) \cdot k_1(N) \cdot k_2(N)) = \omega(\log(N) \cdot c \cdot c) = \omega(\log(N)).$$

For instance $x_0 = 1$ satisfies this condition (more than half of all floating-point numbers have a floating-point inverse, hence $|P_F(x_0)| \geq N/2 = \Theta(N)$) but there are many other points satisfying it. By mean of Theorem 4.28, no under-approximation abstract domain is non-emptying for all multiplications whenever one of these points is representable. ■

¹For instance, this can be proved for $c = 17$.

4.7 Summary

In this chapter, we pointed out some asymmetries between over and under-approximation in Abstract Interpretation, and why those are an obstacle to the design of abstract domains for program analysis via under-approximation Galois connections. The key observation is that the duality between under and over-approximation is broken by the fact that in program analysis over and under-approximations have to be applied to the same transfer functions. The handling of divergence in the abstract domain poses another critical issue.

Building on those ideas, we proposed the novel definition of *non-emptying function* and studied how it plays a crucial role in proving the non-existence of general, useful under-approximation based abstract domains. Indeed, our results prove that an analysis based on such domains will very often answer $\perp$ (representing either absence of information or divergence) for programs that require repeated applications of non-emptying functions. This is a big limitation since recovery from $\perp$ in an under-approximation is not as easy as recovering from $\top$ in an over-approximation: we can say that it is quite impossible. We applied our general results to several concrete domains to conclude that, under some mild assumptions, there are no useful under-approximation abstract domains for program analysis. Then, to show that one of the hypotheses in our result is tight, we proposed a construction to craft an under-approximation abstract domain whenever such a hypothesis is not met.

To summarize, our results hint at the difficulty of designing under-approximation abstract domains. This suggest that it is better to resort to other under-approximation techniques to combine with over-approximation ones, including usual over-approximation abstract interpretation domains. Therefore, in the next chapter we study logical frameworks for under-approximation.

Chapter 5

Sufficient incorrectness logic

In this chapter, we consider triple-based program logics used for both over and under-approximation. We focus on three known examples, namely Hoare Logic (HL), Incorrectness Logic (IL) and Necessary Conditions (NC). We characterize their validity conditions in term of over or under-approximation of forward and backward semantics. First, this allows us to identify the absence of one combination, and thus to define a new program logic, called Sufficient Incorrectness Logic (SIL), for it. Second, this guides a thorough comparison of the four validity conditions, highlighting analogies and differences between over and under-approximation approaches which will prove useful in choosing what over and under-approximation approach to use in the next chapters.

The content of this chapter is based on [ABGL24].

5.1 Taxonomy

As discussed in Sections 2.4 and 2.5, the validity conditions of HL and IL are defined as over and under-approximation of the forward semantics $\llbracket \cdot \rrbracket$. However, other program logics cannot be naturally described in terms of $\llbracket \cdot \rrbracket$. To this end, we consider a backward semantics $\llbracket \cdot \rrbracket^{\leftarrow}$ defined as the converse relation of the forward semantics,¹ that is

$$\llbracket \cdot \rrbracket^{\leftarrow} \sigma' \triangleq \{ \sigma \mid \sigma' \in \llbracket r \rrbracket \sigma \} \quad (5.1)$$

or, equivalently,

$$\sigma \in \llbracket \cdot \rrbracket^{\leftarrow} \sigma' \iff \sigma' \in \llbracket r \rrbracket \sigma \quad (5.2)$$

and we additively lift this definition to set of states by union. Intuitively, the forward semantics $\llbracket r \rrbracket P$ denotes the set of all possible output states of r when execution starts from a state in P (and r terminates). Instead, the backward semantics $\llbracket \cdot \rrbracket^{\leftarrow} Q$ denotes the set of all input states that can lead to a state in Q . We remark that this backward semantics $\llbracket \cdot \rrbracket^{\leftarrow}$ is not Dijkstra **wlp** $[\cdot]$. Instead, it correspond to Hoare's weakest possible precondition [Hoa78]. Particularly, this means that $\llbracket r \rrbracket$ and $\llbracket \cdot \rrbracket^{\leftarrow}$ are not adjoints in the sense that $Q \subseteq \llbracket r \rrbracket P$ if and only if $\llbracket \cdot \rrbracket^{\leftarrow} Q \subseteq P$. For instance, it's easy to build a counterexample using unreachable states (see Lemma 5.12). However, we will show an adjunctive relation between the two in Proposition 5.9.

The backward semantics can also be characterized compositionally, similarly to the forward one:

¹Formally, if we consider a relation $\mathcal{R}_r$ between states defined as $(\sigma, \sigma') \in \mathcal{R}_r$ iff $\sigma' \in \llbracket r \rrbracket \sigma$, the backward semantics $\llbracket \cdot \rrbracket^{\leftarrow}$ defines the converse relation $\mathcal{R}_r^{-1}$.

	Forward	Backward
Over	HL: $\llbracket r \rrbracket P \subseteq Q$	NC: $\llbracket \overleftarrow{r} \rrbracket Q \subseteq P$
Under	IL: $\llbracket r \rrbracket P \supseteq Q$	SIL: $\llbracket \overleftarrow{r} \rrbracket Q \supseteq P$

Figure 5.1: The taxonomy of validity conditions. Columns indicate if validity is based on forward or backward semantics and rows the verse of approximation. HL and NC are equivalent ($\simeq$). SIL is our new logic.

Lemma 5.1. *For any regular commands $r, r_1, r_2 \in \text{Reg}$, the following equalities hold:*

$$\llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket = \llbracket \overleftarrow{r_1} \rrbracket \circ \llbracket \overleftarrow{r_2} \rrbracket \quad \llbracket \overleftarrow{r_1} \oplus \overleftarrow{r_2} \rrbracket = \llbracket \overleftarrow{r_1} \rrbracket \cup \llbracket \overleftarrow{r_2} \rrbracket \quad \llbracket \overleftarrow{r^*} \rrbracket = \bigcup_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n$$

Using $\llbracket \overleftarrow{\cdot} \rrbracket$, we characterize NC with the validity condition $\llbracket \overleftarrow{r} \rrbracket Q \subseteq P$. To see why, assume Q describes good final states. Then, $\llbracket \overleftarrow{r} \rrbracket Q$ defines all states which can reach a good state. Since a precondition $\underline{P}$ is necessary for Q if it contains every state which can reach a state in Q , $\underline{P}$ must contain at least all states in $\llbracket \overleftarrow{r} \rrbracket Q$. More formally, for any initial state $\sigma \in \llbracket \overleftarrow{r} \rrbracket Q$, there exist a $\sigma' \in \llbracket r \rrbracket \sigma$ such that $\sigma' \in Q$. This means that σ has a trace in $\mathcal{T}(\sigma)$ (the one ending in σ'), so it must belong to any necessary precondition $\underline{P}$.

Proposition 5.2 (NC as backward over-approximation). *Given a postcondition Q for the program r , any possible necessary precondition $\underline{P}$ for Q satisfies*

$$\llbracket \overleftarrow{r} \rrbracket Q \subseteq \underline{P}$$

Note that this is not a new understanding of NC (it was already hinted in the work that introduced them, and an analogous characterization appeared in [ZK22, §6.3] in terms of weakest precondition), but the explicit use of the backward semantics in our contest enables a more streamlined comparison with other logics in Section 5.3.

We organize the validity conditions of HL, IL and NC in the taxonomy in Figure 5.1. We classify logics depending on (1) whether the condition is expressed in terms of forward or backward semantics and (2) whether it is an over or an under-approximation. Note that ours is not the first such schematization: others proposed similar taxonomies considering possibly more and different axes. For instance, [Cou24] uses abstract interpretation techniques to obtain many program logics from the composition of a set of small abstractions, [ZK22] compare logics using predicate transformers, [VK23] brings this latter approach further by considering all combinations of forward/backward, non/liberal and over/under-approximation together with a TopKAT formulation. However, our simpler schema clearly points us in the direction of the backward under-approximation condition: we therefore ask ourselves the meaning of such a condition and whether a logic for it has been developed.

Backward under-approximation and Lisbon Triples At POPL'19 in Lisbon, D. Dreyer and R. Jung suggested that P. O'Hearn should look at bug-finding in terms of a logic for proving the presence of faults (as reported in [OHe20; ZDS23]). However, the proposed model of triples did not fit well with a key feature of Pulse, a bug-catching tool developed at Meta, namely its ability to drop the analysis of some program paths, for which IL provides a sound logical foundation instead. The idea of such “Lisbon” triples is that *for any initial state satisfying the pre, there exists some execution trace*

Logic	HL [Hoa69]	IL [OHe20]	NC [CCL11]	OL [ZDS23]	SIL [ABGL24]
Triples	$\{P\} \text{ r } \{Q\}$	$[P] \text{ r } [Q]$	$(P) \text{ r } (Q)$	$\langle P \rangle \text{ r } \langle Q \rangle$	$\langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle$

Figure 5.2: Summary of the notation for different program logics

leading to a final state satisfying the post and it can be dated back to Hoare’s calculus of possible correctness [Hoa78], even if no form of approximation was considered there. Lisbon triples were then briefly discussed in [MOH21, §5] and [Le+22, §3.2] under the name *backwards under-approximate triples*. They were also one of the motivations for OL [ZDS23]: OL recognized the importance of tracking the sources of errors and can encode Lisbon triples together with Hoare triples. Backward under-approximation was also key for the development of a compositional non-termination analysis that has been integrated in Pulse [RVO24], based on the observation that forward and backward under-approximation can be unified in a single logical framework by dropping the respective consequence rules.

Ideally, given an incorrectness specification, the goal of backward under-approximation would be to report to programmers all dangerous input states that lead to bugs. However, all the above proposals are designed according to the forward semantics of programs and thus are best suited to infer postconditions starting from some given precondition. To tackle this issue, we introduce Sufficient Incorrectness Logic (SIL) as a proof system for Lisbon triples with backward-oriented rules.

Since in this chapter we deal with different kinds of program logics, a summary of the notation for the various triples is reported in Figure 5.2.

5.2 Sufficient Incorrectness Logic

SIL is a backward-oriented under-approximation proof system for Lisbon triples that focuses on *finding the sources of incorrectness rather than highlighting the presence of bugs*. Assuming the post Q defines some class of errors, i.e., it is what we call an *incorrectness specification*, a (valid) SIL triple $\langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle$ means that “all input states that satisfy P have at least one execution of the program r leading to a state that satisfies Q ”. For deterministic programs, SIL guarantees that a state satisfying the pre *always* leads to an error. Instead, if r is nondeterministic, SIL guarantees that there *exists* an execution that leads to an error. Sufficient incorrectness preconditions are extremely valuable to programmers: by pointing out the sources of errors, they serve as a starting point to scope down debugging, fuzzing, and testing. Moreover, it is known that, contrary to IL and its extensions, backward under-approximation can expose manifest errors [Le+22, §3.2]: an error Q is manifest iff the SIL triple $\langle\!\langle \text{true} \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle$ is valid. These are bugs that happen regardless of the context and it has been observed experimentally that are more likely to be fixed when reported [Le+22, §5]. Therefore, we study SIL to enhance program analysis frameworks with the ability to identify the source of incorrectness.

SIL goals include: (i) defining a default deduction mechanism that starts from a specification of erroneous outcomes and traces the computation back to some initial states responsible for such errors; (ii) exhibiting a minimal set of rules that are sound and complete for Lisbon triples; and (iii) spelling out, a posteriori, a formalization of the backward analysis step performed by industrial grade analysis tools for security developed at Meta [DFLO19; Gab21; BC19]. Those tools automatically find more than 50% of the security bugs in the Meta family of apps and many severe bugs [DFLO19, Fig. 5].

$\frac{}{\llbracket \overline{c} \rrbracket Q \text{ c } \langle Q \rangle} \langle \text{atom} \rangle$	$\frac{P \subseteq P' \quad \langle P' \rangle \text{ r } \langle Q' \rangle \quad Q' \subseteq Q}{\langle P \rangle \text{ r } \langle Q \rangle} \langle \text{cons} \rangle$
$\frac{\langle P \rangle \text{ r}_1 \langle R \rangle \quad \langle R \rangle \text{ r}_2 \langle Q \rangle}{\langle P \rangle \text{ r}_{1; r_2} \langle Q \rangle} \langle \text{seq} \rangle$	$\frac{\langle P_1 \rangle \text{ r}_1 \langle Q \rangle \quad \langle P_2 \rangle \text{ r}_2 \langle Q \rangle}{\langle P_1 \cup P_2 \rangle \text{ r}_{r_1 \oplus r_2} \langle Q \rangle} \langle \text{choice} \rangle$
$\frac{\forall n \geq 0. \langle Q_{n+1} \rangle \text{ r } \langle Q_n \rangle}{\langle \bigcup_{n \geq 0} Q_n \rangle \text{ r}^* \langle Q_0 \rangle} \langle \text{iter} \rangle$	
Additional rules	
$\frac{}{\langle \emptyset \rangle \text{ r } \langle Q \rangle} \langle \text{empty} \rangle$	$\frac{}{\langle Q \rangle \text{ r}^* \langle Q \rangle} \langle \text{iter0} \rangle$
$\frac{\langle P \rangle \text{ r}^*; \text{r } \langle Q \rangle}{\langle P \rangle \text{ r}^* \langle Q \rangle} \langle \text{unroll} \rangle$	$\frac{\langle P_1 \rangle \text{ r } \langle Q_1 \rangle \quad \langle P_2 \rangle \text{ r } \langle Q_2 \rangle}{\langle P_1 \cup P_2 \rangle \text{ r } \langle Q_1 \cup Q_2 \rangle} \langle \text{disj} \rangle$

Figure 5.3: Sufficient Incorrectness Logic

Roughly, the SIL triple

$$\langle P \rangle \text{ r } \langle Q \rangle$$

requires that all states in P have at least one execution leading to a state in Q . More formally, for all $\sigma \in P$ there must exist a state $\sigma' \in Q$ such that $\sigma' \in \llbracket \text{r} \rrbracket \sigma$. Particularly, in the presence of nondeterminism states in P are required to have one execution leading to Q , not necessarily all of them. Motivated by Figure 5.1, we choose the validity condition

$$\llbracket \overline{r} \rrbracket Q \supseteq P. \quad (\text{SIL})$$

However, the two definition are equivalent:

Proposition 5.3 (Characterization of SIL validity). *For any $\text{r} \in \text{Reg}$, $P, Q \subseteq \Sigma$*

$$\llbracket \overline{r} \rrbracket Q \supseteq P \iff \forall \sigma \in P. \exists \sigma' \in Q. \sigma' \in \llbracket \text{r} \rrbracket \sigma$$

A convenient way to exploit SIL is to assume that the analysis takes as input the incorrectness specification Q , i.e., the set of erroneous final states. Then, any valid SIL triple $\langle P \rangle \text{ r } \langle Q \rangle$ yields a precondition which surely captures erroneous executions. In this sense, P gives a sufficient condition for incorrectness and motivates the name of SIL. This is dual to the interpretation of IL where, for a given precondition P , any IL triple $[P] \text{ r } [Q]$ yields a set Q of final states which are for sure reachable, so that any error state in Q is a true bug reachable from some input in P .

5.2.1 Proof system

The inference rules for SIL are in Figure 5.3. The top five rules form a minimal, sound and complete proof system. The additional rules are other valid rules that can ease program analysis and are discussed in Section 5.2.2. Note that all the rules can be applied to an arbitrary post Q to infer a corresponding pre, in the same way as all the rules of IL can be applied to an arbitrary pre P to infer a corresponding post. This is a key feature of SIL proof system, which facilitates backward reasoning.

Atomic commands are handled by $\langle\langle\text{atom}\rangle\rangle$, which exploits the backward semantics and summarizes cases for **skip**, assignments and Boolean guards. In Section 5.3.2 we discuss further the rule for assignment when pre and postconditions are formulae instead of sets of states. The consequence rule allows to generalize a proof by weakening/strengthening the two conditions P and Q involved. It can readily be derived from the validity condition (SIL). Moreover, $\langle\langle\text{cons}\rangle\rangle$ allows SIL to drop disjuncts in the pre, just as $[\text{cons}]$ allows IL to do it in the post. This feature is crucial in both SIL and IL to increase scalability of tools. Rule $\langle\langle\text{seq}\rangle\rangle$ is standard: SIL triples are composed sequentially just like in all other logics. Rule $\langle\langle\text{choice}\rangle\rangle$ states that if all states in P_1 (resp. P_2) have an execution of r_1 (resp. r_2) ending in Q , they also have an execution of $r_1 \oplus r_2$ ending in Q since the semantics of $r_1 \oplus r_2$ is a superset of that of r_1 (resp. r_2), cf. Figure 2.1. This rule is also reminiscent of the equation for conditionals in the calculus of possible correctness [Hoa78]. For iteration, for each $n \geq 0$ we find inductively the precondition Q_n of executing r exactly n times. The precondition of the whole r^* is then the union of all the Q_n , as formalized by rule $\langle\langle\text{iter}\rangle\rangle$, which first appeared in [MOH21, §5].

The SIL proof system is both correct and complete. Correctness can be proved by induction on the derivation tree of a triple. Intuitively, if the premises of a rule are valid, then its consequence is valid as well, as we briefly observed in above. To prove completeness, we rely on the fact that rules other than $\langle\langle\text{cons}\rangle\rangle$ are exact, that is, if their premises satisfy the equality $\llbracket \check{r} \rrbracket Q = P$, their conclusion does as well. Using this, we prove the triple $\langle\langle \llbracket \check{r} \rrbracket Q \rangle\rangle r \langle\langle Q \rangle\rangle$ for any r and Q . We conclude using $\langle\langle\text{cons}\rangle\rangle$ to get a proof of $\langle\langle P \rangle\rangle r \langle\langle Q \rangle\rangle$ for any $P \subseteq \llbracket \check{r} \rrbracket Q$.

Theorem 5.4 (SIL is sound and complete). *A SIL triple is provable iff it is valid:*

$$\vdash \langle\langle P \rangle\rangle r \langle\langle Q \rangle\rangle \iff \models \langle\langle P \rangle\rangle r \langle\langle Q \rangle\rangle$$

5.2.2 Additional rules for program analysis

The topmost set of five rules in Figure 5.3 is deliberately minimal: if we remove any rule it is no longer complete. However, there are other valid rules that can be useful in practice, and not all of them can be directly derived from the core rules. Some of them are at the bottom of Figure 5.3. Note that, even if the rules themselves are not derivable (for instance the $\langle\langle\text{empty}\rangle\rangle$ rule), by completeness of the core rules they don't allow to prove any additional triple.

Rule $\langle\langle\text{empty}\rangle\rangle$ is used to drop paths backward, just like IL can drop them forward (an analogous axiom $[P] r [\emptyset]$ is valid for IL). Particularly, this allows to ignore one of the branches of $\langle\langle\text{choice}\rangle\rangle$, or to stop the backward iteration of $\langle\langle\text{iter}\rangle\rangle$ without covering all the iterations. An example of such an application is the derived rule $\langle\langle\text{iter0}\rangle\rangle$, which corresponds to not entering the iteration at all. It can be derived from rules $\langle\langle\text{iter}\rangle\rangle$ and $\langle\langle\text{empty}\rangle\rangle$ by taking $Q_0 = Q$ and $Q_n = \emptyset$ for $n \geq 1$. It subsumes HL's rule $\{\text{iter}\}$, which is based on loop invariants: those are a correct but not complete reasoning tool for under-approximation [OHe20]. Rule $\langle\langle\text{unroll}\rangle\rangle$ allows to unroll a loop once. Subsequent applications of this rule allow to simulate (backward) a finite number of iterations, and then rule $\langle\langle\text{iter0}\rangle\rangle$ can be used to ignore the remaining ones. This is on par with IL ability to unroll a loop a finite number of times to find some post, for which analogous rules are valid [OHe20; MOH21]. Rule $\langle\langle\text{disj}\rangle\rangle$ allows to split the analysis and join the results, just like HL and IL. However, while a corresponding rule $\{\text{conj}\}$ which perform intersection is sound for HL, it is unsound for both IL and SIL. We discuss this point further in Section 5.3.3.

All four these additional rules are sound (it can be proved by induction):

$$\begin{array}{c}
\frac{\frac{\frac{\langle T_{2M} \rangle r_w^* \langle T_{2M} \rangle \quad \langle \text{iter0} \rangle \quad \frac{\vdots}{\langle T_{2M} \rangle r_w \langle R_{2M} \rangle}}{\langle T_{2M} \rangle r_w^* \langle R_{2M} \rangle} \quad \langle \text{seq} \rangle}{\langle T_{2M} \rangle r_w^* \langle R_{2M} \rangle} \quad \langle \text{unroll} \rangle \quad \frac{\langle R_{2M} \rangle (n \leq 0)? \langle Q_{2M} \rangle}{\langle R_{2M} \rangle (n \leq 0)? \langle Q_{2M} \rangle} \quad \langle \text{atom} \rangle}{\langle T_{2M} \rangle r_w^* (n \leq 0)? \langle Q_{2M} \rangle} \quad \langle \text{seq} \rangle \\
(*) \\
\frac{\frac{\langle \text{true} \rangle x := 0 \langle x \leq 2000000 \rangle \quad \langle \text{atom} \rangle \quad \frac{\langle x \leq 2000000 \rangle n := \text{nondet}() \langle T_{2M} \rangle}{\langle x \leq 2000000 \rangle n := \text{nondet}() \langle T_{2M} \rangle} \quad \langle \text{atom} \rangle}{\langle \text{true} \rangle x := 0; n := \text{nondet}() \langle T_{2M} \rangle} \quad \langle \text{seq} \rangle}{\langle \text{true} \rangle \text{rloop0} \langle Q_{2M} \rangle} \quad (*) \quad \langle \text{seq} \rangle
\end{array}$$

Figure 5.4: Derivation of the SIL triple $\langle \text{true} \rangle \text{rloop0} \langle Q_{2M} \rangle$ for Example 5.6.

Proposition 5.5. *The additional SIL rules at the bottom of Figure 5.3 are sound, that is, triples provable in SIL extended with those rules are valid.*

Example 5.6. Let us consider the program “loop0” from [OHe20, §6.1]:

```

x := 0;
n := nondet();
while(n > 0) {
  x := x + n;
  n := nondet();
}
// assert(x != 2000000)

```

We can translate it in the syntax of regular commands by letting

$$\begin{aligned}
r_w &\triangleq (n > 0)?; x := x + n; n := \text{nondet}() \\
\text{rloop0} &\triangleq x := 0; n := \text{nondet}(); (r_w)^*; (n \leq 0)?
\end{aligned}$$

Final error states are those in $Q_{2M} \triangleq (x = 2000000)$.

To prove a triple for rloop0 , we have to perform at least one iteration, and we do so using $\langle \text{unroll} \rangle$. We let $R_{2M} \triangleq (x = 2000000 \wedge n \leq 0)$ and $T_{2M} \triangleq (x + n = 2000000 \wedge n > 0)$. It is straightforward to prove $\langle T_{2M} \rangle r_w \langle R_{2M} \rangle$ via $\langle \text{seq} \rangle$ and $\langle \text{atom} \rangle$. Given this triple, we can unroll the loop once and prove the same triple for r_w^* , as shown by the combination of $\langle \text{unroll} \rangle$, $\langle \text{seq} \rangle$ and $\langle \text{iter0} \rangle$ at the top-left of Figure 5.4. This is a property of under-approximation: a nondeterministic number of iterations can be under-approximated by a single iteration [OHe20, §6.1]. With this triple for the loop, we can prove for the whole program the triple $\langle \text{true} \rangle \text{rloop0} \langle Q_{2M} \rangle$ using $\langle \text{seq} \rangle$ and $\langle \text{atom} \rangle$. The full derivation is in Figure 5.4.

Differently than IL, this triple highlights that any initial state can lead to an error: instead of reporting the presence of a true bug, we can prove that this is a manifest error and produce an initial state which causes the bug. ■

5.3 Relations among logics

We first follow the two-dimensional scheme in Figure 5.1 to carry out a duality-driven comparison among the four validity conditions. Then, we realize that analogies and differ-

ences between them can be studied along other axes to obtain interesting insights among the relations between over/under-approximation, forward/backward analysis, reachability/divergence, and others.

We named columns of Figure 5.1 based on which semantics (forward or backward) they use. However, there is not a unique way of fixing the over and under-approximation axes. For instance, it is possible to take the consequence rules as the approximation axis, naming IL and NC as under-approximation because you can always substitute Q for one of its under-approximations $Q' \subseteq Q$. However, we chose to denote approximation depending on the “target” set, that is, Q for forward and P for backward semantics, respectively. We motivate this choice because it classifies both IL and SIL as under-approximation and they share the ability to drop program paths (e.g., by finite unrolling of loops).

5.3.1 Pairwise comparison

We first carry out a comparison of the logics two by two. We skip the pair HL and IL since it was already discussed when IL was introduced in [OHe20].

NC and IL

Sufficient preconditions are properties that imply Dijkstra’s **wlp**: $\bar{P}$ is sufficient for a postcondition Q if and only if $\bar{P} \subseteq \mathbf{wlp}[r](Q)$, which in turn is equivalent to validity of the HL triple $\{\bar{P}\} r \{Q\}$. Necessary and sufficient preconditions are dual, and so are IL and HL. Moreover, NC and IL enjoy the same consequence rule: both can strengthen the post and weaken the pre. This double duality suggests a relation between NC and IL. However, the following example shows this is not the case.

Example 5.7. Consider the nondeterministic program `r42` from Example 2.8, where $Q_{42} \triangleq (z = 42)$. For brevity, we let $Q'_{42} \triangleq (Q_{42} \wedge \text{odd}(y) \wedge \text{even}(x))$. From that Example we know that $[z = 11] \text{ r42 } [Q'_{42}]$ is valid in IL. However, we observe that the NC triple $(z = 11) \text{ r42 } (Q'_{42})$ is not valid because, e.g., the state $[y \mapsto 1, z \mapsto 10]$ has an execution leading to Q'_{42} but doesn’t satisfy $z = 11$. Moreover, take for instance $\underline{P} \triangleq \text{odd}(y)$, which makes the NC triple $(\underline{P}) \text{ r42 } (Q'_{42})$ valid (in any state *not* satisfying $\underline{P}$ y is even, is not changed by `r42` and should be odd to satisfy Q'_{42}). Then it is clear that $(z = 11) \not\models \underline{P}$. This shows that not only IL triples do not yield NC triples, but also that in general there are NC preconditions which are not implied by IL preconditions.

Conversely, consider $\neg Q_{42} = (z \neq 42)$. While the NC triple $(\mathbf{true}) \text{ r42 } (\neg Q_{42})$ is valid, the IL triple $[\mathbf{true}] \text{ r42 } [\neg Q_{42}]$ is not: for instance, the final state $[x \mapsto 11, y \mapsto 11, z \mapsto 11]$ is not reachable from any initial state. It follows that the IL triple $[P] \text{ r42 } [\neg Q_{42}]$ is not valid for any P . ■

Given $\models (\underline{P}) r (Q)$ and $\models [P] r [Q]$, there always are states satisfying both P and $\underline{P}$, i.e., $P \cap \underline{P} \neq \emptyset$. However, in general neither $P \subseteq \underline{P}$ nor $\underline{P} \subseteq P$. The difference between NC and IL becomes apparent when we spell out their validity conditions using quantifiers:

$$\forall \sigma' \in Q. \forall \sigma \in \llbracket \bar{r} \rrbracket \sigma'. \sigma \in P \quad (\text{NC}_{\text{FOL}})$$

$$\forall \sigma' \in Q. \exists \sigma \in \llbracket \bar{r} \rrbracket \sigma'. \sigma \in P \quad (\text{IL}_{\text{FOL}})$$

Initial states are universally quantified in (NC_{FOL}) —*all* initial states with a good run must satisfy the precondition—but they are existentially quantified in (IL_{FOL}) . We also note that, when r is reversible (i.e., $\llbracket r \rrbracket$ is injective) any valid IL triple is also a valid NC triple.

NC and HL

It turns out that NC is strongly connected to weakest liberal preconditions and thus to HL. Let Q be a correctness postcondition: a finite trace is in $\mathcal{T}(\sigma)$ if its final state satisfies Q and in $\mathcal{E}(\sigma)$ otherwise. In general, a necessary precondition has no relationship with $\mathbf{wlp}[r](Q)$. However, if we consider $\neg Q$ instead of Q , we observe that “erroneous” executions becomes those in $\mathcal{T}(\sigma)$ and “correct” ones those in $\mathcal{E}(\sigma)$. This means that $\mathcal{T}(\sigma) = \emptyset$ iff $\sigma \in \mathbf{wlp}[r](\neg Q)$, from which we derive $\neg \underline{P} \subseteq \mathbf{wlp}[r](\neg Q)$ or, equivalently, $\neg \mathbf{wlp}[r](\neg Q) \subseteq \underline{P}$.

Example 5.8. Consider again program `r42` from Example 2.8 and the correctness specification $\neg Q_{42} = (z \neq 42)$. We have that $\mathbf{wlp}[r42](\neg \neg Q_{42}) = Q_{42}$, because if initially $z \neq 42$ then it is possible that x is assigned an odd value and z is not updated. Hence, a condition P is implied by $\neg \mathbf{wlp}[r](\neg \neg Q_{42}) = \neg Q_{42}$ if and only if it is necessary. For instance, $(z \neq 42 \vee \text{odd}(y))$ is necessary but $(z > 42)$ is not. ■

The next bijection establishes the connection between (NC) and (HL): a necessary precondition is just the negation of a sufficient precondition for the negated post. This was also observed using weakest (liberal) preconditions in [ZK22, Theorem 5.4].

Proposition 5.9 (Bijection between NC and HL). *For any $r \in \text{Reg}$ and $P, Q \subseteq \Sigma$:*

$$\llbracket r \rrbracket P \subseteq Q \iff \llbracket \neg r \rrbracket (\neg Q) \subseteq \neg P.$$

Particularly, this establishes an adjunction between $\llbracket r \rrbracket$ and $\neg \circ \llbracket \neg r \rrbracket \circ \neg$.

SIL and IL

Proposition 5.9 highlights an isomorphism between (HL) and (NC). By duality, this naturally sparks the question whether a similar connections between (IL) and (SIL) exists. The next example shows this is not the case.

Example 5.10. Since IL and SIL enjoy different consequence rules, neither of the two can imply the other with the same P and Q . If we take negation into account, consider the program `r1` $\triangleq x := 1$. Both the SIL triple $\langle x \geq 0 \rangle \text{ r1 } \langle x = 1 \rangle$ and the IL triple $[x \geq 0] \text{ r1 } [x = 1]$ are valid. However, neither $[x < 0] \text{ r1 } [x \neq 1]$ nor $\langle x < 0 \rangle \text{ r1 } \langle x \neq 1 \rangle$ are valid. So neither (IL) implies negated (SIL) nor the other way around. ■

To gain some insights on why (SIL) and (IL) are not equivalent, we introduce the following concepts.

Definition 5.11. Given a regular command r , we define the set of states that only diverges D_r and the set of unreachable states U_r :

$$D_r \triangleq \{\sigma \mid \llbracket r \rrbracket \sigma = \emptyset\} \quad U_r \triangleq \{\sigma' \mid \sigma' \notin \llbracket r \rrbracket \Sigma\} = \{\sigma' \mid \llbracket \neg r \rrbracket \sigma' = \emptyset\}.$$

These two definition are dual when we reverse the execution direction: if we consider $\llbracket \neg r \rrbracket$ instead of $\llbracket r \rrbracket$, the roles of D and U are swapped. This means that, in a sense, U_r is the set of states which “diverge” going backward.

Lemma 5.12. *For any regular command $r \in \text{Reg}$ and sets of states $P, Q \subseteq \Sigma$ it holds:*

$$\llbracket \neg r \rrbracket \llbracket r \rrbracket P \supseteq P \setminus D_r \quad \llbracket r \rrbracket \llbracket \neg r \rrbracket Q \supseteq Q \setminus U_r.$$

This lemma highlights the asymmetry between over and under-approximation: the composition of a function with its inverse is increasing (but for non-terminating states). This explains why (HL) and (NC) are related while (IL) and (SIL) are not: for over-approximation, $P \setminus D_r \subseteq \llbracket \overleftarrow{r} \rrbracket P$ can be further exploited if we know $\llbracket r \rrbracket P \subseteq Q$ via (HL), but it cannot when $\llbracket r \rrbracket P \supseteq Q$ via (IL).

Lastly, while IL and SIL are not directly comparable, the preprint [RVO24] introduces a forward-oriented proof system with a core set of rules that are sound for both IL and SIL, therefore proving only triples that are valid for both. It also becomes complete for IL, resp. SIL, when augmented with the corresponding consequence rule.

SIL and HL

In general, HL and SIL are different, but they coincide whenever the program r is deterministic and terminates for every input.

Proposition 5.13. *For any $r \in \text{Reg}$ and $P, Q \subseteq \Sigma$:*

- *if r is deterministic, $\llbracket \overleftarrow{r} \rrbracket Q \supseteq P \implies \llbracket r \rrbracket P \subseteq Q$*
- *if r is terminating, $\llbracket r \rrbracket P \subseteq Q \implies \llbracket \overleftarrow{r} \rrbracket Q \supseteq P$*

Example 5.14. From Example 2.8, we know $\{\text{odd}(y)\} \text{ r42 } \{Q_{42}\}$ is a valid HL triple. Moreover, r42 always terminates, so according to Proposition 5.13 $\llbracket \text{odd}(y) \rrbracket \text{ r42 } \llbracket Q_{42} \rrbracket$ is valid. Indeed, whenever y is odd in the initial state, x can be assigned nondeterministically an even value so that execution enters the if statement and z is assigned 42. ■

5.3.2 Inference rules

In Figure 5.5 we compare the rules of SIL, HL and IL, so to emphasize the similarities and differences among them. HL rule $\{\text{iter}\}$ says that any invariant is acceptable, not necessarily the minimal one, so that HL relies on over-approximation. This is confirmed by the rows for **cons** and **empty**, where on the contrary IL and SIL are shown to rely on under-approximation. The consequence rule is the key rule of all the logics, because it allows to generalize a proof by weakening/strengthening the two conditions P and Q involved. The direction of rules $\llbracket \text{cons} \rrbracket$ of SIL and $\{\text{cons}\}$ of HL is the same and it is exactly the opposite direction of rule $[\text{cons}]$ of IL and NC, which coincide. So the different consequence rules follow the diagonals of Figure 5.1. The row for rules **seq** and **disj** show that in all cases triples can be composed sequentially and additively. Rules **iter0** and **unroll** correspond to finite loop unrolling and are a prerogative of under-approximation: they are the same for SIL and IL, but they are unsound for HL.

The **atom** rule deserves a more in-depth discussion. The presented version using sets shows that HL and IL exploit the forward semantics, and SIL the backward one. However, if we instead use formulae as pre and postconditions, this rule must be instantiated for all atomic construct, particularly for assignments. It is well known that there are two different, valid axioms for assignment in HL: Hoare's backward substitution [Hoa69] and Floyd's forward inference [Flo67] (where $q[a/x]$ denotes the usual capture-avoiding substitution of all free occurrences of x in q with the expression a).

$$\frac{}{\{q[a/x]\} \text{ x } := \text{ a } \{q\}} \{\text{Hoare}\} \qquad \frac{}{\{p\} \text{ x } := \text{ a } \{\exists x'. p[x'/x] \wedge x = a[x'/x]\}} \{\text{Floyd}\}$$

While both axioms are valid in HL, only Floyd's forward axiom is valid in IL [OHe20, §4], already showing a broken symmetry between over and under-approximation. While

Rule	SIL	HL	IL
atom	$\overline{\llbracket \bar{c} \rrbracket Q} \text{ c } \langle Q \rangle$	$\overline{\{P\} \text{ c } \llbracket \bar{c} \rrbracket P}$	$\overline{[P] \text{ c } \llbracket \bar{c} \rrbracket P}$
cons	$\frac{P \subseteq P' \quad \langle P' \rangle \text{ r } \langle Q' \rangle \quad Q' \subseteq Q}{\langle P \rangle \text{ r } \langle Q \rangle}$	$\frac{P \subseteq P' \quad \{P'\} \text{ r } \{Q'\} \quad Q' \subseteq Q}{\{P\} \text{ r } \{Q\}}$	$\frac{P \supseteq P' \quad [P'] \text{ r } [Q'] \quad Q' \supseteq Q}{[P] \text{ r } [Q]}$
seq	$\frac{\langle P \rangle \text{ r}_1 \langle R \rangle \quad \langle R \rangle \text{ r}_2 \langle Q \rangle}{\langle P \rangle \text{ r}_1; \text{r}_2 \langle Q \rangle}$	$\frac{\{P\} \text{ r}_1 \{R\} \quad \{R\} \text{ r}_2 \{Q\}}{\{P\} \text{ r}_1; \text{r}_2 \{Q\}}$	$\frac{[P] \text{ r}_1 [R] \quad [R] \text{ r}_2 [Q]}{[P] \text{ r}_1; \text{r}_2 [Q]}$
choice	$\frac{\forall i \in \{1, 2\} \quad \langle P_i \rangle \text{ r}_i \langle Q \rangle}{\langle P_1 \cup P_2 \rangle \text{ r}_1 \oplus \text{r}_2 \langle Q \rangle}$	$\frac{\forall i \in \{1, 2\} \quad \{P\} \text{ r}_i \{Q\}}{\{P\} \text{ r}_1 \oplus \text{r}_2 \{Q\}}$	$\frac{\forall i \in \{1, 2\} \quad [P] \text{ r}_i [Q_i]}{[P] \text{ r}_1 \oplus \text{r}_2 [Q_1 \cup Q_2]}$
iter	$\frac{\forall n \geq 0. \langle Q_{n+1} \rangle \text{ r } \langle Q_n \rangle}{\langle \bigcup_{n \geq 0} Q_n \rangle \text{ r}^* \langle Q_0 \rangle}$	$\frac{\{P\} \text{ r } \{P\}}{\{P\} \text{ r}^* \{P\}}$	$\frac{\forall n \geq 0. [P_n] \text{ r } [P_{n+1}]}{[P_0] \text{ r}^* [\bigcup_{n \geq 0} P_n]}$
empty	$\overline{\langle \emptyset \rangle \text{ r } \langle Q \rangle}$	$\overline{\{\emptyset\} \text{ r } \{Q\}}$	$\overline{[P] \text{ r } [\emptyset]}$
disj	$\frac{\langle P_1 \rangle \text{ r } \langle Q_1 \rangle \quad \langle P_2 \rangle \text{ r } \langle Q_2 \rangle}{\langle P_1 \cup P_2 \rangle \text{ r } \langle Q_1 \cup Q_2 \rangle}$	$\frac{\{P_1\} \text{ r } \{Q_1\} \quad \{P_2\} \text{ r } \{Q_2\}}{\{P_1 \cup P_2\} \text{ r } \{Q_1 \cup Q_2\}}$	$\frac{[P_1] \text{ r } [Q_1] \quad [P_2] \text{ r } [Q_2]}{[P_1 \cup P_2] \text{ r } [Q_1 \cup Q_2]}$
iter0	$\overline{\langle Q \rangle \text{ r}^* \langle Q \rangle}$	unsound	$\overline{[P] \text{ r}^* [P]}$
unroll	$\frac{\langle P \rangle \text{ r}^*; \text{r } \langle Q \rangle}{\langle P \rangle \text{ r}^* \langle Q \rangle}$	unsound	$\frac{[P] \text{ r}^*; \text{r } [Q]}{[P] \text{ r}^* [Q]}$
conj	unsound	$\frac{\{P_1\} \text{ r } \{Q_1\} \quad \{P_2\} \text{ r } \{Q_2\}}{\{P_1 \cap P_2\} \text{ r } \{Q_1 \cap Q_2\}}$	unsound

Figure 5.5: Comparison of SIL, HL and IL rules. Identical rules are highlighted in purple.

our presentation of SIL rules uses sets of states, we use Hoare’s backward substitution in Separation SIL (see Figure 5.7): dually to the forward IL, the backward axiom is valid for SIL. Surprisingly, Floyd’s forward axiom is valid in SIL as well: this shows that even for under-approximation, forward and backward semantics behave differently. This is possibly rooted in the properties of arithmetic expressions: they are defined for every input but not necessarily surjective. In the terminology of Lemma 5.12, $D_{x:=a}$ is always empty but $U_{x:=a}$ may not be.

5.3.3 Weakest/strongest conditions

Depending on the way in which program analysis is conducted, it can be interesting to derive either the most general or most specific hypotheses for the given property. For instance, given a correctness specification Q , one is typically interested in finding the minimal constraint on the input that guarantees program correctness (this correspond to computing Dijkstra’s **wlp**). Conversely, to infer necessary conditions we can be interested in devising the strongest hypotheses under which some correct run is possible.

To investigate the existence of weakest/strongest pre and post, we find convenient to focus on the validity of the four kinds of triples as shown in Figure 5.1. The concrete semantics is trivially a strongest (HL and NC) or weakest (IL and SIL) condition for the “target” property (i.e., P computing backward and Q forward). However, it turns out that having a best condition on the “source” property is a prerogative of over-approximation, i.e., that over and under-approximation are not dual theories in this respect.

Proposition 5.15 (Existence of weakest conditions). *For any command $r \in \text{Reg}$:*

- given Q , there exists a weakest P such that $\llbracket r \rrbracket P \subseteq Q$ (HL);

- given P , there exists a weakest Q such that $\llbracket \leftarrow r \rrbracket Q \subseteq P$ (NC).

Proposition 5.16 (Non-existence of strongest conditions). *For any command $r \in \text{Reg}$:*

- for some Q , there is no strongest P such that $\llbracket r \rrbracket P \supseteq Q$ (IL);
- for some P , there is no strongest Q such that $\llbracket \leftarrow r \rrbracket Q \supseteq P$ (SIL).

The reason why strongest conditions may not exist for IL and SIL is that collecting semantics (both forward and backward) are additive but not co-additive. In other words, rule $\{\text{disj}\}$ is sound for all triples, while rule $\{\text{conj}\}$ is valid for HL and NC but neither for IL nor SIL, as shown in Figure 5.5. This means that given two IL triples $[P_1] r [Q]$ and $[P_2] r [Q]$, in general $\llbracket r \rrbracket (P_1 \cap P_2) \not\supseteq Q$ in which case $[P_1 \cap P_2] r [Q]$ is not valid, as shown in the following example.

Example 5.17. Consider the program $r1 \triangleq x := 1$. The two IL triples $[x = 0] r1 [x = 1]$ and $[x = 10] r1 [x = 1]$ are valid, but their intersection is $[\emptyset] r1 [x = 1]$, which is not valid.

For SIL, consider the program $\text{rnd} \triangleq x := \text{nondet}()$. Both triples $\langle\langle x = 1 \rangle\rangle \text{rnd} \langle\langle x = 0 \rangle\rangle$ and $\langle\langle x = 1 \rangle\rangle \text{rnd} \langle\langle x = 10 \rangle\rangle$ are valid, but also incomparable and minimal because $\emptyset$ is not a valid postcondition. ■

This can also be observed using the theory of adjunction. It is well known that left adjoints are additive while right adjoints are co-additive [DP02]. The weakest precondition wlp for HL is characterized by the adjunctive property $P \subseteq \text{wlp}[r](Q)$ iff $\llbracket r \rrbracket P \subseteq Q$ (weakest postconditions for NC are defined analogously). Since the forward (resp. backward) semantics is additive we get the existence of its right adjoint, that is exactly HL weakest precondition (resp. NC weakest postcondition). However, a strongest precondition sp for IL would satisfy the adjunctive property $\text{sp}[r](Q) \subseteq P$ iff $Q \subseteq \llbracket r \rrbracket P$, making the non co-additive forward semantics a right adjoint. Similarly, a strongest postcondition for SIL would be a left adjoint of the backward semantics.

5.3.4 Termination and Reachability

Termination and reachability are two sides of the same coin when switching from forward to backward semantics, and over and under-approximation behave differently with respect to these notions.

For HL we can only distinguish a precondition which always causes divergence: if $\{P\} r \{\emptyset\}$, all states in the precondition P will always diverge. However, if just one state in P has one terminating computation, its final state must be in $Q \neq \emptyset$, so we cannot say whether states in P diverge or not. Moreover, because of the over-approximation, a non empty Q does not mean there truly are finite executions, as those may be introduced by the approximation. Dually, NC cannot say much about reachability of Q unless P is empty, in which case Q is unreachable.

On the contrary, under-approximation offers much stronger guarantees on termination/reachability. Any IL triple $[P] r [Q]$ ensures that all states in Q are reachable (in particular, from states in P). Dually, a SIL triple $\langle\langle P \rangle\rangle r \langle\langle Q \rangle\rangle$ means that all states in P have a convergent computation that ends in some state in Q . This observation motivates the design of a forward iteration rule in IL (resp. backward in SIL): a backward (resp. forward) rule would need to prove reachability of all points in the post (resp. pre). Instead, the forward rule of IL (resp. backward rule of SIL) ensures reachability (resp. termination) by construction, as it builds Q (resp. P) only with points known to be reachable (resp. terminating).

$\text{mod}(\text{skip}) = \emptyset$	$\text{mod}(\mathbf{x} := \mathbf{a}) = \{\mathbf{x}\}$
$\text{mod}(\mathbf{b}?) = \emptyset$	$\text{mod}(\mathbf{x} := \text{alloc}()) = \{\mathbf{x}\}$
$\text{mod}(\text{free}(\mathbf{x})) = \emptyset$	$\text{mod}(\mathbf{x} := [\mathbf{y}]) = \{\mathbf{x}\}$
$\text{mod}([\mathbf{x}] := \mathbf{y}) = \emptyset$	$\text{mod}(\mathbf{r}_1; \mathbf{r}_2) = \text{mod}(\mathbf{r}_1) \cup \text{mod}(\mathbf{r}_2)$
$\text{mod}(\mathbf{r}_1 \oplus \mathbf{r}_2) = \text{mod}(\mathbf{r}_1) \cup \text{mod}(\mathbf{r}_2)$	$\text{mod}(\mathbf{r}^*) = \text{mod}(\mathbf{r})$

Figure 5.6: Definition of $\text{mod}(\mathbf{r})$.

5.4 Separation Sufficient Incorrectness Logic

We instantiate SIL to handle pointers and dynamic memory allocation, introducing Separation SIL. The goal of Separation SIL is to identify the causes of memory errors: it takes the backward under-approximation principles of SIL and combines it with the ability to deal with pointers from Separation Logic (SL) [Rey02; ORY01]

5.4.1 Heap regular commands

We denote by HRCmd the set of heap regular commands obtained by plugging the following definition of heap atomic commands in (2.3) (in blue new primitives):

$$\text{HRCmd} \ni c ::= \text{skip} \mid \mathbf{x} := \mathbf{a} \mid \mathbf{b}? \mid \mathbf{x} := \text{alloc}() \mid \text{free}(\mathbf{x}) \mid \mathbf{x} := [\mathbf{y}] \mid [\mathbf{x}] := \mathbf{y}$$

where we assume that $\mathbf{x}$ and $\mathbf{y}$ are syntactically distinct variables. The command $\mathbf{x} := \text{alloc}()$ allocates a new memory location containing a nondeterministic value, $\text{free}(\mathbf{x})$ deallocates memory, and $[\cdot]$ dereferences a variable. The syntax only allows to allocate, free and dereference single variables. To use a value from the heap in an arithmetic $\mathbf{a} \in \text{AExp}$ or Boolean expressions $\mathbf{b} \in \text{BExp}$, it must be assigned to a variable beforehand.

Given a heap command $\mathbf{r} \in \text{HRCmd}$, we let $\text{fv}(\mathbf{r}) \subseteq \text{Var}$ be the set of (free) variables of $\mathbf{r}$ and $\text{mod}(\mathbf{r}) \subseteq \text{Var}$ be the set of variables modified by $\mathbf{r}$ (the latter is a syntactic approximation of the variable possibly modified by the program). The definition of the former is standard, while the latter is defined inductively in Figure 5.6. Note that $\text{free}(\mathbf{x})$ and $[\mathbf{x}] := \mathbf{y}$ do not modify $\mathbf{x}$: they only modify the value *pointed by* $\mathbf{x}$, not the actual value of $\mathbf{x}$ (the memory address itself).

5.4.2 Assertion language

Our assertion language is derived from SL (see Section 2.7) and ISL [Raa+20]:

$$\text{Asl} \ni p, q ::= \text{false} \mid \text{true} \mid p \wedge q \mid p \vee q \mid \exists x. p \mid \mathbf{a} \asymp \mathbf{a} \mid \text{emp} \mid x \mapsto \mathbf{a} \mid x \not\mapsto \mid p * q$$

where $\asymp \in \{=, \neq, \leq, <, \dots\}$ replaces standard comparison operators, $x \in \text{Var}$ is a generic variable (both program and auxiliary logical variables) and $\mathbf{a} \in \text{AExp}$ is an arithmetic expression. The first six constructs describe a fragment of first-order logic, called coherent logic [BC05], which is also the one used in bi-abduction [CDOY09]. The last four describe heaps. emp denotes an empty heap, $x \mapsto a$ represents an heap with a single memory cell pointed by x and whose content is a , $x \not\mapsto$ describes that x points to a previously deallocated memory cell (it was first introduced in [Raa+20]). The separating conjunction $p * q$ is a key feature of Separation Logics and describes an heap which can be divided

$\frac{}{\langle \text{emp} \rangle \text{ skip } \langle \text{emp} \rangle} \langle \text{skip} \rangle$	$\frac{}{\langle q[a/x] \rangle x := a \langle q \rangle} \langle \text{assign} \rangle$
$\frac{}{\langle \text{emp} \rangle x := \text{alloc}() \langle x \mapsto v \rangle} \langle \text{alloc1} \rangle$	$\frac{}{\langle \text{emp} \wedge b \rangle b? \langle \text{emp} \rangle} \langle \text{assume} \rangle$
$\frac{}{\langle \beta \mapsto \rangle x := \text{alloc}() \langle x = \beta \wedge x \mapsto v \rangle} \langle \text{alloc2} \rangle$	$\frac{}{\langle x \mapsto - \rangle \text{free}(x) \langle x \mapsto \rangle} \langle \text{free} \rangle$
$\frac{x \notin \text{fv}(a)}{\langle y \mapsto a * q[a/x] \rangle x := [y] \langle y \mapsto a * q \rangle} \langle \text{load} \rangle$	$\frac{}{\langle x \mapsto - \rangle [x] := y \langle x \mapsto y \rangle} \langle \text{store} \rangle$
$\frac{\langle p \rangle r \langle q \rangle \quad \text{fv}(t) \cap \text{mod}(r) = \emptyset}{\langle p * t \rangle r \langle q * t \rangle} \langle \text{frame} \rangle$	$\frac{\langle p \rangle r \langle q \rangle \quad x \notin \text{fv}(r)}{\langle \exists x. p \rangle r \langle \exists x. q \rangle} \langle \text{exists} \rangle$
$\frac{p \Rightarrow p' \quad \langle p' \rangle r \langle q' \rangle \quad q' \Rightarrow q}{\langle p \rangle r \langle q \rangle} \langle \text{cons} \rangle$	$\frac{\langle p \rangle r_1 \langle t \rangle \quad \langle t \rangle r_2 \langle q \rangle}{\langle p \rangle r_1; r_2 \langle q \rangle} \langle \text{seq} \rangle$
$\frac{\langle p_1 \rangle r_1 \langle q \rangle \quad \langle p_2 \rangle r_2 \langle q \rangle}{\langle p_1 \vee p_2 \rangle r_1 \oplus r_2 \langle q \rangle} \langle \text{choice} \rangle$	$\frac{\forall n \geq 0 \quad \langle q(n+1) \rangle r \langle q(n) \rangle}{\langle \exists n. q(n) \rangle r^* \langle q(0) \rangle} \langle \text{iter} \rangle$
$\frac{}{\langle \text{false} \rangle r \langle q \rangle} \langle \text{empty} \rangle$	$\frac{\langle p_1 \rangle r \langle q_1 \rangle \quad \langle p_2 \rangle r \langle q_2 \rangle}{\langle p_1 \vee p_2 \rangle r \langle q_1 \vee q_2 \rangle} \langle \text{disj} \rangle$
$\frac{}{\langle q \rangle r^* \langle q \rangle} \langle \text{iter0} \rangle$	$\frac{\langle p \rangle r^*; r \langle q \rangle}{\langle p \rangle r^* \langle q \rangle} \langle \text{unroll} \rangle$

Figure 5.7: Proof rules for Separation SIL. The first group replaces SIL rule $\langle \text{atom} \rangle$, the second includes rules peculiar of SL, the third includes rule from SIL.

in two disjoint sub-heaps, one satisfying p and the other q . We let $x \mapsto - \triangleq \exists v. x \mapsto v$ describe that x is allocated without tracking its exact value. Given a formula $p \in \text{Asl}$, we call $\text{fv}(p) \subseteq \text{Var}$ the set of its free variables.

5.4.3 Proof system

We present the rules of Separation SIL in Figure 5.7. $q[a/x]$ is the capture-avoiding substitution. For the sake of presentation, we present rules without explicit error management (see Remark 2.9). However, the extension is straightforward: in Section 5.4.6 we present the error rule for store and its use in Example 5.23, as well as discussing the formal changes to the semantics model.

We split the rules in three groups. The first group gives the rules for atomic commands $c \in \text{HACmd}$, i.e., all instances of the SIL rule $\langle \text{atom} \rangle$. The second one includes rules borrowed from SL, the third one from SIL. Differently than SIL, the post of Separation SIL rules for atomic commands is not a generic assertion q . We formulate rules this way inspired by the “local axiom” style of separation logics, but we will show in Section 5.4.5 how it is always possible to algorithmically rewrite any assertion q to apply these rules, thus allowing for (automated) backward reasoning in Separation SIL.

Rule $\langle \text{skip} \rangle$ is straightforward: whatever is true before and after the `skip` can be added with $\langle \text{frame} \rangle$. Rule $\langle \text{assign} \rangle$ is Hoare’s backward assignment rule [Hoa69]. Floyd’s forward axiom [Flo67] is also valid for SIL (see Section 5.3.2), but we opt for Hoare’s rule because it fits better with the backward analysis of SIL. Rule $\langle \text{assume} \rangle$ conjoins the

assertion **b** to the postcondition: only states satisfying the Boolean guard can reach the post. Rules $\langle\text{alloc1}\rangle$ and $\langle\text{alloc2}\rangle$ allocate a new memory location for x : in the former the location is new, in the latter it reuses the previously deallocated location β . In both variants, if the previous value of x is needed, $x = z$ can be introduced in the premise with $\langle\text{cons}\rangle$. Note that, in theory, this doubles the size of the formulae when allocation appears in the program because each allocation could be analysed with both rules. However, in practice, we propose an algorithm that exploits $\langle\text{alloc2}\rangle$ only, thus avoiding this issue (see Sections 5.4.5 and 5.4.7). Rule $\langle\text{free}\rangle$ requires x to be allocated before freeing it. Rule $\langle\text{load}\rangle$ is similar to $\langle\text{assign}\rangle$, with the addition of the (disjoint) $y \mapsto a$ to make sure that y is allocated. Rule $\langle\text{store}\rangle$ requires that x is allocated, and updates the value it points to. All these rules are local: thanks to $\langle\text{frame}\rangle$, they can specify only pre and post for the modified part of the heap.

Rule $\langle\text{exists}\rangle$ allows to “hide” local variables. Rule $\langle\text{frame}\rangle$ is typical of separation logics [Rey02; Raa+20]: it allows to add a frame around a derivation through the separating conjunction $*$, plugging the proof for a small portion of a program inside a larger heap. In the third group, we instantiated the SIL rules from Figure 5.3 for logical formulae, by replacing set theoretical symbols (such as $\subseteq$ and $\emptyset$) with the corresponding logical symbols (such as $\Rightarrow$ and **false**, respectively). The only notable difference is in rule $\langle\text{iter}\rangle$, where Separation SIL uses a predicate $q(n)$ parametrized by the natural number $n \in \mathbb{N}$ and the precondition $\exists n. q(n)$ in the conclusion of the rule. This is a logical replacement for the infinite union used in SIL rule.

We show in the next example how Separation SIL proof system can infer preconditions ensuring that a provided error can happen.

Example 5.18. Consider the the motivating example of [Raa+20], encoding a use-after-free bug involving C++ vector `push_back` function:

```
// program rclient
x := *v;
push_back(v);
*x := 1;

push_back(v) {
  if (nondet()) {
    free(*v);
    *v := alloc();
  } }
```

We encode the above program as a regular command by letting:

$$\text{rclient} \triangleq x := [v]; (r_b \oplus \text{skip}) \quad r_b \triangleq y := [v]; \text{free}(y); y := \text{alloc}(); [v] := y$$

Since our syntax does not include functions, we inline `push_back`. We cannot free and allocate $*v$ directly, whence the auxiliary variable y . For simplicity, we do not include the last assignment $*x := 1$ in `rclient`: whenever the postcondition $x \not\mapsto$ holds, an error occurs after `rclient`.

We prove the Separation SIL triple

$$\langle\langle v \mapsto z * z \mapsto - * \text{true} \rangle\rangle \text{rclient} \langle\langle x \not\mapsto * \text{true} \rangle\rangle$$

which ensures that *every* state in the precondition reaches the error, thus giving (many) actual witnesses for testing and debugging purposes. Moreover, Separation SIL proof system guides the crafting of the precondition if the proof is done from the error postcondition backward.

Let us fix the following assertions:

$$p \triangleq (v \mapsto z * z \mapsto - * \text{true}) \quad q \triangleq (x \not\mapsto * \text{true}) \quad t \triangleq (v \mapsto z * z \mapsto - * (x = z \vee x \not\mapsto) * \text{true})$$

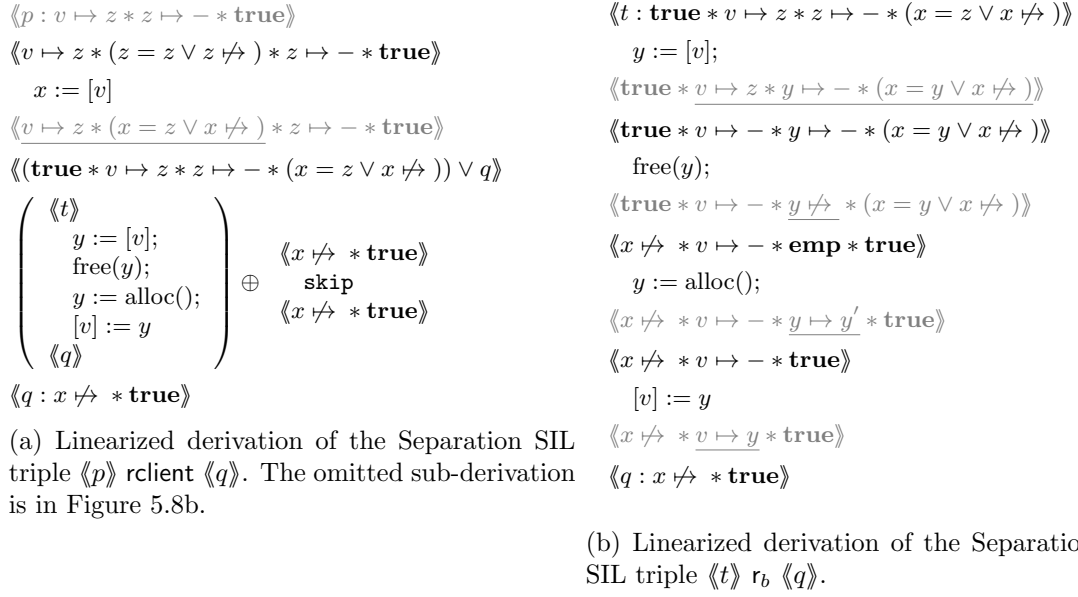


Figure 5.8: The full derivation of $\langle\langle p \rangle\rangle \text{ rclient } \langle\langle q \rangle\rangle$, split in two parts. We write in grey the strengthened conditions obtained using $\langle\langle \text{cons} \rangle\rangle$, and underline the postcondition of the rule for the current atomic command. Everything else is a frame shared between pre and post, using $\langle\langle \text{frame} \rangle\rangle$.

To prove the Separation SIL triple $\langle\langle p \rangle\rangle \text{ rclient } \langle\langle q \rangle\rangle$, we first prove the triple $\langle\langle t \rangle\rangle \text{ r}_b \langle\langle q \rangle\rangle$, whose derivation is in Figure 5.8b. Derivations are best read bottom-up: we start from the post and, for every atomic command, we find a suitable pre to apply the rule. In all cases, we strengthen the post to be able to apply the right rule: this usually means adding some constraint on the shape of the heap. Particularly, to apply the rule $\langle\langle \text{free} \rangle\rangle$ we need y to be deallocated, and this can happen in two different ways: either if $y = x$, since x is deallocated; or if y is a new name. This is captured by the disjunction $x = y \vee x \not\mapsto$. We remark that this can be inferred algorithmically via Lemma 5.21.

Using the derivation in Figure 5.8b, we complete the proof as shown in Figure 5.8a. In the derivation, using rule $\langle\langle \text{load} \rangle\rangle$ for the first assignment $x := [v]$, we get the pre $(v \mapsto z * z \mapsto - * (z = z \vee z \not\mapsto) * \mathbf{true})$, but since $z \mapsto - * z \not\mapsto$ is not satisfiable we remove that disjunct and find p .

Note the use of rule $\langle\langle \text{cons} \rangle\rangle$ in the pre of the nondeterministic choice to remove the disjunct $(x \not\mapsto * \mathbf{true})$, effectively dropping the analysis of that program path. This correspond to IL's ability to drop paths going forward. ■

In the example, we use as error postcondition $x \not\mapsto * \mathbf{true}$. It is necessary to include $(* \mathbf{true})$ because, in final reachable states, x is not the only variable allocated (there are also v and y), so the final heap should talk about them as well. Adding $(* \mathbf{true})$ is a convenient way to focus only on the part of the heap that describes the error, that is $x \not\mapsto$, and just leave everything else unspecified.

5.4.4 Soundness

To prove soundness of Separation SIL, we give a semantic model for heap regular commands. Fixed a finite set Var of variables and an infinite set Loc of memory locations, we define the set of values as $\text{Val} \triangleq \mathbb{Z} \uplus \text{Loc}$ ($\uplus$ is disjoint union). Stores $s \in \text{Store}$ are

$$\begin{aligned}
\llbracket \text{skip} \rrbracket(s, h) &\triangleq \{(s, h)\} \\
\llbracket x := a \rrbracket(s, h) &\triangleq \{(s[x \mapsto \llbracket a \rrbracket s], h)\} \\
\llbracket b? \rrbracket(s, h) &\triangleq \begin{cases} \{(s, h)\} & \text{if } \llbracket b \rrbracket s = \text{tt} \\ \emptyset & \text{otherwise} \end{cases} \\
\llbracket x := \text{alloc}() \rrbracket(s, h) &\triangleq \{(s[x \mapsto l], h[l \mapsto v]) \mid v \in \text{Val}, \text{avail}_h(l)\} \\
\llbracket \text{free}(x) \rrbracket(s, h) &\triangleq \{(s, h[s(x) \mapsto \delta])\} \quad \text{if } h(s(x)) \in \text{Val} \\
\llbracket x := [y] \rrbracket(s, h) &\triangleq \{(s[x \mapsto h(s(y))], h)\} \quad \text{if } h(s(y)) \in \text{Val} \\
\llbracket [x] := y \rrbracket(s, h) &\triangleq \{(s, h[s(x) \mapsto s(y)])\} \quad \text{if } h(s(x)) \in \text{Val}
\end{aligned}$$

Figure 5.9: Semantics of heap atomic commands, where $\text{avail}_h(l) \triangleq (l \notin \text{dom}(h) \vee h(l) = \delta)$ and we assume that $\llbracket c \rrbracket(s, h) \triangleq \{\mathbf{err}\}$ unless differently stated.

$$\begin{aligned}
\llbracket a_1 \prec a_2 \rrbracket &\triangleq \{(s, h) \mid \llbracket a_1 \rrbracket s \prec \llbracket a_2 \rrbracket s\} & \llbracket \mathbf{false} \rrbracket &\triangleq \emptyset \\
\llbracket \exists x. p \rrbracket &\triangleq \{(s, h) \mid \exists v \in \text{Val}. (s[x \mapsto v], h) \in \llbracket p \rrbracket\} & \llbracket \mathbf{true} \rrbracket &\triangleq \Sigma \\
\llbracket x \nrightarrow \rrbracket &\triangleq \{(s, [s(x) \mapsto \delta])\} & \llbracket p \vee q \rrbracket &\triangleq \llbracket p \rrbracket \cup \llbracket q \rrbracket \\
\llbracket x \mapsto a \rrbracket &\triangleq \{(s, [s(x) \mapsto \llbracket a \rrbracket s])\} & \llbracket p \wedge q \rrbracket &\triangleq \llbracket p \rrbracket \cap \llbracket q \rrbracket \\
\llbracket p * q \rrbracket &\triangleq \{(s, h_p \bullet h_q) \mid (s, h_p) \in \llbracket p \rrbracket, (s, h_q) \in \llbracket q \rrbracket, h_p \perp h_q\} & \llbracket \mathbf{emp} \rrbracket &\triangleq \{(s, [])\}
\end{aligned}$$

Figure 5.10: Semantics of the assertion language, similar to Figure 2.4.

(total) functions $s : \text{Var} \rightarrow \text{Val}$; heaps $h \in \text{Heap}$ are partial functions $h : \text{Loc} \rightarrow \text{Val} \uplus \{\delta\}$. If $h(l) = v \in \text{Val}$, location l is allocated and holds value v , if $l \notin \text{dom}(h)$ then it is not allocated. The special value δ describes a deallocated memory location: if $h(l) = \delta$, that location was previously allocated and then deallocated. As notation, we use $s[x \mapsto v]$ for function update, $[]$ for the empty heap and $[l \mapsto v]$ for the heap defined only on l and associating value v to it. We say two heaps are disjoint, written $h_1 \perp h_2$, when $\text{dom}(h_1) \cap \text{dom}(h_2) = \emptyset$, and in that case we define the $\bullet$ operation as the merge of the two: $h_1 \bullet h_2$ coincides with h_1 on $\text{dom}(h_1)$, with h_2 on $\text{dom}(h_2)$ and it is undefined everywhere else.

Let $\Sigma = \text{Store} \times \text{Heap}$, and $\Sigma_e = \Sigma \uplus \{\mathbf{err}\}$: states $\sigma \in \Sigma_e$ are either a pair store/heap or the error state $\mathbf{err}$. The denotational semantics of atomic commands $\llbracket \cdot \rrbracket : \text{HACmd} \rightarrow \wp(\Sigma_e) \rightarrow \wp(\Sigma_e)$ is in Figure 5.9. To simplify the presentation, we define it as $\llbracket \cdot \rrbracket : \text{HACmd} \rightarrow \Sigma \rightarrow \wp(\Sigma_e)$, we let $\llbracket c \rrbracket \mathbf{err} = \{\mathbf{err}\}$, and we lift it to set of states by union. Please note that evaluation of arithmetic a and Boolean expressions b only depends on the store since they cannot dereference variables. We define the forward collecting semantics of heap commands $\llbracket \cdot \rrbracket : \text{HRCmd} \rightarrow \wp(\Sigma_e) \rightarrow \wp(\Sigma_e)$ just as in Figure 2.1 using the different semantics of atomic commands for $c \in \text{HACmd}$.

The semantics $\llbracket \cdot \rrbracket$ of a formula $p \in \text{Asl}$ is a set of states in Σ , and is defined in Figure 5.10.

We say a Separation SIL triple $\llbracket p \rrbracket \mathbf{r} \llbracket q \rrbracket$ is valid if $\llbracket \llbracket p \rrbracket \rrbracket \llbracket q \rrbracket \supseteq \llbracket p \rrbracket$. To prove soundness of Separation SIL, we rely on a stronger lemma, whose proof is by induction on the derivation tree. Then, by taking $t = \mathbf{emp}$ and using $p * \mathbf{emp} \equiv p$, we get the soundness

of the proof system.

Lemma 5.19. *Let $p, q, t \in \text{Asl}$ and $r \in \text{HRCmd}$. If $\vdash \langle p \rangle r \langle q \rangle$ and $\text{fv}(t) \cap \text{mod}(r) = \emptyset$,*

$$\llbracket \ulcorner \rrbracket \{q * t\} \supseteq \{p * t\}$$

Corollary 5.20 (Separation SIL is sound). *Any provable Separation SIL triple is valid:*

$$\vdash \langle p \rangle r \langle q \rangle \implies \models \langle p \rangle r \langle q \rangle$$

5.4.5 Automating Separation SIL

The main issue with automation of Separation SIL is the need to reconcile a generic assertion with the post of axioms for heap-manipulating atomic commands. To do so, we need two observations.

First, for heap-manipulating atomic commands, all the reachable states share the (de)allocation of a given variable. Consider for instance **free**(x): every state reachable by executing it (i.e., any state in $\langle \text{free}(x) \rangle(\Sigma_e)$) is either **err** (which doesn't satisfy any assertion) or a (s, h) such that $h(s(x)) = \delta$. This means (s, h) satisfies $(x \not\mapsto * \text{true})$. Therefore, (s, h) satisfies a q if and only if it satisfies $q \wedge (x \not\mapsto * \text{true})$. This means that, whenever the postcondition for **free**(x) is q , we can strengthen it via $\langle \text{cons} \rangle$ to $q \wedge (x \not\mapsto * \text{true})$ without losing any reachable state and thus any initial state leading to q . Analogous arguments show that similar equivalences hold for all the other atomic commands. This reasoning is also formalized in the proof of completeness of Separation SIL described in Section 5.4.7.

Second, when the postcondition gets strengthened this way, we rewrite it in the shape $\exists \vec{x}. q * t$, where the axiom can be applied to q and then extended to the whole formula using $\langle \text{frame} \rangle$ and $\langle \text{exists} \rangle$. The first step is to lift existential quantifiers to the top level of the assertion (possibly by renaming bound variables). Then, we use the following rewriting:

Lemma 5.21. *Let $q \in \text{Asl}$ be a formula without $\exists$, and let x' be a fresh variable. Then,*

1. *there exists a $\bar{q}$ such that $q \wedge (x \mapsto x' * \text{true}) \equiv x \mapsto x' * \bar{q}$*
2. *there exists a $\bar{q}$ such that $q \wedge (x \not\mapsto * \text{true}) \equiv x \not\mapsto * \bar{q}$*

Proof sketch. $\bar{q}$ is computed inductively on the structure of q as shown in the following table (the last two columns are for points (1) and (2) of the statement, respectively):

q	$\bar{q} \quad (x \mapsto x')$	$\bar{q} \quad (x \not\mapsto)$
$z \mapsto z'$	$z' = x' \wedge z = x \wedge \text{emp}$	false
$z \not\mapsto$	false	$z = x \wedge \text{emp}$
$q_1 * q_2$	$\bar{q}_1 * q_2 \vee q_1 * \bar{q}_2$	
emp	false	
$q_1 \wedge q_2$	$\bar{q}_1 \wedge \bar{q}_2$	
$q_1 \vee q_2$	$\bar{q}_1 \vee \bar{q}_2$	
false	false	
true	true	
$a_1 \asymp a_2$	$a_1 \asymp a_2$	

The proof that these formulae satisfy the desired equalities are in Appendix B. $\square$

In Example 5.18 this algorithm finds precisely the disjunction $x = y \vee x \not\mapsto$ in the post of **free**(y).

5.4.6 Exit conditions in Separation SIL

We now briefly show how to adapt Separation SIL to handle different exit conditions. For this example, following [OHe20], we will consider **ok** and **er**, denoting correct and erroneous termination respectively. As discussed in Remark 2.9, this correspond to use $\{\text{ok}, \text{er}\} \times \Sigma$ as set of states instead of Σ_e . The denotational semantics of regular commands then acts as described for normal states (returning the error version of the current correct state instead of the generic **err**) and as the identity on error states.

The proof system changes accordingly: all the current rules are still valid with the **ok** flag (for atoms) or a generic flag ϵ (for structural rules) in both pre and postconditions. Rules introducing error flags are added for atoms. As an example, we write below the error rule for store.

$$\frac{}{\langle\langle \text{ok} : x \not\mapsto \rangle\rangle [x] := y \langle\langle \text{er} : x \not\mapsto \rangle\rangle} \text{store-er}$$

Moreover, the proof system is augmented with a rule for error propagation, that correspond to the fact that the semantics of programs on error states is the identity:

$$\frac{}{\langle\langle \text{er} : q \rangle\rangle r \langle\langle \text{er} : q \rangle\rangle} \text{er-id}$$

Note that this rule is enough to short-circuit sequence and iterations in case an error happens. For instance, for sequence, you can prove $\langle\langle \text{ok} : p \rangle\rangle r_1 \langle\langle \text{er} : q \rangle\rangle$ and then use $\langle\langle \text{seq} \rangle\rangle$ together with the triple $\langle\langle \text{er} : q \rangle\rangle r_2 \langle\langle \text{er} : q \rangle\rangle$ proved via $\langle\langle \text{er-id} \rangle\rangle$. Similarly for iteration, you can apply rule $\langle\langle \text{unroll} \rangle\rangle$ and use the same derivation for sequence. However, note that this is not useful during a backward analysis: if the postcondition of an iteration is an **er** state, either you skip the loop entirely with $\langle\langle \text{er-id} \rangle\rangle$, or you try to find an **ok** precondition leading to the error. A derivation such as

$$\frac{\begin{array}{c} \vdots \\ \langle\langle \text{ok} : p \rangle\rangle r^* \langle\langle \text{er} : q \rangle\rangle \end{array} \quad \frac{}{\langle\langle \text{er} : q \rangle\rangle r \langle\langle \text{er} : q \rangle\rangle} \text{er-id}}{\frac{\langle\langle \text{ok} : p \rangle\rangle r^*; r \langle\langle \text{er} : q \rangle\rangle}{\langle\langle \text{ok} : p \rangle\rangle r^* \langle\langle \text{er} : q \rangle\rangle} \text{seq}} \text{unroll}$$

is useless. Similarly, it's useless to apply rule $\langle\langle \text{iter} \rangle\rangle$ with $q(1) = q(0) = \text{er} : q$; instead, lower indexes in $q(n)$ by 1 (and drop $q(0)$).

The modified proof system is sound with respect to the modified semantics:

Theorem 5.22. *Any provable triple is valid:*

$$\vdash \langle\langle \epsilon : p \rangle\rangle r \langle\langle \epsilon' : q \rangle\rangle \implies \{\epsilon : p\} \subseteq \llbracket \langle\langle \epsilon' : q \rangle\rangle \rrbracket$$

Example 5.23. Consider a refinement of the program in Example 5.18: here we assume that **len** and **cap** are two variables associated to the vector v describing its current length and capacity, respectively. We can then use them to decide the behavior of **push_back**: the vector gets reallocated only if the length after the insertion would exceed the current capacity. Moreover, we assume that **x** is used to access the element in position 8 of the vector, and therefore it's use after the **push_back** is guarded by a check that the vector is long enough. Therefore, the code becomes

$$\begin{aligned} \text{rclient2} &\triangleq x := [v]; \text{ if } (\text{len} \geq \text{cap}) \{r_{b2}\} \text{ else } \{\text{len} := \text{len} + 1\}; r_{use} \\ r_{b2} &\triangleq y := [v]; \text{ free}(y); y := \text{alloc}(); [v] := y; \text{ len} := \text{len} + 1; \text{ cap} := \text{cap} * 2 \\ r_{use} &\triangleq \text{ if } (\text{len} > 8) \{x := 10\} \end{aligned}$$

```

 $\langle\langle ok : len \geq cap \wedge len > 7 \wedge (v \mapsto z * z \mapsto - * true) \rangle\rangle$ 
 $x := [v]; \quad \langle\langle ok : len \geq cap \wedge len > 7 \wedge (true * v \mapsto z * z \mapsto - * (x = z \vee x \not\mapsto)) \rangle\rangle$ 
 $(len \geq cap)?; \quad \langle\langle ok : len > 7 \wedge (true * v \mapsto z * z \mapsto - * (x = z \vee x \not\mapsto)) \rangle\rangle$ 
 $y := [v]; \quad //$ 
 $free(y); \quad //$  see Figure 5.8b
 $y := alloc(); \quad //$ 
 $[v] := y; \quad \langle\langle ok : len + 1 > 8 \wedge (x \not\mapsto * true) \rangle\rangle$ 
 $len := len + 1; \quad \langle\langle ok : len > 8 \wedge (x \not\mapsto * true) \rangle\rangle$ 
 $cap := cap * 2; \quad \langle\langle ok : len > 8 \wedge (x \not\mapsto * true) \rangle\rangle$ 
 $(len > 8)?; \quad \langle\langle ok : x \not\mapsto * true \rangle\rangle$ 
 $[x] := 10 \quad \langle\langle er : true \rangle\rangle$ 

```

Figure 5.11: Sketch of the derivation of the triple for `rcient2` in Example 5.23. Postconditions to each statement are written on the right of the statement itself.

To analyse this program, we use the error postcondition $\langle\langle er : true \rangle\rangle$. For space constraints, we only consider the code path that goes through the then-branches of both if statements. The proof, linearized, is in Figure 5.11. The omitted part is analogous to the derivation in Figure 5.8b.

We highlight the use of rule $\langle\langle store\text{-}er \rangle\rangle$ to infer the precondition $ok : x \not\mapsto * true$ from the error postcondition $er : true$ and $\langle\langle assume \rangle\rangle$ to conjoin the boolean conditions in the preconditions of the guards. Moreover, the backward substitution of $len := len + 1$ performed by $\langle\langle assign \rangle\rangle$ naturally propagates backward the constraint $len > 8$ to the value preceding the assignment, obtaining $len + 1 > 8$. This way, in the precondition of the whole command we explicitly find the constraints $len \geq cap \wedge len > 7$ on the initial values of `len` and `cap` that lead to the error. ■

5.4.7 Relative completeness of Separation SIL

The proof system in Section 5.4.3 is complete for all atomic commands except `alloc` because $\langle\langle alloc1 \rangle\rangle$ misses the ability to refer to the specific memory location that was allocated. The naive solution to add a constraint $x = \beta$ in the post of $\langle\langle alloc1 \rangle\rangle$ makes the frame rule unsound: it would allow to prove, for instance, the invalid triple

$$\langle\langle emp * \alpha \mapsto - \rangle\rangle x := alloc() \langle\langle (x \mapsto - \wedge x = \alpha) * \alpha \mapsto - \rangle\rangle.$$

To recover the frame rule, just like ISL needs the deallocated assertion in the post [Raa+20, §3], Separation SIL needs a “will be allocated” assertion in the pre. To this end, we use the $\not\mapsto$ assertion, and change the semantic model to only allocate a memory location that is explicitly δ . Intuitively, this means specifying the resources that will be needed by the program execution: in particular, it lists in the pre the memory locations that will be allocated during program execution. Note that memory locations that will be allocated can be specified as logical variables, so they don’t constrain the actual memory layout, but only the amount of locations needed. We formalize this by letting $avail_h(l) \triangleq (h(l) = \delta)$ in Figure 5.9 and removing rule $\langle\langle alloc1 \rangle\rangle$: the only valid rule for allocation in this semantics is $\langle\langle alloc2 \rangle\rangle$. Moreover, we can prove relative completeness [AO19, §4.3] for loop-free programs:

Theorem 5.24 (Relative completeness for loop-free programs). *Suppose to have an oracle to prove implications between formulae in Asl. Let $r \in \text{HRCmd}$ be a regular command without $*$ and $p, q \in \text{Asl}$ such that $\llbracket \neg r \rrbracket \{q\} \supseteq \{p\}$. Then the triple $\langle\langle p \rangle\rangle r \langle\langle q \rangle\rangle$ is provable.*

The proof follows very closely the process described in Section 5.4.5 to automate Separation SIL backward inference, and in fact it was the inspiration for it. The proof uses this process to compute an assertion t whose semantics is precisely the weakest possible pre $\llbracket \overleftarrow{r} \rrbracket \{q\}$ and prove the triple $\langle t \rangle r \langle q \rangle$. Then, using the oracle and $\langle \text{cons} \rangle$, the theorem follows for any p that implies t . Notably, this theorem shows that the weakest (possible) precondition $\llbracket \overleftarrow{r} \rrbracket \{q\}$ of loop-free programs is always expressible as an assertion $t \in \text{Asl}$, namely $\{t\} = \llbracket \overleftarrow{r} \rrbracket \{q\}$. This result is far from trivial, as it depends on the expressiveness of the assertion language: for instance, if we add negation to the assertion language, the same result does not hold anymore because it introduces some new posts for which the precondition is not in the assertion language.

We have several ways to extend the result to loops. Standard techniques include assuming expressivity of the assertion language [Coo78] (i.e., assuming the existence of formulae describing weakest preconditions), or extending the assertion language with a least fixed-point operator [BG87]—both solutions work for Separation SIL. Another option is to focus on single states. Note that, differently than other techniques, the following result does not need the oracle: it is always possible to craft a p whose proof only needs decidable implications.

Theorem 5.25 (State-wise completeness). *Given a heap program $r \in \text{HRCmd}$ and two states σ, σ' such that $\sigma \in \llbracket \overleftarrow{r} \rrbracket \sigma'$, for every assertion q such that $\sigma' \in \{q\}$ there exists an assertion p such that $\sigma \in \{p\}$ and $\langle p \rangle r \langle q \rangle$ is provable in Separation SIL.*

5.4.8 Implementations

To gain confidence in the mechanizability of our results, we developed a proof-of-concept implementation of Separation SIL in OCaml. The prototype is open source and available on GitHub.² While it is not meant to scale up to real programs, it can derive all the examples in this section and validates our intuition on the automated backward inference of Section 5.4.5. The code exploits a routine that follows closely the process described in that section, which is analogous to the proof of relative completeness (Theorem 5.24). The core algorithms are implemented in ~ 400 locs; another big chunk of the code is the simplification of logical formulae, whose complexity is not tied to Separation SIL algorithm. For this implementation, we thank the students and teachers of the Laboratory for Innovative Software 2024 course, Yuri Andriaccio, Samuele Bonini, Andrea Castagna, Marco Antonio Corallo, Andrea Simone Costa, Fabio Federico, Elvis Rossi, Alessandro Scala, Matteo Simone, Chiara Bodei and Gian-Luigi Ferrari.

Moreover, the ideas that lead to SIL represent a *posteriori* formalization and theoretical justification of the parallel, modular, and compositional static analysis implemented in industrial grade static analyzers for security developed and used at Meta, such as Zoncolan [DFLO19], Mariana Trench [Gab21], and Pysa [BC19]. Those tools automatically find more than 50% of the security bugs in the Meta family of apps and many SEVs (“severe bugs” in the Meta jargon) [DFLO19, Figure 5].

In order to scale up to hundreds of millions of lines of code, static analyses need to be parallelizable and henceforth modular and compositional. Modularity implies that the analysis can infer *meaningful* information without full knowledge of the global program. Compositionality means that the *results* of analyzing modules are good enough that one does not lose information in using the inferred triple instead of inlining the code.

The analysis implemented in the aforementioned tools is a modular backward analysis that determines which input states for a function will lead to a security error, likewise the

²<https://github.com/Alex23087/Failure-SSIL-Analyser>

SIL rules described in Figure 5.3. In particular, the analysis infers sufficient incorrectness preconditions (modularity) for callees that can be used by the callers (compositionality) to generate their incorrectness preconditions. When the propagation of the inferred sufficient precondition reaches an attacker-controlled input, the analysers check if that input is included in the propagated error condition. If it is the case, then it emits an error. The function analyses are parallelized and a strategy similar to the iteration rule of Figure 5.3 is used to compute the fixpoint in presence of mutually recursive functions.

5.4.9 Comparisons

There are other logics that address memory errors with an “incorrectness” perspective. Here we compare Separation SIL to ISL and Outcome Separation Logic (OSL) [ZSS24].

The key difference between Separation SIL and ISL is the same between IL and SIL: ISL is more effective for forward analysis and ensures reachability of all final states; Separation SIL is thought for backward analysis and guarantees that all initial states can reach a final state. For instance, the ISL triple in [Raa+20] for the Example 5.18 is $[v \mapsto z * z \mapsto -] \text{ rclient } [v \mapsto y * y \mapsto - * x \not\mapsto]$, which proves that those faulty states are reachable, but tells nothing about which input states actually lead to them. On the other hand, the Separation SIL triple has a more succinct post capturing the error and exposes faulty initial states, but cannot describe which errors are reachable.

The comparison with OSL is more convoluted. If we consider the separation instance of OL, [ZDS23, Figure 6] proves essentially the same triple as Example 5.18 but the deduction process is quite different. OL reasoning is forward oriented, as witnessed by the implication that concludes the proof and by the triple for the `skip` branch, whereas Separation SIL rules naturally guides the backward inference. OSL is more closely related to Separation SIL because it takes pointer programs at its core. For OSL, the authors propose an abduction-based algorithm that produces similar results to Separation SIL. However, the two algorithms differ in their details and work in different settings. To tackle the issue of *non-locality* of allocation, [ZSS24, §2.2] prescribe that an OSL triple should be valid for *every* possible allocation semantics. This prevents triples from talking too precisely about the result of an allocation, and is available even when the execution model doesn’t include nondeterminism. Thus, OSL frame rule works across different execution models, e.g., for probabilistic programs. On the contrary, Separation SIL model is tailored to nondeterminism and can’t be used for other execution models. However, this allows Separation SIL to be more precise about nondeterminism: differently than OSL, Separation SIL is complete and its triples can talk about a *specific* allocation semantics. Thus Separation SIL can prove some special kinds of triples that are not valid in OSL model.

Example 5.26. C standard allocator reuses previously freed locations, and this can cause security leaks. Consider the code in Fig. 5.12: an attacker sets up a memory address by allocating and then freeing it, so that the next user allocation gets exactly that location ℓ . The user then stores a secret in ℓ , that the attacker can read. Separation SIL witnesses the error: in the post of `y := alloc()` it can be seen that the leaky behaviour is caused by $(x = y)$. In contrast, OSL cannot witness this error: since it occurs only for a specific allocation semantics this triple is not valid. ■

This simple example shows that, for some kind of errors, a more specific semantics is needed, thus the usefulness of different techniques for both more general and more specific settings.

<code>// attacker setup</code>	$\langle\langle \mathbf{true} \rangle\rangle$
<code>$x := \text{alloc}();$</code>	$\langle\langle x \mapsto - \rangle\rangle$
<code>$\text{free}(x);$</code>	$\langle\langle (x \not\mapsto) \vee (x \mapsto \text{secret} * \mathbf{true}) \rangle\rangle$
<code>// user code reading the secret</code>	
<code>$y := \text{alloc}();$</code>	$\langle\langle (y \mapsto - * x = y) \vee (x \mapsto \text{secret} * y \mapsto - * \mathbf{true}) \rangle\rangle$
<code>$[y] := \text{secret};$</code>	$\langle\langle (x \mapsto \text{secret} * x = y) \vee (x \mapsto \text{secret} * y \mapsto \text{secret} * \mathbf{true}) \rangle\rangle$
<code>// attacker code leaking the secret</code>	
<code>$\text{leak} := [x]$</code>	$\langle\langle \text{leak} = \text{secret} * x \mapsto - \rangle\rangle$
	$\langle\langle \text{leak} = \text{secret} \rangle\rangle$

Figure 5.12: Sketch of the derivation for Example 5.26. Postconditions to each statement are written on the right of the statement itself.

5.5 Summary

In this chapter, we considered known program logics for over and under-approximation and tried to formalize the relations among them in order to assess their respective strengths and weaknesses. This led to the introduction of the novel proof system for Lisbon triples, that we dubbed Sufficient Incorrectness Logic, and to compare the four logics along several dimensions. We also instantiated the new SIL proof system to handle pointers and memory allocation. The resulting Separation SIL is able to identify the causes of memory errors. We presented a first, simpler version of Separation SIL for which we provide a prototype implementation. We also showed how to manage explicit errors via ok/er flags, and how to modify the logic to become complete. Comparing the logics, we recovered some known results as well as finding new connections and highlighting insights that shed a little more light on the asymmetries between over/under-approximation and forward/backward analysis. The shortest summary of our findings is that there is no silver bullet: each logic has its own strengths and use cases. Moreover, the comparison pointed out an analogy between IL and SIL that will help us seamlessly transfer the techniques of LCL_A to backward analysis in the next Chapter.

Chapter 6

Local Completeness Logic

In this chapter, we extend LCL_A (see Section 3.2) with new capabilities. First, we investigate the possibility of relaxing point (3) of Theorem 3.7 to $\llbracket r \rrbracket^A \alpha(P) = \alpha(Q)$ to achieve extensional soundness, i.e., to untie the set of properties that can be proved about the function computed by the program from the way the program is written. To do so, we follow the idea introduced in [BGGR23, §8] of *changing the abstract domain* during the analysis, possibly in different ways for different portions of the code. While [BGGR23] proposes a single rule for domain refinement, we study here both *refinement* and *simplification* rules for LCL_A . Second, we study here how to weaken the hypothesis of Galois connection: the whole theory of completeness is based on the existence of a best approximation for concrete points, but this is not always available in practical instances [CC92]. Third, we exploit the IL/SIL analogy observed in the previous chapter to study further the possibility of using LCL_A for backward analysis (which was first hinted in [BGGR23, §5.3], although without SIL).

The content of Section 6.3 is based on [ABG23]. Sections 6.4 and 6.5 are included in an extended version of [ABG23] submitted to a journal. The content of Section 6.6 is unpublished work.

6.1 Motivation

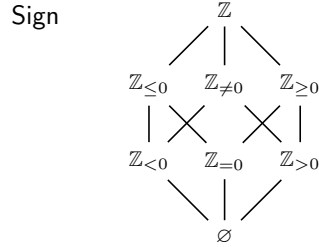
While any LCL_A valid triple allows to prove both correctness and incorrectness, a very powerful ability, proving an LCL_A triple can be challenging. Therefore, it is important to study how to simplify this task.

The strongest of the three properties required by LCL_A is point (3) in Theorem 3.7, which in turn is key to guarantee that point (2) holds. However, requiring the abstract interpreter $\llbracket c \rrbracket_A^\sharp$ to be locally complete is an *intensional* property, because the function $\llbracket c \rrbracket_A^\sharp$ depends on how c is written. In fact, it is well known that the abstract analyses $\llbracket c_1 \rrbracket_A^\sharp$ and $\llbracket c_2 \rrbracket_A^\sharp$ of two programs c_1 and c_2 computing the same function (i.e., $\llbracket c_1 \rrbracket = \llbracket c_2 \rrbracket$) can yield different results.

A weaker requirement that suffices to guarantee the validity of point (2) is the local completeness of the best correct approximation (bca) $\llbracket c \rrbracket^A$ of the function $\llbracket c \rrbracket$, which is an *extensional* property: it only depends on the abstract domain A and on the computed function $\llbracket c \rrbracket$ associated with c , not on the way c is composed. In its original formulation, LCL_A exploits $\llbracket c \rrbracket_A^\sharp$ instead of $\llbracket c \rrbracket^A$ because the second can be as hard to compute as the concrete semantics $\llbracket c \rrbracket$ and because the sequential composition of bcas is not necessarily a bca itself. The difference between $\llbracket c \rrbracket^A$ and $\llbracket c \rrbracket_A^\sharp$ is exemplified below (see also, e.g., [LL09,

Example 1]):

Example 6.1 (Extensional and intensional properties). Consider the concrete domain $\mathcal{P}(\mathbb{Z})$ of sets of integers and the abstract domain of signs given below:



The meaning of each abstract elements of **Sign** is to represent concrete values that satisfy the respective property: for instance, $\gamma(\mathbb{Z}_{<0}) = \{n \in \mathbb{Z} \mid n < 0\}$ and $\alpha(\{0; 1; 100\}) = \mathbb{Z}_{\geq 0}$. The bca of a concrete function $f : \mathcal{P}(\mathbb{Z}) \rightarrow \mathcal{P}(\mathbb{Z})$ is defined as $f^{\text{Sign}} \triangleq \alpha \circ f \circ \gamma : \text{Sign} \rightarrow \text{Sign}$.

Consider the simple program fragment

$$c \triangleq x := x + 1; x := x - 1.$$

It's denotational semantics $\llbracket c \rrbracket$ is the identity $\text{id}_{\mathbb{Z}}$, so its bca (see Definition 2.19)

$$\llbracket c \rrbracket^{\text{Sign}} \triangleq \alpha \circ \text{id}_{\mathbb{Z}} \circ \gamma = \text{id}_{\text{Sign}}$$

is just the abstract identity. We say that $\llbracket c \rrbracket^{\text{Sign}}$ is *extensional* because it only depends on the function computed by c , i.e., its denotational semantics. However, an analyser does not know the semantics of c , so it has to analyse the program syntactically, breaking it down into elementary pieces and gluing the results together. For instance, starting from the concrete point $P = \{1\}$, the analysis first abstracts it to the property $\alpha(P) = \mathbb{Z}_{>0}$, then it computes

$$\begin{aligned} \llbracket c \rrbracket_{\text{Sign}}^{\#}(\mathbb{Z}_{>0}) &= \llbracket x := x - 1 \rrbracket_{\text{Sign}}^{\#} \llbracket x := x + 1 \rrbracket_{\text{Sign}}^{\#}(\mathbb{Z}_{>0}) \\ &= \llbracket x := x - 1 \rrbracket_{\text{Sign}}^{\#}(\mathbb{Z}_{>0}) = \mathbb{Z}_{\geq 0}. \end{aligned}$$

Analogous calculations for all properties in **Sign** yields the abstraction

$$\llbracket c \rrbracket_{\text{Sign}}^{\#}(a) = \begin{cases} \emptyset & \text{if } a = \emptyset \\ \mathbb{Z}_{\geq 0} & \text{if } a \in \{\mathbb{Z}_{=0}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}\} \\ \mathbb{Z}_{< 0} & \text{if } a = \mathbb{Z}_{< 0} \\ \mathbb{Z} & \text{if } a \in \{\mathbb{Z}_{\leq 0}, \mathbb{Z}_{\neq 0}, \mathbb{Z}\} \end{cases}$$

that, albeit sound, is less precise than id_{Sign} (we highlight with a gray background all inputs on which $\llbracket c \rrbracket_{\text{Sign}}^{\#}$ loses accuracy).

The program c is equivalent to the command **skip**,¹ because $\llbracket \text{skip} \rrbracket = \text{id}_{\mathbb{Z}}$, and thus c and **skip** are assigned the same bca $\llbracket \text{skip} \rrbracket^{\text{Sign}} = \llbracket c \rrbracket^{\text{Sign}} = \text{id}_{\text{Sign}}$. However, $\llbracket \text{skip} \rrbracket_{\text{Sign}}^{\#} = \text{id}_{\text{Sign}} \neq \llbracket c \rrbracket_{\text{Sign}}^{\#}$, exposing the *intensional* essence of the abstract interpreter: the abstraction depends on how the program is written and not only on its semantics.² ■

¹In this simplified settings with unbound integers. This is not true for machine integers with overflow.

²While it falls outside the scope of this thesis, we refer the interested reader to, e.g., [Bru+19; BRZ22] for more about intensional and extensional abstract properties.

The possibility of weakening point (3) from being an intensional requirement based on $\llbracket c \rrbracket_A^\sharp$ to an extensional one based on $\llbracket c \rrbracket^A$ has several nice consequences. First, the local completeness of $\llbracket c \rrbracket^A$ is enough to guarantee that points (1-2) hold. Second, while the proof system itself provides an intensional analysis, because its rules are defined inductively on the program syntax, the information it produces is more precise, in the sense that the property associated with triples is extensional: no precision is lost because of the approximation introduced by the intensional abstract interpreter. Third, it allows the proof system to derive more triples than the original one because a bca can be locally complete even when the abstract interpreter is not (but not vice versa). Finally, since extensional properties are independent of how the program is written, this possibility provides the potential for deriving exactly the same triples for equivalent programs.

Another constraint imposed by LCL_A is the need for a Galois connection: the whole theory of completeness in abstract interpretation is based on it. Therefore, LCL_A cannot be applied to known instances of abstract domains lacking an abstraction function α , such as convex polyhedra [CH78] that are widely used in static analysis. However, abstract convexity of local completeness can help mitigate this limitation: even if a point doesn't have a best abstraction, if it can be bound between another point and its abstraction, we can in a sense “prove” local completeness on it thanks to abstract convexity.

Lastly, LCL_A was proposed for forward analysis. In theory, nothing prevents it to be used in a backward fashion, but the classical forward/backward duality requires the use of under-approximation abstract domains, making it impractical (see Chapter 4). However, if we consider the backward semantics $\llbracket \cdot \rrbracket^\leftarrow$ defined in the previous chapter, it turns out that it is possible to combine SIL (Section 5.2) with over-approximation abstract domain in a LCL_A -style.

6.2 Extensional soundness

As anticipated at the beginning of the chapter, one of our goals is to weaken point (3) of the soundness Theorem 3.7 from local completeness of the abstract interpreter $\llbracket r \rrbracket_A^\sharp$ to that of the bca $\llbracket r \rrbracket^A$. The key observation is that the proof of Corollary 3.8 only relies on points (1-2) of Theorem 3.7, so from a program analysis perspective point (3) can be seen as a technical requirement to prove the other two. However, we observe that local completeness of the bca is enough to this aim: we therefore present a slightly weaker³ soundness result for LCL_A , with the bca $\llbracket r \rrbracket^A$ in place of the inductively defined abstract interpreter $\llbracket r \rrbracket_A^\sharp$.

Theorem 6.2 (Extensional soundness). *Let $A_{\alpha,\gamma} \in \text{Abs}(C)$. If $\vdash_A [P] \text{ r } [Q]$ then:*

1. $Q \leq \llbracket r \rrbracket P$,
2. $\alpha(\llbracket r \rrbracket P) = \alpha(Q)$,
3. $\llbracket r \rrbracket^A \alpha(P) = \alpha(Q)$.

Proof. First we remark that points (1) and (3) implies point (2):

$$\begin{aligned}
 \alpha(Q) &\leq \alpha(\llbracket r \rrbracket P) && \text{[(1) and monotonicity of } \alpha\text{]} \\
 &\leq \llbracket r \rrbracket^A \alpha(P) && \text{[soundness of } \llbracket r \rrbracket^A\text{]} \\
 &= \alpha(Q) && \text{[(3)]}
 \end{aligned}$$

³Logically speaking, we prove a stronger conclusion, so the theorem as an implication is weaker.

$$\frac{\vdash_{A'} [P] \text{ r } [Q] \quad A' \preceq A \quad A[\llbracket r \rrbracket^{A'}] A(P) = A(Q)}{\vdash_A [P] \text{ r } [Q]} \text{ (refine-ext)}$$

Figure 6.1: Rule (refine-ext) for LCL_A .

So all the lines are equal, in particular $\alpha(Q) = \alpha(\llbracket r \rrbracket P)$. The proof is then by induction on the derivation tree of $\vdash_A [P] \text{ r } [Q]$, but we only have to prove (1) and (3) because of the observation above. We only include one inductive case as an example; the full proof is in Appendix C.

Case (seq)

(1) $Q \leq \llbracket r_2 \rrbracket R \leq \llbracket r_2 \rrbracket (\llbracket r_1 \rrbracket P) = \llbracket r_1; r_2 \rrbracket P$, where the inequalities follow from inductive hypotheses and monotonicity of $\llbracket r_2 \rrbracket$.

(3) We recall that $\llbracket r_1; r_2 \rrbracket^A \leq \llbracket r_2 \rrbracket^A \llbracket r_1 \rrbracket^A$.

$$\begin{aligned} \alpha(Q) &\leq \alpha(\llbracket r_1; r_2 \rrbracket P) && [(1) \text{ and monotonicity of } \alpha] \\ &\leq \llbracket r_1; r_2 \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \\ &\leq \llbracket r_2 \rrbracket^A \llbracket r_1 \rrbracket^A \alpha(P) && [\text{recalled above}] \\ &= \llbracket r_2 \rrbracket^A \alpha(R) && [\text{inductive hp}] \\ &= \alpha(Q) && [\text{inductive hp}] \end{aligned}$$

So all the lines are equal, in particular $\llbracket r_1; r_2 \rrbracket^A \alpha(P) = \alpha(Q)$. $\square$

Theorem 3.7 involves $\llbracket r \rrbracket_A^\sharp$, an *intensional* property of the program r that depends on how the program is written, while the new statement we propose here relies only on $\llbracket r \rrbracket^A$, an *extensional* property of the computed function $\llbracket r \rrbracket$ and not of r itself. Accordingly, for the rest of this chapter we use the name *intensional soundness* for the former and *extensional soundness* for the latter. Again, we say a triple is *extensionally valid* if it satisfies point (1–3) of Theorem 6.2 above, and *intensionally valid* for the former notion introduced in Section 3.2. We shall write $\models_A [P] \text{ r } [Q]$ for both, but we will make sure to disambiguate the notation when not clear from the context.

6.3 Locally complete refinement

Our aim is to enhance the original LCL_A proof system to handle triples where the extensional abstraction $\llbracket r \rrbracket^A$ is proved to be locally complete w.r.t. the given input, that is $\llbracket r \rrbracket^A \alpha(P) = \alpha(\llbracket r \rrbracket P)$. To achieve this, we extend the proof system with a new inference rule, that is shown in Figure 6.1. It is named after “refine” because it allows to refine the abstract domain A to some $A' \preceq A$ (see Section 2.9) and “ext” since it involves the extensional bca $\llbracket r \rrbracket^{A'}$ of $\llbracket r \rrbracket$ in A' (to distinguish it from the rules we will introduce later).

Using (refine-ext) it is possible to construct a derivation that proves local completeness of portions of the whole program in a more precise abstract domain A' and then carries the result over to the global analysis in a coarser domain A . The only requirement for the application of the rule is that the domain A' satisfies $A[\llbracket r \rrbracket^{A'}] A(P) = A(Q)$.

Formally, given the two abstract domains $A_{\alpha, \gamma}, A'_{\alpha', \gamma'} \in \text{Abs}(C)$, this last premise of rule (refine-ext) should be written as $\alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) = \alpha(Q)$. However we prefer the more concise, albeit a little imprecise, notation in Figure 6.1. That notation is justified by the

following intuitive argument: since $A' \preceq A$ we can consider, with a slight abuse of notation (seeing abstract domains as closures) $A \subseteq A' \subseteq C$, so that for any element $a \in A \subseteq C$ we have $\gamma(a) = \gamma'(a) = a$ and for any $c \in C$ we have $\alpha' A(c) = A(c)$. With these

$$\alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) = \alpha \llbracket r \rrbracket^{A'} A(P) = A \llbracket r \rrbracket^{A'} A(P).$$

With the addition of rule (refine-ext), intensional soundness (Theorem 3.7) does not hold anymore: since this rule allows to perform part of the analysis in a more concrete domain A' , we do not get any information on $\llbracket r \rrbracket_A^\sharp$. However, rule (refine-ext) is sound w.r.t. the bca $\llbracket r \rrbracket^A$, and therefore it makes the proof system extensionally sound:

Theorem 6.3 (Extensional soundness of (refine-ext)). *The proof system in Figure 3.2 with the addition of rule (refine-ext) from Figure 6.1 is extensionally sound.*

Proof sketch. We extend the proof of Theorem 6.2 with a new inductive case. The full details are in Appendix C. $\square$

We remark that a rule like (refine-ext), that allows to carry on part of the proof in a different abstract domain, cannot be unconstrained. We present an example showing that an analogous inference rule only requiring the triple $\vdash [P] \text{ r } [Q]$ to be provable in an abstract domain $A' \preceq A$ without any further constraint would be unsound.

Example 6.4. Consider the concrete domain $C = \mathcal{P}(\mathbb{Z})$ of integers, the point $P = \{-5; -1\}$, the abstract domain **Sign** of Example 6.1 and the program

$$r \triangleq x := x + 10.$$

Then $C \preceq \text{Sign}$ and we can prove $\vdash_C [P] \text{ r } [\{5; 9\}]$ applying (transfer) since all functions are locally complete in the concrete domain. However, if $f = \llbracket r \rrbracket = \langle x := x + 10 \rangle$, it is not the case that $\mathbb{C}_P^{\text{Sign}}(f)$: indeed

$$\text{Sign}(f(\text{Sign}(P))) = \text{Sign}(f(\mathbb{Z}_{<0})) = \text{Sign}(\{n \in \mathbb{Z} \mid n < 10\}) = \top$$

while

$$\text{Sign}(f(P)) = \text{Sign}(\{5, 9\}) = \mathbb{Z}_{>0}.$$

This means that a refinement rule without any additional condition is unsound because it would allow to prove triples which are not locally complete. $\blacksquare$

6.3.1 Logical completeness

Among all the possible conditions that make a refinement rule valid, we believe ours to be very general since (refine-ext) allows us to derive logical completeness, that is, the ability to prove *any* triple satisfying the soundness properties guaranteed by the proof system. Note that this was not the case for the original LCL_A proof system [BGGR21, §5.2].

However, to prove such a result, our extension need an additional rule to handle loops, just like the original LCL_A and other logics (IL, SIL). The necessary infinitary rule, called (limit), allows the proof system to handle Kleene star, and is the same as LCL_A :

$$\frac{\forall n \geq 0 \mid \vdash_A [P_n] \text{ r } [P_{n+1}]}{\vdash_A [P_0] \text{ r }^* [\bigvee_{i \geq 0} P_i]} \text{ (limit)}$$

Theorem 6.5 (Logical completeness of (refine-ext)). *Consider the proof system of Figure 3.2 with the addition of rules (refine-ext) and (limit). If $Q \leq \llbracket r \rrbracket P$ and $\llbracket r \rrbracket^A \alpha(P) = \alpha(Q)$ then $\vdash_A [P] \text{ r } [Q]$.*

Proof. First, the hypotheses of the theorem implies $\mathbb{C}_P^A(\llbracket r \rrbracket)$:

$$\begin{aligned} \llbracket r \rrbracket^A \alpha(P) &= \alpha(Q) && [\text{hp of the theorem}] \\ &\leq \alpha(\llbracket r \rrbracket P) && [\text{monotonicity of } \alpha \text{ and hp } Q \leq \llbracket r \rrbracket P] \\ &\leq \llbracket r \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \end{aligned}$$

Hence $\alpha(\llbracket r \rrbracket P) = \llbracket r \rrbracket^A \alpha(P) = \alpha(\llbracket r \rrbracket \gamma \alpha(P))$, that is local completeness, and $\alpha(Q) = \alpha(\llbracket r \rrbracket P)$.

Now consider r, P, Q satisfying the hypotheses. If $Q < \llbracket r \rrbracket P$, by (relax) we get

$$\frac{P \leq P \leq A(P) \quad \vdash_A [P] \text{ r } [\llbracket r \rrbracket P] \quad Q \leq \llbracket r \rrbracket P \leq A(Q)}{\vdash_A [P] \text{ r } [Q]} \text{ (relax)}$$

But the first condition is trivial, and the third one is made of $Q \leq \llbracket r \rrbracket P$ (the hypothesis) and $\llbracket r \rrbracket P \leq A(Q)$, that follows because $\alpha(\llbracket r \rrbracket P) = \alpha(Q)$ (shown above) and in a Galois connection this implies $\llbracket r \rrbracket P \leq \gamma \alpha(Q) = A(Q)$. Hence, without loss of generality, we can assume $Q = \llbracket r \rrbracket P$.

Now we want to apply (refine-ext) to move to the concrete domain C . Clearly $C \preceq A$. The last hypothesis of the rule can be readily verified recalling that $\llbracket r \rrbracket^C = \llbracket r \rrbracket$ and $\alpha' = \gamma' = \text{id}_C$:

$$\begin{aligned} \alpha \llbracket r \rrbracket^C A(P) &= \alpha \llbracket r \rrbracket A(P) \\ &= \llbracket r \rrbracket^A \alpha(P) \\ &= \alpha(\llbracket r \rrbracket P) \end{aligned}$$

To say that triple $\vdash_C [P] \text{ r } [\llbracket r \rrbracket P]$ is provable we resort to Theorem 5.11 of [BGGR21]. The hypothesis of that theorem are satisfied: all expressions are globally complete in the concrete domain C , $\llbracket r \rrbracket P \leq \llbracket r \rrbracket P$ and $\llbracket r \rrbracket_C^\# \text{id}_C(P) = \llbracket r \rrbracket P = \text{id}_C(\llbracket r \rrbracket P)$, where we used $\alpha' = \text{id}_C$ and $\llbracket r \rrbracket_C^\# = \llbracket r \rrbracket$.

Thus, by applying (refine-ext), we can prove the triple $\vdash_A [P] \text{ r } [\llbracket r \rrbracket P]$:

$$\frac{\vdash_C [P] \text{ r } [\llbracket r \rrbracket P] \quad C \preceq A \quad A \llbracket r \rrbracket^C A(P) = A(\llbracket r \rrbracket P)}{\vdash_A [P] \text{ r } [\llbracket r \rrbracket P]} \text{ (refine-ext)}$$

□

The previous theorem proves the logical completeness of our proof system with respect to extensional validity: indeed, if $Q \leq \llbracket r \rrbracket P$ and $\llbracket r \rrbracket^A \alpha(P) = \alpha(Q)$ we also have $\alpha(\llbracket r \rrbracket P) = \alpha(Q)$ (see, e.g., the proof of Theorem 6.2).

An interesting consequence of this result is the existence of a refinement A' in which it is possible to carry out the proof. In principle, such a refinement could be the concrete domain C (as shown in the proof), that is not computable. However, it is worth nothing that for a sequential fragment (a portion of code without loops) the concrete domain can be actually used (for instance via first-order logic). This opens up the possibility, for instance, to infer a loop invariant on the body using C , and then prove it using an abstract domain. This is reminiscent of the approach in [CL96], where the authors use a (close to) concrete trace-based analysis on loop bodies to infer abstract invariants. In Section 6.3.3 we discuss this issue further.

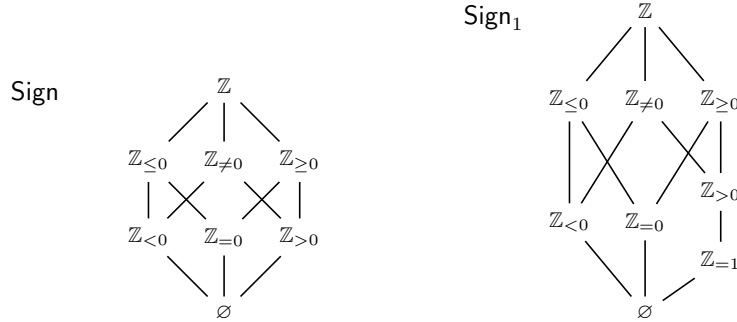
$$\boxed{
\begin{array}{c}
\frac{\vdash_{A'} [P] \text{ r } [Q] \quad A' \preceq A \quad A[[r]]_{A'}^\# A(P) = A(Q)}{\vdash_A [P] \text{ r } [Q]} \text{ (refine-int)} \\
\\
\frac{\vdash_{A'} [P] \text{ r } [Q] \quad A' \preceq A \quad A'(P) = A(P)}{\vdash_A [P] \text{ r } [Q]} \text{ (refine-pre)}
\end{array}
}$$

Figure 6.2: Derived refinement rules for LCL_A .

6.3.2 Derived refinement rules

The hypothesis $A[[r]]^{A'} A(P) = A(Q)$ is added to rule (refine-ext) in order to guarantee soundness: in general, the ability to prove a triple such as $\vdash [P] \text{ r } [Q]$ in a refined domain A' only gives information on $A[[r]]^{A'} A'(P)$ but not on $A[[r]]^{A'} A(P)$. In fact, the example below shows that $A[[r]]^{A'} A'(P)$ and $A[[r]]^{A'} A(P)$ can be different.

Example 6.6. Consider the concrete domain $\mathcal{P}(\mathbb{Z})$, the abstract domain of signs $\text{Sign}_{\alpha,\gamma} \in \text{Abs}(\mathcal{P}(\mathbb{Z}))$ (introduced in Example 6.1) and its refinement Sign_1 below:



For the command $r \triangleq x := x - 1$ and the concrete point $P = \{1\}$ we have

$$\text{Sign}[[r]]^{\text{Sign}_1} \text{Sign}_1(P) = \text{Sign}[[r]]^{\text{Sign}_1}(\mathbb{Z}_{=1}) = \mathbb{Z}_{=0}$$

but

$$\text{Sign}[[r]]^{\text{Sign}_1} \text{Sign}(P) = \text{Sign}[[r]]^{\text{Sign}_1}(\mathbb{Z}_{>0}) = \mathbb{Z}_{\geq 0}.$$

■

Despite being necessary, the hypothesis of rule (refine-ext) cannot be checked algorithmically because, in general, the bca $[[r]]^{A'}$ of a composite command r is not computable. To mitigate this issue, we present in Figure 6.2 two derived rules whose premises imply the premises of (refine-ext), thus ensuring extensional soundness via Theorem 6.3.

The first rule we present replaces the requirement on the extensional bca $[[r]]^{A'}$ with requirements on the intensional compositional abstraction $[[r]]_{A'}^\#$ computed in A' . For this reason, we call this rule (refine-int).

The second derived rule we propose is simpler than (refine-ext): it just requires the abstractions $A(P)$ and $A'(P)$ to coincide, with no reference to the regular command r nor to the postcondition Q . Since the premise is only on the precondition P , we call this rule (refine-pre).

Proposition 6.7. *Both rules (refine-int) and (refine-pre) in Figure 6.2 are extensionally sound.*

Proof sketch. We show that the hypotheses of both rules imply those of (refine-ext). Since the first two hypotheses $\vdash_{A'} [P] \text{ r } [Q]$ and $A' \preceq A$ are shared among the rules, we only have to show that $\alpha\gamma' \llbracket \text{r} \rrbracket^{A'} \alpha'(P) = \alpha(Q)$. The details are in Appendix C.

Because the hypotheses of (refine-int) and (refine-pre) implies those of (refine-ext), whenever we can apply the former we can also apply the latter, so that Theorem 6.3 ensures extensional soundness. $\square$

It is worth noting that the condition in (refine-int) on the compositional abstraction $\llbracket \text{r} \rrbracket_{A'}^\#$ can easily be checked by the analyser, possibly alongside the analysis of r with LCL or using a stand-alone abstract interpreter. Moreover, this rule is as powerful as the original (refine-ext) because it enjoys a logical completeness result akin to Theorem 6.5.

Theorem 6.8 (Logical completeness of (refine-int)). *Consider the proof system of Figure 3.2 with the addition of rules (refine-int) and (limit). If $Q \leq \llbracket \text{r} \rrbracket P$ and $\llbracket \text{r} \rrbracket^A \alpha(P) = \alpha(Q)$ then $\vdash_A [P] \text{ r } [Q]$.*

Proof. The proof is the same as Theorem 6.5, the only difference being that to apply (refine-int) we need to show $A \llbracket \text{r} \rrbracket_C^\# A(P) = A(\llbracket \text{r} \rrbracket P)$ instead of $A \llbracket \text{r} \rrbracket^C A(P) = A(\llbracket \text{r} \rrbracket P)$. However, since in the concrete domain $\llbracket \text{r} \rrbracket_C^\# = \llbracket \text{r} \rrbracket^C = \llbracket \text{r} \rrbracket$ the proof still holds. $\square$

Just like logical completeness of (refine-ext), this result implies the existence of a refinement A' (possibly the concrete domain) in which it is possible to carry out the proof.

Rule (refine-pre) only requires a simple check at the application site instead of an expensive analysis of the program r , so it can be preferable in practice. We present an example to highlight the advantages of this rule (as well as (refine-int)), which allows us to use different domains in the proof derivation of different parts of the program. In the following example(s), we pick the refinement by hand just to show how the rules can be useful. We briefly discuss possible directions on automating the refinement choice in Section 6.3.3, but this is still a largely unexplored question.

Example 6.9 (The use of (refine-pre)). Consider the two program fragments

$$\begin{aligned} r_1 &\triangleq (\text{y} \neq 0)?; \text{y} := \text{abs}(\text{y}) \\ r_2 &\triangleq \text{x} := \text{y}; \text{while } (\text{x} > 1) \{ \text{y} := \text{y} - 1; \text{x} := \text{x} - 1 \} \\ &\quad = \text{x} := \text{y}; ((\text{x} > 1)?; \text{y} := \text{y} - 1; \text{x} := \text{x} - 1)^*; (\text{x} \leq 1)? \end{aligned}$$

and the program $\text{r} \triangleq r_1; r_2$. Here **abs** is a function to compute the absolute value, and we assume, for the sake of simplicity, that the analyser knows its best abstraction. Consider the concrete domain $\mathcal{P}(\mathbb{Z}^2)$ where a pair (n, m) denote a state $\text{x} = n$, $\text{y} = m$, and the initial state $P = (\text{y} \in [-100; 100])$, a logical description of the concrete points $\{(n, m) \mid m \in [-100; 100]\} \in \mathcal{P}(\mathbb{Z}^2)$. The bca $\llbracket \text{r} \rrbracket^{\text{Int}}$ in the abstract domain of intervals is locally complete on P (since P is expressible in Int), but the compositional abstraction $\llbracket \text{r} \rrbracket_{\text{Int}}^\#$ is not:

$$\begin{aligned} \llbracket \text{r} \rrbracket^{\text{Int}} \alpha(P) &= \text{Int}(\llbracket r_2 \rrbracket \llbracket r_1 \rrbracket (\{(n, m) \mid m \in [-100; 100]\})) \\ &= \text{Int}(\llbracket r_2 \rrbracket (\{(n, m) \mid m \in [1; 100]\})) \\ &= \text{Int}(\{(1, 1)\}) \\ &= ([1; 1] \times [1; 1]), \end{aligned}$$

while

$$\begin{aligned}
\llbracket r \rrbracket_{\text{Int}}^\# \alpha(P) &= \llbracket r_2 \rrbracket_{\text{Int}}^\# \llbracket r_1 \rrbracket_{\text{Int}}^\# ([-\infty; +\infty] \times [-100; 100]) \\
&= \llbracket r_2 \rrbracket_{\text{Int}}^\# \llbracket y := \text{abs}(y) \rrbracket_{\text{Int}}^{\text{Int}} ([-\infty; +\infty] \times [-100; 100]) \\
&= \llbracket r_2 \rrbracket_{\text{Int}}^\# ([-\infty; +\infty] \times [0; 100]) \\
&= ([1; 1] \times [0; 100]) \neq ([1; 1] \times [1; 1]).
\end{aligned}$$

The issues are twofold. First, the analysis of r_1 in Int is incomplete, so we need a more concrete domain. For instance, we can select $\text{Int}_{\neq 0}$, the Moore closure of Int with the addition of the element $\mathbb{Z}_{\neq 0}$ representing the property of being nonzero. Intuitively, $\text{Int}_{\neq 0}$ contains all intervals, possibly having a “hole” in 0. Formally

$$\text{Int}_{\neq 0} = \text{Int} \cup \{I_{\neq 0} \mid I \in \text{Int}\}$$

with $\gamma'(I_{\neq 0}) = \gamma(I) \setminus \{0\}$.

While there is no need for a relational domain to analyse r_1 since variable x is never mentioned in it, the analysis of r_2 requires a relational domain to track the information that the value of variable x is equal to the value of variable y . This suggests to use the octagons domain Oct [Min06] to analyse r_2 . It is worth noting that the abstract domain Oct would not be able to perform a locally complete analysis of r_1 for the same reasons that the domain Int could not.

However, rule (**refine-pre**) allows us to combine these different proof derivations. Since the program state between r_1 and r_2 can be precisely represented in Int , we use this domain as a baseline and refine it to $\text{Int}_{\neq 0}$ and to Oct for the two parts, respectively.

Let $R = (y \in \{1; 2; 100\})$ that is an under-approximation of the concrete state in between r_1 and r_2 with the same abstraction in Int , so the triple $\vdash_{\text{Int}} [P] r_1 [R]$ is valid. Note that the concrete point 2 was added to R in order to have local completeness for $(x > 1)?$ in r_2 . However, this triple cannot be proved in Int because $\llbracket r_1 \rrbracket_{\text{Int}}^\#$ is not locally complete on P , so we resort to (**refine-pre**) to change the domain to $\text{Int}_{\neq 0}$. In the derivation below, we let $R_1 = (y \in [-100; 100] \wedge y \neq 0)$ and we omit for simplicity the additional hypothesis of (**relax**):

$$\frac{\frac{\mathbb{C}_P^{\text{Int}_{\neq 0}}(\llbracket y \neq 0? \rrbracket)}{\vdash_{\text{Int}_{\neq 0}} [P] y \neq 0? [R_1]} \text{ (transfer)} \quad \frac{\frac{\mathbb{C}_{R_1}^{\text{Int}_{\neq 0}}(\llbracket y := \text{abs}(y) \rrbracket)}{\vdash_{\text{Int}_{\neq 0}} [R_1] y := \text{abs}(y) [y \in [1; 100]]} \text{ (transfer)}}{\vdash_{\text{Int}_{\neq 0}} [R_1] y := \text{abs}(y) [R]} \text{ (relax)}}{\vdash_{\text{Int}_{\neq 0}} [P] r_1 [R]} \text{ (seq)}$$

Again $\llbracket r_2 \rrbracket$ is locally complete on R in Int , but the compositional analysis $\llbracket r_2 \rrbracket_{\text{Int}}^\#$ is not. Hence to perform the derivation we resort to (**refine-pre**) to introduce relational information in the abstract domain, using Oct instead of Int . Let $Q = (x = 1 \wedge y = 1)$, that is the concrete output of the program, so that we can prove $\vdash_{\text{Int}} [R] r_2 [Q]$. The derivation of this triple is in Appendix C, Figure C.1. However, the proof is just a straightforward application of rules (**seq**), (**iterate**) and (**transfer**).

With those two derivation, we can prove the triple $\vdash_{\text{Int}} [P] r [Q]$ using (**refine-pre**):

$$\frac{\frac{\vdash_{\text{Int}_{\neq 0}} [P] r_1 [R]}{\vdash_{\text{Int}} [P] r_1 [R]} \text{ (refine-pre)} \quad \frac{\frac{\vdash_{\text{Oct}} [R] r_2 [Q]}{\vdash_{\text{Int}} [R] r_2 [Q]} \text{ (refine-pre)}}{\vdash_{\text{Int}} [P] r [Q]} \text{ (seq)}$$

For space constraints, we write here the additional hypotheses of the rules. For the first application, $\text{Int}_{\neq 0} \preceq \text{Int}$ and $\text{Int}_{\neq 0}(P) = P = \text{Int}(P)$. For the second, $\text{Oct} \preceq \text{Int}$ and $\text{Int}(R) = (\mathbf{y} \in [1; 100]) = \text{Oct}(R)$.

It is worth noting that, in this example, all applications of **(refine-pre)** can be replaced by **(refine-int)**. This means that also the latter is able to exploit $\text{Int}_{\neq 0}$ and Oct to prove the triple in the very same way, but its application requires more expensive abstract analyses than the simple checks of **(refine-pre)**. ■

While **(refine-pre)** is simpler than **(refine-ext)** and **(refine-int)**, it is also weaker in both a theoretical and practical sense. On the one hand, LCL_A extended with this rule does not admit a logical completeness result; on the other hand, there are situations in which, even though **(refine-pre)** allows a derivation, **(refine-int)** is more effective. We show these two points by examples. For the first, we propose a valid triple that LCL_A extended with **(refine-pre)** cannot prove.

Example 6.10 (Logical incompleteness of **(refine-pre)**). Consider the program fragments⁴

$$\begin{aligned} r_1 &\triangleq \mathbf{x} := \mathbf{x} + 2 \\ r_w &\triangleq \mathbf{while} \ (\mathbf{true}) \ \{ \ \mathbf{skip} \ \} \\ r &\triangleq r_1; \ r_w \end{aligned}$$

the concrete domain $\mathcal{P}(\mathbb{Z})$, the abstract domains $\text{Int}_{\neq 0}$ (see Example 6.9) and the initial state $P = \{-4, 0\}$. Then $\models_{\text{Int}_{\neq 0}} [P] \ r \ [\emptyset]$ but this triple cannot be proved in LCL_A extended with **(refine-pre)**.

To show that the triple is (intensionally) valid, we observe that

$$\llbracket r \rrbracket_{\text{Int}_{\neq 0}}^\sharp \alpha(P) = \llbracket r_w \rrbracket_{\text{Int}_{\neq 0}}^\sharp \llbracket r_1 \rrbracket_{\text{Int}_{\neq 0}}^\sharp \alpha(P) = \perp$$

because r_w always diverges, so $\llbracket r_w \rrbracket_{\text{Int}_{\neq 0}}^\sharp$ too is the function that always diverge (in the abstract). Therefore,

$$\llbracket r \rrbracket_{\text{Int}_{\neq 0}}^\sharp \alpha(P) = \alpha(\llbracket r \rrbracket P) = \alpha(\emptyset) = \perp.$$

To show that the triple is not provable in LCL_A extended with **(refine-pre)**, we rely on two observations.

The first is that all strict subset $P' \subset P$ are such that $\text{Int}_{\neq 0}(P') \subset P$, and the same property holds for all refinements $A' \preceq \text{Int}_{\neq 0}$. To see this, take $P' \subset P$: there are only three such P' , and for all of them $\text{Int}_{\neq 0}(P') = P' \subset P$. Moreover, $A' \preceq \text{Int}_{\neq 0}$ means that $A'(P') \subseteq \text{Int}_{\neq 0}(P')$, so

$$A'(P') \subseteq \text{Int}_{\neq 0}(P') = P' \subset P.$$

This property is important because it means that we cannot apply **(relax)** to change P : to do it, we would need a $P' \subset P$ such that $P \subseteq A'(P')$.

The second is that $\llbracket r_1 \rrbracket$ is not locally complete on P in $\text{Int}_{\neq 0}$ or any of its refinements

⁴Note that r_w is equivalent to the regular command **false?**.

$A' \preceq \text{Int}_{\neq 0}$ such that $A'(P) = \text{Int}_{\neq 0}(P)$:

$$\begin{aligned}
A'(\llbracket r_1 \rrbracket P) &\subseteq \text{Int}_{\neq 0}(\llbracket r_1 \rrbracket P) \\
&= \{-2, -1, 1, 2\} \\
&\subset \{-2, -1, 0, 1, 2\} \\
&\subseteq A'(\{-2, -1, 0, 1, 2\}) \\
&= A'(\llbracket r_1 \rrbracket (\{-4, -3, -2, -1, 0\})) \\
&= A'(\llbracket r_1 \rrbracket (\text{Int}_{\neq 0}(P))) \\
&= A'(\llbracket r_1 \rrbracket A'(P))
\end{aligned}$$

Now suppose to have a derivation of $\vdash_{\text{Int}_{\neq 0}} [P] \ r \ [\emptyset]$. This proof must use **(seq)** to handle the sequential composition $r_1; r_w$, so it needs a triple for r_1 . By the first observation above, any use of **(relax)** cannot change the precondition of this triple, even if we resort first to **(refine-pre)** to refine the domain. Thus we must have a triple $\vdash_{A'} [P] \ r_1 \ [R]$ for some R and $A' \preceq \text{Int}_{\neq 0}$ satisfying $A'(P) = \text{Int}_{\neq 0}(P)$. However, by soundness, any such triple would imply local completeness of $\llbracket r_1 \rrbracket$ on P in A' , which is a contradiction by the second observation above. $\blacksquare$

Another example of a sound triple which is not provable using **(refine-pre)**, which does not rely on divergence, is in Appendix C, Example C.3. A corollary of these examples (and more in general of logical incompleteness) is that there may not exist a refinement A' to carry out the proof using **(refine-pre)**. Another consequence of this incompleteness result is the fact that, even when a command is locally complete in an abstract domain A , we may need to reason about properties that are not expressible in A in order to prove it, as **(refine-pre)** may not be sufficient.

We now present an example to illustrate that there are situations in which **(refine-int)** is more practical than **(refine-pre)**, even though they are both able to prove the same triple.

Example 6.11. Consider the two program fragments

$$\begin{aligned}
r_1 &\triangleq (y \neq 0)?; \ x := y; \ y := \text{abs}(y) \\
r_2 &\triangleq x := y; \ \text{while } (x > 1) \ \{ \ y := y - 1; \ x := x - 1 \}
\end{aligned}$$

and the program $r \triangleq r_1; r_2$. Consider also the initial state $P = y \in [-100; 100]$.

This example is a variation of Example 6.9: the difference is the introduction of the relational dependency $x := y$ in r_1 , that is partially stored in the postcondition R of r_1 . Because of this, $\text{Oct}(R)$ and $\text{Int}(R)$ are different, so we cannot apply **(refine-pre)** to prove $\vdash [R] \ r_2 \ [Q]$ for some Q .

Following Example 6.9, the domain $\text{Int}_{\neq 0}$ is able to infer on r_1 a subset R of the strongest postcondition $y \in [1; 100] \wedge y = \text{abs}(x)$ with the same abstraction $\text{Int}_{\neq 0}(R) = [-100; 100]_{\neq 0} \times [1; 100]$. However, for any such R we cannot use **(refine-pre)** to prove the triple $\vdash_{\text{Int}} [R] \ r_2 \ [x = 1 \wedge y = 1]$ via **Oct** because $\text{Int}(R) = x \in [-100; 100] \wedge y \in [1; 100]$ while $\text{Oct}(R) = 1 \leq y \leq 100 \wedge -y \leq x \leq y$. More in general, any subset of the strongest postcondition contains the relational information $y = \text{abs}(x)$, so relational domains like octagons and polyhedra [CH78] do not have the same abstraction as the non-relational Int , preventing the use of **(refine-pre)**. However, we can apply **(refine-int)**: considering $R = (y \in \{1; 2; 100\} \wedge y = \text{abs}(x))$, $Q = (x = 1 \wedge y = 1)$ and

$r_w \triangleq \text{while } (x > 1) \{ y := y - 1; x := x - 1 \}$, we have

$$\begin{aligned}
 \text{Int} \llbracket r_2 \rrbracket_{\text{Oct}}^{\#} \text{Int}(R) &= \text{Int} \llbracket r_2 \rrbracket_{\text{Oct}}^{\#} (x \in [-100; 100] \wedge y \in [1; 100]) \\
 &= \text{Int} \llbracket r_w \rrbracket_{\text{Oct}}^{\#} \llbracket x := y \rrbracket_{\text{Oct}}^{\#} (x \in [-100; 100] \wedge y \in [1; 100]) \\
 &= \text{Int} \llbracket r_w \rrbracket_{\text{Oct}}^{\#} (1 \leq y \leq 100, y = x) \\
 &= \text{Int}(x = 1 \wedge y = 1) \\
 &= \text{Int}(Q).
 \end{aligned}$$

In this example, rule (refine-pre) can be applied to prove the triple, but it requires to have relational information from the assignment $x := y$ in r_1 , hence forcing the use of a relational domain (e.g. the reduced product [CC79] of Oct and $\text{Int}_{\neq 0}$) for the whole r , making the analysis more expensive. ■

6.3.3 Choosing the refinement

Thanks to the three new rules (refine-ext), (refine-int) and (refine-pre) we can now combine different domains in the same derivation. However, in order to obtain an algorithm that automatises the search of a provable LCL_A triple we are left with the problem of the selection of the right refinement to use each time. A crucial point to the applicability of refine rules is a strategy to find the most convenient refined abstract domain. While we have not addressed this problem yet, we believe there are some interesting starting points in the literature that we discuss below.

In previous sections, we settled the question from a theoretical point of view. Logical completeness results for (refine-ext) (Theorem 6.5) and (refine-int) (Theorem 6.8) implies the existence of a domain in which it is possible to complete the proof (if this were not the case, then the proof could not be completed in any domain, contradicting logical completeness). However, the proofs of those theorems exhibit the concrete domain C as an example, which is unfeasible in general. On the contrary, as (refine-pre) is logically incomplete (Example 6.10), there are triples that cannot be proved in any domain with it.

As more practical alternatives, we envisage some possibilities. First, we are studying relationships with counterexample-guided abstraction refinement (CEGAR) [Cla+00], which is a technique that exploits refinement in the context of abstract model checking. However, CEGAR and our approach seem complementary. On the one hand, our refinement rules allow a dynamic change of domain, during the analysis and only for a part of it, while CEGAR performs a static refinement and then a new analysis of the whole transition system in the new, more precise domain. On the other hand, our rules lack an instantiation technique, while for CEGAR there are effective algorithms available to pick a suitable refinement.

Second, local completeness shell [BGGR22] were proposed as an analogous of (global) completeness shell [GRS00]. In the article, the authors proposed to use local completeness shells to perform abstract interpretation repair, a technique to refine the abstract domain depending on the program to analyse, just like CEGAR does for abstract model checking. Abstract interpretation repair works well with LCL_A , and could be a way to decide the best refinement for one of our rules in presence of a failed local completeness proof obligation. The advantage of combining repair with our new rules is given by the possibility of discarding the refined domain just after its use in a subderivation instead of using it to carry out the whole derivation. Investigations in this direction is ongoing.

A third related approach, which shares some common ground with CEGAR, is Lazy (Predicate) Abstraction [HJMS02; McM06]. Both ours and this approach exploits different

$$\frac{\vdash_{A'} [P] \text{ r } [Q] \quad A' \succeq A \quad A'(Q) = A(Q)}{\vdash_A [P] \text{ r } [Q]} \text{ (simplify)}$$

Figure 6.3: Rule (simplify) for LCL_A .

abstract domains for different parts of the proof, refining it as needed. The key difference is that Lazy Abstraction unwinds the control flow graph of the program (with techniques to handle loops) while we work inductively on the syntax. This means that, when Lazy Abstraction refines a domain, it must use it from that point onward (unless it finds a loop invariant). On the other hand, our method can change abstract domain even for different parts of sequential code. However, the technique used in Lazy Abstraction (basically to trace a counterexample back with a theorem prover until it is either found to be spurious or proved to be true) could be applicable to LCL_A : a failed local completeness proof obligation in (transfer) can be traced back with a theorem prover and the failed proof can be used to understand how to refine the abstract domain.

A fourth practically implemented technique is the static analyser Astreé [Cou+05]. In Astreé, different abstract domains cooperate sharing information via message passing, implicitly defining a partially reduced product domain. Moreover, this message passing is not statically fixed. Instead, an abstract domain may ask other domains for information to refine its own result, building a hierarchy of cooperating domains [Cou+06]. This on-demand partially reduced product can be formalized as a domain refinement, that in theory could be integrated in our refinement framework, but further investigation is needed.

6.4 Locally complete simplification

We now turn our attention to *simplification* of the abstract domain in the LCL_A proof system. It is known that (global) completeness can be achieved both by refining and by simplifying the abstract domain (these constructions are called completeness shell and core, respectively [GRS00]). Therefore, we do the same for local completeness. We propose the rule (simplify), in Figure 6.3. This is a dual of (refine-pre): while the latter requires A' to be a refinement of A with the same abstraction on the precondition P , the former requires A' to be a simplification of A with the same abstraction on the postcondition Q . We remark that this rule is independent of the refinement ones: it can be added to LCL_A both with and without any of the refinement rules.

The proposed rule (simplify) is sound, but differently than the refinement rules, it is so *intensionally*:

Theorem 6.12 (Intensional soundness of rule simplify). *The proof system in Figure 3.2 with the addition of rule (simplify) from Figure 6.3 is intensionally sound.*

Proof sketch. Since the proof of Theorem 3.7 in [BGGR21] is by rule induction, we extend its proof with a new inductive case. The full details are in Appendix C. $\square$

This result is somewhat surprising: for completeness, refinement and simplification have the same power; instead, for local completeness, the former appears to be stronger than the latter.

Even though rule (simplify) does not allow the proof system to prove triples which are not intensionally sound, it still allows to prove more triples, as shown in the following example.

Example 6.13. This example builds on the previous Example 6.10. Consider the same program fragments r_1 , r_w and r , concrete domain $\mathcal{P}(\mathbb{Z})$, abstract domain $\text{Int}_{\neq 0}$, initial state $P = \{-5, 0\}$ and final state $\emptyset$. We already showed that the triple $\vdash_{\text{Int}_{\neq 0}} [P] \ r \ [\emptyset]$ cannot be proved using LCL_A (extended with (refine-pre)) in Example 6.10. Hence, we only need to show that it is provable using (simplify).

Consider the simplified domain $\text{Div} = \{\perp, \top\} \supseteq \text{Int}_{\neq 0}$. The domain Div separates the empty set from any other set, and can be used for divergence analysis. $\llbracket r_1 \rrbracket$ is locally complete on P in Div :

$$\text{Div}(\llbracket r_1 \rrbracket(P)) = \top = \text{Div}(\llbracket r_1 \rrbracket(\top)) = \text{Div}(\llbracket r_1 \rrbracket(\text{Div}(P)))$$

This means we can prove the triple $\vdash_{\text{Div}} [P] \ r_1 \ [\llbracket r_1 \rrbracket(P)]$ by just applying (transfer). Moreover, $\llbracket r_w \rrbracket$ is globally complete since its output is always $\emptyset$. With these two observations, we can derive the triple $\vdash_{\text{Int}_{\neq 0}} [P] \ r \ [\emptyset]$ using (simplify) with the following proof tree:

$$\frac{\frac{\frac{\mathbb{C}_P^{\text{Div}}(\llbracket r_1 \rrbracket)}{\vdash_{\text{Div}} [P] \ r_1 \ [\llbracket r_1 \rrbracket(P)]} \text{ (transfer)} \quad \frac{\frac{\mathbb{C}_{\llbracket r_1 \rrbracket(P)}^{\text{Div}}(\llbracket r_w \rrbracket)}{\vdash_{\text{Div}} [\llbracket r_1 \rrbracket(P)] \ r_w \ [\emptyset]} \text{ (transfer)}}{\text{Div} \supseteq \text{Int}_{\neq 0} \quad \vdash_{\text{Div}} [P] \ r \ [\emptyset] \quad \text{Int}_{\neq 0}(\emptyset) = \emptyset = \text{Div}(\emptyset)} \text{ (seq)} \quad \text{ (simplify)} \\ \vdash_{\text{Int}_{\neq 0}} [P] \ r \ [\emptyset]$$

■

Intuitively, in the previous example the incompleteness is caused by the precision of $\text{Int}_{\neq 0}$ on the output of r_1 . However, this precision is not needed because the details of the intermediate state are discarded by r_w . The simpler domain Div is able to discard such precision, thus proving local completeness of the composite command.

In this example, we showed that $\vdash_{\text{Int}_{\neq 0}} [P] \ r \ [\emptyset]$ can be proved in LCL_A extended with (simplify), but by Example 6.10 we know it is not provable by (refine-pre). We remark that the opposite is true as well: LCL_A extended with (refine-pre) can prove extensionally valid triples (cf. Example 6.9) that it cannot prove when extended with (simplify), because the latter is bound by intensional soundness. Together, these two facts means that (refine-pre) and (simplify) extend the logic in two incomparable ways. If we instead include (refine-int), we know by logical completeness (Theorem 6.8) that it can prove all (extensionally) valid triples, including all those provable with (simplify).

However, even though LCL_A extended with (simplify) can prove more triples, it is logically incomplete:

Theorem 6.14 (Intrinsic incompleteness). *Consider the concrete domain of stores $C = \mathcal{P}(\Sigma)$. Assume Reg is a Turing complete language, and $A \in \text{Abs}(C)$ is not trivial. Then there exists $P, Q \in C$ and $r \in \text{Reg}$ such that $Q \leq \llbracket r \rrbracket P$, $\llbracket r \rrbracket_A^\# \alpha(P) = \alpha(Q)$ but the triple $\vdash_A [P] \ r \ [Q]$ is not provable in LCL_A extended with (simplify).*

Proof sketch. The proof follows closely that of intrinsic incompleteness of LCL_A [BGGR21, Theorem 5.12] and is reported in Appendix C. $\square$

This theorem shows that the strength of rule (simplify) on the logical level is quite thin. However, this rule could be extremely helpful in practice because it allows to perform part of the analysis in a simpler and possibly much more efficient domain. For instance, consider variable partitioning, a kind of domain simplification. In a series of papers, Singh

et al. [SPV15; SPV17b; SPV17a] showed that it leads to great speedups in relational numerical abstract domains, such as octagons and polyhedra. Variable partitioning is a technique that divides variables in subsets such that relations only exists among variables in the same subset. This allows to perform many operations separately on different partitions, reducing the cost that is superlinear in the number of variables (eg. cubic for octagons, exponential for polyhedra). Moreover, partitions are chosen dynamically, so that (1) they change during the analysis and (2) they are guaranteed not to lose precision w.r.t. the non partitioned domain. Let us consider the domain **Poly** of polyhedra as an example, but we remark these observations are general enough to be applied to many relational numerical domains. Formally, given a set of variables **Var** and one of its partitions $\pi = \{\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_m\}$, a polyhedron P can be expressed in the partition π if and only if, for all constraints k of P all variables involved in k are in the same element $\mathcal{X}_i \in \pi$. Given a variable partitioning π we denote by Poly_π the abstract domain of polyhedra that can be expressed in the partition π . Clearly $\gamma(\text{Poly}_\pi) \subseteq \gamma(\text{Poly})$, so that $\text{Poly} \preceq \text{Poly}_\pi$ for all π . Since the partition is changed during the analysis, the abstract domain changes too. Rule (refine-ext) cannot accommodate for this change. A domain coarser than all partitions correspond to the maximal partition, in which each variable is on its own, and is non relational: for **Poly** this domain is **Int**. In general **Int** is not precise enough to prove the condition $A[\mathbf{r}]^{A'} A(P) = A(Q)$ (instantiated with $A = \text{Int}$ and $A' = \text{Poly}_\pi$ it becomes $\text{Int}[\mathbf{r}]^{\text{Poly}_\pi} \text{Int}(P) = \text{Int}(Q)$) at every partition change, since those occur during the analysis and must keep track of relational informations computed up to that point. On the other hand, rule (simplify) perform the “global” analysis in **Poly**, and simplify locally to Poly_π for the computation, taking advantage of the better performances in the simpler domain. Note that, since the partition is chosen in order not to lose precision w.r.t. the non partitioned domain **Poly**, the condition $A'(Q) = A(Q)$ is satisfied. This means that variable partitioning can be plugged in LCL_A with our rule (simplify), allowing it to benefit from all the performance increase. While it is intuitive that variable partitioning is sound since it is as precise as **Poly**, our rule formally justify this claim.

Example 6.15. Consider the program fragments

$$\begin{aligned}
 r_1 &\triangleq x := y; y := y - 3; x := x - 4 \\
 r_2 &\triangleq z := y; y := 0 \\
 r_3 &\triangleq w := z - x \\
 r &\triangleq r_1; r_2; r_3 \\
 &= x := y; y := y - 3; x := x - 4; z := y; y := 0; w := z - x
 \end{aligned}$$

and the initial state $P = -100 \leq y \leq 100$. At the end, the value for **w** is always 1. To prove it, the analysis must track the dependency between variables using a relational domain such as polyhedra. However, for the first three assignments (i.e., the fragment r_1) we do not need to track any dependency involving **z** and **w**, and after the assignment $y := 0$ in r_2 there is no dependency to track with **y**.

Consider the final set of states

$$Q \triangleq (y = 0 \wedge x \in \{-104, 96\} \wedge z = x + 1 \wedge w = 1)$$

and partitions $\pi_1 = \{\{x, y\}, \{z\}, \{w\}\}$ and $\pi_2 = \{\{x, z\}, \{y\}, \{w\}\}$. A proof for the triple $\vdash_{\text{Poly}} [P] r [Q]$ can exploit (simplify) to work in Poly_{π_1} and Poly_{π_2} for parts of the program, therefore simplifying the computation of the local completeness proof obligations for (transfer). Fixed $R_1 \triangleq (y \in \{-103, 97\} \wedge x = y - 1)$ and $R_2 \triangleq (x \in \{-104, 96\} \wedge y = 0 \wedge z = x + 1)$, the proof sketch looks like

$$\frac{\frac{\dots}{\vdash_{\text{Poly}_{\pi_1}} [P] \ r_1 \ [R_1]} \quad \text{(simplify)} \quad \frac{\dots}{\vdash_{\text{Poly}} [R_1] \ r_2 \ [R_2]} \quad \frac{\dots}{\vdash_{\text{Poly}_{\pi_2}} [R_2] \ r_3 \ [Q]} \quad \text{(simplify)}}{\vdash_{\text{Poly}} [P] \ r \ [Q]} \quad \text{(seq)}$$

Note that $\text{Poly}(R_1) = \text{Poly}_{\pi_1}(R_1)$ and $\text{Poly}(Q) = \text{Poly}_{\pi_2}(Q)$ because R_1 (resp. Q) only contains constraints between variables in the same partition of π_1 (resp. π_2). ■

Since (simplify) is analogous to (refine-pre), a natural question is whether there exists one simplification rule similar to the stronger (refine-ext). We believe this is not the case. A dual of (refine-ext) for simplification should involve $\llbracket r \rrbracket^{A'}$, because the goal of the rule is to perform the analysis in A' . The only reasonable input to which we can apply $\llbracket r \rrbracket^{A'}$ is $\alpha'(P)$. Note that, since $A' \succeq A$, abstracting with A before does not change the result: $\alpha'A(P) = \alpha'(P)$. The hypothesis $\vdash_{A'} [P] \ r \ [Q]$ implies, by soundness, $\llbracket r \rrbracket^{A'} \alpha'(P) = \alpha'(Q)$. Moreover, for the rule to be sound its hypotheses must ensure that $\alpha(Q) = \alpha(\llbracket r \rrbracket P) = \llbracket r \rrbracket^A \alpha(P)$, so any condition involving $\llbracket r \rrbracket^{A'} \alpha'(P)$ and any of those three is equivalent to $A'(Q) = A(Q)$. Based on this argument, we don't expect a weaker condition (that is, a more general rule) for simplification to exist.

6.5 Exploiting convexity

LCL_A requires the existence of a best correct abstraction $\alpha : C \rightarrow A$, but there are domains for which only the concretisation map γ is defined (e.g., for polyhedra). If this is the case, LCL_A is not applicable because we cannot even write the (local) completeness equation $Af(P) = AfA(P)$. However, local completeness enjoys abstract convexity (Lemma 3.6). This suggests that, even if a concrete point X cannot be abstracted exactly, we can use a different point P that can be abstracted such that $P \leq X \leq A(P)$. Intuitively, if f is locally complete on P , abstract convexity implies that f is locally complete on x , too. This idea is the basis of the development in this section.

Formally, we introduce an intermediate domain L between C and A such that (1) there is a Galois connection between L and A and (2) we have a monotone concretisation function $\gamma_0 : L \rightarrow C$, as in the following diagram:

$$C \xleftarrow{\gamma_0} L \xrightleftharpoons[\alpha]{\gamma} A$$

Intuitively, we take L as the set of all the assertions we can write as pre and postconditions of LCL_A triples, and we limit this to be only a subset of C for which there is an abstraction function $\alpha : L \rightarrow A$. For instance, if $A = \text{Poly}$, we can take L to be the set of finite unions of polyhedra, and limit the logic to only use those as pre and postconditions of triples, instead of any possible concrete state. To apply LCL_A using L as the concrete domain, we fix an abstraction $\llbracket r \rrbracket_L : L \rightarrow L$ of $\llbracket r \rrbracket$. As it is standard in the absence of the abstraction function [CC92], we require $\llbracket r \rrbracket_L$ to satisfy the soundness condition $\llbracket r \rrbracket \gamma_0 \leq \gamma_0 \llbracket r \rrbracket_L$. Since there is a Galois connection between L and A , we can exploit the LCL_A framework (and all of its extensions) using L as the concrete domain and $\llbracket r \rrbracket_L$ as the concrete semantics. Then, to transfer the nice properties of LCL_A from L to C , we require the concrete point $X \in C$ to be in between the current assertion $P \in L$ and its abstraction $\alpha(P) \in A$. Formally, at any program point with concrete state X and assertion P , we require that

$$\gamma_0(P) \leq X \leq \gamma_0 \gamma \alpha(P) \tag{I}$$

Please note that, if this invariant holds after the execution of a program r , then the Proofs of Verification (Corollary 3.8) holds for the concrete value X .

However, in general this invariant is not preserved by program execution. Given a triple $\vdash_A [P] \text{ r } [Q]$ (w.r.t. the concrete domain L and semantics $\llbracket r \rrbracket_L$) such that the concrete state before r is X satisfying (I), in general the corresponding invariant after the program $\gamma_0(Q) \leq \llbracket r \rrbracket X \leq \gamma_0 \gamma \alpha(Q)$ does not hold. This is because $\llbracket r \rrbracket X$ may not be comparable with $\gamma_0(\llbracket r \rrbracket_L P)$ even if $\llbracket r \rrbracket_L$ is sound. To solve this issue, we impose the additional condition of forward completeness [GQ01] of $\llbracket r \rrbracket_L$, but only on the single point P , namely

$$\gamma_0(\llbracket r \rrbracket_L P) = \llbracket r \rrbracket \gamma_0(P).$$

Under this hypothesis, we show the following proposition:

Proposition 6.16. *Assume that $\llbracket r \rrbracket_L$ is forward complete on P , that $\vdash_A [P] \text{ r } [Q]$ is valid taking L as the concrete domain and that invariant (I) holds. Then*

$$\gamma_0(Q) \leq \llbracket r \rrbracket X \leq \gamma_0 \gamma \alpha(Q).$$

Proof. We prove the two inequalities separately. We recall that $\llbracket r \rrbracket$ and γ_0 are monotone, and we use this fact implicitly in the chains of inequalities below.

$$\begin{aligned} \gamma_0(Q) &\leq \gamma_0(\llbracket r \rrbracket_L P) && [\text{validity of } \vdash_A [P] \text{ r } [Q], \text{ pt. (1)}] \\ &= \llbracket r \rrbracket \gamma_0(P) && [\text{forward completeness}] \\ &\leq \llbracket r \rrbracket X && [\gamma_0(P) \leq X] \end{aligned}$$

and

$$\begin{aligned} \llbracket r \rrbracket X &\leq \llbracket r \rrbracket \gamma_0 \gamma \alpha(P) && [X \leq \gamma_0 \gamma \alpha(P)] \\ &\leq \gamma_0(\llbracket r \rrbracket_L \gamma \alpha(P)) && [\text{soundness of } \llbracket r \rrbracket_L] \\ &\leq \gamma_0 \gamma ((\llbracket r \rrbracket_L)^A \alpha(P)) && [\text{soundness of } (\llbracket r \rrbracket_L)^A] \\ &= \gamma_0 \gamma \alpha(Q) && [\text{validity of } \vdash_A [P] \text{ r } [Q], \text{ pt. (3)}] \end{aligned}$$

□

This lemma allows us to use LCL_A even in the absence of a best abstraction function. However, to do so we need to identify a set L of assertions which is concrete enough to be forward complete and abstract enough to have an abstraction function $\alpha : L \rightarrow A$.

Intuitively, the (local) forward completeness requirement $\gamma_0(\llbracket r \rrbracket_L P) = \llbracket r \rrbracket \gamma_0(P)$ implies that all the concrete state traversed are “close enough” to have an abstraction that a proof in LCL_A yields useful information. In other words, the local forward completeness is a “sanity check” that the program did not traverse a concrete state for which the LCL_A theory is not applicable, that are states X for which there exists no $P \in L$ s.t. (I) holds. For instance, with the previous example of polyhedra and finite unions of polyhedra, any shape with a curved “edge” (e.g., a circle) does not satisfy the requirement (I), and the local forward completeness condition ensures such states are not traversed.

Example 6.17. Consider the concrete domain $\mathbb{R}^2$ of two real variables x and y , the polyhedra domain Poly as A and the set of finite unions of polyhedra $\text{Poly}_\cup$ as L . Consider the program fragment

$$r_r \triangleq x := (\sqrt{2} / 2) * x - (\sqrt{2} / 2) * y; y := x + \sqrt{2} * y$$

$$\begin{aligned}
\llbracket \overleftarrow{e} \rrbracket_A^\# a &\triangleq \llbracket \overleftarrow{e} \rrbracket^A a = \alpha \llbracket \overleftarrow{e} \rrbracket \gamma(a) \\
\llbracket \overleftarrow{r_1}; r_2 \rrbracket_A^\# a &\triangleq \llbracket \overleftarrow{r_2} \rrbracket_A^\# \llbracket \overleftarrow{r_1} \rrbracket_A^\# (a) \\
\llbracket \overleftarrow{r_1} \oplus r_2 \rrbracket_A^\# a &\triangleq \llbracket \overleftarrow{r_1} \rrbracket_A^\# a \vee A \llbracket \overleftarrow{r_2} \rrbracket_A^\# a \\
\llbracket \overleftarrow{r^*} \rrbracket_A^\# a &\triangleq \bigvee_{n \geq 0} (\llbracket \overleftarrow{r} \rrbracket_A^\#)^n a
\end{aligned}$$

Figure 6.4: Backward abstract semantics of regular commands.

the program $r \triangleq r_r^*$ and the set of states $X = -5 \leq x \leq 5 \wedge -5 \leq y \leq 5$, describing a square in the Cartesian plane. Please note that the program r_r rotates the point (x, y) in the Cartesian plane by $\pi/4$ radians, therefore applying r_r to X twice returns X . In particular, letting $X' \triangleq \llbracket r_r \rrbracket X$, this means that $\llbracket r \rrbracket X = X \cup X'$.

Consider the initial assertion $P = \{(5, 5), (-5, 5), (5, -5), (-5, -5)\}$, where a pair denote the value for variables x and y , that is the union of four polyhedra, each containing a single point. It holds that $\gamma_0(P) \subseteq X = \gamma_0(\text{Poly}(P))$. We also define $P' = \{(5\sqrt{2}, 0), (-5\sqrt{2}, 0), (0, 5\sqrt{2}), (0, -5\sqrt{2})\}$ and $Q = P \cup P'$. Using LCL_{Poly} over $L = \text{Poly}_\cup$, we can first prove the two triples $\vdash_{\text{Poly}} [P] r_r [P']$ and $\vdash_{\text{Poly}} [P \cup P'] r_r [Q]$ using (seq) and (transfer). Then, we compose them using (iterate) and (req) to conclude:

$$\frac{\frac{\vdash_{\text{Poly}} [P] r_r [P']}{\vdash_{\text{Poly}} [P] r_r [P']} \quad \frac{\frac{\vdash_{\text{Poly}} [P \cup P'] r_r [Q]}{\vdash_{\text{Poly}} [P \cup P'] r_r^* [Q]} \quad (P \cup P') \leq \text{Poly}(Q)}{\vdash_{\text{Poly}} [P] r [P \cup Q]} \text{ (iterate)} \quad \text{(rec)}$$

This tells us that the triple $\vdash_{\text{Poly}} [P] r [P \cup Q]$ is valid relative to the semantics in $\text{Poly}_\cup$. To transfer the same analysis to the concrete domain $\mathbb{R}^2$, we check forward completeness. In this case, the check is trivial because the semantics of r in L on P , P' and Q is exactly the same as the concrete semantics on $\gamma_0(P)$, $\gamma_0(P')$ and $\gamma_0(Q)$. Note that this triple allows us to identify errors: given the specification $\text{Spec} = x \leq 7$, we observe that $Q \not\subseteq \text{Spec}$. Since Spec is expressible in Poly , this in turn highlights the error $(5\sqrt{2}, 0) \in Q \setminus \text{Spec}$, which is a true alert because $(5\sqrt{2}, 0) \in X \cup X'$. ■

6.6 Backward analysis

In principle, the theory of Abstract Interpretation does not rely on the analysis being forward. This suggests that LCL can be used for backward analysis, as discussed in [BGGR23, §5.3]. However, they explored the use of **wlp** as the reference backwards semantics, requiring the use of under-approximation abstract domains, that are hard to exploit in practice (see Chapter 4). Instead, our key insight is that by using $\llbracket \overleftarrow{\cdot} \rrbracket$ (see equation (5.1)) the proof system can be turned over by duality for backward inference *using classical, over-approximation abstract domains*.

We define inductively in Figure 6.4 the backward abstract semantics or regular commands. This definition is analogous to the forward one (Figure 2.6). Thanks to Lemma 5.1 giving $\llbracket \overleftarrow{\cdot} \rrbracket$ the same inductive definition as the forward semantics $\llbracket \cdot \rrbracket$, we can straightforwardly adapt Proposition 2.22 to obtain that $\llbracket \overleftarrow{\cdot} \rrbracket_A^\#$ is a sound abstraction of $\llbracket \overleftarrow{\cdot} \rrbracket$.

We can then define the proof system for Converse Local Completeness Logic (CLCL) in Figure 6.5 by just swapping the roles of pre and post in the LCL rules.

$\frac{\mathbb{C}_Q^A(\llbracket \overleftarrow{c} \rrbracket)}{\vdash_A \langle \llbracket \overleftarrow{c} \rrbracket Q \rangle \text{ c } \langle Q \rangle} \text{ (transfer)} \quad \frac{P \leq P' \leq A(P) \quad \vdash_A \langle P' \rangle \text{ r } \langle Q' \rangle \quad Q' \leq Q \leq A(Q')}{\vdash_A \langle P \rangle \text{ r } \langle Q \rangle} \text{ (relax)}$	
$\frac{\vdash_A \langle P \rangle \text{ r}_1 \langle R \rangle \quad \vdash_A \langle R \rangle \text{ r}_2 \langle Q \rangle}{\vdash_A \langle P \rangle \text{ r}_1; \text{ r}_2 \langle Q \rangle} \text{ (seq)}$	$\frac{\vdash_A \langle P_1 \rangle \text{ r}_1 \langle Q \rangle \quad \vdash_A \langle P_2 \rangle \text{ r}_2 \langle Q \rangle}{\vdash_A \langle P_1 \vee P_2 \rangle \text{ r}_1 \oplus \text{ r}_2 \langle Q \rangle} \text{ (join)}$
$\frac{\vdash_A \langle P \rangle \text{ r}^* \langle R \vee Q \rangle \quad \vdash_A \langle R \rangle \text{ r } \langle Q \rangle}{\vdash_A \langle P \rangle \text{ r}^* \langle Q \rangle} \text{ (rec)}$	$\frac{P \leq A(Q) \quad \vdash_A \langle P \rangle \text{ r } \langle Q \rangle}{\vdash_A \langle P \vee Q \rangle \text{ r}^* \langle Q \rangle} \text{ (iterate)}$

Figure 6.5: The proof system for CLCL.

Theorem 6.18 (CLCL is sound). *If the CLCL triple $\vdash_A \langle P \rangle \text{ r } \langle Q \rangle$ is provable, then*

1. $P \leq \llbracket \overleftarrow{r} \rrbracket Q$,
2. $\alpha(P) = \alpha(\llbracket \overleftarrow{r} \rrbracket Q)$,
3. $\alpha(P) = \llbracket \overleftarrow{r} \rrbracket_A^\dagger \alpha(Q)$.

Just like regular LCL, any provable CLCL triple ensures that $\llbracket \overleftarrow{r} \rrbracket Q$ is between P and $A(P)$ and that $\mathbb{C}_Q^A(\llbracket \overleftarrow{r} \rrbracket)$. Particularly, this means that if $\llbracket \overleftarrow{r} \rrbracket Q \neq \emptyset$ and A is a non-trivial abstraction, then also $P \neq \emptyset$. In other words, given an error Q , the analysis either finds some (non empty) precondition P leading to that Q , or shows that Q is unreachable. On the other end of the spectrum, the abstraction $A(P)$ always exhibits a necessary precondition for Q since $\llbracket \overleftarrow{r} \rrbracket Q \subseteq A(\llbracket \overleftarrow{r} \rrbracket Q) = A(P)$.

Example 6.19. Expanding on Example 5.6, we observe that SIL can also prove the triple $\langle R_{2M} \rangle \text{ r}_w^* \langle R_{2M} \rangle$ via $\langle \text{iter0} \rangle$. However, this would produce the triple $\langle \text{false} \rangle \text{ rloop0 } \langle Q_{2M} \rangle$ for the whole program because it finds the post $x = 2000000$ for $\mathbf{x} := 0$, that has **false** as the only valid precondition.

To rule out such derivations, we can use CLCL with the octagons domain Oct [Min06]: the triple $\vdash_{\text{Oct}} \langle R_{2M} \rangle \text{ r}_w^* \langle R_{2M} \rangle$ is not valid because $\text{Oct}(R_{2M}) \neq \text{Oct}(\llbracket \overleftarrow{r}_w^* \rrbracket R_{2M})$. This way, CLCL forces the analysis to unroll the loop twice, proving the triple

$$\vdash_{\text{Oct}} \langle (n > 0 \wedge x + n \leq 2000000) \vee R_{2M} \rangle \text{ r}_w^* \langle R_{2M} \rangle$$

whose full derivation is in Appendix C, Figure C.3. Using this triple, we conclude as in Example 5.6: both applications of $\langle \text{atom} \rangle$ in Figure 5.4 can be replaced by (transfer) since the backward semantics of both assignments is locally complete. Thus, we prove the triple $\vdash_{\text{Oct}} \langle \text{true} \rangle \text{ rloop0 } \langle Q_{2M} \rangle$ for the full program, which exposes a manifest error.

Note that, while loop unrolling is a standard technique used in abstract analysers, the use of CLCL forces the unrolling until precision is ensured instead of relying on a fixed number of unrolls or some heuristics. ■

Other than the original LCL_A proof system, we can adapt all the other extensions proposed in this chapter to CLCL. The fundamental reason that allows this is the fact that $\llbracket \cdot \rrbracket$ and $\llbracket \overleftarrow{\cdot} \rrbracket$ enjoy the same inductive definition. For instance, (refine-ext) can be adapted with the additional condition $A(P) = A(\llbracket \overleftarrow{r} \rrbracket^{A'} A(Q))$. All refinement and simplification rules are presented in Figure 6.6. The refinement rules can be proved *extensionally* sound (whose definition is analogous to LCL_A), while the simplification rule is (intensionally) sound.

$$\begin{array}{c}
\frac{\vdash_{A'} \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle \quad A' \preceq A \quad A(P) = A[\llbracket \cdot \rrbracket^{A'}] A(Q)}{\vdash_A \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle} \text{ (refine-ext)} \\
\\
\frac{\vdash_{A'} \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle \quad A' \succeq A \quad A'(P) = A(P)}{\vdash_A \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle} \text{ (simplify)} \\
\\
\frac{\vdash_{A'} \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle \quad A' \preceq A \quad A(P) = A[\llbracket \cdot \rrbracket_A^\#] A(Q)}{\vdash_A \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle} \text{ (refine-int)} \\
\\
\frac{\vdash_{A'} \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle \quad A' \preceq A \quad A'(Q) = A(Q)}{\vdash_A \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle} \text{ (refine-pre)}
\end{array}$$

Figure 6.6: Refinement and simplification rules for CLCL.

Proposition 6.20. *The proof system in Figure 6.5 with the addition of any rule from Figure 6.6 is sound.*

Proof sketch. The proof is analogous to those for LCL_A . It extends the soundness proof with new inductive cases for the new rules. $\square$

Similarly, if the abstract domain doesn't have an abstraction function α , we can adapt Proposition 6.16 assuming that the invariant holds in for the final state Q and prove it for P , by replacing $\llbracket \cdot \rrbracket$ with $\llbracket \cdot \rrbracket^{\leftarrow}$ in the proof and using validity of the $\vdash_A \langle\!\langle P \rangle\!\rangle \text{ r } \langle\!\langle Q \rangle\!\rangle$.

6.7 Summary

In this chapter, we started from LCL_A [BGGR21], a logical framework to prove both correctness and incorrectness of a program combining over and under-approximation in the form of (locally complete) abstractions and Incorrectness Logic. The original work was *intensionally* sound, based on Galois connection and more suitable for forward analysis. We tried to relax those constraints.

For the first, we followed the idea of [BGGR23] to exploit different abstract domains to analyse different portions of the whole program. We propose four new rules that can be independently added to the proof system. Three of them are based on domain *refinement*, with different complexity-precision trade-off, and the last is based on domain *simplification*. With any of this rule, we are able to prove many triples that the original LCL_A could not because of how the program is written. For the second, in the absence of α we investigated the possibility to introduce an intermediate domain L between C and A . Under the hypothesis of local forward completeness, we showed that this is enough to recover the properties of LCL_A . For the third, we explored the use of the backward SIL together with over-approximation abstract domains instead of IL with under-approximation abstract domains, and we showed that we can recast all results for LCL_A to CLCL.

We present a pictorial comparison among the expressiveness of the various proof systems in Figure 6.7. The bottom node of the diagram represents the original proof system LCL_A . Each other node represents the proof system extended with the single rule mentioned in the balloon. Each arrow corresponds to an expressivity result: all triples provable in the target system are also provable in the source system, which is thus more powerful. The labels reference the result that justifies the claim. For simplicity, we omit arrows

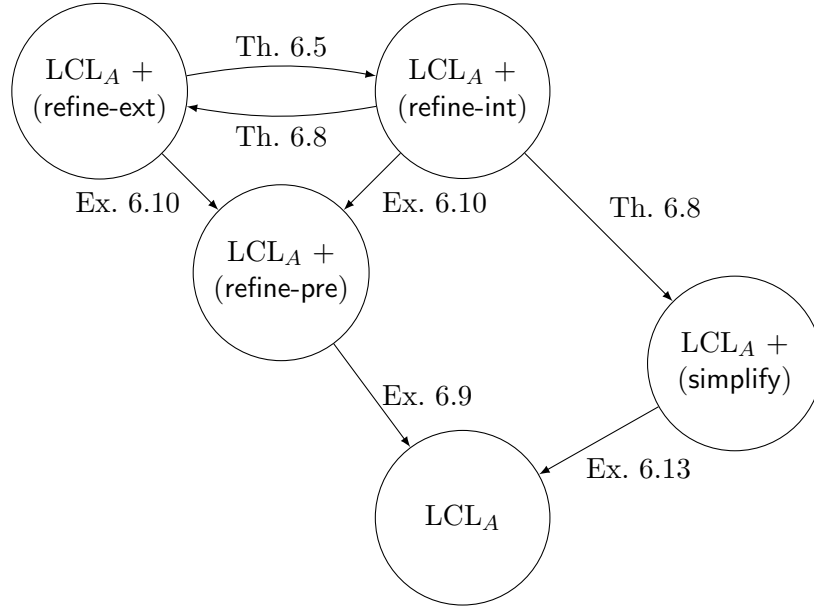


Figure 6.7: Relations between the new proof systems

obtained by transitivity. The two mutual arrows between the two topmost nodes indicate that the two proof systems are logically equivalent (i.e., they can prove the same triples).

To relax the Galois connection hypothesis, we further exploit the knowledge of both an over and an under-approximation of the concrete state via abstract convexity. Instead of bounding the concrete state with any under-approximation, we limit ourselves to those properties that have a best abstraction. If the program then satisfies a sanity check, namely local forward completeness, this ensures the result of LCL_A is valid for the concrete state, too.

Lastly, the forward/backward duality between IL and SIL highlighted in Chapter 5 allowed us to straightforwardly adapt LCL_A for backward analysis, combining the under-approximation of SIL with abstract interpretation in CLCL. Moreover, the same duality allowed us to adapt all the other results from this Chapter for CLCL: all the refinement rules, the simplification rule and Proposition 6.16 when there is no best abstraction.

Chapter 7

AdjointPDR

In this chapter we study a new PDR-like algorithm (see Section 3.3). The novelty with respect to this kind of algorithms is our extensive use of *adjunctions*. We propose a first algorithm, **AdjointPDR**, which exploits an adjoint g to the function f (which roughly identify the backward semantics of f) to quicken the counterexample search. This first algorithm allows us to devise a theory of heuristics to better understand and compare them. However, to apply **AdjointPDR** the right adjoint g to the forward semantics f must exist, and this is not always the case. To get rid of this constraint, we propose **AdjointPDR[↓]**, a variation of **AdjointPDR** which lift the problem to lower sets, where it is always possible to define this adjoint. Lastly, we propose yet another variation of the algorithm, **AdjointPDR^{AI}**, which can instantiate both **AdjointPDR** and **AdjointPDR[↓]**. We implemented this latter algorithm, and compared it against other PDR-like algorithms and state-of-the-art tools with encouraging results. In their generic variants, both over and under-approximation are just elements of the lattice. In the specific instance for probabilistic systems, the ingenuity of our new heuristics is precisely to always compute under-approximations that can be efficiently represented.

The content of this chapter is based on [Kor+23] and its extended version, invited to the conference special issue.

7.1 Overview

Category theory has recognized adjunctions $f \dashv g$ as fundamental concepts appearing across various mathematical domains [Law69]. Adjointness is prevalent in various branches of computer science as well, including abstract interpretation and functional programming [Lev04]. In our development, we employ adjoints in two distinct ways:

- (Forward-Backward Adjoint) f characterizes the *forward semantics* of a transition system, while g represents the *backward semantics*.
- (Abstraction-Concretization Adjoint) C denotes a concrete semantic domain, while A is an abstract one, akin to abstract interpretation. An adjoint allows us to translate a fixed-point problem from C to A .

The problem we address is the standard lattice-theoretical formulation of safety problems, namely whether the least fixed point of a continuous map b over a complete lattice $(L, \leq, \perp, \top, \vee, \wedge)$ is below a given element $p \in L$: $\text{lfp}(b) \leq p$.

The first algorithm we present, **AdjointPDR**, assumes the existence of an element $i \in L$ and two adjoints $f \dashv g: L \rightarrow L$, representing intuitively initial states i , forward semantics f and backward semantics g . In such cases, we assume $b(x) = f(x) \vee i$ for all $x \in L$.

$$L \begin{array}{c} \xleftarrow{f} \\ \perp \\ \xrightarrow{g} \end{array} L$$

Under this assumption, Tarski Theorem 2.1 yields the well-known equivalences:

$$\text{lfp}(b) \leq p \iff \text{lfp}(f \vee i) \leq p \iff i \leq \text{gfp}(g \wedge p),$$

where $\text{lfp}(f \vee i)$ and $\text{gfp}(g \wedge p)$ are, by Tarski-Scott Theorem 2.3, the limits of the *initial* and *final* chains illustrated below.

$$\perp \leq i \leq f(i) \vee i \leq \dots \qquad \dots \leq g(p) \wedge p \leq p \leq \top$$

The distinguishing feature of **AdjointPDR** is to take as a negative sequence (that is a sequential construction of potential counterexamples) an over-approximation of the final chain. This crucially differs from the negative sequence of other PDR-like algorithm, which is an under-approximation of the computed positive chain.

AdjointPDR is sound (Theorem 7.5) and does not loop (Proposition 7.7), but since the problem $\text{lfp}(b) \leq? p$ is not always decidable, we cannot prove termination. Nevertheless, **AdjointPDR** allows for a formal theory of heuristics that are essential when instantiating the algorithm to concrete problems. The theory prescribes the choices to obtain the boundary executions, using initial and final chains (Proposition 7.10); it thus identifies a class of heuristics guaranteeing termination when answers are negative (Theorem 7.15).

In general, **AdjointPDR**'s assumption of a forward-backward adjoint $f \dashv g$ does not hold, especially in probabilistic settings. Our second algorithm **AdjointPDR**[↓] circumvents this problem by extending the lattice for the negative sequence, from L to the lattice $L^\downarrow$ of *lower sets* in L . Specifically, by using the second form of adjoints, namely an abstraction-concretization pair, the problem $\text{lfp}(b) \leq? p$ in L can be translated to an equivalent problem on $b^\downarrow$ in $L^\downarrow$, for which an adjoint $b^\downarrow \dashv b_r^\downarrow$ always exists.

$$b \begin{array}{c} \circlearrowleft \\ \circlearrowright \end{array} L \begin{array}{c} \xleftarrow{\sqcup} \\ \perp \\ \xrightarrow{(-)^\downarrow} \end{array} L^\downarrow \begin{array}{c} \circlearrowleft \\ \circlearrowright \end{array} b^\downarrow \dashv b_r^\downarrow$$

This allows us to run **AdjointPDR** in the lattice $L^\downarrow$. We then notice that the search for a positive chain can be conveniently restricted to principals in $L^\downarrow$, which have representatives in L . The resulting algorithm, using L for positive chains and $L^\downarrow$ for negative sequences, is **AdjointPDR**[↓].

The use of lower sets for the negative sequence is a key advantage. It not only avoids the restrictive assumption of backward adjoint g , but also enables a more thorough search for counterexamples. **AdjointPDR**[↓] can simulate stepwise LT-PDR (Theorem 7.23), but it is more general since a single negative sequence in **AdjointPDR**[↓] potentially represents multiple (Proposition 7.24) or even all (Proposition 7.25) negative sequences of LT-PDR.

Our lattice-theoretic algorithms yield many concrete instances: the original IC3/PDR as well as Reverse PDR [SS17] are instances of **AdjointPDR** with L being the powerset of the state space; since LT-PDR can be simulated by **AdjointPDR**[↓], the latter generalizes all instances in [Kor+22]. As a notable instance, we apply **AdjointPDR**[↓] to MDPs, specifically to decide if the maximum reachability probability [BK08] is below a given threshold. Here the lattice $L = [0, 1]^S$ is that of fuzzy predicates over the state space S . Our theory provides guidance to devise two heuristics, for which we prove negative termination (Corollary 7.26).

$$\begin{array}{llll}
& \forall j \in [k, n-1], x_j \not\leq y_j & \text{(PN)} \\
x_0 = \perp & \text{(I0)} & \forall j \in [0, n-1], (f \vee i)^j(\perp) \leq x_j \leq (g \wedge p)^{n-1-j}(\top) & \text{(A1)} \\
1 \leq k \leq n & \text{(I1)} & \forall j \in [1, n-1], x_{j-1} \leq g^{n-1-j}(p) & \text{(A2)} \\
\forall j \in [0, n-2], x_j \leq x_{j+1} & \text{(I2)} & \forall j \in [k, n-1], g^{n-1-j}(p) \leq y_j & \text{(A3)} \\
\\
i \leq x_1 & \text{(P1)} & & \\
x_{n-2} \leq p & \text{(P2)} & \text{If } \vec{y} \neq \varepsilon \text{ then } p \leq y_{n-1} & \text{(N1)} \\
\forall j \in [0, n-2], f(x_j) \leq x_{j+1} & \text{(P3)} & \forall j \in [k, n-2], g(y_{j+1}) \leq y_j & \text{(N2)} \\
\forall j \in [0, n-2], x_j \leq g(x_{j+1}) & \text{(P3a)} & &
\end{array}$$

Figure 7.1: Invariants of `AdjointPDR`.

We implement this latter instance in Haskell. However, the implementation is not based on `AdjointPDR↓` directly, but rather on a third algorithm, `AdjointPDRAI`. This can be understood as a generalisation of both `AdjointPDR` and `AdjointPDR↓` to a more abstract setting:

$$b \bigcirc (L, \leq_L) \xrightarrow{\gamma} (C, \leq_C) \bigcirc \bar{b} \bar{\leq}_r$$

where $\gamma: L \rightarrow C$ is an order embedding and $b, \bar{b}$ and γ are required to satisfy a condition that is known in the setting of abstract interpretation as *forward completeness* [GRS00].

We experimentally evaluate our implementation. We compare it against existing probabilistic PDR algorithms (PrIC3 [Bat+20], LT-PDR [Kor+22]) and a non-PDR one (Storm [DJKV17]). The performance of `AdjointPDR↓` is encouraging—it supports the potential of PDR algorithms in probabilistic model checking. The experiments also indicate the importance of having a variety of heuristics, and thus the value of our adjoint framework that helps in coming up with those. Additionally, we found that abstraction features of Haskell allow us to code lattice-theoretic algorithms almost literally (~ 100 lines). Implementing a few heuristics takes another ~ 240 lines. This way, we found that mathematical abstraction can directly help in easing implementation effort.

7.2 Adjoint PDR

In this section we introduce `AdjointPDR`, an algorithm that takes in input a tuple (i, f, g, p) with $i, p \in L$ and $f \dashv i, g: L \rightarrow L$ and, if it terminates, it returns true whenever $\text{lfp}(f \vee i) \leq p$ and false otherwise. The algorithm manipulates two sequences of elements of L :

$$\vec{x} \triangleq x_0, \dots, x_{n-1} \quad \vec{y} \triangleq y_0, \dots, y_{n-1}$$

of length n and $n - k$, respectively. These satisfy, through the executions of `AdjointPDR`, the invariants in Figure 7.1. By (A1), x_j over-approximates the j -th element of the initial chain, namely $(f \vee i)^j(\perp) \leq x_j$, while, by (A3), the j -indexed element y_j of $\vec{y}$ over-approximates $g^{n-j-1}(p)$ that, borrowing the terminology of Example 3.11, is the set of states which are safe in $n - j - 1$ transitions. Moreover, by (PN), the element y_j witnesses that x_j is unsafe, i.e., that $x_j \not\leq g^{n-1-j}(p)$ or equivalently $f^{n-j-1}(x_j) \not\leq p$. Notably, $\vec{x}$ is a positive chain and $\vec{y}$ a negative sequence, according to the definitions below.

Definition 7.1 (positive chain). A *positive chain* for $\text{lfp}(f \vee i) \leq p$ is a finite chain $x_0 \leq \dots \leq x_{n-1}$ in L of length $n \geq 2$ which satisfies (P1), (P2), (P3) in Figure 7.1. It is *conclusive* if $x_{j+1} \leq x_j$ for some $j \leq n - 2$.

AdjointPDR (i, f, g, p)

```

<INITIALISATION>
  ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\perp, \top \parallel \varepsilon$ )2,2
<ITERATION>                                     %  $\vec{x}, \vec{y}$  not conclusive
  case ( $\vec{x} \parallel \vec{y}$ )n,k of
     $\vec{y} = \varepsilon$  and  $x_{n-1} \leq p$  :                % (Unfold)
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x}, \top \parallel \varepsilon$ )n+1,n+1
     $\vec{y} = \varepsilon$  and  $x_{n-1} \not\leq p$  :              % (Candidate)
      choose  $z \in L$  such that  $x_{n-1} \not\leq z$  and  $p \leq z$ ;
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x} \parallel z$ )n,n-1
     $\vec{y} \neq \varepsilon$  and  $f(x_{k-1}) \not\leq y_k$  :        % (Decide)
      choose  $z \in L$  such that  $x_{k-1} \not\leq z$  and  $g(y_k) \leq z$ ;
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x} \parallel z, \vec{y}$ )n,k-1
     $\vec{y} \neq \varepsilon$  and  $f(x_{k-1}) \leq y_k$  :        % (Conflict)
      choose  $z \in L$  such that  $z \leq y_k$  and  $(f \vee i)(x_{k-1} \wedge z) \leq z$ ;
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x} \wedge_k z \parallel \text{tail}(\vec{y})$ )n,k+1
  endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. x_{j+1} \leq x_j$  then return true %  $\vec{x}$  conclusive
  if  $i \not\leq y_1$  then return false %  $\vec{y}$  conclusive

```

Figure 7.2: AdjointPDR algorithm checking $\text{lfp}(f \vee i) \leq p$.

In a conclusive positive chain, x_{j+1} provides an invariant for $f \vee i$ and thus, by (2.2), $\text{lfp}(f \vee i) \leq p$ holds. So, when $\vec{x}$ is conclusive, **AdjointPDR** returns true.

Definition 7.2 (negative sequence). A *negative sequence* for $\text{lfp}(f \vee i) \leq p$ is a finite sequence $y_k, \dots, y_{n-1}$ in L with $1 \leq k \leq n$ which satisfies (N1) and (N2) in Figure 7.1. It is *conclusive* if $k = 1$ and $i \not\leq y_1$.

When $\vec{y}$ is conclusive, **AdjointPDR** returns false as y_1 provides a counterexample: (N1) and (N2) entail (A3) and thus $i \not\leq y_1 \geq g^{n-2}(p)$, so that $g^{n-2}(p) \geq \text{gfp}(g \wedge p)$ and thus $i \not\leq \text{gfp}(g \wedge p)$. By (2.1), $\text{lfp}(f \vee i) \not\leq p$.

The pseudocode of the algorithm is in Figure 7.2, where we write $(\vec{x} \parallel \vec{y})_{n,k}$ to compactly represent the state of the algorithm: the pair (n, k) is called the *index* of the state, with $\vec{x}$ of length n and $\vec{y}$ of length $n-k$. When $k = n$, $\vec{y}$ is the empty sequence ε . For any $z \in L$, we write $\vec{x}, z$ for the chain $x_0, \dots, x_{n-1}, z$ of length $n+1$ and $z, \vec{y}$ for the sequence $z, y_k, \dots, y_{n-1}$ of length $n-(k-1)$. Moreover, we write $\vec{x} \wedge_j z$ for the chain $x_0 \wedge z, \dots, x_j \wedge z, x_{j+1}, \dots, x_{n-1}$. Finally, $\text{tail}(\vec{y})$ stands for the tail of $\vec{y}$, namely $y_{k+1}, \dots, y_{n-1}$ of length $n-(k+1)$.

The algorithm starts in the initial state $s_0 \triangleq (\perp, \top \parallel \varepsilon)_{2,2}$ and, unless one of $\vec{x}$ and $\vec{y}$ is conclusive, iteratively applies one of the four mutually exclusive rules: (Unfold), (Candidate), (Decide) and (Conflict). The rule (Unfold) extends the positive chain by one element when the negative sequence is empty and the positive chain is under p ; since the element introduced by (Unfold) is $\top$, its application typically triggers rule (Candidate) that starts the negative sequence with an over-approximation of p . Recall that the role of y_j is to witness that x_j is unsafe. After (Candidate) either (Decide) or (Conflict) are possible: if y_k witnesses that, besides x_k , also $f(x_{k-1})$ is unsafe, then (Decide) is used to further extend the negative sequence to witness that x_{k-1} is unsafe; otherwise, the rule (Conflict) improves the precision of the positive chain in such a way that y_k no longer witnesses $x_k \wedge z$ unsafe and, thus, the negative sequence is shortened. Note that, in (Candidate), (Decide)

and (Conflict), the element $z \in L$ is chosen among a set of possibilities, thus **AdjointPDR** is nondeterministic.

To illustrate the executions of the algorithm, we adopt a labeled transition system notation. Let $\mathcal{S} \triangleq \{(\vec{x} \parallel \vec{y})_{n,k} \mid n \geq 2, k \leq n, \vec{x} \in L^n \text{ and } \vec{y} \in L^{n-k}\}$ be the set of all possible states of **AdjointPDR**. We call $(\vec{x} \parallel \vec{y})_{n,k} \in \mathcal{S}$ *conclusive* if $\vec{x}$ or $\vec{y}$ are such. When $s \in \mathcal{S}$ is not conclusive, we write $s \xrightarrow{D}$ to mean that s satisfies the guards in the rule (Decide), and $s \xrightarrow{D}_z s'$ to mean that, being (Decide) applicable, **AdjointPDR** moves from state s to s' by choosing z . Similarly for the other rules: the labels Ca , Co and U stands for (Candidate), (Conflict) and (Unfold), respectively. When irrelevant we omit to specify labels and choices and we just write $s \rightarrow s'$. As usual $\rightarrow^+$ stands for the transitive closure of $\rightarrow$ and $\rightarrow^*$ stands for the reflexive and transitive closure of $\rightarrow$.

Example 7.3. Consider the safety problem in Example 3.11. Below we illustrate two possible computations of **AdjointPDR** that differ for the choice of z in (Conflict). The first run is conveniently represented as the following series of transitions.

$$\begin{aligned} & (\emptyset, S \parallel \varepsilon)_{2,2} \xrightarrow{Ca}_P (\emptyset, S \parallel P)_{2,1} \xrightarrow{Co}_I (\emptyset, I \parallel \varepsilon)_{2,2} \\ & \xrightarrow{U} (\emptyset, I, S \parallel \varepsilon)_{3,3} \xrightarrow{Ca}_P (\emptyset, I, S \parallel P)_{3,2} \xrightarrow{Co}_{S_2} (\emptyset, I, S_2 \parallel \varepsilon)_{3,3} \\ & \xrightarrow{U} \xrightarrow{Ca}_P (\emptyset, I, S_2, S \parallel P)_{4,3} \xrightarrow{Co}_{S_3} (\emptyset, I, S_2, S_3 \parallel \varepsilon)_{4,4} \\ & \xrightarrow{U} \xrightarrow{Ca}_P (\emptyset, I, S_2, S_3, S \parallel P)_{5,4} \xrightarrow{Co}_{S_4} (\emptyset, I, S_2, S_3, S_4 \parallel \varepsilon)_{5,5} \\ & \xrightarrow{U} \xrightarrow{Ca}_P (\emptyset, I, S_2, S_3, S_4, S \parallel P)_{6,5} \xrightarrow{Co}_{S_4} (\emptyset, I, S_2, S_3, S_4, S_4 \parallel \varepsilon)_{6,6} \end{aligned}$$

The last state returns true since $x_4 = x_5 = S_4$. Observe that the chain $\vec{x}$, with the exception of its last element x_{n-1} , is exactly the initial chain of $(T \cup I)$, i.e., x_j is the set of states reachable in at most $j - 1$ steps. In the second computation, the elements of $\vec{x}$ are roughly those of the final chain of $(G \cap P)$. More precisely, after (Unfold) or (Candidate), x_{n-j} for $j < n - 1$ is the set of states which only reach safe states within j steps.

$$\begin{aligned} & (\emptyset, S \parallel \varepsilon)_{2,2} \xrightarrow{Ca}_P (\emptyset, S \parallel P)_{2,1} \xrightarrow{Co}_P (\emptyset, P \parallel \varepsilon)_{2,2} \\ & \xrightarrow{U} \xrightarrow{Ca}_P (\emptyset, P, S \parallel P)_{3,2} \xrightarrow{D}_{S_4} (\emptyset, P, S \parallel S_4, P)_{3,1} \xrightarrow{Co}_{S_4} (\emptyset, S_4, S \parallel P)_{3,2} \xrightarrow{Co}_P (\emptyset, S_4, P \parallel \varepsilon)_{3,3} \\ & \xrightarrow{U} \xrightarrow{Ca}_P (\emptyset, S_4, P, S \parallel P)_{4,3} \xrightarrow{D}_{S_4} (\emptyset, S_4, P, S \parallel S_4, P)_{4,2} \xrightarrow{Co}_{S_4} (\emptyset, S_4, S_4, S \parallel P)_{4,3} \end{aligned}$$

Observe that, by invariant (A1), the values of $\vec{x}$ in the two runs are, respectively, the least and the greatest values for all possible computations of **AdjointPDR**. ■

7.3 Properties of AdjointPDR

In this section we prove the main properties of **AdjointPDR**: (1) any returned result is valid (soundness); (2) although **AdjointPDR** can diverge, any state is never visited twice (called progression); (3) certain heuristics can be used to guarantee termination when a counterexample exists (called negative termination).

7.3.1 Invariants

The proofs of the properties of **AdjointPDR** rely on the properties in Figure 7.1. In this section, we prove that such properties are invariants:

Proposition 7.4. *For any possible choice performed by **AdjointPDR**, the properties in Figure 7.1 hold in all reachable states of the algorithm.*

In proving the invariants, some observations on the choice of element z naturally emerge. First, the proofs of the three invariants (I0), (I1) and (I2) do not rely on the properties of the chosen element $z \in L$. For proving the invariants of the positive chain ((P1), (P2), (P3) and (P3a)) and of the negative sequence ((N1) and (N2)) we only exploit the *second* constraints on z of each rule of the algorithm, namely $p \leq z$ in (Candidate), $g(y_k) \leq z$ in (Decide), and $(f \vee i)(x_{k-1} \wedge z) \leq z$ in (Conflict). Lastly, the *first* constraint on z in each rule ensures the remaining invariants ((PN), (A1), (A2) and (A3)), which in turn are key to the proof of progression.

To make the proofs more uniform and compact, we adopt the following notation: for a state s and a property (Q) we will write $s \models (Q)$ to mean that (Q) holds in s . We will often show that (Q) is an invariant inductively: namely, we will prove

- (a) $s_0 \models (Q)$ and
- (b) if $s \models (Q)$ and $s \rightarrow s'$, then $s' \models (Q)$.

Hereafter, we fix $s = (\vec{x} \parallel \vec{y})_{n,k}$ and $s' = (\vec{x}' \parallel \vec{y}')_{n',k'}$. As usual we will write x_j and y_j for the elements of $\vec{x}$ and $\vec{y}$. For the elements of $\vec{x}'$ and $\vec{y}'$, we will write x'_j and y'_j . Throughout the proofs, we will avoid to repeat every time in (b) that $s \models (Q)$, and we will just write $\stackrel{(Q)}{=}$ or $\stackrel{(Q)}{\leq}$ whenever using such hypothesis. Moreover in (b) we will avoid to specify those cases that are trivial: for instance, for the properties that only concerns the positive chain $\vec{x}$, e.g., (I0) and (P3), it is enough to check the property (b) for $s \xrightarrow{U} s'$ and $s \xrightarrow{Co} s'$, since $s \xrightarrow{D} s'$ and $s \xrightarrow{Ca} s'$ only modify the negative sequence $\vec{y}$. We illustrate below only the most interesting cases. The remaining ones are in Appendix D.

Proof sketch. Case (I0): $x_0 = \perp$

- (a) In s_0 , $x_0 = \perp$.
- (b) If $s \xrightarrow{U} s'$, then $x'_0 = x_0 \stackrel{(I0)}{=} \perp$.
If $s \xrightarrow{Co}_z s'$, then $x'_0 = x_0 \wedge z \stackrel{(I0)}{=} \perp \wedge z = \perp$.

Case (I1): $1 \leq k \leq n$

- To prove that $1 \leq k$, observe that k is initialised at 2 and that it is only decremented by 1. When $k = 1$, $\vec{y} \neq \varepsilon$. By (I0) $x_0 = \perp$. Since f is a left adjoint, $f(\perp) = \perp$. Thus, $f(x_0) \leq y_1$. This means that either the state is conclusive and the algorithm returns, or (Conflict) is enabled and thus k is incremented.
- To prove that $k \leq n$, observe that k is incremented only by 1. When $k = n$, the algorithm does either (Unfold) or (Candidate). In the latter case, k is decremented. In the former, both n and k are incremented.

Case (P3): $\forall j \in [0, n-2], f(x_j) \leq x_{j+1}$

- (a) In s_0 , since $n = 2$ one needs to check only the case $j = 0$: $f(x_0) \leq \top = x_1$.
- (b) If $s \xrightarrow{U} s'$, then $f(x'_j) = f(x_j) \stackrel{(I2)}{\leq} x_{j+1} = x'_{j+1}$ for all $j \in [0, n-2]$. For $j = n-1$, $f(x'_{n-1}) = f(x_{n-1}) \leq \top = x'_{j+1}$. Since $n' = n+1$, then $\forall j \in [0, n'-2], f(x'_j) \leq x'_{j+1}$.
If $s \xrightarrow{Co}_z s'$, since $f(x_{k-1} \wedge z) \leq z$, then by (I2) and monotonicity of f it holds that $\forall j \in [0, k-1], f(x_j \wedge z) \leq z$. Since $f(x_j \wedge z) \leq f(x_j) \stackrel{(P3)}{\leq} x_{j+1}$, it holds that $f(x_j \wedge z) \leq x_{j+1} \wedge z$ for all $j \in [0, k-1]$. With this observation is immediate to conclude that $\forall j \in [0, n'-2], f(x'_j) \leq x'_{j+1}$.

Case (N2): $\forall j \in [k, n-2], g(y_{j+1}) \leq y_j$

The case of (Conflict) is trivial: the negative sequence $\vec{y}$ is truncated in the rule (Conflict), and if the invariant holds for $\vec{y}$ then it holds for its tail $\text{tail}(\vec{y})$ as well.

(a) In s_0 , $k = 2$ and $n = 2$. Thus (N2) trivially holds.

(b) If $s \xrightarrow{C_a} s'$, then $k' = n - 1$ and thus (N2) trivially holds.

If $s \xrightarrow{D_z} s'$, since $z \geq g(y_k)$ and $k' = k - 1$, then $y'_{k'} = y'_{k-1} = z \geq g(y_k) = g(y'_k) = g(y'_{k'+1})$. For $j \in [k' + 1, n - 2]$, namely for $j \in [k, n - 2]$, it holds that $y'_j = y_j \stackrel{(N2)}{\geq} g(y_{j+1}) = g(y'_{j+1})$. Thus, $\forall j \in [k', n - 2], g(y'_{j+1}) \leq y'_j$.

Case (PN): $\forall j \in [k, n - 1], x_j \not\leq y_j$

(a) In s_0 , $k = n$ and thus (PN) trivially holds.

(b) If $s \xrightarrow{U} s'$, then $k' = n'$ and thus (PN) trivially holds.

If $s \xrightarrow{C_a} s'$, since $x_{n-1} \not\leq z$, $x'_{n-1} = x_{n-1}$ and $k' = n' - 1 = n - 1$, then $x'_{n'-1} = x_{n-1} \not\leq z = y'_{n'-1}$.

If $s \xrightarrow{D_z} s'$, since $x_{k-1} \not\leq z$, then $x'_{k-1} = x_{k-1} \not\leq z = y'_{k-1}$. Moreover, $\forall j \in [k, n - 1]$, $x'_j = x_j \stackrel{(PN)}{\not\leq} y_j = y'_j$. Thus, $\forall j \in [k', n' - 1], x'_j \not\leq y'_j$.

If $s \xrightarrow{C_o} s'$, then $k' = k + 1$ and $n' = n$. Observe that for $j \in [k + 1, n - 1]$, $x'_j = x_j \stackrel{(PN)}{\not\leq} y_j = y'_j$. Thus $\forall j \in [k', n' - 1], x'_j \not\leq y'_j$.

□

7.3.2 Soundness

Once the properties in Figure 7.1 are proved to be invariants, the proof of soundness of **AdjointPDR** is rather straightforward: it only appeals to the Knaster-Tarski fixed-point theorem for the positive case, and to the Kleene one for the negative case.

Theorem 7.5 (Soundness). *AdjointPDR is sound, namely,*

1. If **AdjointPDR** returns true then $\text{lfp}(f \vee i) \leq p$.
2. If **AdjointPDR** returns false then $\text{lfp}(f \vee i) \not\leq p$.

Proof. We prove the two items separately.

1. Observe that **AdjointPDR** returns true if $x_{j+1} \leq x_j$. By (P3), we thus have $f(x_j) \leq x_{j+1} \leq x_j$. Moreover, by (P1) and (I2), it holds that $i \leq x_j$ and $x_j \leq p$. Therefore, it holds that

$$(f \vee i)x_j \leq x_j \leq p.$$

By (2.2), we have that $\text{lfp}(f \vee i) \leq p$.

2. Observe that **AdjointPDR** returns false if $i \not\leq y_1$. By (A3), $g^{n-2}(p) \leq y_1$. Thus $i \not\leq g^{n-2}(p)$. Moreover

$$\begin{aligned} g^{n-2}p &\leq \bigwedge_{j \in \omega} g^j(p) \\ &= \text{gfp}(g \wedge p) \end{aligned}$$

Thus $i \not\leq \text{gfp}(g \wedge p)$. By (2.1), $\text{lfp}(f \vee i) \not\leq p$.

□

7.3.3 Progression

It is necessary to prove that in any step of the execution, if the algorithm does not return true or false, then it can progress to a new state, not yet visited. To this aim we must deal with the subtleties of the non-deterministic choice of the element z in (Candidate), (Decide) and (Conflict). The following proposition ensures that, for any of these three rules, there is always a possible choice.

Proposition 7.6 (Canonical choices). *The following choices of z are always possible:*

1. in (Candidate) $z = p$;
2. in (Decide) $z = g(y_k)$;
3. in (Conflict) $z = y_k$;
4. in (Conflict) $z = (f \vee i)(x_{k-1})$.

Thus, for all non-conclusive $s \in \mathcal{S}$, if $s_0 \rightarrow^* s$ then $s \rightarrow$.

Proof. For each rule, we prove that if the guard of the rule is satisfied then the choice of z satisfies the required constraints.

1. The guard of (Candidate) is $x_{n-1} \not\leq p$. By choosing $z = p$, one has that $x_{n-1} \not\leq z$ and $p \leq z$ are trivially satisfied;
2. The guard of (Decide) is $f(x_{k-1}) \not\leq y_k$ thus, by $f \dashv g$, $x_{k-1} \not\leq g(y_k)$. By choosing $z = g(y_k)$, one has that $x_{k-1} \not\leq z$ and $g(y_k) \leq z$;

The proofs for the choices in (Conflict) are more subtle. First of all, observe that if $k = 1$, then $i \leq y_1$ otherwise the algorithm would have returned false. Moreover, for $k \geq 2$, we have that $i \leq x_{k-1} \leq y_k$: the first inequality holds by (P1) and the second by (A2) and (A3). In summary,

$$\forall j \geq 1. i \leq y_j. \quad (7.1)$$

We can then proceed as follows.

3. The guard of (Conflict) is $f(x_{k-1}) \leq y_k$. By choosing $z = y_k$, one has that $z \leq y_k$ trivially holds. For $(f \vee i)(x_{k-1} \wedge z) \leq z$ observe that

$$\begin{aligned} (f \vee i)(x_{k-1} \wedge z) &= f(x_{k-1} \wedge z) \vee i && [\text{def.}] \\ &\leq f(x_{k-1}) \vee i && [\text{monotonicity}] \\ &\leq z \vee i && [\text{guard}] \\ &= z && [(7.1)] \end{aligned}$$

4. The guard of (Conflict) is $f(x_{k-1}) \leq y_k$. By choosing $z = (f \vee i)(x_{k-1})$, one has that $(f \vee i)(x_{k-1} \wedge z) \leq (f \vee i)(x_{k-1}) = z$ holds by monotonicity. For $z \leq y_k$, by using the guard and (7.1), we have that $z = (f \vee i)(x_{k-1}) = f(x_{k-1}) \vee i \leq y_k$.

□

The following proposition ensures that **AdjointPDR** always traverses new states.

Proposition 7.7 (Impossibility of loops). *If $s_0 \rightarrow^* s \rightarrow^+ s'$, then $s \neq s'$.*

Proof. Let us consider the following partial order on positive chains: given two sequences $\vec{x} = x_0, \dots, x_{n-1}$ and $\vec{x}' = x'_0, \dots, x'_{n'-1}$, we say $\vec{x} \preceq \vec{x}'$ if

$$n \leq n' \wedge x_j \geq x'_j \text{ for each } j \in [0, n-1]$$

We extend the order to states by letting $(\vec{x} \parallel \vec{y})_{n,k} \preceq (\vec{x}' \parallel \vec{y}')_{n',k'}$ with $\vec{x} \prec \vec{x}'$ or $\vec{x} = \vec{x}'$ and $k \geq k'$.

We prove the statement by showing that applying a rule strictly increases the state in that partial order. As before, we use non-primed variables such as $\vec{x}$ for values before the application of a rule, and primed variables such as $\vec{x}'$ after.

For (Unfold), we have that $n < n' = n + 1$ and $x_j = x'_j$ for each $j \in [0, n-1]$.

For (Candidate), we have $\vec{x}' = \vec{x}$ and $k' = n - 1 < n = k$.

For (Decide), we have $\vec{x}' = \vec{x}$ and $k' = k - 1 < k$.

For (Conflict), $n = n'$, and

$$x'_j = \begin{cases} x_j & \text{if } j > k \\ x_j \wedge z & \text{if } j \leq k \end{cases}$$

So for $j \in [k+1, n-1]$ we have $x_j = x'_j$, and for $j \in [0, k]$ we have $x_j \geq x_j \wedge z = x'_j$. So $\vec{x} \preceq \vec{x}'$. Assume by contradiction that $x'_k = x_k$. Since $x'_k = x_k \wedge z$, this is equivalent to $x_k \leq z$. The choice of z in (Conflict) satisfies $z \leq y_k$, that would imply $x_k \leq z \leq y_k$. However, this is a contradiction, since by (PN) we know $x_k \not\leq y_k$. Hence $x_k \geq x'_k$, meaning $\vec{x} \prec \vec{x}'$. $\square$

Observe that the above propositions entail that **AdjointPDR** terminates whenever the lattice L is finite, since the set of reachable states is finite in this case.

Example 7.8. For (I, T, G, P) as in Example 3.11, **AdjointPDR** behaves essentially as IC3/PDR [Bra11], solving reachability problems for transition systems with finite state space S . Since the lattice $\mathcal{PS}$ is also finite, **AdjointPDR** always terminates. $\blacksquare$

7.3.4 Heuristics

The nondeterministic choices of the algorithm can be resolved by using heuristics. Intuitively, a heuristic chooses for any states $s \in \mathcal{S}$ an element $z \in L$ to be possibly used in (Candidate), (Decide) or (Conflict), so it is just a function $h: \mathcal{S} \rightarrow L$. When defining a heuristic, we will avoid to specify its values on conclusive states or in those performing (Unfold), as they are clearly irrelevant.

With a heuristic, one can instantiate **AdjointPDR** by making the choice of z as prescribed by h . Syntactically, this means to erase from the code of Figure 7.2 the three lines of **choose** and replace them with $z := h((\vec{x} \parallel \vec{c})_{n,k})$. We call **AdjointPDR_h** the resulting deterministic algorithm and write $s \rightarrow_h s'$ to mean that **AdjointPDR_h** moves from state s to s' . We let $\mathcal{S}^h \triangleq \{s \in \mathcal{S} \mid s_0 \rightarrow_h^* s\}$ be the sets of all states reachable by **AdjointPDR_h**.

Definition 7.9 (legit heuristic). A heuristic $h: \mathcal{S} \rightarrow L$ is called *legit* whenever for all $s, s' \in \mathcal{S}^h$, if $s \rightarrow_h s'$ then $s \rightarrow s'$.

When h is legit, the only execution of the deterministic algorithm **AdjointPDR_h** is one of the possible executions of the non-deterministic algorithm **AdjointPDR**.

The canonical choices provide two legit heuristics: first, we call *simple* any legit heuristic h that chooses z in (Candidate) and (Decide) as in Proposition 7.6:

$$(\vec{x} \parallel \vec{y})_{n,k} \mapsto \begin{cases} p & \text{if } (\vec{x} \parallel \vec{y})_{n,k} \xrightarrow{Ca} \\ g(y_k) & \text{if } (\vec{x} \parallel \vec{y})_{n,k} \xrightarrow{D} \end{cases} \quad (7.2)$$

Then, if the choice in (Conflict) is like in Proposition 7.6.4, we call h *initial*; if it is like in Proposition 7.6.3, we call h *final*. Shortly, the two legit heuristics are:

<i>simple initial</i>	(7.2) and $(\vec{x} \parallel \vec{y})_{n,k} \mapsto (f \vee i)(x_{k-1})$	if $(\vec{x} \parallel \vec{y})_{n,k} \in Co$
<i>simple final</i>	(7.2) and $(\vec{x} \parallel \vec{y})_{n,k} \mapsto y_k$	if $(\vec{x} \parallel \vec{y})_{n,k} \in Co$

Interestingly, with any simple heuristic, the sequence $\vec{y}$ takes a familiar shape:

Proposition 7.10. *Let $h: \mathcal{S} \rightarrow L$ be any simple heuristic. For all $(\vec{x} \parallel \vec{y})_{n,k} \in \mathcal{S}^h$, invariant (A3) holds as an equality, namely for all $j \in [k, n-1]$, $y_j = g^{n-1-j}(p)$.*

Proof. As for the invariants, we prove this equality by induction showing

- (a) it holds for s_0 and
- (b) if it holds for s and $s \rightarrow s'$, then it holds for s' .

In s_0 and after (Unfold), since $k = n$ there is no $j \in [k, n-1]$.

For (Conflict), since the property holds on $\vec{y}$ it also holds on $\vec{y}' = \text{tail}(\vec{y})$.

For (Candidate), $\vec{y}' = p$ and $k' = n-1$, so the thesis holds because $y_{n-1} = p = g^{n-1-(n-1)}p$.

For (Decide), $\vec{y}' = g(y_k), \vec{y}$ and $k' = k-1$. For all $j \in [k'+1, n-1]$ the thesis holds because $y'_j = y_j$. For $j = k'$, we have $y_{k'} = g(y_k) = g(g^{n-1-k}(p)) = g^{n-1-k'}(p)$. $\square$

By the above proposition and (A3), the negative sequence $\vec{y}$ occurring in the execution of AdjointPDR_h , for a simple heuristic h , is the least amongst all the negative sequences occurring in any execution of AdjointPDR . Instead, invariant (A1) informs us that the positive chain $\vec{x}$ is always in between the initial chain of $f \vee i$ and the final chain of $g \wedge p$. Such values of $\vec{x}$ are obtained by, respectively, simple initial and simple final heuristic. This is formally shown in Propositions 7.12 and 7.13 below.

Example 7.11. Consider the two runs of AdjointPDR in Example 7.3. The first one exploits the simple initial heuristic and indeed, the positive chain $\vec{x}$ coincides with the initial chain. Analogously, the second run uses the simple final heuristic. $\blacksquare$

Proposition 7.12. *Assume $p \neq \top$ and let $h: \mathcal{S} \rightarrow L$ be any simple initial heuristic. For all $(\vec{x} \parallel \vec{y})_{n,k} \in \mathcal{S}^h$, the first inequality in (A1) holds as an equality for all $j \in [0, n-2]$, namely $x_j = (f \vee i)^j(\perp)$.*

Proposition 7.13. *Assume $p \neq \top$ and let $h: \mathcal{S} \rightarrow L$ be any simple final heuristic. If $s_0 \xrightarrow{*} \xrightarrow{U} (\vec{x} \parallel \vec{y})_{n,k}$ then the second inequality in (A1) holds as an equality, namely for all $j \in [1, n-1]$, $x_j = (g \wedge p)^{n-1-j}(\top)$.*

7.3.5 Negative Termination

When the lattice L is not finite, AdjointPDR may not terminate, since checking $\text{lfp}(f \vee i) \leq p$ is not always decidable. In this section, we show that the use of certain heuristics can guarantee termination whenever $\text{lfp}(f \vee i) \not\leq p$. The key insight is the following: if $\text{lfp}(f \vee i) \not\leq p$ then by (2.3), there should exist some $\tilde{n} \in \mathbb{N}$ such that $(f \vee i)^{\tilde{n}}(\perp) \not\leq p$. By (A1), the rule (Unfold) can be applied only when $(f \vee i)^{n-1}(\perp) \leq x_{n-1} \leq p$. Since (Unfold) increases n and n is never decreased by other rules, then (Unfold) can be applied at most $\tilde{n}$ times. Therefore, we can guarantee termination whenever the number of steps between two (Unfold) is finite.

The first observation for termination is the following lemma. It states that an element z cannot be added twice to negative sequence until n is increased, i.e., until (Unfold) is applied.

Lemma 7.14. *If $s_0 \rightarrow^* s \xrightarrow{D}_z \rightarrow^* s' \xrightarrow{D}_{z'}$ and s and s' carry the same index (n, k) , then $z' \neq z$. Similarly, if $s_0 \rightarrow^* s \xrightarrow{Ca}_z \rightarrow^* s' \xrightarrow{Ca}_{z'}$ and s and s' carry the same index (n, k) , then $z' \neq z$.*

Elements of negative sequences are introduced by rules (Candidate) and (Decide). If we guarantee that for any index (n, k) the heuristic in such cases returns a finite number of values for z , then we can prove termination. To make this formal, we fix $CaD_{n,k}^h \triangleq \{(\vec{x} \parallel \vec{y})_{n,k} \in \mathcal{S}^h \mid (\vec{x} \parallel \vec{y})_{n,k} \xrightarrow{Ca} \text{ or } (\vec{x} \parallel \vec{y})_{n,k} \xrightarrow{D}\}$, i.e., the set of all (n, k) -indexed states reachable by AdjointPDR_h that trigger (Candidate) or (Decide), and $h(CaD_{n,k}^h) \triangleq \{h(s) \mid s \in CaD_{n,k}^h\}$, i.e., the set of all possible values returned by h in such states.

Theorem 7.15 (Negative termination). *Let h be a legit heuristic. If $h(CaD_{n,k}^h)$ is finite for all n, k and $\text{lfp}(f \vee i) \not\leq p$, then AdjointPDR_h terminates.*

Corollary 7.16. *Let h be a simple heuristic. If $\text{lfp}(f \vee i) \not\leq p$, then AdjointPDR_h terminates.*

Note that this corollary ensures negative termination whenever we use the canonical choices in (Candidate) and (Decide) *irrespective of the choice for (Conflict)*, therefore it holds for both simple initial and simple final heuristics.

7.3.6 The meet-semilattice of positive chains and the join-semilattice of negative sequences

We conclude this section with two results illustrating some algebraic properties of positive chains and negative sequences. These are not necessary for proving properties of AdjointPDR , but they will be quite convenient in Section 7.4.2.

We observe that positive chains of a fixed length n form a join-semilattice and negative sequences a meet-semilattices, where joins and meets are defined point-wise, i.e., for two positive chains $\vec{x}^1, \vec{x}^2$ their join is defined as $(\vec{x}^1 \vee \vec{x}^2)_j \triangleq x_j^1 \vee x_j^2$, and similarly for negative sequences. To show this it suffices to prove that the join of an arbitrary set of positive chains (resp. the meet of an arbitrary set of negative sequences) is still a positive chain (resp. negative sequence).

Lemma 7.17. *Let I be a set. For all $m \in I$, let $\vec{x}^m = x_0^m, \dots, x_{n-1}^m$ be a positive chain. Then, the chain $\bigvee_{m \in I} \vec{x}^m$ defined for all $j \in [0, n-1]$ as*

$$(\bigvee_{m \in I} \vec{x}^m)_j \triangleq \bigvee_{m \in I} x_j^m$$

is a positive chain.

Proof. Since $i \leq x_1^m$ for all $m \in I$, then $i \leq \bigvee_{m \in I} x_1^m$. Since $x_{n-2}^m \leq p$ for all $m \in I$, then $\bigvee_{m \in I} x_{n-2}^m \leq p$.

To show that $f((\bigvee_{m \in I} x^{\vec{m}})_j) \leq (\bigvee_{m \in I} x^{\vec{m}})_{j+1}$ we just observe the following

$$\begin{aligned}
 f((\bigvee_{m \in I} x^{\vec{m}})_j) &= f(\bigvee_{m \in I} x_j^m) && [\text{def.}] \\
 &= \bigvee_{m \in I} f(x_j^m) && [f \dashv g] \\
 &\leq \bigvee_{m \in I} x_{j+1}^m && [(P3)] \\
 &= (\bigvee_{m \in I} x^{\vec{m}})_{j+1} && [\text{def.}]
 \end{aligned}$$

Thus (P1), (P2) and (P3) hold for $\bigvee_{m \in I} x^{\vec{m}}$. $\square$

Lemma 7.18. *Let I be a set. For all $m \in I$, let $y^{\vec{m}} = y_k^m, \dots, y_{n-1}^m$ be a negative sequence. Then, the sequence $\bigwedge_{m \in I} y^{\vec{m}}$ defined for all $j = 0, \dots, n-1$ as*

$$(\bigwedge_{m \in I} y^{\vec{m}})_j \triangleq \bigwedge_{m \in I} y_j^m$$

is a negative sequence. Moreover, if $y^{\vec{m}}$ is conclusive for all $m \in I$, then also $\bigwedge_{m \in I} y^{\vec{m}}$ is conclusive.

Proof. Since $p \leq y_{n-1}^m$ for all $m \in I$, then $p \leq \bigwedge_{m \in I} y_{n-1}^m$.

To show that $g(\bigwedge_{m \in I} y^{\vec{m}})_{j+1} \leq (\bigwedge_{m \in I} y^{\vec{m}})_j$ we proceed as follows

$$\begin{aligned}
 g(\bigwedge_{m \in I} y^{\vec{m}})_{j+1} &= g(\bigwedge_{m \in I} y_j^m) && [\text{def.}] \\
 &= \bigwedge_{m \in I} g(y_{j+1}^m) && [f \dashv g] \\
 &\leq \bigwedge_{m \in I} y_j^m && [(N2)] \\
 &= (\bigwedge_{m \in I} y^{\vec{m}})_j && [\text{def.}]
 \end{aligned}$$

For conclusive sequences, observe that, since $i \not\leq y_1^m$ for all $m \in I$, then $i \not\leq \bigwedge_{m \in I} y_1^m = (\bigwedge_{m \in I} y^{\vec{m}})_1$. $\square$

The bottom element of the meet-semilattice of negative sequences is given by $g^{n-1-j}(p)$ for all $j \in [k, n-1]$, and is exactly the one in invariant (A3). The top element of the join-semilattice of positive chains is the chain defined as $(g \wedge p)^{n-1-j}(\top)$ for all $j \in [0, n-1]$; its bottom element is the chain $(f \vee i)^j(\perp)$. Again, these are exactly the bounds that appear in invariant (A1). Note that, if $y^{\vec{1}}$ and $y^{\vec{2}}$ are conclusive, also $y^{\vec{1}} \wedge y^{\vec{2}}$ is conclusive. An analogous property for positive chains does not hold.

7.4 AdjointPDR $^\downarrow$

In Section 7.2, we have introduced an algorithm for checking $\text{lfp}(b) \leq p$ whenever b is of the form $f \vee i$ for an element $i \in L$ and a left-adjoint $f: L \rightarrow L$. This, unfortunately, is not the case for several interesting problems, like the max reachability problem for Markov Decision Processes [BK08] that we will illustrate in Section 7.5.

The next result informs us that, under standard assumptions, one can transfer the problem of checking $\text{lfp}(b) \leq p$ to lower sets, where adjoints can always be defined. Recall that, for a lattice $(L, \leq)$, a *lower set* is a subset $X \subseteq L$ such that if $x \in X$ and $x' \leq x$ then $x' \in X$; the set of lower sets of L forms a complete lattice $(L^\downarrow, \subseteq)$ with joins and meets given by union and intersection; as expected $\perp$ is $\emptyset$ and $\top$ is L . Given $b: L \rightarrow L$, one can define two functions $b^\downarrow, b_r^\downarrow: L^\downarrow \rightarrow L^\downarrow$ as $b^\downarrow(X) \triangleq b(X)^\downarrow$ and $b_r^\downarrow(X) \triangleq \{x \mid b(x) \in X\}$. It holds that $b^\downarrow \dashv b_r^\downarrow$.

$$b \circlearrowleft (L, \leq) \xleftarrow[\begin{smallmatrix} \sqcup \\ \perp \\ (-)^\downarrow \end{smallmatrix}]{\sqcup} (L^\downarrow, \subseteq) \circlearrowright b^\downarrow \dashv b_r^\downarrow \quad (7.3)$$

In the diagram above, $(-)^\downarrow: x \mapsto \{x' \mid x' \leq x\}$ and $\sqcup: L^\downarrow \rightarrow L$ maps a lower set X into $\sqcup\{x \mid x \in X\}$. The maps $\sqcup$ and $(-)^\downarrow$ form a *Galois insertion*, namely $\sqcup \dashv (-)^\downarrow$ and $\sqcup(-)^\downarrow = id$, and thus one can think of (7.3) in terms of abstract interpretation: $L^\downarrow$ represents the concrete domain, L the abstract domain and b is a sound abstraction of $b^\downarrow$. Moreover, b is *forward-complete* [GRS00; BGGP18] w.r.t. $b^\downarrow$, namely:

$$(-)^\downarrow \circ b = b^\downarrow \circ (-)^\downarrow \quad (7.4)$$

Proposition 7.19. *Let $(L, \leq)$ be a complete lattice, $p \in L$ and $b: L \rightarrow L$ be a ω -continuous map. Then $\text{lfp}(b) \leq p$ iff $\text{lfp}(b^\downarrow \cup \perp^\downarrow) \subseteq p^\downarrow$.*

Proof. A simple inductive argument using (7.4) confirms that

$$(b^n x)^\downarrow = (b^\downarrow)^n x^\downarrow \quad (7.5)$$

for all $x \in L$. The following sequence of logical equivalences

$$\begin{aligned} \text{lfp}(b) \leq p &\Leftrightarrow \forall n \in \mathbb{N}. b^n \perp \leq p && [\text{Theorem 2.3}] \\ &\Leftrightarrow \forall n \in \mathbb{N}. (b^n \perp)^\downarrow \subseteq p^\downarrow && [\text{mon. of } (-)^\downarrow, \sqcup \text{ and } \sqcup(-)^\downarrow = id] \\ &\Leftrightarrow \bigcup_{n \in \mathbb{N}} (b^n \perp)^\downarrow \subseteq p^\downarrow && [\text{def. of } \bigcup] \\ &\Leftrightarrow \bigcup_{n \in \mathbb{N}} (b^\downarrow)^n \perp^\downarrow \subseteq p^\downarrow && [(7.5)] \\ &\Leftrightarrow \text{lfp}(b^\downarrow \cup \perp^\downarrow) \subseteq p^\downarrow && [\text{Theorem 2.3}]. \end{aligned}$$

concludes the proof of the main statement. $\square$

By means of Proposition 7.19, we can thus solve $\text{lfp}(b) \leq p$ in L by running **AdjointPDR** on $(\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow)$. Hereafter, we tacitly assume that b is ω -continuous.

7.4.1 AdjointPDR[↓]: Positive Chain in L , Negative Sequence in $L^\downarrow$

While **AdjointPDR** on $(\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow)$ might be computationally expensive, it is the first step toward an efficient algorithm that exploits a convenient form of the positive chain.

A lower set $X \in L^\downarrow$ is said to be a *principal* if $X = x^\downarrow$ for some $x \in L$. Observe that the top of the lattice $(L^\downarrow, \subseteq)$ is a principal, namely $\top^\downarrow$, and that the meet (intersection) of two principals $x^\downarrow$ and $y^\downarrow$ is the principal $(x \wedge y)^\downarrow$.

Suppose that, in (Conflict), **AdjointPDR** $(\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow)$ always chooses principals rather than arbitrary lower sets. This suffices to guarantee that all the elements of $\vec{x}$ are principals (with the only exception of x_0 which is constantly the bottom element of $L^\downarrow$ that, note,

AdjointPDR[↓] (b, p)

```

<INITIALISATION>
   $(\vec{x} \parallel \vec{Y})_{n,k} := (\emptyset, \perp, \top \parallel \varepsilon)_{3,3}$ 
<ITERATION>
  case  $(\vec{x} \parallel \vec{Y})_{n,k}$  of
    %  $\vec{x}, \vec{Y}$  not conclusive
     $\vec{Y} = \varepsilon$  and  $x_{n-1} \leq p$  :                               % (Unfold)
       $(\vec{x} \parallel \vec{Y})_{n,k} := (\vec{x}, \top \parallel \varepsilon)_{n+1, n+1}$ 
     $\vec{Y} = \varepsilon$  and  $x_{n-1} \not\leq p$  :                               % (Candidate)
      choose  $Z \in L^\downarrow$  such that  $x_{n-1} \notin Z$  and  $p \in Z$ ;
       $(\vec{x} \parallel \vec{Y})_{n,k} := (\vec{x} \parallel Z)_{n, n-1}$ 
     $\vec{Y} \neq \varepsilon$  and  $b(x_{k-1}) \notin Y_k$  :                               % (Decide)
      choose  $Z \in L^\downarrow$  such that  $x_{k-1} \notin Z$  and  $b_r^\downarrow(Y_k) \subseteq Z$ ;
       $(\vec{x} \parallel \vec{Y})_{n,k} := (\vec{x} \parallel Z, \vec{Y})_{n, k-1}$ 
     $\vec{Y} \neq \varepsilon$  and  $b(x_{k-1}) \in Y_k$  :                               % (Conflict)
      choose  $z \in L$  such that  $z \in Y_k$  and  $b(x_{k-1} \wedge z) \leq z$ ;
       $(\vec{x} \parallel \vec{Y})_{n,k} := (\vec{x} \wedge_k z \parallel \text{tail}(\vec{Y}))_{n, k+1}$ 
  endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. x_{j+1} \leq x_j$  then return true %  $\vec{x}$  conclusive
  if  $Y_1 = \emptyset$  then return false %  $\vec{Y}$  conclusive

```

Figure 7.3: The algorithm $\text{AdjointPDR}^\downarrow$ for checking $\text{lfp}(b) \leq p$: the elements of negative sequence are in $L^\downarrow$, while those of the positive chain are in L , with the only exception of x_0 which is constantly the bottom lower set $\emptyset$. For x_0 , we fix $b(x_0) = \perp$.

is $\emptyset$ and not $\perp^\downarrow$). In fact, the elements of $\vec{x}$ are all obtained by (Unfold), that adds the principal $\top^\downarrow$, and by (Conflict), that takes their meets with the chosen principal.

Since principals are in bijective correspondence with the elements of L , by imposing to $\text{AdjointPDR}(\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow)$ to choose a principal in (Conflict), we obtain an algorithm, named $\text{AdjointPDR}^\downarrow$, where the elements of the positive chain are drawn from L , while the negative sequence is taken in $L^\downarrow$. The algorithm is reported in Figure 7.3 where we use the notation $(\vec{x} \parallel \vec{Y})_{n,k}$ to emphasize that the elements of the negative sequence are lower sets of elements in L .

All definitions and results illustrated in Sections 7.2 and 7.3 for AdjointPDR are inherited by $\text{AdjointPDR}^\downarrow$, with the only exception of Proposition 7.6.3. This does not hold because it prescribes a choice for (Conflict) that may not be a principal. In contrast, the choice in Proposition 7.6.4 is, thanks to (7.4), a principal. This means in particular that the simple initial heuristic is always applicable.

Theorem 7.20. *All results in Section 7.3, but Proposition 7.6.3, hold for $\text{AdjointPDR}^\downarrow$.*

7.4.2 $\text{AdjointPDR}^\downarrow$ simulates LT-PDR

The closest approach to AdjointPDR and $\text{AdjointPDR}^\downarrow$ is the lattice-theoretic extension of the original PDR, called LT-PDR [Kor+22]. While these algorithms exploit essentially the same positive chain to find an invariant, the main difference lies in the sequence used to witness the existence of some counterexamples.

Definition 7.21 (Kleene sequence, from [Kor+22]). A sequence $\vec{c} = c_k, \dots, c_{n-1}$ of elements of L is a *Kleene sequence* if the conditions (C1) and (C2) below hold. It is *conclusive* if also condition (C0) holds.

$$(C0) \ c_1 \leq b(\perp), \quad (C1) \ c_{n-1} \not\leq p, \quad (C2) \ \forall j \in [k, n-2]. \ c_{j+1} \leq b(c_j).$$

LT-PDR tries to construct an under-approximation c_{n-1} of $b^{n-2}(\perp)$ that violates the property p . The Kleene sequence is constructed by trial and error, starting by some arbitrary choice of c_{n-1} .

AdjointPDR crucially differs from LT-PDR in the search for counterexamples: LT-PDR under-approximates the final chain while AdjointPDR over-approximates it. However, we can draw a formal correspondence between AdjointPDR[↓] and LT-PDR by showing that AdjointPDR[↓] simulates LT-PDR, but cannot be simulated by LT-PDR. In fact, AdjointPDR[↓] exploits the existence of the adjoint to start from an over-approximation Y_{n-1} of $p^\downarrow$ and computes backward an over-approximation of the set of safe states. Thus, the key difference comes from the strategy to look for a counterexample: to prove $\text{lfp}(b) \not\leq p$, AdjointPDR[↓] tries to find Y_{n-1} satisfying $p \in Y_{n-1}$ and $\text{lfp}(b) \notin Y_{n-1}$ while LT-PDR tries to find c_{n-1} s.t. $c_{n-1} \not\leq p$ and $c_{n-1} \leq \text{lfp}(b)$.

Theorem 7.23 below states that AdjointPDR[↓] can mimic any execution of LT-PDR. The proof exploits a map from LT-PDR's Kleene sequences $\vec{c}$ to AdjointPDR[↓]'s negative sequences $\text{neg}(\vec{c})$ of a particular form. Let $(L^\uparrow, \supseteq)$ be the complete lattice of upper sets, namely subsets $X \subseteq L$ such that $X = X^\uparrow \triangleq \{x' \in L \mid \exists x \in X. x \leq x'\}$. There is an isomorphism $\neg: (L^\uparrow, \supseteq) \xrightarrow{\cong} (L^\downarrow, \subseteq)$ mapping each $X \subseteq S$ into its complement. For a Kleene sequence $\vec{c} = c_k, \dots, c_{n-1}$ of LT-PDR, the sequence $\text{neg}(\vec{c}) \triangleq \neg(\{c_k\}^\uparrow), \dots, \neg(\{c_{n-1}\}^\uparrow)$ is a negative sequence, in the sense of Definition 7.2, for AdjointPDR[↓].

Proposition 7.22. *Let $\vec{c}$ be a Kleene sequence. Then $\text{neg}(\vec{c})$ is a negative sequence for AdjointPDR[↓].*

Proof. First, we show that $p \in \text{neg}(\vec{c})_{n-1}$. Since $c_{n-1} \not\leq p$, by (C1), then $p \notin \{c_{n-1}\}^\uparrow$. Thus $p \in \neg(\{c_{n-1}\}^\uparrow)$, that is $p \in \text{neg}(\vec{c})_{n-1}$.

Then, we show that $b_r^\downarrow(\text{neg}(\vec{c})_{j+1}) \subseteq \text{neg}(\vec{c})_j$.

$$\begin{aligned} b_r^\downarrow(\text{neg}(\vec{c})_{j+1}) &= b_r^\downarrow(\neg(\{c_{j+1}\}^\uparrow)) && [\text{def.}] \\ &= \{x \mid b(x) \notin (\{c_{j+1}\}^\uparrow)^\uparrow\} && [\text{def.}] \\ &= \{x \mid c_{j+1} \not\leq b(x)\} && [\text{def.}] \\ &\subseteq \{x \mid b(c_j) \not\leq b(x)\} && [(C2)] \\ &\subseteq \{x \mid c_j \not\leq x\} && [\text{mon. of } b] \\ &= \neg(\{c_j\}^\uparrow) && [\text{def.}] \\ &= \text{neg}(\vec{c})_j && [\text{def.}] \end{aligned}$$

□

Most importantly, the assignment $\vec{c} \mapsto \text{neg}(\vec{c})$ extends to a function, from the states of LT-PDR to those of AdjointPDR[↓], that is proved to be a *strong simulation* [Mil89].

Theorem 7.23. *AdjointPDR[↓] simulates LT-PDR.*

Remarkably, AdjointPDR[↓]'s negative sequences are not limited to the images of LT-PDR's Kleene sequences: they are more general than the complement of the upper closure

of a singleton. In fact, a single negative sequence of $\text{AdjointPDR}^\downarrow$ can represent *multiple* Kleene sequences of LT-PDR at once. Intuitively, this means that a single execution of $\text{AdjointPDR}^\downarrow$ can correspond to multiple runs of LT-PDR. We can make this formal by means of the following result.

Proposition 7.24. *Let $\{\vec{c}^m\}_{m \in M}$ be a family of Kleene sequences. Then its pointwise intersection $\bigcap_{m \in M} \text{neg}(\vec{c}^m)$ is a negative sequence.*

Proof. Since each of the $\vec{c}^m$ is a Kleene sequence, for all $m \in M$, $\text{neg}(\vec{c}^m)$ is, by Proposition 7.22, a negative sequence. Since negative sequences form a meet-semilattice, their intersection is also a negative sequence (Lemma 7.18). $\square$

The above intersection is pointwise in the sense that, for all $j \in [k, n-1]$, it holds $(\bigcap_{m \in M} \text{neg}(\vec{c}^m))_j \triangleq \bigcap_{m \in M} (\text{neg}(\vec{c}^m))_j = \neg(\{c_j^m \mid m \in M\}^\uparrow)$: intuitively, this is (up to $\text{neg}(\cdot)$) a set containing all the M counterexamples. Note that, if the negative sequence of $\text{AdjointPDR}^\downarrow$ makes (A3) hold as an equality, as it is possible with any simple heuristic (see Proposition 7.10), then its complement contains *all* Kleene sequences possibly computed by LT-PDR.

Proposition 7.25. *Let $\vec{c}$ be a Kleene sequence and $\vec{Y}$ be the negative sequence s.t. $Y_j = (b_r^\downarrow)^{n-1-j}(p^\downarrow)$ for all $j \in [k, n-1]$. Then $c_j \in \neg(Y_j)$ for all $j \in [k, n-1]$.*

Proof. Since $\vec{c} = c_0, \dots, c_{n-1}$ is a Kleene sequence, $\text{neg}(\vec{c}) = \neg(\{c_k\}^\uparrow), \dots, \neg(\{c_{n-1}\}^\uparrow)$ is, by Proposition 7.22, a negative sequence. By (A3), for all $j \in [k, n-1]$ we have $(b_r^\downarrow)^{n-1-j}(p^\downarrow) \subseteq \neg(\{c_j\}^\uparrow)$. Therefore, $\neg(b_r^\downarrow)^{n-1-j}(p^\downarrow) \supseteq \{c_j\}^\uparrow$, so $c_j \in \neg(b_r^\downarrow)^{n-1-j}(p^\downarrow) = \neg(Y_j)$. $\square$

While the previous result suggests that simple heuristics are always the best in theory, as they can carry all counterexamples, this is often not the case in practice, since they might be computationally hard and outperformed by some smart over-approximations. An example is given by (7.7) in the next section.

7.5 Instantiating $\text{AdjointPDR}^\downarrow$ for MDPs

In this section we illustrate how to use $\text{AdjointPDR}^\downarrow$ to address the max reachability problem [BK08] for Markov Decision Processes.

7.5.1 The max reachability problem

A *Markov Decision Process* (MDP) is a tuple (A, S, s_i, δ) where A is a set of labels, S is a set of states, $s_i \in S$ is an initial state, and $\delta: S \times A \rightarrow \mathcal{DS} + 1$ is a transition function. Here $\mathcal{DS}$ is the set of probability distributions over S , namely functions $d: S \rightarrow [0, 1]$ such that $\sum_{s \in S} d(s) = 1$, and $\mathcal{DS} + 1$ is the disjoint union of $\mathcal{DS}$ and $1 = \{*\}$. The transition function δ assigns to every label $a \in A$ and to every state $s \in S$ either a distribution of states or $* \in 1$. We assume that both S and A are finite sets and that the set $\text{Act}(s) \triangleq \{a \in A \mid \delta(s, a) \neq *\}$ of actions enabled at s is non-empty for all states.

An MDP (A, S, s_i, δ) mixes nondeterministic and probabilistic computations. The notion of a *scheduler*, also known as *adversary*, *policy* or *strategy*, is used to resolve non-deterministic choices. Below, we write S^+ for the set of non-empty sequence over S , intuitively representing runs of the MDP. A scheduler is a function $\alpha: S^+ \rightarrow A$ such that $\alpha(s_0 s_1 \dots s_n) \in \text{Act}(s_n)$: given the states visited so far, the scheduler decides which action

to trigger among the enabled ones so that the MDP behaves as a Markov chain. A scheduler α is called *memoryless* if it always selects the same action in a given state, namely, if $\alpha(s_0s_1 \dots s_n) = \alpha(s_n)$ for any sequence $s_0s_1 \dots s_n \in S^+$. Memoryless schedulers can thus be represented just as functions $\alpha: S \rightarrow A$ such that $\alpha(s) \in \text{Act}(s)$ for any $s \in S$.

Given an MDP, the *max reachability problem* requires to check whether the probability of reaching some bad states $\beta \subseteq S$ is less than or equal to a given threshold $\lambda \in [0, 1]$ for all possible schedulers. Thus, to solve this problem, one should compute the supremum over infinitely many schedulers. Notably, it is known that there always exists one memoryless scheduler that maximizes the probabilities to reach β (see e.g. [BK08]). As the memoryless schedulers are finitely many (although their number can grow exponentially), the supremum can thus be replaced by a maximum.

7.5.2 Applying AdjointPDR[↓] to the max reachability problem

The max-reachability problem, namely checking for a MDP (A, S, s_ι, δ) whether the probability of reaching some bad states $\beta \subseteq P$ is less than a threshold $\lambda \in [0, 1]$, enjoys a convenient lattice-theoretical formulation [BK08].

Consider the lattice $([0, 1]^S, \leq)$ of all functions $d: S \rightarrow [0, 1]$, often called frames or fuzzy predicates, ordered pointwise. The max reachability problem is equivalent to check that $\text{lfp}(b) \leq p$ for $p \in [0, 1]^S$ and $b: [0, 1]^S \rightarrow [0, 1]^S$, defined for all $d \in [0, 1]^S$ and $s \in S$, as

$$p(s) \triangleq \begin{cases} \lambda & \text{if } s = s_\iota, \\ 1 & \text{if } s \neq s_\iota, \end{cases} \quad b(d)(s) \triangleq \begin{cases} 1 & \text{if } s \in \beta, \\ \max_{a \in \text{Act}(s)} \sum_{s' \in S} d(s') \cdot \delta(s, a)(s') & \text{if } s \notin \beta. \end{cases}$$

Since b is not of the form $f \vee i$ for a left adjoint f (see e.g. [Kor+22]), we can't use **AdjointPDR**, but we can use **AdjointPDR[↓]**. Beyond the simple initial heuristic, which is always applicable and enjoys negative termination, we illustrate now two additional heuristics that are experimentally tested in Section 7.7.

The two novel heuristics make the same choices in (Candidate) and (Decide). They exploit memoryless schedulers $\alpha: S \rightarrow A$, and the function $b_\alpha: [0, 1]^S \rightarrow [0, 1]^S$ defined for all $d \in [0, 1]^S$ and $s \in S$ as follows:

$$b_\alpha(d)(s) \triangleq \begin{cases} 1 & \text{if } s \in \beta, \\ \sum_{s' \in S} d(s') \cdot \delta(s, \alpha(s))(s') & \text{otherwise.} \end{cases} \quad (7.6)$$

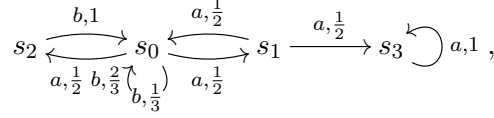
Since for all $D \in ([0, 1]^S)^\downarrow$, $b_r^\downarrow(D) = \{d \mid b(d) \in D\} = \bigcap_\alpha \{d \mid b_\alpha(d) \in D\}$ and since **AdjointPDR[↓]** executes (Decide) only when $b(x_{k-1}) \notin Y_k$, there should exist some α such that $b_\alpha(x_{k-1}) \notin Y_k$. One can thus fix

$$(\vec{x} \parallel \vec{Y})_{n,k} \mapsto \begin{cases} p^\downarrow & \text{if } (\vec{x} \parallel \vec{Y})_{n,k} \xrightarrow{C_a} \\ \{d \mid b_\alpha(d) \in Y_k\} & \text{if } (\vec{x} \parallel \vec{Y})_{n,k} \xrightarrow{D} \end{cases} \quad (7.7)$$

Intuitively, such choices are smart refinements of those in (7.2): for (Candidate) they are exactly the same; for (Decide) rather than taking $b_r^\downarrow(Y_k)$, we consider a larger lower-set determined by the labels chosen by α . This allows to represent each Y_j as a set of $d \in [0, 1]^S$ satisfying a *single* linear inequality, while using $b_r^\downarrow(Y_k)$ would yield a systems of possibly exponentially many inequalities (see Example 7.27 below). Moreover, from Theorem 7.15, it follows that such choices ensures negative termination.

Corollary 7.26. *Let h be a legit heuristic defined for (Candidate) and (Decide) as in (7.7). If $\text{lfp}(b) \not\leq p$, then $\text{AdjointPDR}_h^\downarrow$ terminates.*

Example 7.27. Consider the maximum reachability problem with threshold $\lambda = \frac{1}{4}$ and $\beta = \{s_3\}$ for the following MDP on alphabet $A = \{a, b\}$ and $s_i = s_0$.



Hereafter we write $d \in [0, 1]^S$ as column vectors with four entries $v_0 \dots v_3$ and we will use $\cdot$ for the usual matrix multiplication. With this notation, the lower set $p^\downarrow \in ([0, 1]^S)^\downarrow$ and $b: [0, 1]^S \rightarrow [0, 1]^S$ can be written as

$$p^\downarrow = \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid [1 \ 0 \ 0 \ 0] \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \left[\frac{1}{4}\right] \right\} \quad \text{and} \quad b\left(\begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix}\right) = \begin{bmatrix} \max(\frac{v_1+v_2}{2}, \frac{v_0+2v_2}{3}) \\ \frac{v_0+v_3}{2} \\ v_0 \\ 1 \end{bmatrix}.$$

Amongst the several memoryless schedulers, only two are relevant for us:

$$\zeta \triangleq (s_0 \mapsto a, s_1 \mapsto a, s_2 \mapsto b, s_3 \mapsto a) \text{ and} \quad (7.8)$$

$$\xi \triangleq (s_0 \mapsto b, s_1 \mapsto a, s_2 \mapsto b, s_3 \mapsto a). \quad (7.9)$$

By using the definition of $b_\alpha: [0, 1]^S \rightarrow [0, 1]^S$ in (7.6), we have that

$$b_\zeta\left(\begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix}\right) = \begin{bmatrix} \frac{v_1+v_2}{2} \\ \frac{v_0+v_3}{2} \\ v_0 \\ 1 \end{bmatrix} \quad \text{and} \quad b_\xi\left(\begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix}\right) = \begin{bmatrix} \frac{v_0+2v_2}{3} \\ \frac{v_0+v_3}{2} \\ v_0 \\ 1 \end{bmatrix}.$$

It is immediate to see that the problem has negative answer, since using ζ in 4 steps or less, s_0 can reach s_3 already with probability $\frac{1}{4} + \frac{1}{8}$.

To illustrate the advantages of (7.7), we run $\text{AdjointPDR}^\downarrow$ with the simple initial heuristic and with the heuristic that only differs for the choice in (Decide), taken as in (7.7). For both heuristics, the first iterations are the same: several repetitions of (Candidate), (Conflict) and (Unfold) exploiting elements of the positive chain that form the initial chain (except for the last element x_{n-1}).

$$(\emptyset \parallel \varepsilon)_{3,3} \xrightarrow{CaCo} (\emptyset \parallel \varepsilon)_{3,3} \xrightarrow{U} \xrightarrow{CaCo} \xrightarrow{U} \xrightarrow{CaCo} \xrightarrow{U} \xrightarrow{CaCo} \xrightarrow{U} \xrightarrow{Ca} (\emptyset \parallel \varepsilon)_{7,6} \parallel p^\downarrow.$$

In the latter state the algorithm has to perform (Decide), since $b(x_5) \notin p^\downarrow$. Now the choice of z in (Decide) is different for the two heuristics: the former uses $b_r^\downarrow(p^\downarrow) = \{d \mid b(d) \in p^\downarrow\}$, the latter uses $\{d \mid b_\zeta(d) \in p^\downarrow\}$. Despite the different choices, both the heuristics proceed with 6 steps of (Decide):

$$(\emptyset \parallel \varepsilon)_{7,6} \xrightarrow{\mathcal{F}^0} \xrightarrow{\mathcal{F}^1} \xrightarrow{\mathcal{F}^2} \xrightarrow{\mathcal{F}^3} \xrightarrow{\mathcal{F}^4} \xrightarrow{\mathcal{F}^5} (\emptyset \parallel \varepsilon)_{7,1}$$

The element of the negative sequence $\mathcal{F}^i$ are illustrated in Figure 7.4 for both the heuristics. In both cases, $\mathcal{F}^5 = \emptyset$ and thus $\text{AdjointPDR}^\downarrow$ returns false.

To appreciate the advantages provided by (7.7), it is enough to compare the two columns for the $\mathcal{F}^i$ in Figure 7.4: in the central column, the number of inequalities defining $\mathcal{F}^i$ grows significantly, while in the rightmost column is always 1. ■

$$\begin{array}{lcl}
\mathcal{F}^0 \triangleq & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} \frac{1}{4} \end{bmatrix} \right\} & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} \frac{1}{4} \end{bmatrix} \right\} \\
\mathcal{F}^1 \triangleq & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 2 & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} \frac{1}{2} \\ \frac{3}{4} \end{bmatrix} \right\} & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} \frac{1}{4} \end{bmatrix} \right\} \\
\mathcal{F}^2 \triangleq & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 3 & 0 & 0 & 1 \\ 2 & 1 & 1 & 0 \\ 4 & 0 & 2 & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} 1 \\ \frac{3}{2} \\ \frac{9}{4} \end{bmatrix} \right\} & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} \frac{3}{4} & 0 & 0 & \frac{1}{4} \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} \frac{1}{4} \end{bmatrix} \right\} \\
\mathcal{F}^3 \triangleq & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 0 & \frac{3}{2} & \frac{3}{2} & 0 \\ 1 & 0 & 2 & 0 \\ \frac{3}{2} & 1 & 1 & \frac{1}{2} \\ \frac{13}{6} & 0 & \frac{4}{3} & \frac{1}{2} \\ 2 & 2 & 2 & 0 \\ \frac{10}{3} & 0 & \frac{8}{3} & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} 0 \\ 0 \\ \frac{3}{2} \\ \frac{3}{2} \\ \frac{9}{4} \\ \frac{9}{4} \end{bmatrix} \right\} & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 0 & \frac{3}{8} & \frac{3}{8} & 0 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} 0 \end{bmatrix} \right\} \\
\mathcal{F}^4 \triangleq & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right\} = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right\} & \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid \begin{bmatrix} \frac{9}{16} & 0 & 0 & \frac{3}{16} \end{bmatrix} \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq \begin{bmatrix} 0 \end{bmatrix} \right\} \\
\mathcal{F}^5 \triangleq & \emptyset & \emptyset
\end{array}$$

Figure 7.4: The elements of the negative sequences computed by $\text{AdjointPDR}^\downarrow$ for the MDP in Example 7.27. In the central column, these elements are computed by means of the simple initial heuristics, that is $\mathcal{F}^i = (b_r^\downarrow)^i(p^\downarrow)$. In the rightmost column, these elements are computed using the heuristic in (7.7). In particular $\mathcal{F}^i = \{d \mid b_\zeta(d) \in \mathcal{F}^{i-1}\}$ for $i \leq 3$, while for $i \geq 4$ these are computed as $\mathcal{F}^i = \{d \mid b_\xi(d) \in \mathcal{F}^{i-1}\}$.

The fact that using (7.7) ensures that Y_k is generated by a single linear inequality is quite convenient. Indeed, in this case

$$Y_k = \{d \in [0, 1]^S \mid \sum_{s \in S} (r_s \cdot d(s)) \leq r\}$$

for suitable non-negative real numbers r and r_s for all $s \in S$. The convex set Y_k is generated by finitely many $d \in [0, 1]^S$ enjoying a useful property: $d(s)$ is different from 0 and 1 only for at most one $s \in S$. The set of its generators, denoted by $\mathcal{G}_k$, can thus be easily computed. We exploit this property to resolve the choice for (Conflict). We consider its subset $\mathcal{Z}_k \triangleq \{d \in \mathcal{G}_k \mid b(x_{k-1}) \leq d\}$ and define $z_B, z_{01} \in [0, 1]^S$ for all $s \in S$ as

$$z_B(s) \triangleq \begin{cases} (\bigwedge \mathcal{Z}_k)(s) & \text{if } r_s \neq 0, \mathcal{Z}_k \neq \emptyset \\ b(x_{k-1})(s) & \text{otherwise} \end{cases} \quad z_{01}(s) \triangleq \begin{cases} \lceil z_B(s) \rceil & \text{if } r_s = 0, \mathcal{Z}_k \neq \emptyset \\ z_B(s) & \text{otherwise} \end{cases} \quad (7.10)$$

where, for $u \in [0, 1]$, $\lceil u \rceil$ denotes 0 if $u = 0$ and 1 otherwise. We call **hCoB** and **hCo01** the heuristics defined as in (7.7) for (Candidate) and (Decide) and as z_B , respectively z_{01} , for (Conflict). The heuristics **hCo01** can be seen as a Boolean modification of **hCoB**, rounding up positive values to 1 to accelerate convergence.

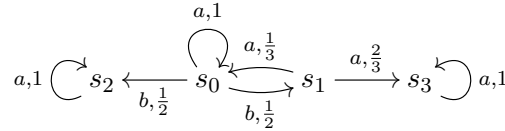
Proposition 7.28. *The heuristics **hCoB** and **hCo01** are legit.*

By Corollary 7.26, $\text{AdjointPDR}^\downarrow$ terminates for negative answers with both **hCoB** and **hCo01**. We conclude this section with a last example.

$$\begin{aligned}
& \left(\begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \parallel \varepsilon \right)_{2,2} \xrightarrow{C_a} \left(\begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix} \parallel p^\downarrow \right)_{2,1} \xrightarrow{C_o} \left(\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \parallel \varepsilon \right)_{2,2} \\
& \xrightarrow{U} \xrightarrow{C_a} \left(\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \parallel p^\downarrow \right)_{3,2} \xrightarrow{C_o} \left(\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix} \parallel \varepsilon \right)_{3,3} \\
& \xrightarrow{U} \xrightarrow{C_a} \left(\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \parallel p^\downarrow \right)_{4,3} \xrightarrow{C_o} \left(\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} \parallel \varepsilon \right)_{4,4} \\
& \xrightarrow{U} \xrightarrow{C_a} \left(\begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix} \parallel p^\downarrow \right)_{5,4} \xrightarrow{C_o} \left(\begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix} \parallel \varepsilon \right)_{4,4} \\
& \xrightarrow{U} \xrightarrow{C_a} \left(\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \parallel p^\downarrow \right)_{6,5} \xrightarrow{C_o} \left(\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \parallel \varepsilon \right)_{6,6} \\
& \xrightarrow{U} \xrightarrow{C_a} \left(\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \parallel p^\downarrow \right)_{7,6} \xrightarrow{C_o} \left(\begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \parallel \varepsilon \right)_{7,7} \dots
\end{aligned}$$

Figure 7.5: The non-terminating execution of $\text{AdjointPDR}^\downarrow$ with the simple initial heuristics for the max reachability problem of Example 7.29. The elements of the positive chain, with the exception of the last one x_{n-1} are those of the initial chain.

Example 7.29. Consider the following MDP with alphabet $A = \{a, b\}$ and $s_\ell = s_0$



and the max reachability problem with threshold $\lambda = \frac{2}{5}$ and $\beta = \{s_3\}$. The lower set $p^\downarrow \in ([0, 1]^S)^\downarrow$ and $b: [0, 1]^S \rightarrow [0, 1]^S$ can be written as

$$p^\downarrow = \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid [1 \ 0 \ 0 \ 0] \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq [\frac{2}{5}] \right\} \quad \text{and} \quad b\left(\begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix}\right) = \begin{bmatrix} \max(v_0, \frac{v_1+v_2}{2}) \\ \frac{v_0+2 \cdot v_3}{3} \\ v_2 \\ 1 \end{bmatrix}$$

Consider also the scheduler $\xi: S \rightarrow A$ defined as $\xi \triangleq [s_0 \mapsto b, s_1 \mapsto a, s_2 \mapsto a, s_3 \mapsto a]$, for which we illustrate

$$b_\xi\left(\begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix}\right) = \begin{bmatrix} \frac{v_1+v_2}{2} \\ \frac{v_0+2 \cdot v_3}{3} \\ v_2 \\ 1 \end{bmatrix} \quad \text{and} \quad \mathcal{F}_\xi^\downarrow \triangleq \{d \mid b_\xi(d) \in p^\downarrow\} = \left\{ \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \mid [0 \ 1 \ 1 \ 0] \cdot \begin{bmatrix} v_0 \\ v_1 \\ v_2 \\ v_3 \end{bmatrix} \leq [\frac{4}{5}] \right\} \quad (7.11)$$

With the simple initial heuristic, $\text{AdjointPDR}^\downarrow$ does not terminate (Figure 7.5). With the heuristic hCo01 using scheduler ξ , it returns true in 14 steps (Figure 7.6), while with hCoB in 8 (Figure 7.7). The first 4 steps of both hCoB and hCo01 are the same: in the first (Conflict) $z_B = z_{01}$, while in the second $z_{01}(s_1) = 1$ and $z_B(s_1) = \frac{4}{5}$, leading to the two different executions. Observe that this difference is due to the fact that in $p^\downarrow$, the coefficient corresponding to s_1 , namely r_{s_1} , is 0 and, since $\mathcal{Z}_3 \neq \emptyset$, then $z_{01}(s_1) = \lceil z_B(s_1) \rceil$. ■

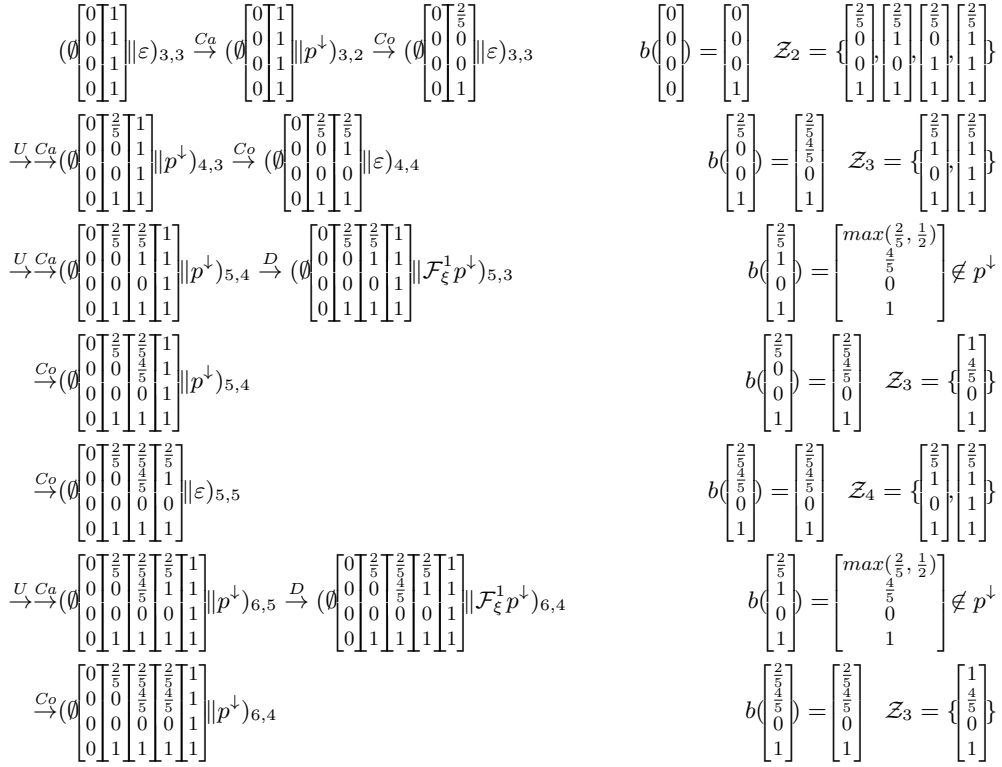


Figure 7.6: On the left, the execution of $\text{AdjointPDR}^{\downarrow}_{\text{nc001}}$ for the max reachability problem of Example 7.29: in the last state, it returns true since $x_3 = x_4$. On the right, the data explaining the choices of (Conflict) and (Decide). Note that, in the two (Decide) steps, the guard $b(x_{k-1}) \notin Y_k$ holds because of the possibility of choosing the label b in state s_0 . This explain why Z is taken as $\mathcal{F}_\xi^1(p^\downarrow)$ for the scheduler ξ defined in (7.11).

7.6 AdjointPDR^{AI}

In Section 7.2 we have introduced AdjointPDR and in Section 7.4 $\text{AdjointPDR}^\downarrow$: the latter is inspired by the former but takes the positive chain in some lattice L and the negative sequence in $L^\downarrow$. In this section, we introduce $\text{AdjointPDR}^{\text{AI}}$, a third algorithm that generalises both by allowing to manipulate positive and negative sequences in two different lattices. The interest in this generalisation is not just theoretical, but it is also convenient in practice since it allows us in the next section to use an implementation of $\text{AdjointPDR}^{\text{AI}}$ as a common template for both AdjointPDR and $\text{AdjointPDR}^\downarrow$.

The framework for $\text{AdjointPDR}^{\text{AI}}$ is a diagram

$$b \bigcirc_{\leftarrow} (L, \leq_L) \xrightarrow{\gamma} (C, \leq_C) \bigcirc_{\leftarrow} \bar{b} \bar{b}_r \quad (7.12)$$

where $(L, \leq_L)$ and $(C, \leq_C)$ are complete lattices, $b: L \rightarrow L$ is an ω -continuous function, $\bar{b} \vdash \bar{b}_r: C \rightarrow C$, and $\gamma: L \rightarrow C$ is a function satisfying

1. *order-embeddingness*: $x \leq y$ if and only if $\gamma(x) \leq \gamma(y)$ for all $x, y \in L$, and
2. *forward completeness*: $\bar{b}\gamma = \gamma b$.

In this setting the problem $\text{lfp}(b) \leq p$ in L has an equivalent formulation in C :

$$\begin{array}{lcl}
\begin{array}{l}
\begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \end{array} \parallel \varepsilon \end{array} \end{array}_{3,3} \xrightarrow{C_a} \begin{array}{l} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \end{array} \parallel p^\perp \end{array} \end{array}_{3,2} \xrightarrow{C_o} \begin{array}{l} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \end{bmatrix} \end{array} \parallel \varepsilon \end{array} \end{array}_{3,3} \\
\begin{array}{l} \xrightarrow{U} \xrightarrow{C_a} \begin{array}{l} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{array} \parallel p^\perp \end{array} \end{array}_{4,3} \xrightarrow{C_o} \begin{array}{l} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{array} \parallel \varepsilon \end{array} \end{array}_{4,4} \\
\begin{array}{l} \xrightarrow{U} \xrightarrow{C_a} \begin{array}{l} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{array} \parallel p^\perp \end{array} \end{array}_{5,4} \xrightarrow{C_o} \begin{array}{l} \begin{array}{c} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \\ 0 \\ 0 \end{bmatrix} \end{array} \parallel p^\perp \end{array} \end{array}_{5,4}
\end{array} \quad \begin{array}{l}
b\left(\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \mathcal{Z}_2 = \left\{ \begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right\} \\
b\left(\begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \mathcal{Z}_3 = \left\{ \begin{bmatrix} 2 \\ 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right\} \\
b\left(\begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 2 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \mathcal{Z}_4 = \mathcal{Z}_3
\end{array}$$

Figure 7.7: On the left, the execution of $\text{AdjointPDR}_{\text{hCoB}}^\downarrow$ for the max reachability problem of Example 7.29: in the last state, it returns true since $x_3 = x_4$. On the right, the data explaining the three choices of (Conflict).

Proposition 7.30. *Consider the framework (7.12) and let $p \in L$. Then $\text{lfp}(b) \leq p$ is equivalent to $\text{lfp}(\bar{b} \vee \gamma \perp) \leq \gamma p$.*

Proof.

$$\begin{aligned}
\text{lfp}(b) \leq p &\iff \forall n \in \mathbb{N}. b^n \perp \leq p \\
&\iff \forall n \in \mathbb{N}. \gamma b^n \perp \leq \gamma p && [\text{because } \gamma \text{ is order-embedding}] \\
&\iff \forall n \in \mathbb{N}. \bar{b}^n \gamma \perp \leq \gamma p && [\text{by the forward completeness}] \\
&\iff \text{lfp}(\bar{b} \vee \gamma \perp) \leq \gamma p && [\text{since } \bar{b} \dashv \bar{b}_r]
\end{aligned}$$

□

The reader may have recognised some similarities with the proof of Proposition 7.19. Indeed, as we will show in Example 7.32, the latter result is an instance of Proposition 7.30.

Example 7.31. Consider a Galois insertion with a forward complete abstract interpretation $b: L \rightarrow L$ to a concrete semantic function $\bar{b}: C \rightarrow C$. If the function $\bar{b}$ has a right adjoint, then this is an instance of (7.12). ■

Example 7.32. It is easy to check that (7.3) is an instance of (7.12). Hereafter, we illustrate how this can be understood in categorical terms. For a complete lattice L and an ω -continuous function $b: L \rightarrow L$, we shall see L as a **Bool**-enriched category where **Bool** is the monoidal category with two objects $\perp, \top$, one arrow $\perp \rightarrow \top$ and monoidal product given by $\wedge$. Then the left Kan extension $\text{Lan}_y(y \circ b)$ along the Yoneda embedding $y: L \rightarrow \mathbf{Bool}^{L^{\text{op}}}$ is a left adjoint of the induced functor $b^* \triangleq (-) \circ b^{\text{op}}$.

$$b \circlearrowleft L \xrightarrow{y} \mathbf{Bool}^{L^{\text{op}}} \circlearrowright_{\text{Lan}_y(y \circ b) \dashv b^*}$$

The above diagram is the same as (7.3) under the isomorphism $\mathbf{Bool}^{L^{\text{op}}} \cong L^\downarrow$. It is an instance of (7.12) because:

1. y is order-embedding since y is full and faithful and
2. y is forward complete since the unit $b \Rightarrow \text{Lan}_y(y \circ b) \circ y$ is an isomorphism by [Kel82, Proposition 4.23] and full and faithfulness of y .

■

AdjointPDR^{AI} ($b, p, \gamma: L \rightarrow C, \bar{b}, \bar{b}_r$)

```

<INITIALISATION>
  ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\perp, \top \parallel \varepsilon$ )2,2
<ITERATION>                                     %  $\vec{x}, \vec{y}$  not conclusive
  case ( $\vec{x} \parallel \vec{y}$ )n,k of
     $\vec{y} = \varepsilon$  and  $x_{n-1} \leq p$  :                % (Unfold)
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x}, \top \parallel \varepsilon$ )n+1,n+1
     $\vec{y} = \varepsilon$  and  $x_{n-1} \not\leq p$  :                % (Candidate)
      choose  $z \in C$  such that  $\gamma x_{n-1} \not\leq z$  and  $\gamma p \leq z$ ;
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x} \parallel z$ )n,n-1
     $\vec{y} \neq \varepsilon$  and  $\bar{b} \gamma x_{k-1} \not\leq y_k$  :        % (Decide)
      choose  $z \in C$  such that  $\gamma x_{k-1} \not\leq z$  and  $\bar{b}_r(y_k) \leq z$ ;
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x} \parallel z, \vec{y}$ )n,k-1
     $\vec{y} \neq \varepsilon$  and  $\gamma b x_{k-1} \leq y_k$  :          % (Conflict)
      choose  $z \in L$  such that  $\gamma z \leq y_k$  and  $b(x_{k-1} \wedge z) \leq z$ ;
      ( $\vec{x} \parallel \vec{y}$ )n,k := ( $\vec{x} \wedge_k z \parallel \text{tail}(\vec{y})$ )n,k+1
  endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. x_{j+1} \leq x_j$  then return true %  $\vec{x}$  conclusive
  if  $y_0$  is defined then return false %  $\vec{y}$  conclusive

```

Figure 7.8: AdjointPDR^{AI} algorithm checking $\text{lfp}(b) \leq p$.

The algorithm AdjointPDR^{AI} checking $\text{lfp}(b) \leq p$ in L is illustrated in Figure 7.8. It is an adaptation of AdjointPDR algorithm checking $\text{lfp}(\bar{b} \vee \gamma \perp) \leq \gamma p$ in C (that, by Proposition 7.30, is equivalent to $\text{lfp}(b) \leq p$) that takes positive sequences in L .

More precisely, AdjointPDR^{AI} manipulates pairs $(\vec{x} \parallel \vec{y})_{n,k}$ of sequences $\vec{x}$ in L and $\vec{y}$ in C while AdjointPDR manipulates pairs of sequences in C . The pairs in AdjointPDR^{AI} satisfy invariants (I0), (I1), (I2) and (PN'): $\forall j \in [k, n-1], \gamma x_j \not\leq y_j$. Observe that $(\perp, \gamma \vec{x}) \triangleq (\perp, \gamma x_0, \dots, \gamma x_{n-1})$ forms a positive chain (Definition 7.1) and $\vec{y}$ forms a negative sequence (Definition 7.2) for $\text{lfp}(\bar{b} \vee \gamma \perp) \leq \gamma p$ in C . The algorithm returns true if $\vec{x}$ is conclusive (i.e., $\exists j \in [0, n-2]. x_{j+1} \leq x_j$) which implies $(\perp, \gamma \vec{x})$ is also conclusive. It returns false if y_0 is defined, or equivalently, if $\vec{y}$ in the pair $(\perp, \gamma \vec{x} \parallel \vec{y})$ is conclusive. This equivalence is deduced from (I0) and $\gamma x_0 \not\leq y_0$ by (PN').

The above discussion immediately yields the soundness of AdjointPDR^{AI}. One can also prove the properties of canonical choices, impossibility of loops, and negative termination for AdjointPDR^{AI} in the same way as for AdjointPDR[↓], yielding the following result.

Theorem 7.33. *All results in Section 7.3, but Proposition 7.6.3, hold for AdjointPDR^{AI}.*

We conclude this section by illustrating how the algorithm AdjointPDR^{AI} generalises both AdjointPDR and AdjointPDR[↓].

Proposition 7.34. *AdjointPDR is an instance of AdjointPDR^{AI}.*

Proof. Consider the setting for AdjointPDR with (i, f, g, p) , i.e. $i, p \in L$ and $f \dashv g: L \rightarrow L$. Let $L_{i\uparrow}$ be the complete lattice $\{x \in L \mid i \leq x\}$ (with the same order as L) and let $s: L_{i\uparrow} \hookrightarrow L$ be the inclusion function. Then AdjointPDR^{AI} with parameters $(f \vee i, p, s: L_{i\uparrow} \hookrightarrow L, f, g)$ defines exactly AdjointPDR with parameters (i, f, g, p) and starting state $(\perp, i, \top \parallel \varepsilon)_{3,3}$ (reachable in AdjointPDR applying (Candidate), (Conflict) and (Unfold)). $\square$

Proposition 7.35. $\text{AdjointPDR}^\downarrow$ is an instance of $\text{AdjointPDR}^{\text{AI}}$.

Proof. Consider the setting for $\text{AdjointPDR}^\downarrow$, i.e. (b, p) with an ω -continuous function $b: L \rightarrow L$ and $p \in L$. Then $\text{AdjointPDR}^{\text{AI}}$ with parameters $(b, p, (-)^\downarrow: L \rightarrow L^\downarrow, b^\downarrow, b_r^\downarrow)$ defines exactly $\text{AdjointPDR}^\downarrow$ with parameters (b, p) under the correspondence between intermediate data $(\vec{x} \parallel \vec{y})_{n,k}$ in $\text{AdjointPDR}^{\text{AI}}$ and $(\emptyset, \vec{x} \parallel \vec{Y})$ in $\text{AdjointPDR}^\downarrow$ where $Y_{i+1} := y_i$ for each $i \in \{k, \dots, n-1\}$. Please note that $x^\downarrow \subseteq Z$ if and only if $x \in Z$ for a lower set Z . $\square$

7.7 Implementation

We first developed, using Haskell and exploiting its abstraction features, a common template of $\text{AdjointPDR}^{\text{AI}}$ that accommodates both AdjointPDR and $\text{AdjointPDR}^\downarrow$. It is a program parametrized by two lattices—used for positive chains and negative sequences, respectively—and by a heuristic.

For our experiments, we instantiated the template to $\text{AdjointPDR}^\downarrow$ for MDPs (letting $L = [0, 1]^S$), with three different heuristics: **hCoB** and **hCo01** from Proposition 7.28; and **hCoS** introduced below. Besides the template (~ 100 lines), we needed ~ 140 lines to account for **hCoB** and **hCo01**, and additional ~ 100 lines to further obtain **hCoS**. All this indicates a clear benefit of our abstract theory: a general template can itself be coded succinctly; instantiation to concrete problems is easy, too, thanks to an explicitly specified interface of heuristics.

Our implementation accepts MDPs expressed in a symbolic format inspired by Prism models [KNP11], in which states are variable valuations and transitions are described by symbolic functions (they can be segmented with symbolic guards $\{\text{guard}_i\}_i$). We use rational arithmetic (**Rational** in Haskell) for probabilities to avoid rounding errors.

Heuristics. The three heuristics (**hCoB**, **hCo01**, **hCoS**) use the same choices in (Candidate) and (Decide), as defined in (7.7), but different ones in (Conflict).

The third heuristics **hCoS** is a *symbolic* variant of **hCoB**; it relies on our symbolic model format. It uses z_S for z in (Conflict), where $z_S(s) = z_B(s)$ if $r_s \neq 0$ or $\mathcal{Z}_k = \emptyset$. The definition of $z_S(s)$ otherwise is notable: we use a piecewise affine function $(t_i \cdot s + u_i)_i$ for $z_S(s)$, where the affine functions $(t_i \cdot s + u_i)_i$ are guarded by the same guards $\{\text{guard}_i\}_i$ of the MDP’s transition function. We let the SMT solver Z3 [MB08] search for the values of the coefficients t_i, u_i , so that z_S satisfies the requirements of (Conflict) (namely $b(x_{k-1})(s) \leq z_S(s) \leq 1$ for each $s \in S$ with $r_s = 0$), together with the condition $b(z_S) \leq z_S$ for faster convergence. If the search is unsuccessful, we give up **hCoS** and fall back on the **hCoB**.

As a task common to the three heuristics, we need to calculate $\mathcal{Z}_k = \{d \in \mathcal{G}_k \mid b(x_{k-1}) \leq d\}$ in (Conflict) (see (7.10)). Rather than computing the whole set $\mathcal{G}_k$ of generating points of the linear inequality that defines Y_k , we implemented an ad-hoc algorithm that crucially exploits the condition $b(x_{k-1}) \leq d$ for pruning.

Experiment Settings. We conducted the experiments on Ubuntu 18.04 and AWS t2.xlarge (4 CPUs, 16 GB memory, up to 3.0 GHz Intel Scalable Processor). We used several Markov chain (MC) benchmarks and a couple of MDP ones.

Research Questions. We wish to address the following questions.

RQ1 Does $\text{AdjointPDR}^\downarrow$ advance the state-of-the-art performance of *PDR* algorithms for probabilistic model checking?

Table 7.1: Experimental results on MC benchmarks. $|S|$ is the number of states, P is the reachability probability (calculated by manual inspection), λ is the threshold in the problem $P \leq ? \lambda$ (shaded if the answer is no). The other columns show the average execution time in seconds; TO is timeout (900 s); MO is out-of-memory. For **AdjointPDR**[↓] and **LT-PDR** we used the **tasty-bench** Haskell package and repeated executions until std. dev. is $< 5\%$ (at least three execs). For **PrIC3** and **Storm**, we made five executions. Storm’s execution does not depend on λ : it seems to answer queries of the form $P \leq ? \lambda$ by calculating P . We observed a wrong answer for the entry with (†) (Storm, sp.-num., Haddad-Monnege); see the discussion of RQ2.

Benchmark	S	P	λ	AdjointPDR [↓]			LT-PDR		PrIC3			Storm		
				hCoB	hCo01	hCoS	none	lin.	pol.	hyb.	sp.-num.	sp.-rat.	sp.-sd.	
Grid	10 ²	0.033	0.3	0.013	0.022	0.659	0.343	1.383	23.301	MO	MO			
			0.2	0.013	0.031	0.657	0.519	1.571	26.668	TO	MO	0.010	0.010	0.010
	10 ³	<0.001	0.3	1.156	2.187	5.633	126.441	TO	TO	TO	MO			
			0.2	1.146	2.133	5.632	161.667	TO	TO	TO	MO	0.010	0.017	0.011
BRP	10 ³	0.035	0.1	12.909	7.969	55.788	TO	TO	TO	MO	MO			
			0.01	1.977	8.111	5.645	21.078	60.738	626.052	524.373	823.082	0.012	0.018	0.011
			0.005	0.604	2.261	2.709	1.429	12.171	254.000	197.940	318.840			
			0.9	1.217	68.937	0.196	TO	19.765	136.491	0.630	0.468			
Zero-Conf	10 ²	0.5	0.75	1.223	68.394	0.636	TO	19.782	132.780	0.602	0.467			
			0.52	1.228	60.024	0.739	TO	19.852	136.533	0.608	0.474	0.010	0.018	0.011
			0.45	<0.001	0.001	0.001	<0.001	0.035	0.043	0.043	0.043			
			0.9	MO	TO	7.443	TO	TO	TO	0.602	0.465			
Chain	10 ³	0.394	0.9	MO	TO	15.223	TO	TO	TO	0.599	0.470			
			0.75	MO	TO	TO	TO	TO	TO	0.488	0.475	0.037	262.193	0.031
			0.52	MO	TO	TO	TO	TO	TO	0.040	0.040			
			0.45	0.108	0.119	0.169	0.016	0.035	0.040	0.040	0.040			
Chain	10 ³	0.394	0.9	36.083	TO	0.478	TO	269.801	TO	0.938	0.686			
			0.4	35.961	TO	394.955	TO	271.885	TO	0.920	TO	0.010	0.014	0.011
			0.35	101.351	TO	454.892	435.199	238.613	TO	TO				
			0.3	62.036	463.981	120.557	209.346	124.829	746.595	TO	TO			
Double-Chain	10 ³	0.215	0.9	12.122	7.318	TO	TO	TO	1.878	2.053				
			0.3	12.120	20.424	TO	TO	TO	TO	1.953	2.058			
			0.216	12.096	19.540	TO	TO	TO	TO	172.170	TO	0.011	0.018	0.010
			0.15	12.344	16.172	TO	16.963	TO	TO	TO	TO			
Haddad-Monnege	41	0.7	0.9	0.004	0.009	8.528	TO	1.188	31.915	TO	MO			
			0.75	0.004	0.011	2.357	TO	1.209	32.143	TO	712.086	0.011	0.011	1.560
			0.9	59.721	61.777	TO	TO	TO	TO	TO	TO			
			0.75	60.413	63.050	TO	TO	TO	TO	TO	TO	0.013 (†)	0.043	TO

Table 7.2: Experimental results on MDP benchmarks. The legend is the same as Table 7.1, except that P is now the maximum reachability probability.

Benchmark	$ S $	P	λ	AdjointPDR [↓]			Storm		
				hCoB	hCo01	hCoS	sp.-num	sp.-rat.	sp.-sd.
CDrive2	38	0.865	0.9	MO	0.172	TO	0.019	0.019	0.018
			0.75	MO	0.058	TO			
			0.5	0.015	0.029	86.798			
TireWorld	8670	0.233	0.9	MO	3.346	TO	0.070	0.164	0.069
			0.75	MO	3.337	TO			
			0.5	MO	6.928	TO			
			0.2	4.246	24.538	TO			

RQ2 How does AdjointPDR[↓]'s performance compare against *non-PDR* algorithms for probabilistic model checking?

RQ3 Does the theoretical framework of AdjointPDR[↓] successfully guide the discovery of various heuristics with practical performance?

RQ4 Does AdjointPDR[↓] successfully manage nondeterminism in MDPs (that is absent in MCs)?

Experiments on MCs (Table 7.1). We used six benchmarks: Haddad-Monmege is from [Har+19]; the others are from [Bat+20; Kor+22]. We compared AdjointPDR[↓] (with three heuristics) against LT-PDR [Kor+22], PrIC3 (with four heuristics *none*, *lin.*, *pol.*, *hyb.*, see [Bat+20]), and Storm 1.5 [DJKV17]. Storm is a recent comprehensive toolsuite that implements different algorithms and solvers. Among them, our comparison is against *sparse-numeric*, *sparse-rational*, and *sparse-sound*. The *sparse* engine uses explicit state space representation by sparse matrices; this is unlike another representative *dd* engine that uses symbolic BDDs. (We did not use *dd* since it often reported errors, and was overall slower than *sparse*.) *Sparse-numeric* is a value-iteration (VI) algorithm; *sparse-rational* solves linear (in)equations using rational arithmetic; *sparse-sound* is a sound VI algorithm [QK18].¹

Experiments on MDPs (Table 7.2). We used two benchmarks from [Har+19]. We compared AdjointPDR[↓] only against Storm, since RQ1 is already addressed using MCs (besides, PrIC3 did not run for MDPs).

Discussion. The experimental results suggest the following answers to the RQs.

RQ1. The performance advantage of AdjointPDR[↓], over both LT-PDR and PrIC3, was clearly observed throughout the benchmarks. AdjointPDR[↓] outperformed LT-PDR, thus confirming empirically the theoretical observation in Section 7.4.2. The profit is particularly evident in those instances whose answer is positive. AdjointPDR[↓] generally outperformed PrIC3, too. Exceptions are in ZeroConf, Chain and DoubleChain, where PrIC3 with polynomial (*pol.*) and hybrid (*hyb.*) heuristics performs well. This seems to be thanks to the expressivity of the polynomial template in PrIC3, which is a possible

¹There are two more sound algorithms in Storm: one that utilizes interval iteration [Bai+17] and the other does optimistic VI [HK20]. We have excluded them from the results since we observed that they returned incorrect answers.

enhancement we are yet to implement (currently our symbolic heuristic **hCoS** uses only the affine template).

RQ2. The comparison with Storm is interesting. Note first that Storm’s *sparse-numeric* algorithm is a VI algorithm that gives a guaranteed lower bound *without guaranteed convergence*. Therefore its positive answer to $P \leq? \lambda$ may not be correct. Indeed, for Haddad-Monmege with $|S| \sim 10^3$, it answered $P = 0.5$ which is wrong ($\dagger$) in Table 7.1). This is in contrast with PDR algorithms that discovers an explicit witness for $P \leq \lambda$ via their positive chain.

Storm’s *sparse-rational* algorithm is precise. It was faster than PDR algorithms in many benchmarks, although **AdjointPDR**[↓] was better or comparable in ZeroConf (10^4) and Haddad-Monmege (41), for λ such that $P \leq \lambda$ is true. We believe this suggests a general advantage of PDR algorithms, namely to accelerate the search for an invariant-like witness for safety.

Storm’s *sparse-sound* algorithm is a sound VI algorithm that returns correct answers aside numerical errors. Its performance was similar to that of sparse-numeric, except for the two instances of Haddad-Monmege: sparse-sound returned correct answers but was much slower than sparse-numeric. For these two instances, **AdjointPDR**[↓] outperformed sparse-sound.

It seems that a big part of Storm’s good performance is attributed to the sparsity of state representation. This is notable in the comparison of the two instances of Haddad-Monmege (41 vs. 10^3): while Storm handles both of them easily, **AdjointPDR**[↓] struggles a bit in the bigger instance. Our implementation can be extended to use sparse representation, too; this is future work.

RQ3. We derived the three heuristics (**hCoB**, **hCo01**, **hCoS**) exploiting the theory of **AdjointPDR**[↓]. The experiments show that each heuristic has its own strength. For example, **hCo01** is slower than **hCoB** for MCs, but it is much better for MDPs. In general, there is no silver bullet heuristic, so coming up with a variety of them is important. The experiments suggest that our theory of **AdjointPDR**[↓] provides great help in doing so.

RQ4. Table 7.2 shows that **AdjointPDR**[↓] can handle nondeterminism well: once a suitable heuristic is chosen, its performances on MDPs and on MCs of similar size are comparable. It is also interesting that better-performing heuristics vary, as we discussed above.

7.8 Summary

In this chapter, we presents **AdjointPDR**, an algorithm that generalizes Bradley’s PDR [Bra11] to address the least fixpoint problem $\text{lfp}(f \vee i) \leq p$. The novelty in the algorithm lies in the use of a right adjoint $g: L \rightarrow L$ of the map f to search for counterexamples: the function f is used in the search of an over-approximation to show that the least fixpoint is below p , while the adjoint g produces candidate counterexamples – that is, under-approximations – to witness the violation of the property.

Similar to other PDR-like algorithms, **AdjointPDR** depends on some non-deterministic choices of elements $z \in L$. Therefore, not only we proved soundness of the algorithm for any possible resolution of nondeterminism, but we also showed that the algorithm always progresses to a new state. In general, **AdjointPDR** is not guaranteed to terminate since the problem $\text{lfp}(f \vee i) \leq p$ is undecidable. However, we showed that certain well-behaved heuristics, which essentially fix a legitimate choice of z , ensure termination whenever the problem has a negative answer.

The assumption that f possesses a right adjoint g is not satisfied in several interesting

cases, particularly those involving probabilistic systems. To address these, we introduce a variation of the algorithm, called $\text{AdjointPDR}^\downarrow$, that employs lower sets to guarantee the presence of a right adjoint. In this algorithm, a witness for the positive answer is sought in the lattice L , while a counterexample in the lattice of lower sets $L^\downarrow$. We demonstrated that most properties of AdjointPDR hold for $\text{AdjointPDR}^\downarrow$ and that $\text{AdjointPDR}^\downarrow$ simulates LT-PDR [Kor+22], a preceding lattice-theoretical generalization of PDR. Conversely, LT-PDR cannot simulate $\text{AdjointPDR}^\downarrow$: indeed, the use of $L^\downarrow$ enables $\text{AdjointPDR}^\downarrow$ to simultaneously search for multiple – even all – counterexamples computed by LT-PDR at the same time.

We instantiated $\text{AdjointPDR}^\downarrow$ to address the max-reachability problem of Markov Decision Processes. Specifically, we devised two heuristics, named hCoB and hCo01 , which cleverly reduce the search space and are guaranteed to terminate in the case of a negative answer. We conducted experimental comparisons, using $\text{AdjointPDR}^{\text{AI}}$ as a basis for implementation, of the two heuristics with other tools, yielding promising results: our implementation clearly outperforms other PDR-based algorithms in many benchmarks, and even compares favourably with Storm—a highly sophisticated toolsuite—in a couple of benchmarks. These are notable especially given that $\text{AdjointPDR}^\downarrow$ currently lacks enhancing features such as richer symbolic templates and sparse representation (adding which is future work). Overall, we believe that $\text{AdjointPDR}^\downarrow$ *confirms the potential of PDR algorithms in probabilistic model checking*. Through the three heuristics, we also observed the value of an abstract general theory in devising heuristics in PDR, which is probably true of verification algorithms in general besides PDR.

Chapter 8

Conclusions

Since proving *incorrectness* via under-approximation is a concept that recently attracted lots of attention, in this thesis we explored the possibility of combining over and under-approximation to improve the ability of both at proving a program correct or incorrect.

Surveying the literature, we found that most works in this direction (the embedding in KATs, LCL_A and OL) are very recent, but PDR stood out as older. While it was probably not understood in these terms at the time, the success of PDR as a model checking technique shows the potential of this approach.

Given the impact of over-approximation based abstract interpretation in the established field of program analysis, our first question was whether under-approximation based abstract interpretation could have the same potential. It turns out that, for a wide class of analyses, the classical formulation of abstraction based on Galois connections is hard to turn effectively for under-approximation. Given this limitation, we then considered logical frameworks for under-approximation, and introduced a backward analogous to IL, which we dubbed SIL. Moreover, by comparing SIL, IL and two over-approximation logics (HL and NC) we further consolidated our understanding of the relations between over and under-approximation.

We apply the insight gained from the above studies in extending two known techniques. First, we propose a new framework for LCL_A that is able to prove intensional properties of programs by changing the domain in which (part of) the analysis is performed. We also present a way to circumvent the requirements of best abstraction, basically by performing a sanity check which guarantees soundness. Lastly, we turn LCL_A over for backward analysis by using SIL instead of IL as the underlying program logic: the duality between the two proof systems streamlines the transfer of concepts. Second, we introduce a new PDR-like algorithm which uses an adjoint (roughly corresponding to the backward semantics) to speed up the search for counterexamples. This algorithm allowed us to develop a theory of heuristics in which we can schematize, compare and have some reference points for devising new heuristics. We instantiate our algorithm for probabilistic systems: the experiments suggest that our approach is overall better than other PDR-like algorithms.

The main message of the thesis is that the interaction between over and under-approximation is far from trivial but shows great potential. Efficient exchange of meaningful information between over and under-approximation is sometimes elusive. In our opinion, this is partly due to the broken symmetry between over and under-approximation. However, if understood correctly, this information sharing can improve the combined search for correctness and incorrectness: under-approximation can guide the refinement process of the over-approximation, while over-approximation prevents under-approximation from forgetting too much.

Our theoretical investigations have been complemented with experimental prototypes, that made possible to double check our examples, gain confidence in our methodologies and compare to the state of the art.

Overall, the results in this thesis put new clarity in the relations and asymmetries between over and under-approximation which can serve as a basis for future investigations and applications. We outline here some possible research direction to pursue in the future.

Future work. By analogy with PDR, where forward and backward analysis interact positively, we are thinking of a combination of IL and SIL. More precisely, we observe that the two logics share the structural rules (eg. sequential composition, finite loop unrolling). Therefore, given a proof tree in one of the two logics, the other is able to follow the same shape to prove a triple for the same program. For instance, suppose to perform a forward analysis with IL which highlights some reachable error state. To pinpoint inputs leading to such errors, you could use SIL. Instead of following all possible program paths backward to find such sources, the IL proof tree already identifies program paths leading to those errors, that can thus be followed with SIL by mimicking the proof tree shape.

In our opinion, the ability of LCL_A and $CLCL$ to prove both correctness and incorrectness of a program at the same time shows potential for a useful tool. Indeed, our extensions allow them to be applied more easily in practice. Therefore, we are interested in exploring an implementation which combines domain refinement and abstract interpretation repair [BGGR22] for practical analyses.

Cousot’s calculational design of program logics [Cou24] opens the way to the possibility of comparing and combining over and under-approximation approaches from the abstract interpretation perspective. The most appealing aspect of this approach is the ability to automatically derive the combination, just like Cousot derives proof systems from a given composition of abstraction.

Another aspects highlighted by Cousot’s calculational design is that negation of HL (in a different flavour than the isomorphisms in Proposition 5.9) correspond to testing. It is interesting to study the meaning of the negation of NC, IL and SIL, too.

PDR presents an interesting framework which combines a forward search of a safe invariant with a backward search of a counterexample. We are considering other instantiations of this general idea, for instance via abstract interpretation for the over-approximation and abstract interpretation repair as the under. Specifically for program analysis, we are also considering a similar approach using HL for the forward over-approximation and SIL for the backward search for counterexamples, tailoring the algorithm to the stepwise program execution similarly to IC3 software model checking [LNNK20].

Bibliography

- [Apt81] Krzysztof R. Apt. “Ten Years of Hoare’s Logic: A Survey—Part I”. In: *ACM Trans. Program. Lang. Syst.* 3.4 (Oct. 1981), pp. 431–483. ISSN: 0164-0925. DOI: 10.1145/357146.357150.
- [Apt84] Krzysztof R. Apt. “Ten Years of Hoare’s Logic: A Survey Part II: Nondeterminism”. In: *Theor. Comput. Sci.* 28 (1984), pp. 83–109. DOI: 10.1016/0304-3975(83)90066-X.
- [AO19] Krzysztof R. Apt and Ernst-Rüdiger Olderog. “Fifty years of Hoare’s logic”. In: *Formal Aspects Comput.* 31.6 (2019), pp. 751–807. DOI: 10.1007/S00165-019-00501-3.
- [ABG22] Flavio Ascari, Roberto Bruni, and Roberta Gori. “Limits and difficulties in the design of under-approximation abstract domains”. In: *Foundations of Software Science and Computation Structures - 25th International Conference, FOSSACS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*. Ed. by Patricia Bouyer and Lutz Schröder. Vol. 13242. Lecture Notes in Computer Science. Springer, 2022, pp. 21–39. DOI: 10.1007/978-3-030-99253-8_2.
- [ABG23] Flavio Ascari, Roberto Bruni, and Roberta Gori. “Logics for Extensional, Locally Complete Analysis via Domain Refinements”. In: *Programming Languages and Systems - 32nd European Symposium on Programming, ESOP 2023, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2023, Paris, France, April 22-27, 2023, Proceedings*. Ed. by Thomas Wies. Vol. 13990. Lecture Notes in Computer Science. Springer, 2023, pp. 1–27. DOI: 10.1007/978-3-031-30044-8_1.
- [ABG24] Flavio Ascari, Roberto Bruni, and Roberta Gori. “Limits and Difficulties in the Design of Under-Approximation Abstract Domains”. In: *ACM Trans. Program. Lang. Syst.* (June 2024). Just Accepted. ISSN: 0164-0925. DOI: 10.1145/3666014.
- [ABGL24] Flavio Ascari, Roberto Bruni, Roberta Gori, and Francesco Logozzo. *Sufficient Incorrectness Logic: SIL and Separation SIL*. Preprint. 2024. arXiv: 2310.18156 [cs.LG].
- [Ass+17] Mounir Assaf, David A. Naumann, Julien Signoles, Éric Totel, and Frédéric Tronel. “Hypercollecting Semantics and Its Application to Static Analysis of Information Flow”. In: *SIGPLAN Not.* 52.1 (Jan. 2017), pp. 874–887. ISSN: 0362-1340. DOI: 10.1145/3093333.3009889.
- [BK08] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT Press, 2008. ISBN: 978-0-262-02649-9.

- [Bai+17] Christel Baier, Joachim Klein, Linda Leuschner, David Parker, and Sascha Wunderlich. “Ensuring the Reliability of Your Model Checker: Interval Iteration for Markov Decision Processes”. In: *Proc. of CAV 2017, Part I*. Ed. by Rupak Majumdar and Viktor Kuncak. Vol. 10426. Lecture Notes in Computer Science. Springer, 2017, pp. 160–180. DOI: 10.1007/978-3-319-63387-9_8.
- [BRZ22] Paolo Baldan, Francesco Ranzato, and Linpeng Zhang. “Intensional Kleene and Rice theorems for abstract program semantics”. In: *Inf. Comput.* 289.Part (2022), p. 104953. DOI: 10.1016/J.IC.2022.104953.
- [BKY05] Thomas Ball, Orna Kupferman, and Greta Yorsh. “Abstraction for Falsification”. In: *Computer Aided Verification, 17th International Conference, CAV 2005, Edinburgh, Scotland, UK, July 6-10, 2005, Proceedings*. Ed. by Kousha Etessami and Sriram K. Rajamani. Vol. 3576. Lecture Notes in Computer Science. Springer, 2005, pp. 67–81. DOI: 10.1007/11513988_8.
- [BR01] Thomas Ball and Sriram K. Rajamani. “The SLAM Toolkit”. In: *Computer Aided Verification, 13th International Conference, CAV 2001, Paris, France, July 18-22, 2001, Proceedings*. Ed. by Gérard Berry, Hubert Comon, and Alain Finkel. Vol. 2102. Lecture Notes in Computer Science. Springer, 2001, pp. 260–264. DOI: 10.1007/3-540-44585-4_25.
- [Bat+20] Kevin Batz, Sebastian Junges, Benjamin Lucien Kaminski, Joost-Pieter Katoen, Christoph Matheja, and Philipp Schröder. “PrIC3: Property Directed Reachability for MDPs”. In: *Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21-24, 2020, Proceedings, Part II*. Ed. by Shuvendu K. Lahiri and Chao Wang. Vol. 12225. Lecture Notes in Computer Science. Springer, 2020, pp. 512–538. DOI: 10.1007/978-3-030-53291-8_27. URL: https://doi.org/10.1007/978-3-030-53291-8%5C_27.
- [BC05] Marc Bezem and Thierry Coquand. “Automating Coherent Logic”. In: *Logic for Programming, Artificial Intelligence, and Reasoning, 12th International Conference, LPAR 2005, Montego Bay, Jamaica, December 2-6, 2005, Proceedings*. Ed. by Geoff Sutcliffe and Andrei Voronkov. Vol. 3835. Lecture Notes in Computer Science. Springer, 2005, pp. 246–260. DOI: 10.1007/11591191_18.
- [Bla+03] Bruno Blanchet, Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. “A Static Analyzer for Large Safety-Critical Software”. In: *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation 2003, San Diego, California, USA, June 9-11, 2003*. Ed. by Ron Cytron and Rajiv Gupta. ACM, 2003, pp. 196–207. DOI: 10.1145/781131.781153.
- [BG87] Andreas Blass and Yuri Gurevich. “Existential Fixed-Point Logic”. In: *Computation Theory and Logic, In Memory of Dieter Rödding*. Ed. by Egon Börger. Vol. 270. Lecture Notes in Computer Science. Springer, 1987, pp. 20–36. DOI: 10.1007/3-540-18170-9_151.
- [BC19] Graham Bleaney and Sinan Cepel. *Pysa: An Open Source Static Analysis Tool to Detect and Prevent Security Issues in Python Code*. <https://engineering.fb.com/2021/09/29/security/mariana-trench/>. 2019.

- [BGGP18] Filippo Bonchi, Pierre Ganty, Roberto Giacobazzi, and Dusko Pavlovic. “Sound up-to techniques and Complete abstract domains”. In: *Proc. of LICS 2018*. Ed. by Anuj Dawar and Erich Grädel. ACM, 2018, pp. 175–184. DOI: 10.1145/3209108.3209169.
- [Bra11] Aaron R. Bradley. “SAT-Based Model Checking without Unrolling”. In: *Verification, Model Checking, and Abstract Interpretation - 12th International Conference, VMCAI 2011, Austin, TX, USA, January 23-25, 2011. Proceedings*. Ed. by Ranjit Jhala and David A. Schmidt. Vol. 6538. Lecture Notes in Computer Science. Springer, 2011, pp. 70–87. DOI: 10.1007/978-3-642-18275-4_7.
- [Bra12] Aaron R. Bradley. “Understanding IC3”. In: *Theory and Applications of Satisfiability Testing - SAT 2012 - 15th International Conference, Trento, Italy, June 17-20, 2012. Proceedings*. Ed. by Alessandro Cimatti and Roberto Sebastiani. Vol. 7317. Lecture Notes in Computer Science. Springer, 2012, pp. 1–14. DOI: 10.1007/978-3-642-31612-8_1.
- [Bru+19] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, Isabel Garcia-Contreras, and Dusko Pavlovic. “Abstract Extensionality: On the Properties of Incomplete Abstract Interpretations”. In: *Proc. ACM Program. Lang.* 4.POPL (Dec. 2019). DOI: 10.1145/3371096.
- [BGGR21] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. “A Logic for Locally Complete Abstract Interpretations”. In: *36th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2021, Rome, Italy, June 29 - July 2, 2021*. IEEE, 2021, pp. 1–13. DOI: 10.1109/LICS52264.2021.9470608.
- [BGGR22] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. “Abstract interpretation repair”. In: *PLDI ’22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022*. Ed. by Ranjit Jhala and Isil Dillig. ACM, 2022, pp. 426–441. DOI: 10.1145/3519939.3523453.
- [BGGR23] Roberto Bruni, Roberto Giacobazzi, Roberta Gori, and Francesco Ranzato. “A Correctness and Incorrectness Program Logic”. In: *J. ACM* 70.2 (2023), 15:1–15:45. DOI: 10.1145/3582267.
- [CDOY09] Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. “Compositional shape analysis by means of bi-abduction”. In: *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2009, Savannah, GA, USA, January 21-23, 2009*. Ed. by Zhong Shao and Benjamin C. Pierce. ACM, 2009, pp. 289–300. DOI: 10.1145/1480881.1480917.
- [Car+22] Quentin Carbonneaux, Noam Zilberstein, Christoph Klee, Peter W. O’Hearn, and Francesco Zappa Nardelli. “Applying Formal Verification to Microkernel IPC at Meta”. In: *CPP ’22: 11th ACM SIGPLAN International Conference on Certified Programs and Proofs, Philadelphia, PA, USA, January 17 - 18, 2022*. Ed. by Andrei Popescu and Steve Zdancewic. ACM, 2022, pp. 116–129. DOI: 10.1145/3497775.3503681.

- [CG12] Alessandro Cimatti and Alberto Griggio. “Software Model Checking via IC3”. In: *Computer Aided Verification - 24th International Conference, CAV 2012, Berkeley, CA, USA, July 7-13, 2012 Proceedings*. Ed. by P. Madhusudan and Sanjit A. Seshia. Vol. 7358. Lecture Notes in Computer Science. Springer, 2012, pp. 277–293. DOI: 10.1007/978-3-642-31424-7_23.
- [Cla+00] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. “Counterexample-Guided Abstraction Refinement”. In: *Proc. of CAV’00*. Ed. by E. Allen Emerson and Aravinda Prasad Sistla. Springer, 2000, pp. 154–169. ISBN: 978-3-540-45047-4. DOI: 10.1007/10722167_15.
- [Coh+09] Ernie Cohen, Markus Dahlweid, Mark A. Hillebrand, Dirk Leinenbach, Michal Moskal, Thomas Santen, Wolfram Schulte, and Stephan Tobies. “VCC: A Practical System for Verifying Concurrent C”. In: *Theorem Proving in Higher Order Logics, 22nd International Conference, TPHOLs 2009, Munich, Germany, August 17-20, 2009. Proceedings*. Ed. by Stefan Berghofer, Tobias Nipkow, Christian Urban, and Makarius Wenzel. Vol. 5674. Lecture Notes in Computer Science. Springer, 2009, pp. 23–42. DOI: 10.1007/978-3-642-03359-9_2.
- [CL96] Christopher Colby and Peter Lee. “Trace-Based Program Analysis”. In: *Conference Record of POPL’96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, St. Petersburg Beach, Florida, USA, January 21-24, 1996*. Ed. by Hans-Juergen Boehm and Guy L. Steele Jr. ACM Press, 1996, pp. 195–207. DOI: 10.1145/237721.237776.
- [Coo78] Stephen A. Cook. “Soundness and Completeness of an Axiom System for Program Verification”. In: *SIAM J. Comput.* 7.1 (1978), pp. 70–90. DOI: 10.1137/0207005.
- [Cou24] Patrick Cousot. “Calculational Design of [In]Correctness Transformational Program Logics by Abstract Interpretation”. In: *Proc. ACM Program. Lang.* 8.POPL (2024), pp. 175–208. DOI: 10.1145/3632849.
- [CC77] Patrick Cousot and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints”. In: *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. POPL ’77. Los Angeles, California: Association for Computing Machinery, 1977, pp. 238–252. ISBN: 9781450373500. DOI: 10.1145/512950.512973.
- [CC79] Patrick Cousot and Radhia Cousot. “Systematic Design of Program Analysis Frameworks”. In: *Proceedings of the 6th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. POPL ’79. San Antonio, Texas: Association for Computing Machinery, 1979, pp. 269–282. ISBN: 9781450373579. DOI: 10.1145/567752.567778.
- [CC92] Patrick Cousot and Radhia Cousot. “Abstract Interpretation Frameworks”. In: *J. Log. Comput.* 2.4 (1992), pp. 511–547. DOI: 10.1093/LOGCOM/2.4.511.
- [CCFL13] Patrick Cousot, Radhia Cousot, Manuel Fähndrich, and Francesco Logozzo. “Automatic Inference of Necessary Preconditions”. In: *Verification, Model Checking, and Abstract Interpretation, 14th International Conference, VMCAI 2013, Rome, Italy, January 20-22, 2013. Proceedings*. Ed. by Roberto Giacobazzi, Josh Berdine, and Isabella Mastroeni. Vol. 7737. Lecture Notes

- in Computer Science. Springer, 2013, pp. 128–148. DOI: 10.1007/978-3-642-35873-9_10.
- [Cou+05] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. “The ASTREÉ Analyzer”. In: *Programming Languages and Systems, 14th European Symposium on Programming, ESOP 2005, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2005, Edinburgh, UK, April 4-8, 2005, Proceedings*. Ed. by Shmuel Sagiv. Vol. 3444. Lecture Notes in Computer Science. Springer, 2005, pp. 21–30. DOI: 10.1007/978-3-540-31987-0_3.
- [Cou+06] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. “Combination of Abstractions in the ASTRÉE Static Analyzer”. In: *Advances in Computer Science - ASIAN 2006. Secure Software and Related Issues, 11th Asian Computing Science Conference, Tokyo, Japan, December 6-8, 2006, Revised Selected Papers*. Ed. by Mitsu Okada and Ichiro Satoh. Vol. 4435. Lecture Notes in Computer Science. Springer, 2006, pp. 272–300. DOI: 10.1007/978-3-540-77505-8_23.
- [CCL11] Patrick Cousot, Radhia Cousot, and Francesco Logozzo. “Precondition Inference from Intermittent Assertions and Application to Contracts on Collections”. In: *Verification, Model Checking, and Abstract Interpretation - 12th International Conference, VMCAI 2011, Austin, TX, USA, January 23-25, 2011. Proceedings*. Ed. by Ranjit Jhala and David A. Schmidt. Vol. 6538. Lecture Notes in Computer Science. Springer, 2011, pp. 150–168. DOI: 10.1007/978-3-642-18275-4_12.
- [CCLB12] Patrick Cousot, Radhia Cousot, Francesco Logozzo, and Michael Barnett. “An Abstract Interpretation Framework for Refactoring with Application to Extract Methods with Contracts”. In: *Proceedings of the 27th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2012, part of SPLASH 2012, Tucson, AZ, USA, October 21-25, 2012*. Ed. by Gary T. Leavens and Matthew B. Dwyer. ACM, 2012, pp. 213–232. DOI: 10.1145/2384616.2384633.
- [CH78] Patrick Cousot and Nicolas Halbwachs. “Automatic Discovery of Linear Constraints among Variables of a Program”. In: *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. POPL ’78. Tucson, Arizona: Association for Computing Machinery, 1978, pp. 84–96. ISBN: 9781450373487. DOI: 10.1145/512760.512770.
- [DP02] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. 2nd ed. Cambridge University Press, 2002. ISBN: 9780511809088. DOI: 10.1017/CB09780511809088.
- [DJKV17] Christian Dehnert, Sebastian Junges, Joost-Pieter Katoen, and Matthias Volk. “A Storm is Coming: A Modern Probabilistic Model Checker”. In: *Proc. of CAV 2017, Part II*. Ed. by Rupak Majumdar and Viktor Kuncak. Vol. 10427. Lecture Notes in Computer Science. Springer, 2017, pp. 592–600. DOI: 10.1007/978-3-319-63390-9_31.
- [DMS06] Jules Desharnais, Bernhard Möller, and Georg Struth. “Kleene Algebra with Domain”. In: *ACM Trans. Comput. Logic* 7.4 (Oct. 2006), pp. 798–833. ISSN: 1529-3785. DOI: 10.1145/1183278.1183285.

- [Dij70] Edsger W. Dijkstra. “Notes on Structured Programming”. circulated privately. Apr. 1970. URL: <http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD249.PDF>.
- [Dij75] Edsger W. Dijkstra. “Guarded Commands, Nondeterminacy and Formal Derivation of Programs”. In: *Commun. ACM* 18.8 (Aug. 1975), pp. 453–457. ISSN: 0001-0782. DOI: 10.1145/360933.360975.
- [DFLO19] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W. O’Hearn. “Scaling Static Analyses at Facebook”. In: *Commun. ACM* 62.8 (2019), pp. 62–70. DOI: 10.1145/3338112.
- [Flo67] Robert W. Floyd. “Assigning Meanings to Programs”. In: *Proceedings of Symposium on Applied Mathematics* 19 (1967), pp. 19–32. URL: <http://laser.cs.umass.edu/courses/cs521-621.Spr06/papers/Floyd.pdf>.
- [Gab21] Dominik Gabi. *Open-sourcing Mariana Trench: Analyzing Android and Java App Security in Depth*. <https://engineering.fb.com/2021/09/29/security/mariana-trench/>. 2021.
- [GLR15] Roberto Giacobazzi, Francesco Logozzo, and Francesco Ranzato. “Analyzing Program Analyses”. In: *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*. Ed. by Sriram K. Rajamani and David Walker. ACM, 2015, pp. 261–273. DOI: 10.1145/2676726.2676987.
- [GQ01] Roberto Giacobazzi and Elisa Quintarelli. “Incompleteness, Counterexamples, and Refinements in Abstract Model-Checking”. In: *Static Analysis, 8th International Symposium, SAS 2001, Paris, France, July 16-18, 2001, Proceedings*. Ed. by Patrick Cousot. Vol. 2126. Lecture Notes in Computer Science. Springer, 2001, pp. 356–373. DOI: 10.1007/3-540-47764-0_20.
- [GRS00] Roberto Giacobazzi, Francesco Ranzato, and Francesca Scozzari. “Making Abstract Interpretations Complete”. In: *J. ACM* 47.2 (Mar. 2000), pp. 361–416. ISSN: 0004-5411. DOI: 10.1145/333979.333989.
- [GS97] Susanne Graf and Hassen Saïdi. “Construction of Abstract State Graphs with PVS”. In: *Computer Aided Verification, 9th International Conference, CAV ’97, Haifa, Israel, June 22-25, 1997, Proceedings*. Ed. by Orna Grumberg. Vol. 1254. Lecture Notes in Computer Science. Springer, 1997, pp. 72–83. DOI: 10.1007/3-540-63166-6_10.
- [Gra91] Philippe Granger. “Static Analysis of Linear Congruence Equalities among Variables of a Program”. In: *TAPSOFT’91: Proceedings of the International Joint Conference on Theory and Practice of Software Development, Brighton, UK, April 8-12, 1991, Volume 1: Colloquium on Trees in Algebra and Programming (CAAP’91)*. Ed. by Samson Abramsky and T. S. E. Maibaum. Vol. 493. Lecture Notes in Computer Science. Springer, 1991, pp. 169–192. DOI: 10.1007/3-540-53982-4_10.
- [Har79] David Harel. *First-Order Dynamic Logic*. Vol. 68. Lecture Notes in Computer Science. Springer, 1979. ISBN: 3-540-09237-4. DOI: 10.1007/3-540-09237-4.
- [HK20] Arnd Hartmanns and Benjamin Lucien Kaminski. “Optimistic Value Iteration”. In: *Proc. of CAV 2020, Part II*. Ed. by Shuvendu K. Lahiri and Chao Wang. Vol. 12225. Lecture Notes in Computer Science. Springer, 2020, pp. 488–511. DOI: 10.1007/978-3-030-53291-8_26.

- [Har+19] Arnd Hartmanns, Michaela Klauck, David Parker, Tim Quatmann, and Enno Ruijters. “The Quantitative Verification Benchmark Set”. In: *Proc. of TACAS 2019, Part I*. Ed. by Tomás Vojnar and Lijun Zhang. Vol. 11427. Lecture Notes in Computer Science. Springer, 2019, pp. 344–350. DOI: 10.1007/978-3-030-17462-0_20.
- [HJMS02] Thomas A. Henzinger, Ranjit Jhala, Rupak Majumdar, and Grégoire Sutre. “Lazy abstraction”. In: *Proc. of POPL’02*. Ed. by John Launchbury and John C. Mitchell. ACM, 2002, pp. 58–70. DOI: 10.1145/503272.503279.
- [Hoa69] C. A. R. Hoare. “An Axiomatic Basis for Computer Programming”. In: *Commun. ACM* 12.10 (1969), pp. 576–580. DOI: 10.1145/363235.363259.
- [Hoa78] C. A. R. Hoare. “Some Properties of Predicate Transformers”. In: *J. ACM* 25.3 (1978), pp. 461–480. DOI: 10.1145/322077.322088.
- [HL74] C. A. R. Hoare and Peter E. Lauer. “Consistent and Complementary Formal Theories of the Semantics of Programming Languages”. In: *Acta Informatica* 3 (1974), pp. 135–153. DOI: 10.1007/BF00264034.
- [Kel82] Gregory Maxwell Kelly. *Basic concepts of enriched category theory*. Vol. 64. CUP Archive, 1982.
- [Kor+23] Mayuko Kori, Flavio Ascari, Filippo Bonchi, Roberto Bruni, Roberta Gori, and Ichiro Hasuo. “Exploiting Adjoints in Property Directed Reachability Analysis”. In: *Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II*. Ed. by Constantin Enea and Akash Lal. Vol. 13965. Lecture Notes in Computer Science. Springer, 2023, pp. 41–63. DOI: 10.1007/978-3-031-37703-7_3.
- [Kor+22] Mayuko Kori, Natsuki Urabe, Shin-ya Katsumata, Kohei Suenaga, and Ichiro Hasuo. “The Lattice-Theoretic Essence of Property Directed Reachability Analysis”. In: *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part I*. Ed. by Sharon Shoham and Yakir Vizel. Vol. 13371. Lecture Notes in Computer Science. Springer, 2022, pp. 235–256. DOI: 10.1007/978-3-031-13185-1_12.
- [Koz94] Dexter Kozen. “A Completeness Theorem for Kleene Algebras and the Algebra of Regular Events”. In: *Inf. Comput.* 110.2 (1994), pp. 366–390. DOI: 10.1006/INCO.1994.1037.
- [Koz97] Dexter Kozen. “Kleene Algebra with Tests”. In: *ACM Trans. Program. Lang. Syst.* 19.3 (May 1997), pp. 427–443. ISSN: 0164-0925. DOI: 10.1145/256167.256195.
- [Koz00] Dexter Kozen. “On Hoare Logic and Kleene Algebra with Tests”. In: *ACM Trans. Comput. Logic* 1.1 (July 2000), pp. 60–76. ISSN: 1529-3785. DOI: 10.1145/343369.343378.
- [KNP11] Marta Z. Kwiatkowska, Gethin Norman, and David Parker. “PRISM 4.0: Verification of Probabilistic Real-Time Systems”. In: *Proc. of CAV 2011*. Ed. by Ganesh Gopalakrishnan and Shaz Qadeer. Vol. 6806. Lecture Notes in Computer Science. Springer, 2011, pp. 585–591. DOI: 10.1007/978-3-642-22110-1_47.

- [LNNK20] Tim Lange, Martin R. Neuhäuser, Thomas Noll, and Joost-Pieter Katoen. “IC3 software model checking”. In: *Int. J. Softw. Tools Technol. Transf.* 22.2 (2020), pp. 135–161. DOI: 10.1007/s10009-019-00547-x.
- [LL09] Vincent Laviron and Francesco Logozzo. “Refining Abstract Interpretation-Based Static Analyses with Hints”. In: *Proc. of APLAS’09*. Ed. by Zhenjiang Hu. Vol. 5904. LNCS. Springer, 2009, pp. 343–358. DOI: 10.1007/978-3-642-10672-9_24.
- [Law69] F. William Lawvere. “Adjointness in foundations”. In: *Dialectica* (1969), pp. 281–296.
- [Le+22] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. “Finding Real Bugs in Big Programs with Incorrectness Logic”. In: *Proc. ACM Program. Lang.* 6.OOPSLA1 (2022), pp. 1–27. DOI: 10.1145/3527325.
- [Ler09] Xavier Leroy. “Formal Verification of a Realistic Compiler”. In: *Commun. ACM* 52.7 (2009), pp. 107–115. DOI: 10.1145/1538788.1538814.
- [LSRG07] Tal Lev-Ami, Mooly Sagiv, Thomas Reps, and Sumit Gulwani. “Backward analysis for inferring quantified preconditions”. In: *Tr-2007-12-01, Tel Aviv University* (2007).
- [Lev04] Paul Blain Levy. *Call-By-Push-Value: A Functional/Imperative Synthesis*. Vol. 2. Semantics Structures in Computation. Springer, 2004. ISBN: 1-4020-1730-8.
- [McM06] Kenneth L. McMillan. “Lazy Abstraction with Interpolants”. In: *Proc. of CAV’06*. Ed. by Thomas Ball and Robert B. Jones. Vol. 4144. LNCS. Springer, 2006, pp. 123–136. DOI: 10.1007/11817963_14.
- [MR22] Marco Milanese and Francesco Ranzato. “Local Completeness Logic on Kleene Algebra with Tests”. In: *Static Analysis - 29th International Symposium, SAS 2022, Auckland, New Zealand, December 5-7, 2022, Proceedings*. Ed. by Gagandeep Singh and Caterina Urban. Vol. 13790. Lecture Notes in Computer Science. Springer, 2022, pp. 350–371. DOI: 10.1007/978-3-031-22308-2_16.
- [Mil89] R. Milner. *Communication and Concurrency*. USA: Prentice-Hall, Inc., 1989. ISBN: 0131149849.
- [Min06] A. Miné. “The octagon abstract domain”. In: *High. Order Symb. Comput.* 19.1 (2006), pp. 31–100. DOI: 10.1007/s10990-006-8609-1.
- [Min14] Antoine Miné. “Backward Under-Approximations in Numeric Abstract Domains to Automatically Infer Sufficient Program Conditions”. In: *Sci. Comput. Program.* 93 (Nov. 2014), pp. 154–182. ISSN: 0167-6423. DOI: 10.1016/j.scico.2013.09.014.
- [MOH21] Bernhard Möller, Peter W. O’Hearn, and Tony Hoare. “On Algebra of Program Correctness and Incorrectness”. In: *Relational and Algebraic Methods in Computer Science - 19th International Conference, RAMiCS 2021, Marseille, France, November 2-5, 2021, Proceedings*. Ed. by Uli Fahrenberg, Mai Gehrke, Luigi Santocanale, and Michael Winter. Vol. 13027. Lecture Notes in Computer Science. Springer, 2021, pp. 325–343. DOI: 10.1007/978-3-030-88701-8_20.

- [MB08] Leonardo Mendonça de Moura and Nikolaj S. Bjørner. “Z3: An Efficient SMT Solver”. In: *Proc. of TACAS 2008*. Ed. by C. R. Ramakrishnan and Jakob Rehof. Vol. 4963. Lecture Notes in Computer Science. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24.
- [OHe20] Peter W. O’Hearn. “Incorrectness logic”. In: *Proc. ACM Program. Lang.* 4.POPL (2020), 10:1–10:32. DOI: 10.1145/3371078.
- [ORY01] Peter W. O’Hearn, John C. Reynolds, and Hongseok Yang. “Local Reasoning about Programs that Alter Data Structures”. In: *Computer Science Logic, 15th International Workshop, CSL 2001. 10th Annual Conference of the EACSL, Paris, France, September 10-13, 2001, Proceedings*. Ed. by Laurent Fribourg. Vol. 2142. Lecture Notes in Computer Science. Springer, 2001, pp. 1–19. DOI: 10.1007/3-540-44802-0_1.
- [QK18] Tim Quatmann and Joost-Pieter Katoen. “Sound Value Iteration”. In: *Proc. of CAV 2018, Part I*. Ed. by Hana Chockler and Georg Weissenbacher. Vol. 10981. Lecture Notes in Computer Science. Springer, 2018, pp. 643–661. DOI: 10.1007/978-3-319-96145-3_37.
- [Raa+20] Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter W. O’Hearn, and Jules Villard. “Local Reasoning About the Presence of Bugs: Incorrectness Separation Logic”. In: *Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21-24, 2020, Proceedings, Part II*. Ed. by Shuvendu K. Lahiri and Chao Wang. Vol. 12225. Lecture Notes in Computer Science. Springer, 2020, pp. 225–252. DOI: 10.1007/978-3-030-53291-8_14.
- [RBDO22] Azalea Raad, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. “Concurrent Incorrectness Separation Logic”. In: *Proc. ACM Program. Lang.* 6.POPL (2022), pp. 1–29. DOI: 10.1145/3498695.
- [RVBO23] Azalea Raad, Julien Vanegue, Josh Berdine, and Peter W. O’Hearn. “A General Approach to Under-Approximate Reasoning About Concurrent Programs”. In: *34th International Conference on Concurrency Theory, CONCUR 2023, September 18-23, 2023, Antwerp, Belgium*. Ed. by Guillermo A. Pérez and Jean-François Raskin. Vol. 279. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023, 25:1–25:17. DOI: 10.4230/LIPICS.CONCUR.2023.25.
- [RVO24] Azalea Raad, Julien Vanegue, and Peter O’Hearn. *Compositional Non-Termination Proving*. Preprint. 2024. URL: <https://www.soundandcomplete.org/papers/Unter.pdf>.
- [Rey02] John C. Reynolds. “Separation Logic: A Logic for Shared Mutable Data Structures”. In: *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*. IEEE Computer Society, 2002, pp. 55–74. DOI: 10.1109/LICS.2002.1029817.
- [Sch07] David A. Schmidt. “A calculus of logical relations for over- and underapproximating static analyses”. In: *Sci. Comput. Program.* 64.1 (2007), pp. 29–53. DOI: 10.1016/j.scico.2006.03.008.
- [SS17] Tobias Seufert and Christoph Scholl. “Sequential Verification Using Reverse PDR”. In: *Proc. of MBMV 2017*. Ed. by Daniel Große and Rolf Drechsler. Shaker Verlag, 2017, pp. 79–90.

- [SPV15] Gagandeep Singh, Markus Püschel, and Martin Vechev. “Making Numerical Program Analysis Fast”. In: *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI ’15. Portland, OR, USA: Association for Computing Machinery, 2015, pp. 303–313. ISBN: 9781450334686. DOI: 10.1145/2737924.2738000.
- [SPV17a] Gagandeep Singh, Markus Püschel, and Martin Vechev. “A Practical Construction for Decomposing Numerical Abstract Domains”. In: 2.POPL (Dec. 2017). DOI: 10.1145/3158143.
- [SPV17b] Gagandeep Singh, Markus Püschel, and Martin Vechev. “Fast Polyhedra Abstract Domain”. In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. POPL 2017. Paris, France: Association for Computing Machinery, 2017, pp. 46–59. ISBN: 9781450346603. DOI: 10.1145/3009837.3009885.
- [VK23] Lena Verscht and Benjamin Kaminski. *Hoare-Like Triples and Kleene Algebras with Top and Tests: Towards a Holistic Perspective on Hoare Logic, Incorrectness Logic, and Beyond*. Preprint. 2023. arXiv: 2312.09662 [cs.LO].
- [VK11] Edsko de Vries and Vasileios Koutavas. “Reverse Hoare Logic”. In: *Software Engineering and Formal Methods - 9th International Conference, SEFM 2011, Montevideo, Uruguay, November 14-18, 2011. Proceedings*. Ed. by Gilles Barthe, Alberto Pardo, and Gerardo Schneider. Vol. 7041. Lecture Notes in Computer Science. Springer, 2011, pp. 155–171. DOI: 10.1007/978-3-642-24690-6_12.
- [Win93] G. Winskel. *The Formal Semantics of Programming Languages: an Introduction*. MIT press, 1993.
- [ZAG22] Cheng Zhang, Arthur Azevedo de Amorim, and Marco Gaboardi. “On Incorrectness Logic and Kleene Algebra with Top and Tests”. In: *Proc. ACM Program. Lang.* 6.POPL (Jan. 2022). DOI: 10.1145/3498690.
- [ZK22] Linpeng Zhang and Benjamin Lucien Kaminski. “Quantitative strongest post: a calculus for reasoning about the flow of quantitative information”. In: *Proc. ACM Program. Lang.* 6.OOPSLA1 (2022), pp. 1–29. DOI: 10.1145/3527331.
- [Zil24] Noam Zilberstein. *A Relatively Complete Program Logic for Effectful Branching*. Preprint. 2024. arXiv: 2401.04594 [cs.LO].
- [ZDS23] Noam Zilberstein, Derek Dreyer, and Alexandra Silva. “Outcome Logic: A Unifying Foundation for Correctness and Incorrectness Reasoning”. In: *Proc. ACM Program. Lang.* 7.OOPSLA1 (2023), pp. 522–550. DOI: 10.1145/3586045.
- [ZSS24] Noam Zilberstein, Angelina Saliling, and Alexandra Silva. “Outcome Separation Logic: Local Reasoning for Correctness and Incorrectness with Computational Effects”. In: *Proc. ACM Program. Lang.* 8.OOPSLA1 (2024), pp. 276–304. DOI: 10.1145/3649821. URL: <https://doi.org/10.1145/3649821>.

Appendix A

Under-approximation abstract domains supplementary materials

This appendix contains technical details of proofs and examples for Chapter 4.

Proof of Lemma 4.7. By hypothesis c is representable but the pair $\{c, \tilde{c}\}$ is not representable. Since α is monotone and c is representable we have $\alpha(\{c, \tilde{c}\}) \supseteq \alpha(\{c\}) = \{c\}$. Since by correctness $\{c, \tilde{c}\} \supseteq \alpha(\{c, \tilde{c}\})$ and $\alpha(\{c, \tilde{c}\}) \neq \{c, \tilde{c}\}$ because this pair is not representable and hence not in the image of α , it must be the case that $\alpha(\{c, \tilde{c}\}) = \{c\}$.

Now

$$\alpha(f(\{c, \tilde{c}\})) = \alpha(\{f(c), f(\tilde{c})\}) \supseteq \alpha(\{f(\tilde{c})\}) = \{f(\tilde{c})\}$$

where the last equality follows by the hypothesis that $f(\tilde{c}) \in R$. This in particular means that $\alpha(f(\{c, \tilde{c}\})) \neq \emptyset$, and together with the fact that $\alpha(\{c, \tilde{c}\}) = \{c\} \neq \emptyset$, we get that $f^A(\alpha(\{c, \tilde{c}\})) \neq \emptyset$, because f is non-emptying.

From this it follows:

$$\begin{aligned} \emptyset &\subset f^A(\alpha(\{c, \tilde{c}\})) && \text{[shown above]} \\ &= f^A(\{c\}) && [\alpha(\{c, \tilde{c}\}) = \{c\}] \\ &= \alpha(f(\{c\})) && \text{[definition of } f^A\text{]} \\ &= \alpha(\{f(c)\}) && \text{[additivity of } f\text{]} \\ &\subseteq \{f(c)\} && \text{[correctness]} \end{aligned}$$

Since $\alpha(\{f(c)\})$ cannot be empty it must be exactly $\alpha(\{f(c)\}) = \{f(c)\}$, that is $f(c) \in R$. $\square$

Proof of Lemma 4.9. By union closure of the abstract domain, any set $S \cup T$ for $T \subseteq R(S)$ is representable too, since it can be expressed as the union of representable sets:

$$S \cup T = \bigcup_{x \in T} (S \cup \{x\})$$

and each $S \cup \{x\}$ is representable because $x \in T \subseteq R(S)$. The number of those sets is given by the cardinality of $\mathcal{P}(R(S))$. If $R(S)$ were infinite, it would be at least countable, so its powerset $\mathcal{P}(R(S))$ would have a greater cardinality. However, this would conflict with the Assumption 4.8 saying that A is at most countable. Therefore, $R(S)$ must be finite. $\square$

Proof of Lemma 4.12. For any $N \geq N_0$, as in the proof of Lemma 4.9, by union closure any set $S \cup T$ for $T \subseteq R_N(S)$ is representable in A_N . Hence we have

$$\text{poly}(N) = |A_N| \geq |\mathcal{P}(R_N(S))| = 2^{|R_N(S)|}$$

so, taking log at both sides, $|R_N(S)| = O(\log(N))$. $\square$

Proof of Theorem 4.15. Assume by contradiction that A is non-emptying for all $f \in F$. By hypothesis, R is not empty and thus we can take a $c_0 \in R$. We then define recursively a sequence of representable elements c_n .

Given an element $c \in C$, let

$$NR(c) = C \setminus R(c) = \{\tilde{c} \in C \mid \{c, \tilde{c}\} \text{ is not representable}\}$$

be the set of elements that are *not* representable with c . To ease presentation, we define here E_n , a set of elements of C that depends on the sequence c_n that we construct along the proof. Note that the definition of E_n only depends on elements of the sequence up to c_n .

$$\begin{aligned} E_n = \{ \tilde{c} \in C \mid & \forall 0 \leq i \leq n . \tilde{c} \in NR(c_i), \\ & f_{\tilde{c}}(\tilde{c}) = c_0, \\ & \forall 0 \leq i \leq n-1 . f_{\tilde{c}}(c_i) = c_{i+1} \} \end{aligned}$$

To improve readability, the conditions on $\tilde{c}$ are separated on three different lines. The first line is needed to make Lemma 4.7 applicable to the pair $\{c_i, \tilde{c}\}$ and conclude that $f_{\tilde{c}}(c_i) = c_{i+1}$ is representable. The second line means just that $\tilde{c}$ represents $f_{\tilde{c}}$, and the last line expresses the requirement that $f_{\tilde{c}}$ coincides on the prefix of the sequence up to c_n .

We observe that the sequence E_n can also be defined inductively by

$$\begin{aligned} E_0 &= \{ \tilde{c} \in C \mid \tilde{c} \in NR(c_0), f_{\tilde{c}}(\tilde{c}) = c_0 \} \\ &= NR(c_0) \cap P_F(c_0) \end{aligned}$$

$$\begin{aligned} E_{n+1} &= \{ \tilde{c} \in E_n \mid \tilde{c} \in NR(c_{n+1}), f_{\tilde{c}}(c_n) = c_{n+1} \} \\ &= NR(c_{n+1}) \cap \{ \tilde{c} \in E_n \mid f_{\tilde{c}}(c_n) = c_{n+1} \} \end{aligned} \tag{A.1}$$

E_0 is the intersection of $NR(c_0)$ and the set of $\tilde{c}$ for which there exists $f_{\tilde{c}}$. Using Lemma 4.9 to say $R(c_0)$ is finite and recalling that $P_F(c_0)$ is infinite by high surjectivity, we observe that

$$E_0 = P_F(c_0) \cap NR(c_0) = P_F(c_0) \setminus (C \setminus NR(c_0)) = P_F(c_0) \setminus R(c_0)$$

is infinite too.

We then prove by induction on n the following three statements:

1. c_n is representable, i.e., $c_n \in R$;
2. E_n is infinite;
3. c_n is different from all c_i for $0 \leq i \leq n-1$.

We have already proved the base case for $n = 0$: c_0 is representable by hypothesis, E_0 is infinite as shown above and the third condition is vacuous since there are no $0 \leq i \leq -1$.

For the inductive step, assume the three hypothesis hold for n and let us prove them for $n + 1$. Consider the set

$$S = \{f_{\tilde{c}}(c_n) \mid \tilde{c} \in E_n\}$$

of possible candidates for the role of c_{n+1} . Since $c_n \in R$, $\tilde{c} \in E_n \subseteq NR(c_n)$ and $f_{\tilde{c}}(\tilde{c}) = c_0 \in R$ (by inductive hypotheses) we can apply Lemma 4.7 to get $f_{\tilde{c}}(c_n) \in R$ too, hence S is a subset of R . Since R is finite also S must be, and by inductive hypothesis we know E_n is infinite, so there must be an element c_{n+1} in S such that an infinite amount of $\tilde{c} \in E_n$ satisfies $f_{\tilde{c}}(c_n) = c_{n+1}$. We observe that, as shown above, $c_{n+1} \in R$. Moreover we chose c_{n+1} such that

$$\{\tilde{c} \in E_n \mid f_{\tilde{c}}(c_n) = c_{n+1}\}$$

is infinite, so we get

$$\{\tilde{c} \in E_n \mid f_{\tilde{c}}(c_n) = c_{n+1}\} \cap NR(c_{n+1}) = \{\tilde{c} \in E_n \mid f_{\tilde{c}}(c_n) = c_{n+1}\} \setminus R(c_{n+1})$$

is infinite too because $R(c_{n+1})$ is finite. But this, by Equation (A.1) above, is exactly E_{n+1} .

We only have to show that $c_{n+1} \neq c_i$ for all $0 \leq i \leq n$. Assume by contradiction that this is not the case, so that for some $0 \leq j \leq n$ it holds $c_{n+1} = c_j$. If $f_{\tilde{c}}$ is acyclic this is a contradiction because it would create the cycle $f_{\tilde{c}}^{n+2-j}(c_j) = f_{\tilde{c}}(c_{n+1}) = c_j$. If $f_{\tilde{c}}$ is injective, let us distinguish two cases. If $j = 0$ we get $f_{\tilde{c}}(c_n) = c_{n+1} = c_0 = f_{\tilde{c}}(\tilde{c})$, that would imply $c_n = \tilde{c}$: this is not possible, because the former is representable and the latter is not. If otherwise $j > 0$ we get $f_{\tilde{c}}(c_n) = c_{n+1} = c_j = f_{\tilde{c}}(c_{j-1})$, that would imply $c_n = c_{j-1}$, that is not the case by inductive hypothesis. So $c_{n+1} \neq c_j$, and this concludes the inductive proof.

By the above induction we conclude that all the infinitely many c_n are elements of R and are all distinct. This yields the desired contradiction because R is finite by Lemma 4.9. $\square$

Proof of Theorem 4.19. Towards a contradiction, let us assume that A is non-emptying for all $f \in F$. By hypothesis, R is not empty and thus we can take a $c_0 \in R$.

Since F is a highly surjective family, $P_F(c_0)$ is infinite. By Lemma 4.9 we know that $R(c_0)$ is finite. Therefore we get that the set

$$\begin{aligned} E &= \{\tilde{c} \in C \mid \tilde{c} \notin R(c_0), \exists f_{\tilde{c}} \in F . f_{\tilde{c}}(\tilde{c}) = c_0\} \\ &= P_F(c_0) \setminus R(c_0) \end{aligned}$$

is infinite. By Lemma 4.7, for all $\tilde{c} \in E$ we have $f_{\tilde{c}}(c_0)$ is representable.

Now fix a function $f \in F$, and let $J(f)$ be the set of $\tilde{c}$ for which f can play the role of $f_{\tilde{c}}$:

$$J(f) = \{\tilde{c} \in C \setminus R(c_0) \mid f(\tilde{c}) = c_0\}$$

By hypothesis (2), the set $J(f)$ is finite.

Now let G be the set of functions in F that can play the role of $f_{\tilde{c}}$ for some $\tilde{c} \in E$:

$$G = \{f \in F \mid \exists \tilde{c} \in E . f(\tilde{c}) = c_0\}$$

Clearly

$$E = \bigcup_{f \in G} J(f)$$

Since E is infinite while each $J(f)$ is finite, the set G must be infinite too.

The above observation about $f_{\bar{c}}(c_0)$ being representable can be equivalently restated by saying that for all $f \in G$, $f(c_0)$ is representable. So consider the set I of all possible images of c_0 through functions in G :

$$I = \{f(c_0) \mid f \in G\}$$

This set is a subset of R because all its elements are representable.

Clearly

$$G = \bigcup_{d \in I} \{f \in G \mid f(c_0) = d\}$$

Now observe that for any $d \in C$, by hypothesis (1) the set

$$\{f \in F \mid f(c_0) = d\}$$

is finite, and this is a superset of $\{f \in G \mid f(c_0) = d\}$, which must be finite too. Since we know that G is infinite, the set I must be infinite too.

This leads to the desired contradiction: R is finite by Lemma 4.9 but we found that I is an infinite set of representable elements. $\square$

Proof of Proposition 4.22. Since F is not highly surjective, there exists $c_0 \in C$ such that $P_F(c_0)$ is finite. We then define A_F as follows. The only element of C representable on its own is c_0 itself, ie. $R = \{c_0\}$. A pair of elements of C is representable if and only if one of its elements is c_0 and the other is in $P_F(c_0)$. This also means that $R(c_0) = P_F(c_0)$. Subsets of C with at least three elements are representable if and only if they are unions of representable pairs. The complete definition of A_F is then

$$A_F = \{\emptyset\} \cup \{\{c_0\} \cup T \mid T \subseteq P_F(c_0)\}$$

A_F is an opposite Moore family with respect to $\mathcal{P}(C)$: it contains the minimal element, that is $\emptyset$, and is closed by union, that are lubs. Hence A_F is an under-approximation abstract domain. Moreover, since $R(c_0) = P_F(c_0)$ is finite, we get that A_F is finite too:

$$|A_F| = 1 + 2^{|P_F(c_0)|}.$$

Now we want to show that an arbitrary f in F is non-emptying in A_F . We first observe that a subset $S \subseteq C$ is such that $\alpha(S) \neq \emptyset$ if and only if $c_0 \in S$. Suppose that $c_0 \in S$, then

$$\alpha(S) \supseteq \alpha(\{c_0\}) = \{c_0\} \supset \emptyset.$$

Conversely, suppose that $\alpha(S) \neq \emptyset$. Since all elements of A_F but the empty set contains c_0 , by correctness we have

$$c_0 \in \alpha(S) \subseteq S.$$

So fix now $S \subseteq C$ an element of the concrete domain, and assume that both $\alpha(S) \neq \emptyset$ and $\alpha(f(S)) \neq \emptyset$. These two assumptions are equivalent to the conditions $c_0 \in S$ and $c_0 \in f(S)$, respectively, and the second can be rewritten as $\exists d \in S . f(d) = c_0$. By definition of A_F we know this is equivalent to $d \in P_F(c_0) = R(c_0)$. Hence

$$\begin{aligned} S &\supseteq \{c_0, d\} \\ \implies \alpha(S) &\supseteq \alpha(\{c_0, d\}) = \{c_0, d\} && [d \in R(c_0)] \\ \implies f(\alpha(S)) &\supseteq f(\{c_0, d\}) \supseteq \{f(d)\} = \{c_0\} && [f(d) = c_0] \\ \implies f^A(\alpha(S)) &= \alpha(f(\alpha(S))) \supseteq \alpha(\{c_0\}) = \{c_0\} && [c_0 \in R] \end{aligned}$$

where in the second line $d \in R(c_0)$ entails $\alpha(\{c_0, d\}) = \{c_0, d\}$. The last line implies $f^A(\alpha(S)) \neq \emptyset$, so f is non-emptying in A_F . $\square$

Proof of Proposition 4.24. Since the proof is very similar to that of Proposition 4.22 above, we gloss over some details.

First, we define a *basis* for the abstract domain as

$$B_F = \{S_0\} \cup \{S_0 \cup S \mid S \in P_F(S_0)\}$$

and then consider its closure under union

$$A_F = \left\{ \bigcup_{T \in \Gamma} T \mid \Gamma \subseteq B_F \right\}.$$

This is an opposite Moore family and hence is an under-approximation abstract domain for $\mathcal{P}(C)$, and is finite because

$$|A_F| \leq |\mathcal{P}(B_F)|$$

and

$$|B_F| \leq 1 + |\mathcal{P}(P_F(S_0))|$$

with $P_F(S_0)$ finite by hypothesis.

Again we observe that a subset $T \subseteq C$ is such that $\alpha(T) \neq \emptyset$ if and only if $S_0 \subseteq T$ because all elements of the abstract domain but the empty set contains S_0 . Then the proof proceeds as above: fix $f \in F$ and $T \subseteq C$ such that $\alpha(T) \neq \emptyset$ and $\alpha(f(T)) \neq \emptyset$, that in turn are equivalent to $S_0 \subseteq T$ and $\exists S \subseteq T. f(S) = S_0$. Then by definition of A_F this means $S_0 \cup S \in A_F$, so

$$\begin{aligned} T &\supseteq S_0 \cup S \\ \implies \alpha(T) &\supseteq \alpha(S_0 \cup S) = S_0 \cup S && [S_0 \cup S \in A_F] \\ \implies f(\alpha(T)) &\supseteq f(S_0 \cup S) \supseteq f(S) = S_0 && [f(S) = S_0] \\ \implies f^A(\alpha(T)) &= \alpha(f(\alpha(T))) \supseteq \alpha(S_0) = S_0 && [S_0 \in A_F] \end{aligned}$$

that is f is non-emptying. □

Proof of Proposition 4.26. Define a *basis* for the abstract domain

$$B_F = \{S_0\} \cup \{S_0 \cup S \mid S \in I_F(S_0)\}$$

and consider its closure under union

$$A_F = \left\{ \bigcup_{T \in \Gamma} T \mid \Gamma \subseteq B_F \right\}.$$

Again this is a correct finite under-approximation abstract domain for $\mathcal{P}(C)$ satisfying that $\alpha(T) \neq \emptyset$ if and only if $S_0 \subseteq T$ (this can be shown in the very same way as in the proof of Proposition 4.24).

Taken an arbitrary $f \in F$, let us show that it is non-emptying. Fix a set $T \subseteq C$ such that $\alpha(T) \neq \emptyset$. This equivalently means that $S_0 \subseteq T$, so

$$\begin{aligned} \alpha(T) &\supseteq \alpha(S_0) = S_0 \\ \implies f(\alpha(T)) &\supseteq f(S_0) \\ \implies f^A(\alpha(T)) &= \alpha(f(\alpha(T))) \supseteq \alpha(f(S_0)) \end{aligned}$$

But $f(S_0) \in B_F$, so $\alpha(f(S_0)) = f(S_0) \neq \emptyset$ by the hypothesis that f is total, and so $f^A(\alpha(T)) \neq \emptyset$, from which we conclude that f is non-emptying. □

Proof of Theorem 4.28. Fix an N such that all $f \in F_N$ are non-emptying in A_N .

Define

$$\begin{aligned} E &= \{\tilde{c} \in C_N \mid \tilde{c} \notin R_N(c_0), \exists f_{\tilde{c}} \in F_N \cdot f_{\tilde{c}}(\tilde{c}) = c_0\} \\ &= P_{F_N}(c_0) \setminus R_N(c_0). \end{aligned}$$

Now fix a function $f \in F_N$, and let $J(f)$ be the set of $\tilde{c}$ for which f can play the role of $f_{\tilde{c}}$, namely

$$J(f) = \{\tilde{c} \in C_N \setminus R_N(c_0) \mid f(\tilde{c}) = c_0\}.$$

By hypothesis (2), $|J(f)| \leq k_2(N)$.

Now let G be the set of functions in F_N that can play the role of $f_{\tilde{c}}$ for some $\tilde{c} \in E$:

$$G = \{f \in F_N \mid \exists \tilde{c} \in E \cdot f(\tilde{c}) = c_0\}.$$

Clearly

$$E = \bigcup_{f \in G} J(f).$$

But we know that $|J(f)| \leq k_2(N)$ for all f , so

$$|E| \leq \sum_{f \in G} |J(f)| \leq |G| \cdot k_2(N).$$

By Lemma 4.7, for all $\tilde{c} \in E$ we have $f_{\tilde{c}}(c_0)$ is representable. This can be equivalently restated saying that for all $f \in G$, $f(c_0)$ is representable. So consider the set I of all possible images of c_0 through functions in G :

$$I = \{f(c_0) \mid f \in G\}.$$

This set is a subset of R_N because all its elements are representable.

Clearly,

$$G = \bigcup_{d \in I} \{f \in G \mid f(c_0) = d\}.$$

Now observe that, for any $d \in C$, by hypothesis (1) we have

$$|\{f \in G \mid f(c_0) = d\}| \leq k_1(N)$$

so

$$|G| \leq \sum_{d \in I} |\{f \in G \mid f(c_0) = d\}| \leq |I| \cdot k_1(N)$$

that in turn implies

$$|E| \leq |G| \cdot k_2(N) \leq |I| \cdot k_1(N) \cdot k_2(N).$$

Since I is a subset of R_N , we get

$$|E| \leq |I| \cdot k_1(N) \cdot k_2(N) \leq |R_N| \cdot k_1(N) \cdot k_2(N).$$

Lastly, we recall that $E = P_{F_N}(c_0) \setminus R_N(c_0)$. Therefore, if all $f \in F_N$ are non-emptying in A_N , then

$$|P_{F_N}(c_0)| \leq |E| \leq |R_N| \cdot k_1(N) \cdot k_2(N).$$

The last hypothesis of the theorem states that $|P_{F_N}(c_0)| = \omega(\log(N) \cdot k_1(N) \cdot k_2(N))$. Lemma 4.12 states that $|R_N| = O(\log(N))$. Therefore, there exists an N_0 such that the inequality

$$\omega(\log(N) \cdot k_1(N) \cdot k_2(N)) = |P_{F_N}(c_0)| \leq |R_N| \cdot k_1(N) \cdot k_2(N) = O(\log(N) \cdot k_1(N) \cdot k_2(N))$$

does not hold for any $N > N_0$. This in turn implies that for all $N > N_0$ it is not possible that all the functions $f \in F_N$ are non-emptying in A_N . $\square$

Appendix B

Logics comparison supplementary materials

This appendix contains technical details of proofs and examples for Chapter 5.

Proof of Lemma 5.1. In the proof, we assume Q to be any set of states, and $\sigma' \in Q$ to be any of its elements.

Case $\llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket$

By (5.2), $\sigma \in \llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket \sigma'$ if and only if $\sigma' \in \llbracket r_1; r_2 \rrbracket \sigma$.

$$\llbracket r_1; r_2 \rrbracket \sigma = \llbracket r_2 \rrbracket (\llbracket r_1 \rrbracket \sigma) = \bigcup_{\sigma'' \in \llbracket r_1 \rrbracket \sigma} \llbracket r_2 \rrbracket \sigma''$$

so $\sigma' \in \llbracket r_1; r_2 \rrbracket \sigma$ if and only if there exists a $\sigma'' \in \llbracket r_1 \rrbracket \sigma$ such that $\sigma' \in \llbracket r_2 \rrbracket \sigma''$. Again by (5.2), these are equivalent to $\sigma \in \llbracket \overleftarrow{r_1} \rrbracket \sigma''$ and $\sigma'' \in \llbracket \overleftarrow{r_2} \rrbracket \sigma'$, respectively. Hence

$$\sigma' \in \llbracket r_1; r_2 \rrbracket \sigma \iff \exists \sigma'' \in \llbracket \overleftarrow{r_2} \rrbracket \sigma' . \sigma \in \llbracket \overleftarrow{r_1} \rrbracket \sigma''$$

Since $\llbracket \overleftarrow{\cdot} \rrbracket$ is defined on sets by union

$$\llbracket \overleftarrow{r_1} \rrbracket (\llbracket \overleftarrow{r_2} \rrbracket \sigma') = \bigcup_{\sigma'' \in \llbracket \overleftarrow{r_2} \rrbracket \sigma'} \llbracket \overleftarrow{r_1} \rrbracket \sigma''$$

which means $\exists \sigma'' \in \llbracket \overleftarrow{r_2} \rrbracket \sigma' . \sigma \in \llbracket \overleftarrow{r_1} \rrbracket \sigma''$ if and only if $\sigma \in \llbracket \overleftarrow{r_1} \rrbracket (\llbracket \overleftarrow{r_2} \rrbracket \sigma')$. Putting everything together, we get $\sigma \in \llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket \sigma'$ if and only if $\sigma \in \llbracket \overleftarrow{r_1} \rrbracket (\llbracket \overleftarrow{r_2} \rrbracket \sigma')$, so the two are the same set. The thesis follows easily lifting the equality by union on $\sigma' \in Q$ and by the arbitrariness of Q .

Case $\llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket$

By (5.2), $\sigma \in \llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket \sigma'$ if and only if $\sigma' \in \llbracket r_1 \oplus r_2 \rrbracket \sigma$.

$$\llbracket r_1 \oplus r_2 \rrbracket \sigma = \llbracket r_1 \rrbracket \sigma \cup \llbracket r_2 \rrbracket \sigma$$

so $\sigma' \in \llbracket r_1 \oplus r_2 \rrbracket \sigma$ if and only if $\exists i \in \{1, 2\}$ such that $\sigma' \in \llbracket r_i \rrbracket \sigma$. This is again equivalent to $\sigma \in \llbracket \overleftarrow{r_i} \rrbracket \sigma'$, and

$$\exists i \in \{1, 2\} . \sigma \in \llbracket \overleftarrow{r_i} \rrbracket \sigma' \iff \sigma \in \llbracket \overleftarrow{r_1} \rrbracket \sigma' \cup \llbracket \overleftarrow{r_2} \rrbracket \sigma'$$

Putting everything together, we get $\sigma \in \llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket \sigma'$ if and only if $\sigma \in \llbracket \overleftarrow{r_1} \rrbracket \sigma' \cup \llbracket \overleftarrow{r_2} \rrbracket \sigma'$, which implies the thesis as in point 1.

Case $\llbracket \overleftarrow{r^*} \rrbracket$

To prove this last equality, we define r^n inductively as the sequential composition of r with itself n times: $r^1 = r$ and $r^{n+1} = r^n; r$. Clearly $\llbracket r^n \rrbracket = \llbracket r \rrbracket^n$. For simplicity, we also define $\llbracket r^0 \rrbracket = \llbracket \overleftarrow{r^0} \rrbracket = \llbracket r \rrbracket^0$. We prove by induction on n that $\llbracket \overleftarrow{r^n} \rrbracket = \llbracket \overleftarrow{r} \rrbracket^n$. For $n = 1$ we have $\llbracket \overleftarrow{r^1} \rrbracket = \llbracket \overleftarrow{r} \rrbracket^1$. If we assume it holds for n we have

$$\begin{aligned}
\llbracket \overleftarrow{r^{n+1}} \rrbracket &= \llbracket \overleftarrow{r^n}; r \rrbracket && [\text{def. of } r^n] \\
&= \llbracket \overleftarrow{r^n} \rrbracket \circ \llbracket \overleftarrow{r} \rrbracket && [\text{pt. 1 of this lemma}] \\
&= \llbracket \overleftarrow{r} \rrbracket^n \circ \llbracket \overleftarrow{r} \rrbracket && [\text{inductive hp}] \\
&= \llbracket \overleftarrow{r} \rrbracket^{n+1}
\end{aligned}$$

We then observe that

$$\begin{aligned}
\llbracket \overleftarrow{r^*} \rrbracket \sigma' &= \{\sigma \mid \sigma' \in \llbracket r^* \rrbracket \sigma\} && [\text{def. of } \llbracket \overleftarrow{\cdot} \rrbracket] \\
&= \{\sigma \mid \sigma' \in \bigcup_{n \geq 0} \llbracket r^n \rrbracket \sigma\} && [\text{def. of } \llbracket r^* \rrbracket] \\
&= \bigcup_{n \geq 0} \{\sigma \mid \sigma' \in \llbracket r \rrbracket^n \sigma\} \\
&= \bigcup_{n \geq 0} \{\sigma \mid \sigma' \in \llbracket r^n \rrbracket \sigma\} && [\text{observed above}] \\
&= \bigcup_{n \geq 0} \{\sigma \mid \sigma \in \llbracket \overleftarrow{r^n} \rrbracket \sigma'\} && [(5.2)] \\
&= \bigcup_{n \geq 0} \llbracket \overleftarrow{r^n} \rrbracket \sigma' \\
&= \bigcup_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n \sigma' && [\text{shown above}]
\end{aligned}$$

As in the cases above, the thesis follows. $\square$

Proof of Proposition 5.3. By definition of $\llbracket \overleftarrow{\cdot} \rrbracket$ we have

$$\llbracket \overleftarrow{r} \rrbracket Q = \bigcup_{\sigma' \in Q} \{\sigma \mid \sigma \in \llbracket \overleftarrow{r} \rrbracket \sigma'\} = \{\sigma \mid \exists \sigma' \in Q. \sigma' \in \llbracket r \rrbracket \sigma\}$$

Using this,

$$P \subseteq \llbracket \overleftarrow{r} \rrbracket Q \iff \forall \sigma \in P. \sigma \in \{\sigma \mid \exists \sigma' \in Q. \sigma' \in \llbracket r \rrbracket \sigma\} \iff \forall \sigma \in P. \exists \sigma' \in Q. \sigma' \in \llbracket r \rrbracket \sigma$$

$\square$

We split the proof between soundness and completeness.

Proposition B.1 (SIL is sound). *Any provable SIL triple is valid.*

Proof. The proof is by structural induction on the derivation tree.

Case $\langle\langle \text{atom} \rangle\rangle$

This case is trivial since $\llbracket \overleftarrow{c} \rrbracket Q \subseteq \llbracket \overleftarrow{c} \rrbracket Q$.

Case $\langle\text{cons}\rangle$

We have that

$$P \subseteq P' \subseteq \llbracket \overleftarrow{r} \rrbracket Q' \subseteq \llbracket \overleftarrow{r} \rrbracket Q$$

The inequalities above are justified, in order, by the hypothesis of the rule, by the inductive hypothesis on $\langle P' \rangle \text{ r } \langle Q' \rangle$, by monotonicity of $\llbracket \overleftarrow{r} \rrbracket$ and the hypothesis of the rule.

Case $\langle\text{seq}\rangle$

We have that

$$P \subseteq \llbracket \overleftarrow{r_1} \rrbracket R \subseteq \llbracket \overleftarrow{r_1} \rrbracket \llbracket \overleftarrow{r_2} \rrbracket Q = \llbracket \overleftarrow{r_1; r_2} \rrbracket Q$$

The inequalities above are justified, in order, by the inductive hypothesis on $\langle P \rangle \text{ r}_1 \langle R \rangle$, by the inductive hypothesis on $\langle R \rangle \text{ r}_2 \langle Q \rangle$, by Lemma 5.1.

Case $\langle\text{choice}\rangle$

We have that

$$P_1 \cup P_2 \subseteq \llbracket \overleftarrow{r_1} \rrbracket Q \cup \llbracket \overleftarrow{r_2} \rrbracket Q = \llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket Q$$

The (in)equalities above are justified, in order, by the two inductive hypotheses, by Lemma 5.1.

Case $\langle\text{iter}\rangle$

We first prove by induction on n that $Q_n \subseteq \llbracket \overleftarrow{r} \rrbracket^n Q_0$. The base case $n = 0$ is trivial because $Q_0 \subseteq \llbracket \overleftarrow{r} \rrbracket^0 Q_0$. For the inductive case, we have

$$Q_{n+1} \subseteq \llbracket \overleftarrow{r} \rrbracket Q_n \subseteq \llbracket \overleftarrow{r} \rrbracket \llbracket \overleftarrow{r} \rrbracket^n Q_0 = \llbracket \overleftarrow{r} \rrbracket^{n+1} Q_0$$

The (in)equalities above are justified, in order, by inductive hypothesis on $\langle Q_{n+1} \rangle \text{ r } \langle Q_n \rangle$, the inductive hypothesis for n and monotonicity of $\llbracket \overleftarrow{r} \rrbracket$, by definition of $\llbracket \overleftarrow{r} \rrbracket^{n+1}$.

With this, we prove

$$\bigcup_{n \geq 0} Q_n \subseteq \bigcup_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n Q_0 = \llbracket \overleftarrow{r} \rrbracket^* Q_0$$

The (in)equalities above are justified, in order, by the proof above and by Lemma 5.1. $\square$

Proposition B.2 (SIL is complete). *Any valid SIL triple is provable.*

Proof. First we show that, for any Q , the triple $\langle \llbracket \overleftarrow{r} \rrbracket Q \rangle \text{ r } \langle Q \rangle$ is provable by induction on the structure of r .

Case $r = c$

We can prove $\langle \llbracket \overleftarrow{c} \rrbracket Q \rangle \text{ c } \langle Q \rangle$ using $\langle\text{atom}\rangle$.

Case $r = r_1; r_2$

We can prove $\langle \llbracket \overleftarrow{r} \rrbracket Q \rangle \text{ r}_1; r_2 \langle Q \rangle$ with

$$\frac{\langle \llbracket \overleftarrow{r_1} \rrbracket \llbracket \overleftarrow{r_2} \rrbracket Q \rangle \text{ r}_1 \langle \llbracket \overleftarrow{r_2} \rrbracket Q \rangle \quad \langle \llbracket \overleftarrow{r_2} \rrbracket Q \rangle \text{ r}_2 \langle Q \rangle}{\langle \llbracket \overleftarrow{r_1} \rrbracket \llbracket \overleftarrow{r_2} \rrbracket Q \rangle \text{ r}_1; r_2 \langle Q \rangle} \langle\text{seq}\rangle$$

where the two premises can be proved by inductive hypothesis, and $\llbracket \overleftarrow{r_1; r_2} \rrbracket Q = \llbracket \overleftarrow{r_1} \rrbracket \llbracket \overleftarrow{r_2} \rrbracket Q$ by Lemma 5.1.

Case $r = r_1 \oplus r_2$

We can prove $\langle \llbracket \overleftarrow{r} \rrbracket Q \rangle \text{ r}_1 \oplus r_2 \langle Q \rangle$ with

$$\frac{\forall i \in \{1, 2\} \quad \langle \llbracket \overleftarrow{r_i} \rrbracket Q \rangle \text{ r}_i \langle Q \rangle}{\langle \llbracket \overleftarrow{r_1} \rrbracket Q \cup \llbracket \overleftarrow{r_2} \rrbracket Q \rangle \text{ r}_1 \oplus r_2 \langle Q \rangle} \langle\text{choice}\rangle$$

where the two premises can be proved by inductive hypothesis, and $\llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket Q = \llbracket \overleftarrow{r_1} \rrbracket Q \cup \llbracket \overleftarrow{r_2} \rrbracket Q$ by Lemma 5.1.

Case $r = r^*$

We can prove $\llbracket r^* \rrbracket Q \text{ } r^* \llbracket Q \rrbracket$ with

$$\frac{\forall n \geq 0. \llbracket r^* \rrbracket^{n+1} Q \text{ } r \llbracket r^* \rrbracket^n Q}{\llbracket \bigcup_{n \geq 0} \llbracket r^* \rrbracket^n Q \rrbracket \text{ } r \llbracket Q \rrbracket} \text{iter}$$

where the premises can be proved by inductive hypothesis since $\llbracket r^* \rrbracket^{n+1} Q = \llbracket r^* \rrbracket \llbracket r^* \rrbracket^n Q$, and $\llbracket r^* \rrbracket Q = \bigcup_{n \geq 0} \llbracket r^* \rrbracket^n Q$ by Lemma 5.1.

To conclude the proof, take a triple $\langle P \rangle \text{ } r \llbracket Q \rrbracket$ such that $\llbracket r^* \rrbracket Q \supseteq P$. Then we can first prove the triple $\langle \llbracket r^* \rrbracket Q \rangle \text{ } r \llbracket Q \rrbracket$, and then using rule $\langle \text{cons} \rangle$ we derive $\langle P \rangle \text{ } r \llbracket Q \rrbracket$. $\square$

The proof of Theorem 5.4 is a corollary of Proposition B.1–B.2.

Proof of Proposition 5.5. The proof is by structural induction on the derivation tree and extends that of Proposition B.1 with inductive cases for the new rules.

Case $\langle \text{empty} \rangle$

Clearly $\emptyset \subseteq \llbracket r^* \rrbracket Q$.

Case $\langle \text{disj} \rangle$

We have that

$$P_1 \cup P_2 \subseteq \llbracket r^* \rrbracket Q_1 \cup \llbracket r^* \rrbracket Q_2 = \llbracket r^* \rrbracket (Q_1 \cup Q_2)$$

The (in)equalities above are justified, in order, by inductive hypotheses on $\langle P_1 \rangle \text{ } r \llbracket Q_1 \rrbracket$ and $\langle P_2 \rangle \text{ } r \llbracket Q_2 \rrbracket$, by additivity of $\llbracket r^* \rrbracket$.

Case $\langle \text{iter0} \rangle$

We have that

$$Q = \llbracket r^* \rrbracket^0 Q \subseteq \bigcup_{n \geq 0} \llbracket r^* \rrbracket^n Q = \llbracket r^* \rrbracket Q$$

The last equality above is justified by Lemma 5.1.

Case $\langle \text{unroll} \rangle$

We have that

$$P \subseteq \llbracket r^*; r \rrbracket Q = \llbracket r^* \rrbracket \llbracket r \rrbracket Q = \bigcup_{n \geq 0} \llbracket r^* \rrbracket^n (\llbracket r \rrbracket Q) = \bigcup_{n \geq 0} \llbracket r^* \rrbracket^{n+1} Q \subseteq \bigcup_{n \geq 0} \llbracket r^* \rrbracket^n Q = \llbracket r^* \rrbracket Q$$

The first inequality is justified by the inductive hypothesis on $\langle P \rangle \text{ } r^*; r \llbracket Q \rrbracket$, other equalities are justified by Lemma 5.1. $\square$

Proof of Proposition 5.9. We prove the left-to-right implication, so assume $\llbracket r \rrbracket P \subseteq Q$. Take a state $\sigma' \in \neg Q$. This means $\sigma' \notin Q$, that implies $\sigma' \notin \llbracket r \rrbracket P$. So, for any state $\sigma \in P$, we have $\sigma' \notin \llbracket r \rrbracket \sigma$, which is equivalent to $\sigma \notin \llbracket r^* \rrbracket \sigma'$ by (5.2). This being true for all $\sigma \in P$ means $P \cap \llbracket r^* \rrbracket \sigma' = \emptyset$, that is equivalent to $\llbracket r^* \rrbracket \sigma' \subseteq \neg P$. Since this holds for all states $\sigma' \in \neg Q$, we have $\llbracket r^* \rrbracket (\neg Q) \subseteq \neg P$.

The other implication is analogous. $\square$

Proof of Proposition 5.13. To prove the first point, assume $\llbracket r^* \rrbracket Q \supseteq P$ and take $\sigma' \in \llbracket r \rrbracket P$. Then there exists $\sigma \in P$ such that $\sigma' \in \llbracket r \rrbracket \sigma$. Since r is deterministic, $\llbracket r \rrbracket \sigma$ can contain at most one element, hence $\llbracket r \rrbracket \sigma = \{\sigma'\}$. Moreover, since $\sigma \in P \subseteq \llbracket r^* \rrbracket Q$ there must exist a $\sigma'' \in Q$ such that $\sigma'' \in \llbracket r \rrbracket \sigma = \{\sigma'\}$, which means $\sigma' \in Q$. Again, by arbitrariness of $\sigma' \in \llbracket r \rrbracket P$, this implies $\llbracket r \rrbracket P \subseteq Q$.

To prove the second point, assume $\llbracket r \rrbracket P \subseteq Q$ and take a state $\sigma \in P$. Since r is terminating, $\llbracket r \rrbracket \sigma$ is not empty, hence we can take $\sigma' \in \llbracket r \rrbracket \sigma$. The hypothesis $\llbracket r \rrbracket P \subseteq Q$

implies that $\sigma' \in Q$. Then, by (5.2), $\sigma \in \llbracket \overleftarrow{r} \rrbracket \sigma' \subseteq \llbracket \overleftarrow{r} \rrbracket Q$. By arbitrariness of $\sigma \in P$, this implies $P \subseteq \llbracket \overleftarrow{r} \rrbracket Q$. $\square$

Proof of Lemma 5.12. We first prove that $\llbracket \overleftarrow{r} \rrbracket \llbracket r \rrbracket P \supseteq P \setminus D_r$. Take a $\sigma \in P \setminus D_r$. Because $\sigma \notin D_r$, $\llbracket r \rrbracket \sigma \neq \emptyset$, so take $\sigma' \in \llbracket r \rrbracket \sigma$. Since $\sigma \in P$ we have $\sigma' \in \llbracket r \rrbracket P$. Moreover, by (5.2), we get $\sigma \in \llbracket \overleftarrow{r} \rrbracket \sigma' \subseteq \llbracket r \rrbracket P$. By arbitrariness of $\sigma \in P \setminus D_r$ we have the thesis. $\square$

The proof for $\llbracket r \rrbracket \llbracket \overleftarrow{r} \rrbracket Q \supseteq Q \setminus U_r$ is analogous. $\square$

Proof of Proposition 5.15. By definition, $\llbracket r \rrbracket$ is additive, that is $\llbracket r \rrbracket (P_1 \cup P_2) = \llbracket r \rrbracket P_1 \cup \llbracket r \rrbracket P_2$. Take all P such that $\llbracket r \rrbracket P \subseteq Q$. By additivity of $\llbracket r \rrbracket$, their union satisfies the same inequality, hence it is the weakest such P .

By definition, $\llbracket \overleftarrow{r} \rrbracket$ is additive. Analogously, take all Q such that $\llbracket \overleftarrow{r} \rrbracket Q \subseteq P$. By additivity of $\llbracket \overleftarrow{r} \rrbracket$, their union is the weakest Q satisfying that inequality. $\square$

Proof of Proposition 5.16. The proof is given by the counterexamples in Example 5.17. For IL, the example shows that for $x = 1$ there is no strongest P such that $\llbracket r \rrbracket P \supseteq (x = 1)$: $x = 0$ and $x = 10$ are incomparable and are both minimal, as $\emptyset$ is not a valid precondition. The argument for SIL is analogous with precondition $x = 1$. $\square$

B.1 Proofs about Separation SIL

Given two stores $s, s' \in \text{Store}$ and a heap command $r \in \text{HRCmd}$, we use the notation $s \sim_r s'$ to indicate that they coincide on all variables not modified by r : $\forall x \notin \text{mod}(r). s(x) = s'(x)$. Please note that $\sim_r$ is an equivalence relation.

Lemma B.3. *Let $(s, h) \in \Sigma$, $r \in \text{HRCmd}$. If $(s', h') \in \llbracket r \rrbracket (s, h)$ then $s \sim_r s'$.*

Proof. The proof is by induction on the syntax of r . We prove here only some relevant cases.

Case $x := a$

$(s', h') \in \llbracket x := a \rrbracket (s, h)$ means that $s' = s[x \mapsto \langle a \rangle s]$. Particularly, this means that for all variables $y \neq x$, $s'(y) = s(y)$, which is the thesis because $\text{mod}(x := a) = \{x\}$.

Case $\text{free}(x)$

$(s', h') \in \llbracket \text{free}(x) \rrbracket (s, h)$ means that $s' = s$, which is the thesis because $\text{mod}(\text{free}(x)) = \emptyset$.

Case $r_1; r_2$

$(s', h') \in \llbracket r_1; r_2 \rrbracket (s, h)$ means that there exists $(s'', h'') \in \llbracket r_1 \rrbracket (s, h)$ such that $(s', h') \in \llbracket r_2 \rrbracket (s'', h'')$. By inductive hypothesis, since $\text{mod}(r_1) \subseteq \text{mod}(r_1; r_2)$, we have $s'' \sim_{r_1; r_2} s$. Analogously, $\text{mod}(r_2) \subseteq \text{mod}(r_1; r_2)$ implies $s' \sim_{r_1; r_2} s''$. From these, we get $s' \sim_{r_1; r_2} s$. $\square$

The next technical proposition states some semantic properties of the assertion language to be exploited in the proof of Lemma 5.19.

Proposition B.4. *Let $p \in \text{Asl}$, $s, s' \in \text{Store}$, $h \in \text{Heap}$ and $a \in \text{AExp}$.*

1. *If $\forall x \in \text{fv}(p). s(x) = s'(x)$ and $(s, h) \in \llbracket p \rrbracket$ then $(s', h) \in \llbracket p \rrbracket$.*
2. *If $(s, h) \in \llbracket p[a/x] \rrbracket$ then $(s[x \mapsto \langle a \rangle s], h) \in \llbracket p \rrbracket$.*

Proof. By structural induction on the syntax of assertions. $\square$

Proof of Lemma 5.19. First, we observe that Proposition 5.3 does not depend on the specific definition of $\llbracket \cdot \rrbracket$, thus it holds for separation SIL as well. Thanks to this, we prove the thesis through the equivalent condition

$$\forall (s, h) \in \{p * t\}. \exists (s', h') \in \{q * t\}. (s', h') \in \llbracket r \rrbracket(s, h)$$

The proof is by induction on the derivation tree of the provable triple $\langle p \rangle \text{ r } \langle q \rangle$. We prove here only some relevant cases.

Case $\langle \text{assign} \rangle$

Take $(s, h) \in \{q[a/x] * t\}$. Then we can split $h = h_p \bullet h_t$ such that $(s, h_p) \in \{q[a/x]\}$ and $(s, h_t) \in \{t\}$. Let $s' = s[x \mapsto \langle a \rangle s]$, so that $(s', h) \in \llbracket x := a \rrbracket \{q[a/x] * t\}$. Since $\text{fv}(t) \cap \text{mod}(r) = \emptyset$, $x \notin \text{fv}(t)$. Thus, by Proposition B.4.1, $(s', h_t) \in \{t\}$. Moreover, $(s', h_p) \in \{q\}$ by Proposition B.4.2. Hence, $(s', h_p \bullet h_t) = (s', h) \in \{q * t\}$.

Case $\langle \text{alloc1} \rangle$

Take $(s, h) \in \{\text{emp} * t\} = \{t\}$. Take a location $l \notin \text{dom}(h)$, and let $s' = s[x \mapsto l]$, $h' = h[l \mapsto s(v)]$, so that $(s', h') \in \llbracket x := \text{alloc}() \rrbracket(s, h)$. We can split $h' = [l \mapsto s(v)] \bullet h$ because $l \notin \text{dom}(h)$. Since $\text{fv}(t) \cap \text{mod}(r) = \emptyset$, $x \notin \text{fv}(t)$. Thus, by Proposition B.4.1, $(s', h) \in \{t\}$. Moreover, $(s', [l \mapsto s(v)]) = (s', [s'(x) \mapsto s'(v)])$, which satisfies $(s', [s'(x) \mapsto s'(v)]) \in \{x \mapsto v\}$. Hence $(s', h') \in \{x \mapsto v * t\}$.

Case $\langle \text{load} \rangle$

Take $(s, h) \in \{y \mapsto a * q[a/x] * t\}$. Then we know $x \notin \text{fv}(t)$ and $h = [s(y) \mapsto \langle a \rangle s] \bullet h_p \bullet h_t$, $(s, h_p) \in \{q[a/x]\}$, $(s, h_t) \in \{t\}$. Let $s' = s[x \mapsto h(s(y))] = s[x \mapsto \langle a \rangle s]$. By Proposition B.4.1, $(s', h_t) \in \{t\}$. By Proposition B.4.2, $(s', h_p) \in \{q\}$. Lastly, since $x \notin \text{fv}(a)$, $\langle a \rangle s' = \langle a \rangle s$ and $s'(y) = s(y)$, so we have $(s', [s'(y) \mapsto \langle a \rangle s']) \in \{y \mapsto a\}$. Combining these, $(s', h) = (s', [s(y) \mapsto \langle a \rangle s] \bullet h_p \bullet h_t) \in \{y \mapsto a * q * t\}$. The thesis follows observing that $(s', h) \in \llbracket x := [y] \rrbracket(s, h)$.

Case $\langle \text{store} \rangle$

Take $(s, h) \in \{x \mapsto - * t\}$. Then $x \notin \text{fv}(t)$ and exists $v \in \text{Val}$ such that $h = [s(x) \mapsto v] \bullet h_t$, $(s, h_t) \in \{t\}$. Let $h' = h[s(x) \mapsto s(y)]$. Clearly $h' = [s(x) \mapsto s(y)] \bullet h_t$ and $(s, [s(x) \mapsto s(y)]) \in \{x \mapsto y\}$. Hence $(s, h') \in \{x \mapsto y * t\}$ and $(s, h') \in \llbracket [x] := y \rrbracket(s, h)$, which is the thesis.

Case $\langle \text{exists} \rangle$

Take $(s, h) \in \{(\exists x.p) * t\}$. Then there exists a value $v \in \text{Val}$ and decomposition $h = h_p \bullet h_t$ such that $(s[x \mapsto v], h_p) \in \{p\}$ and $(s, h_t) \in \{t\}$. Without loss of generality, we can assume $x \notin \text{fv}(t)$; otherwise, we just rename it using a fresh name neither in t nor in r . Hence, by Proposition B.4.1, $(s[x \mapsto v], h_t) \in \{t\}$. So $(s[x \mapsto v], h) \in \{p * t\}$. By inductive hypothesis on the provable triple $\langle p \rangle \text{ r } \langle q \rangle$ and formula t , there is $(s', h') \in \{q * t\}$ such that $(s', h') \in \llbracket r \rrbracket(s[x \mapsto v], h)$. Because $x \notin \text{fv}(r)$, we also have $(s', h') \in \llbracket r \rrbracket(s, h)$, and clearly $(s', h') \in \{(\exists x.q) * t\}$, that is the thesis.

Case $\langle \text{frame} \rangle$

By hypothesis, $(\text{fv}(t' * t)) \cap \text{mod}(r) = (\text{fv}(t') \cup \text{fv}(t)) \cap \text{mod}(r) = \emptyset$. Then, applying the inductive hypothesis on the provable triple $\langle p \rangle \text{ r } \langle q \rangle$ and the formula $t' * t$ (which satisfies the hypothesis of the theorem) we get exactly the thesis.

Case $\langle \text{seq} \rangle$

Because of name clashes, here we assume the hypotheses of rule $\langle \text{seq} \rangle$ to be $\langle p \rangle \text{ r}_1 \langle p' \rangle$ and $\langle p' \rangle \text{ r}_2 \langle q \rangle$. Since $\text{mod}(r_1) \cup \text{mod}(r_2) = \text{mod}(r_1; r_2)$, we know that $\text{fv}(t) \cap \text{mod}(r_1) = \text{fv}(t) \cap \text{mod}(r_2) = \emptyset$. Take $(s, h) \in \{p * t\}$. By inductive hypothesis on provable triple $\langle p \rangle \text{ r}_1 \langle p' \rangle$ and formula t we get that there exists $(s'', h'') \in \{p' * t\}$ such that $(s'', h'') \in \llbracket r_1 \rrbracket(s, h)$. Then, by inductive hypothesis on the provable triple $\langle p' \rangle \text{ r}_2 \langle q \rangle$ and formula t again, we get $(s', h') \in \{q * t\}$ such that $(s', h') \in \llbracket r_2 \rrbracket(s'', h'')$. The thesis follows since $(s', h') \in \llbracket r_2 \rrbracket(s'', h'') \subseteq \llbracket r_2 \rrbracket(\llbracket r_1 \rrbracket(s, h)) = \llbracket r_1; r_2 \rrbracket(s, h)$. $\square$

Proof of Theorem 5.22. The proof follows the same line as the soundness proof of “plain” Separation SIL. For notation, we write (ϵ, s, h) for state $(\epsilon, (s, h))$ from domain $\{\text{ok}, \text{er}\} \times \Sigma$, where ϵ is the flag. Following the proof of Lemma 5.19, we prove by induction on the derivation tree of $\llbracket \epsilon : p \rrbracket \text{ r } \llbracket \epsilon : q \rrbracket$ the condition

$$\forall (\epsilon, s, h) \in \llbracket \epsilon : p * t \rrbracket . \exists (\epsilon', s', h') \in \llbracket \epsilon' : q * t \rrbracket . (\epsilon', s', h') \in \llbracket r \rrbracket (\epsilon, s, h)$$

We prove here only some relevant cases.

Case $\llbracket \text{assign} \rrbracket$

Rule $\llbracket \text{assign} \rrbracket$ requires both flags ϵ and ϵ' to be *ok*. Take $(\text{ok}, s, h) \in \llbracket \text{ok} : p * t \rrbracket$. Since the semantics of assignments $(x := a)$ never fails, on *ok* states it behaves exactly as in the Separation SIL model without flags. Therefore, the proof concludes as in Lemma 5.19.

Case $\llbracket \text{frame} \rrbracket$

By hypothesis, $(\text{fv}(t' * t)) \cap \text{mod}(r) = (\text{fv}(t') \cup \text{fv}(t)) \cap \text{mod}(r) = \emptyset$. Then, applying the inductive hypothesis on the provable triple $\llbracket \epsilon : p \rrbracket \text{ r } \llbracket \epsilon' : q \rrbracket$ and the formula $t' * t$ (which satisfies the hypothesis of the theorem) we get exactly the thesis.

Case $\llbracket \text{seq} \rrbracket$

Because of name clashes, we let the hypotheses of rule $\llbracket \text{seq} \rrbracket$ be $\llbracket \epsilon : p \rrbracket \text{ r } \llbracket \epsilon' : p' \rrbracket$ and $\llbracket \epsilon' : p' \rrbracket \text{ r } \llbracket \epsilon'' : q \rrbracket$. Since $\text{mod}(r_1) \cup \text{mod}(r_2) = \text{mod}(r_1; r_2)$, we know that $\text{fv}(t) \cap \text{mod}(r_1) = \text{fv}(t) \cap \text{mod}(r_2) = \emptyset$. Take $(\epsilon, s, h) \in \llbracket \epsilon : p * t \rrbracket$. By inductive hypothesis on provable triple $\llbracket \epsilon : p \rrbracket \text{ r } \llbracket \epsilon' : p' \rrbracket$ and formula t we get that there exists $(\epsilon', s', h') \in \llbracket \epsilon' : p' * t \rrbracket$ such that $(\epsilon', s', h') \in \llbracket r_1 \rrbracket (\epsilon, s, h)$. Then, by inductive hypothesis on the provable triple $\llbracket \epsilon' : p' \rrbracket \text{ r } \llbracket \epsilon'' : q \rrbracket$ and formula t again, we get $(\epsilon'', s'', h'') \in \llbracket \epsilon'' : q * t \rrbracket$ such that $(\epsilon'', s'', h'') \in \llbracket r_2 \rrbracket (\epsilon', s', h')$. The thesis follows since $(\epsilon'', s'', h'') \in \llbracket r_2 \rrbracket (\epsilon', s', h') \subseteq \llbracket r_2 \rrbracket (\llbracket r_1 \rrbracket (\epsilon, s, h)) = \llbracket r_1; r_2 \rrbracket (\epsilon, s, h)$.

Case $\llbracket \text{store-er} \rrbracket$

Take $(\text{ok}, s, h) \in \llbracket \text{ok} : x \not\mapsto * t \rrbracket$. $h(s(x)) = \delta \notin \text{Val}$, so that $(\text{er}, s, h) \in \llbracket [x] := y \rrbracket (\text{ok}, s, h)$. Moreover, since $(s, h) \in \llbracket x \not\mapsto * t \rrbracket$, we have $(\text{er}, s, h) \in \llbracket \text{er} : x \not\mapsto * t \rrbracket$, which is the thesis.

Case $\llbracket \text{er-id} \rrbracket$

Take $(\text{er}, s, h) \in \llbracket \text{er} : q \rrbracket$. Since $\llbracket \cdot \rrbracket$ always acts as the identity on *er* states, we have $(\text{er}, s, h) \in \llbracket r \rrbracket (\text{er}, s, h)$ and $(\text{er}, s, h) \in \llbracket \text{er} : q \rrbracket$. $\square$

Some of the following equivalences are standard, but we collect them all here for convenience.

Lemma B.5. *For all assertions p_1, p_2, q , variables $x, x' \in \text{Var}$ and arithmetic expressions $a_1, a_2 \in \text{AExp}$, the following equivalences hold:*

1. $(p_1 \vee p_2) \wedge q \equiv (p_1 \wedge q) \vee (p_2 \wedge q)$
2. $(p_1 \vee p_2) * q \equiv (p_1 * q) \vee (p_2 * q)$
3. $\exists x.(p \vee q) \equiv (\exists x.p) \vee (\exists x.q)$
4. $\exists x.(p \wedge q) \equiv (\exists x.p) \wedge q \quad \text{if } x \notin \text{fv}(q)$
5. $\exists x.(p * q) \equiv (\exists x.p) * q \quad \text{if } x \notin \text{fv}(q)$
6. $a_1 \prec a_2 \wedge (p_1 * p_2) \equiv (a_1 \prec a_2 \wedge p_1) * p_2$
7. $(p_1 * x \mapsto x') \wedge (p_2 * x \mapsto x') \equiv (p_1 \wedge p_2) * x \mapsto x'$
8. $(p_1 * x \not\mapsto) \wedge (p_2 * x \not\mapsto) \equiv (p_1 \wedge p_2) * x \not\mapsto$

Proof. Point (1), (3) and (4) are standard in first-order logic. Point (2), (5) and (6) are standard in separation logic [Rey02].

For point (7) we observe

$$\begin{aligned}
& \llbracket (p_1 * x \mapsto x') \wedge (p_2 * x \mapsto x') \rrbracket \\
&= \{(s, h) \mid (s, h) \in \llbracket p_1 * x \mapsto x' \rrbracket, (s, h) \in \llbracket p_2 * x \mapsto x' \rrbracket\} \\
&= \{(s, h) \mid h = h_1 \bullet [s(x) \mapsto s(x')], (s, h_1) \in \llbracket p_1 \rrbracket, h = h_2 \bullet [s(x) \mapsto s(x')], (s, h_2) \in \llbracket p_2 \rrbracket\} \\
&= \{(s, h) \mid h = h' \bullet [s(x) \mapsto s(x')], (s, h') \in \llbracket p_1 \rrbracket, (s, h') \in \llbracket p_2 \rrbracket\} \\
&= \llbracket x \mapsto x' * (p_1 \wedge p_2) \rrbracket
\end{aligned}$$

Point (8) is analogous. $\square$

Lemma B.6. *Let $c \in \text{HACmd}$, $z \notin \text{fv}(c)$ be a fresh variable, and $(s, h), (s', h') \in \Sigma$ be two states such that $(s, h) \in \llbracket \overline{c} \rrbracket(s', h')$. Then, for any value $v \in \text{Val}$,*

$$(s[z \mapsto v], h) \in \llbracket \overline{c} \rrbracket(s'[z \mapsto v], h')$$

Proof. By definition of $\llbracket \overline{c} \rrbracket$, we know that $(s', h') \in \llbracket c \rrbracket(s, h)$ and that the thesis is equivalent to $(s'[z \mapsto v], h') \in \llbracket c \rrbracket(s[z \mapsto v], h)$. The proof is by cases on c .

Case skip

Since $\llbracket \text{skip} \rrbracket$ is the identity, $(s, h) = (s', h')$. Therefore, $(s'[z \mapsto v], h') \in \llbracket \text{skip} \rrbracket(s[z \mapsto v], h)$.

Case $x := a$

By definition of $\llbracket x := a \rrbracket$, $s' = s[x \mapsto \llbracket a \rrbracket s]$ and $h' = h$. Since $z \notin \text{fv}(x := a)$, $z \neq x$ and $z \notin \text{fv}(a)$, therefore

$$s[z \mapsto v][x \mapsto \llbracket a \rrbracket(s[z \mapsto v])] = s[z \mapsto v][x \mapsto \llbracket a \rrbracket s] = s[x \mapsto \llbracket a \rrbracket s][z \mapsto v] = s'[z \mapsto v]$$

With this, we have $(s'[z \mapsto v], h') \in \llbracket x := a \rrbracket(s[z \mapsto v], h)$.

Case $\text{free}(x)$

By definition of $\llbracket \text{free}(x) \rrbracket$, $s' = s$, $h' = h[s(x) \mapsto \delta]$. Since $z \notin \text{fv}(\text{free}(x))$ then $z \neq x$. With this, we have $(s'[z \mapsto v], h') \in \llbracket \text{free}(x) \rrbracket(s[z \mapsto v], h)$. $\square$

Proof of Lemma 5.21. The proof is by induction on the structure of q .

Case $q = \text{false}$

Take $\bar{q} = \text{false}$.

Case $q = \text{true}$

Take $\bar{q} = \text{true}$.

Case $q = q_1 \wedge q_2$

We consider point (1) first. By inductive hypothesis, there exists $\bar{q}_1$ and $\bar{q}_2$ such that

$$\begin{aligned}
q_1 \wedge (x \mapsto x' * \text{true}) &\equiv x \mapsto x' * \bar{q}_1 \\
q_2 \wedge (x \mapsto x' * \text{true}) &\equiv x \mapsto x' * \bar{q}_2
\end{aligned}$$

so that

$$\begin{aligned}
q \wedge (x \mapsto x' * \text{true}) &\equiv q_1 \wedge q_2 \wedge (x \mapsto x' * \text{true}) \\
&\equiv q_1 \wedge (x \mapsto x' * \text{true}) \wedge q_2 \wedge (x \mapsto x' * \text{true}) \\
&\equiv (x \mapsto x' * \bar{q}_1) \wedge (x \mapsto x' * \bar{q}_2) \\
&\equiv x \mapsto x' * (\bar{q}_1 \wedge \bar{q}_2)
\end{aligned}$$

where we used Lemma B.5.7 for the last equivalence. The case for point (2) is analogous using Lemma B.5.8 instead.

Case $q = q_1 \vee q_2$

We consider point (1) first. By inductive hypothesis, there exists $\bar{q}_1$ and $\bar{q}_2$ such that

$$\begin{aligned} q_1 \wedge (x \mapsto x' * \mathbf{true}) &\equiv x \mapsto x' * \bar{q}_1 \\ q_2 \wedge (x \mapsto x' * \mathbf{true}) &\equiv x \mapsto x' * \bar{q}_2 \end{aligned}$$

so that

$$\begin{aligned} q \wedge (x \mapsto x' * \mathbf{true}) &\equiv (q_1 \vee q_2) \wedge (x \mapsto x' * \mathbf{true}) \\ &\equiv (q_1 \wedge (x \mapsto x' * \mathbf{true})) \vee (q_2 \wedge (x \mapsto x' * \mathbf{true})) \\ &\equiv (x \mapsto x' * \bar{q}_1) \vee (x \mapsto x' * \bar{q}_2) \\ &\equiv x \mapsto x' * (\bar{q}_1 \vee \bar{q}_2) \end{aligned}$$

where we used Lemma B.5.1 for the second equivalence and Lemma B.5.2 for the last one. The case for point (2) is analogous.

Case $q = a_1 \asymp a_2$

If we take $\bar{q} = a_1 \asymp a_2$, both points follow from Lemma B.5.6 by taking $p_1 = \mathbf{true}$ and $p_2 = x \mapsto x'$ (resp. $p_2 = x \not\mapsto$).

Case $q = \mathbf{emp}$

Both formulae $\mathbf{emp} \wedge (x \mapsto x' * \mathbf{true})$ and $\mathbf{emp} \wedge (x \not\mapsto * \mathbf{true})$ are not satisfiable. Therefore we get the thesis with $\bar{q} = \mathbf{false}$.

Case $q = z \mapsto z'$

For point (1):

$$\begin{aligned} &\{ \{ z \mapsto z' \wedge (x \mapsto x' * \mathbf{true}) \} \} \\ &= \{ (s, h) \mid (s, h) \in \{ \{ z \mapsto z' \} \}, (s, h) \in \{ \{ x \mapsto x' * \mathbf{true} \} \} \} \\ &= \{ (s, h) \mid h = [s(z) \mapsto s(z')], h = [s(x) \mapsto s(x')] \bullet h_t \} \\ &= \{ (s, h) \mid h = [s(x) \mapsto s(x')], s(z) = s(x), s(z') = s(x') \} \\ &= \{ \{ z' = x' \wedge z = x \wedge x \mapsto x' \} \} \end{aligned}$$

For point (2), we observe that $z \mapsto z' \wedge (x \not\mapsto * \mathbf{true})$ is not satisfiable, so we get the thesis with $\bar{q} = \mathbf{false}$.

Case $q = z \not\mapsto$

For point (1), we observe that $z \not\mapsto \wedge (x \mapsto x' * \mathbf{true})$ is not satisfiable, so we get the thesis with $\bar{q} = \mathbf{false}$.

For point (2):

$$\begin{aligned} &\{ \{ z \not\mapsto \wedge (x \not\mapsto * \mathbf{true}) \} \} \\ &= \{ (s, h) \mid (s, h) \in \{ \{ z \not\mapsto \} \}, (s, h) \in \{ \{ x \not\mapsto * \mathbf{true} \} \} \} \\ &= \{ (s, h) \mid h = [s(z) \mapsto \delta], h = [s(x) \mapsto \delta] \bullet h_t \} \\ &= \{ (s, h) \mid h = [s(x) \mapsto \delta], s(z) = s(x) \} \\ &= \{ \{ z = x \wedge x \not\mapsto \} \} \end{aligned}$$

Case $q = q_1 * q_2$

We consider point (1) first. By inductive hypothesis, there exists $\bar{q}_1$ and $\bar{q}_2$ such that

$$\begin{aligned} q_1 \wedge (x \mapsto x' * \mathbf{true}) &\equiv x \mapsto x' * \bar{q}_1 \\ q_2 \wedge (x \mapsto x' * \mathbf{true}) &\equiv x \mapsto x' * \bar{q}_2 \end{aligned}$$

Take $\bar{q} = \bar{q}_1 * q_2 \vee q_1 * \bar{q}_2$. We have

$$\{x \mapsto x' * p\} = \{x \mapsto x' * \bar{q}_1 * q_2\} \cup \{x \mapsto x' * q_1 * \bar{q}_2\}$$

Now consider

$$\begin{aligned} & \{q \wedge (x \mapsto x' * \mathbf{true})\} \\ &= \{(q_1 * q_2) \wedge (x \mapsto x' * \mathbf{true})\} \\ &= \{(s, h) \mid (s, h) \in \{x \mapsto x' * \mathbf{true}\}, h = h_1 \bullet h_2, (s, h_1) \in \{q_1\}, (s, h_2) \in \{q_2\}\} \\ &= \{(s, h) \mid h(s(x)) = s(x'), h = h_1 \bullet h_2, (s, h_1) \in \{q_1\}, (s, h_2) \in \{q_2\}\} \end{aligned}$$

For every state (s, h) in this set, either $s(x) \in \text{dom}(h_1)$ or $s(x) \in \text{dom}(h_2)$: it can't be in neither because $h(s(x)) = s(x')$. Consider the former case: then $(s, h_1) \in \{x \mapsto x' * \mathbf{true}\}$, so that $(s, h_1) \in \{q_1 \wedge (x \mapsto x' * \mathbf{true})\} = \{x \mapsto x' * \bar{q}_1\}$, so that $(s, h_1 \bullet h_2) \in \{x \mapsto x' * \bar{q}_1 * q_2\}$. Analogously, in the latter case $(s, h_1 \bullet h_2) \in \{x \mapsto x' * q_1 * \bar{q}_2\}$. Therefore, $\{q \wedge (x \mapsto x' * \mathbf{true})\} \subseteq \{x \mapsto x' * \bar{q}\}$.

For the other inclusion, consider

$$\begin{aligned} & \{x \mapsto x' * \bar{q}_1 * q_2\} \\ &= \{(s, h) \mid h = h_1 \bullet h_2, (s, h_1) \in \{x \mapsto x' * \bar{q}_1\}, (s, h_2) \in \{q_2\}\} \\ &= \{(s, h) \mid h = h_1 \bullet h_2, (s, h_1) \in \{q_1 \wedge (x \mapsto x' * \mathbf{true})\}, (s, h_2) \in \{q_2\}\} \\ &= \{(s, h) \mid h = h_1 \bullet h_2, h_1(s(x)) = s(x'), (s, h_1) \in \{q_1\}, (s, h_2) \in \{q_2\}\} \\ &\subseteq \{(s, h) \mid h = h_1 \bullet h_2, h(s(x)) = s(x'), (s, h_1) \in \{q_1\}, (s, h_2) \in \{q_2\}\} \\ &= \{q \wedge (x \mapsto x' * \mathbf{true})\} \end{aligned}$$

and analogously for $\{x \mapsto x' * q_1 * \bar{q}_2\} \subseteq \{q \wedge (x \mapsto x' * \mathbf{true})\}$.

The case for point (2) is analogous. $\square$

Lemma B.7. *Let $q \in \text{Asl}$ be an assertion without $\vee$ and $\exists$, and let $c \in \text{HACmd}$. Then there exists $p \in \text{Asl}$ such that $\{p\} = \llbracket \bar{c} \rrbracket \{q\}$, and $\langle\!\langle p \rangle\!\rangle \subset \langle\!\langle q \rangle\!\rangle$ is provable.*

Proof. We recall that

$$\llbracket \bar{c} \rrbracket \{q\} = \{(s, h) \mid \langle\!\langle c \rangle\!\rangle(s, h) \cap \{q\} \neq \emptyset\}$$

and that $\mathbf{err} \notin \{q\}$ for any q . In the proof below, we will use the following equivalence: given a state (s, h) such that $s(h(x)) \in \text{Val}$, $(s, h) \in \{x \mapsto - * \mathbf{true}\}$. Therefore, $(s, h) \in \{q\}$ if and only if $(s, h) \in \{q \wedge (x \mapsto - * \mathbf{true})\}$. Using Lemma 5.21.1, there exists a q' (which depends on q and x but not on (s, h)) such that this is true if and only if $(s, h) \in \{\exists x'. (x \mapsto x' * q')\}$. Analogously (using Lemma 5.21.2), if $s(h(x)) = \delta$, $(s, h) \in \{q\}$ if and only if $(s, h) \in \{x \mapsto \delta * q'\}$ for some q' .

We now proceed by cases on the heap atomic command c .

Case skip

We have

$$\llbracket \bar{c} \rrbracket \{q\} = \{(s, h) \mid \langle\!\langle \text{skip} \rangle\!\rangle(s, h) \cap \{q\} \neq \emptyset\} = \{(s, h) \mid (s, h) \in \{q\}\}$$

So we have the thesis taking $p = q$, and we prove it by using $\langle\!\langle \text{skip} \rangle\!\rangle$ and $\langle\!\langle \text{frame} \rangle\!\rangle$.

Case $x := a$

We have

$$\begin{aligned} \llbracket \bar{c} \rrbracket \{q\} &= \{(s, h) \mid \langle\!\langle x := a \rangle\!\rangle(s, h) \cap \{q\} \neq \emptyset\} \\ &= \{(s, h) \mid (s[x \mapsto \langle\!\langle a \rangle\!\rangle s], h) \in \{q\}\} \\ &= \{(s, h) \mid (s, h) \in \{q[a/x]\}\} \end{aligned}$$

So we have the thesis taking $p = q[a/x]$, and we prove it by using $\llbracket \text{assign} \rrbracket$.

Case b?

We have

$$\begin{aligned} \llbracket \overleftarrow{\mathbf{c}} \rrbracket \{q\} &= \{(s, h) \mid (\mathbf{b}?) (s, h) \cap \{q\} \neq \emptyset\} \\ &= \{(s, h) \mid (\mathbf{b}) s = \mathbf{tt}, (s, h) \in \{q\}\} \\ &= \{q \wedge b\} \end{aligned}$$

So we have the thesis taking $p = q \wedge b$, and we prove it by using $\llbracket \text{assume} \rrbracket$.

Case $x := \text{alloc}()$

We have

$$\begin{aligned} \llbracket \overleftarrow{\mathbf{c}} \rrbracket \{q\} &= \{(s, h) \mid (x := \text{alloc}()) (s, h) \cap \{q\} \neq \emptyset\} \\ &= \{(s, h) \mid \exists l, v. h(l) = \delta, (s[x \mapsto l], h[l \mapsto v]) \in \{q\}\} \\ &= \{(s, h) \mid \exists l, v. h(l) = \delta, (s[x \mapsto l], h[l \mapsto v]) \in \{\exists x'. x \mapsto x' * q'\}\} \\ &= \{(s, h) \mid \exists l, v. h(l) = \delta, \exists v'. (s[x \mapsto l][x' \mapsto v'], h[l \mapsto v]) \in \{x \mapsto x' * q'\}\} \\ &= \{(s, h) \mid \exists l, v. h = [l \mapsto \delta] \bullet h_q, \exists v'. \\ &\quad (s[x \mapsto l][x' \mapsto v'], [l \mapsto v]) \in \{x \mapsto x'\}, \\ &\quad (s[x \mapsto l][x' \mapsto v'], h_q) \in \{q'\}\} \\ &= \{(s, h) \mid \exists l, v. h = [l \mapsto \delta] \bullet h_q, (s[x \mapsto l][x' \mapsto v], h_q) \in \{q'\}\} \\ &= \{\exists i. \exists x'. i \not\mapsto * q'[i/x]\} \end{aligned}$$

for fresh variables i . So we have the thesis taking $p = \exists i. \exists x'. i \not\mapsto * q'[i/x]$. To prove the triple $\llbracket p \rrbracket x := \text{alloc}() \llbracket q \rrbracket$ we first observe the following chain of implications:

$$\begin{aligned} q &\Leftarrow \exists x'. x \mapsto x' * q' && [\text{Lemma 5.21.1}] \\ &\equiv \exists i. \exists x'. x \mapsto x' * q' && [i \text{ fresh}] \\ &\Leftarrow \exists i. \exists x'. x = i \wedge (x \mapsto x' * q') \\ &\Leftarrow \exists i. \exists x'. x = i \wedge (x \mapsto x' * q'[i/x]) && [\text{replacing } i = x] \\ &\Leftarrow \exists i. \exists x'. (x = i \wedge x \mapsto x') * q'[i/x] && [\text{Lemma B.5.6}] \end{aligned}$$

Then we prove the triple with the following derivation tree:

$$\frac{\frac{\frac{x \notin \text{fv}(q'[i/x]) \quad \overline{\llbracket i \not\mapsto \rrbracket} \mathbf{c} \llbracket x = i \wedge x \mapsto x' \rrbracket} \llbracket i \not\mapsto * q'[i/x] \rrbracket \mathbf{c} \llbracket (x = i \wedge x \mapsto x') * q'[i/x] \rrbracket} \llbracket p \rrbracket \mathbf{c} \llbracket \exists i. \exists x'. (x = i \wedge x \mapsto x') * q'[i/x] \rrbracket} \llbracket p \rrbracket \mathbf{c} \llbracket q \rrbracket \begin{array}{l} \llbracket \text{alloc} \rrbracket \\ \llbracket \text{frame} \rrbracket \\ \llbracket \text{exists} \rrbracket \text{ x2} \\ \llbracket \text{cons} \rrbracket \end{array}$$

Case $\text{free}(x)$

We have

$$\begin{aligned} \llbracket \overleftarrow{\mathbf{c}} \rrbracket \{q\} &= \{(s, h) \mid (\mathbf{free}(x)) (s, h) \cap \{q\} \neq \emptyset\} \\ &= \{(s, h) \mid h(s(x)) \in \text{Val}, (s, h[s(x) \mapsto \delta]) \in \{q\}\} \\ &= \{(s, h) \mid h(s(x)) \in \text{Val}, (s, h[s(x) \mapsto \delta]) \in \{x \not\mapsto * q'\}\} \\ &= \{(s, h) \mid h(s(x)) \in \text{Val}, h = [s(x) \mapsto h(s(x))] \bullet h_q, (s, h_q) \in \{q'\}\} \\ &= \{x \mapsto - * q'\} \end{aligned}$$

So we have the thesis taking $p = x \mapsto - * q'$, and we prove it by using $\llbracket \text{free} \rrbracket$ and $\llbracket \text{frame} \rrbracket$ with frame q' (this is always possible because $\text{mod}(\text{free}(x)) = \emptyset$).

Case $x := [y]$

We have

$$\begin{aligned}
\llbracket \text{c} \rrbracket \{q\} &= \{(s, h) \mid ([x] := [y])(s, h) \cap \{q\} \neq \emptyset\} \\
&= \{(s, h) \mid h(s(y)) \in \text{Val}, (s[x \mapsto h(s(y))], h) \in \{q\}\} \\
&= \{(s, h) \mid h(s(y)) \in \text{Val}, (s[x \mapsto h(s(y))], h) \in \{\exists y'. y \mapsto y' * q'\}\} \\
&= \{(s, h) \mid h(s(y)) \in \text{Val}, h = [s(y) \mapsto h(s(y))] \bullet h_q, (s[x \mapsto h(s(y))], h_q) \in \{q'\}\} \\
&= \{\exists y'. (y \mapsto y' * q'[y'/x])\}
\end{aligned}$$

So we have the thesis taking $p = \exists y'. (y \mapsto y' * q'[y'/x])$, and we prove it by using $\llbracket \text{load} \rrbracket$ with $a = y'$ and $\llbracket \text{exists} \rrbracket$ because y' is fresh.

Case $[x] := y$

We have

$$\begin{aligned}
\llbracket \text{c} \rrbracket \{q\} &= \{(s, h) \mid ([x] := y)(s, h) \cap \{q\} \neq \emptyset\} \\
&= \{(s, h) \mid h(s(x)) \in \text{Val}, (s, h[s(x) \mapsto s(y)]) \in \{q\}\} \\
&= \{(s, h) \mid h(s(x)) \in \text{Val}, (s, h[s(x) \mapsto s(y)]) \in \{\exists x'. x \mapsto x' * q'\}\} \\
&= \{(s, h) \mid h(s(x)) \in \text{Val}, h = [s(x) \mapsto h(s(x))] \bullet h_q, (s, h_q) \in \{q'\}\} \\
&= \{x \mapsto - * q'\}
\end{aligned}$$

So we have the thesis taking $p = x \mapsto - * q'$. To prove the triple $\langle p \rangle [x] := y \langle q \rangle$, we first prove $\langle p \rangle [x] := y \langle x \mapsto y * q' \rangle$ by using $\llbracket \text{store} \rrbracket$ and $\llbracket \text{frame} \rrbracket$ with frame q' (this is always possible because $\text{mod}([x] := y) = \emptyset$). Then we observe that $x \mapsto y \implies \exists x'. x \mapsto x'$, so that we get $\langle p \rangle [x] := y \langle q \rangle$ by using $\llbracket \text{cons} \rrbracket$. $\square$

Lemma B.8. *Let $p, q \in \text{Asl}$ be two assertions and $c \in \text{HACmd}$ such that $\{p\} = \llbracket \text{c} \rrbracket \{q\}$. Then, for $z \in \text{Var}$ fresh, $\{\exists z. p\} = \llbracket \text{c} \rrbracket \{\exists z. q\}$*

Proof. In this proof, we use the following notation: given a state $(s, h) = \sigma \in \Sigma$ and a value $v \in \text{Val}$, we denote with σ_v the state $(s[z \mapsto v], h)$, where we performed in the store s the substitution of value v for the variable z of the statement of the lemma. We prove the two inclusions separately.

To show that $\{\exists z. p\} \supseteq \llbracket \text{c} \rrbracket \{\exists z. q\}$, take $\sigma \in \llbracket \text{c} \rrbracket \{\exists z. q\}$. Then, by definition of $\llbracket \text{c} \rrbracket$, there exist $\sigma' \in \{\exists z. q\}$ such that $\sigma \in \llbracket \text{c} \rrbracket \sigma'$. By definition of $\{\exists z. q\}$, there exists a value $v \in \text{Val}$ such that $\sigma'_v \in \{q\}$. Then, since z is fresh, by Lemma B.6 $\sigma_v \in \llbracket \text{c} \rrbracket \sigma'_v \subseteq \llbracket \text{c} \rrbracket \{q\}$. Since by hypothesis $\{p\} = \llbracket \text{c} \rrbracket \{q\}$ we have $\sigma_v \in \{p\}$, and thus $\sigma \in \{\exists z. p\}$.

To show that $\{\exists z. p\} \subseteq \llbracket \text{c} \rrbracket \{\exists z. q\}$, take $(s, h) = \sigma \in \{\exists z. p\}$. By definition of $\{\exists z. p\}$, there exists a value v such that $\sigma_v \in \{p\}$, and $\{p\} = \llbracket \text{c} \rrbracket \{q\}$ by hypothesis. By definition of $\llbracket \text{c} \rrbracket$, there exist $\sigma' \in \{q\}$ such that $\sigma_v \in \llbracket \text{c} \rrbracket \sigma'$. Let $w = s(z)$: clearly $\sigma = (\sigma_v)_w$. Moreover, $\sigma_v \in \llbracket \text{c} \rrbracket \sigma'$ and z is fresh, so by Lemma B.6 we have $\sigma = (\sigma_v)_w \in \llbracket \text{c} \rrbracket \sigma'_w$. Lastly, since $\sigma' \in \{q\}$ we have $\sigma'_w \in \{\exists z. q\}$, thus $\sigma \in \llbracket \text{c} \rrbracket \{\exists z. q\}$. $\square$

Proof of Theorem 5.24. First we fix q and prove, by induction on the structure of r , that there exists $p \in \text{Asl}$ such that $\{p\} = \llbracket \text{r} \rrbracket \{q\}$, and $\langle p \rangle r \langle q \rangle$ is provable.

Case $r = c$

First, we transform q in a normal form: we rename all quantified variables to fresh names, and use Lemma B.5 (points 1-5) to lift disjunctions to the top, then existential quantifiers. Thus, without loss of generality, we can assume that q is a disjunction of existentially

quantified formulae that don't contain $\vee$ and $\exists$. Moreover, if we have a proof for each one of these formulae without $\vee$ and $\exists$, we can combine them using rules $\langle\langle \text{disj} \rangle\rangle$ and $\langle\langle \text{exists} \rangle\rangle$ to get a proof for the original q , and by Lemma B.8 and additivity of $\llbracket \overleftarrow{c} \rrbracket$ that is the weakest precondition for q . Therefore, again without loss of generality, we can consider only the case in which q does not contain $\vee$ and $\exists$. This case is exactly Lemma B.7, so we conclude the inductive step.

Case $r = r_1; r_2$

By inductive hypothesis on r_2 , we know that there exists an assertion $t \in \text{Asl}$ such that $\{t\} = \llbracket \overleftarrow{r_2} \rrbracket \{q\}$ and $\langle\langle t \rangle\rangle r_2 \langle\langle q \rangle\rangle$ is provable. By inductive hypothesis on r_1 , we know that there exists an assertion $p \in \text{Asl}$ such that $\{p\} = \llbracket \overleftarrow{r_1} \rrbracket \{t\}$ and $\langle\langle p \rangle\rangle r_1 \langle\langle t \rangle\rangle$ is provable. Now $\{p\} = \llbracket \overleftarrow{r_1} \rrbracket \{t\} = \llbracket \overleftarrow{r_1} \rrbracket \llbracket \overleftarrow{r_2} \rrbracket \{q\} = \llbracket \overleftarrow{r_1; r_2} \rrbracket \{q\}$ and we can prove $\langle\langle p \rangle\rangle r_1; r_2 \langle\langle q \rangle\rangle$ using $\langle\langle \text{seq} \rangle\rangle$ and the two proofs given by the inductive hypothesis.

Case $r = r_1 \oplus r_2$

For $i = 1, 2$, by inductive hypothesis on r_i we know that there exists an assertion $p_i \in \text{Asl}$ such that $\{p_i\} = \llbracket \overleftarrow{r_i} \rrbracket \{q\}$ and $\langle\langle p_i \rangle\rangle r_i \langle\langle q \rangle\rangle$ is provable. Therefore $\{p_1 \vee p_2\} = \{p_1\} \cup \{p_2\} = \llbracket \overleftarrow{r_1} \rrbracket \{q\} \cup \llbracket \overleftarrow{r_2} \rrbracket \{q\} = \llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket \{q\}$ and we can prove $\langle\langle p_1 \vee p_2 \rangle\rangle r_1 \oplus r_2 \langle\langle q \rangle\rangle$ using $\langle\langle \text{choice} \rangle\rangle$ and the two proofs given by the inductive hypothesis.

Now take any $p, q \in \text{Asl}$ such that $\llbracket \overleftarrow{r} \rrbracket \{q\} \supseteq \{p\}$. By the proof above we know that there exists p' such that $\llbracket \overleftarrow{r} \rrbracket \{q\} = \{p'\}$ and $\langle\langle p' \rangle\rangle r \langle\langle q \rangle\rangle$ is provable. Since $\{p\} \subseteq \{p'\}$, the implication $p \implies p'$ holds. Using the oracle for this implication we can prove the triple $\langle\langle p \rangle\rangle r \langle\langle q \rangle\rangle$ using $\langle\langle \text{cons} \rangle\rangle$ and the proof of $\langle\langle p' \rangle\rangle r \langle\langle q \rangle\rangle$. $\square$

Proof of Theorem 5.25. The proof is by induction on the structure of r .

Case $r = c$

This is a special case of completeness for loop-free programs (Theorem 5.24).

Case $r = r_1; r_2$

By inductive hypothesis, given any σ'' such that $\sigma'' \in \llbracket \overleftarrow{r_2} \rrbracket \sigma'$ there exists an assertion t such that $\{t\} = \llbracket \overleftarrow{r_2} \rrbracket \sigma'$ and $\langle\langle t \rangle\rangle r_2 \langle\langle q \rangle\rangle$ is provable and $\sigma'' \in \{t\}$. Particularly, we can take a σ'' such that $\sigma \in \llbracket \overleftarrow{r_1} \rrbracket \sigma''$: this exists because $\sigma \in \llbracket \overleftarrow{r_1; r_2} \rrbracket \sigma' = \llbracket \overleftarrow{r_1} \rrbracket \llbracket \overleftarrow{r_2} \rrbracket \sigma'$ (by Lemma 5.1). Then, again by inductive hypothesis, we get the assertion p such that $\sigma \in \{p\}$ and $\langle\langle p \rangle\rangle r_1 \langle\langle t \rangle\rangle$ is provable. We conclude the inductive case by proving $\langle\langle p \rangle\rangle r_1; r_2 \langle\langle q \rangle\rangle$ via rule $\langle\langle \text{seq} \rangle\rangle$.

Case $r = r_1 \oplus r_2$

Since $\sigma \in \llbracket \overleftarrow{r_1 \oplus r_2} \rrbracket \sigma' = \llbracket \overleftarrow{r_1} \rrbracket \sigma' \cup \llbracket \overleftarrow{r_2} \rrbracket \sigma'$ (by Lemma 5.1), there must exist an $i \in \{1, 2\}$ such that $\sigma \in \llbracket \overleftarrow{r_i} \rrbracket \sigma'$. Without loss of generality, we assume $i = 1$. By inductive hypothesis, we get an assertion p such that $\sigma \in \{p\}$ and $\langle\langle p \rangle\rangle r_1 \langle\langle q \rangle\rangle$ is provable. We conclude the inductive case with the proof

$$\frac{\text{(induction)} \quad \frac{\langle\langle p \rangle\rangle r_1 \langle\langle q \rangle\rangle \quad \langle\langle \text{false} \rangle\rangle r_2 \langle\langle q \rangle\rangle}{\langle\langle p \rangle\rangle r_1 \oplus r_2 \langle\langle q \rangle\rangle} \quad \langle\langle \text{empty} \rangle\rangle}{\langle\langle p \rangle\rangle r_1 \oplus r_2 \langle\langle q \rangle\rangle} \langle\langle \text{choice} \rangle\rangle$$

Case $r = r^*$

Since $\sigma \in \llbracket \overleftarrow{r^*} \rrbracket \sigma' = \bigcup_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n \sigma'$ (by Lemma 5.1), there must exist an $m \geq 0$ such that

$\sigma \in \llbracket \overleftarrow{r} \rrbracket^m \sigma'$. Therefore, there must exist a sequence of states $\{\sigma_i\}_{0 \leq i \leq m}$ such that $\sigma_0 = \sigma'$, $\sigma_m = \sigma$ and $\sigma_{i+1} \in \llbracket \overleftarrow{r} \rrbracket \sigma_i$ for all $0 \leq i < m$. By inductive hypothesis, fixed $q_0 = q$, there exists a corresponding sequence of assertions $\{q_i\}_{0 \leq i \leq m}$ such that $\sigma_i \in \{q_i\}$ and $\langle\langle q_{i+1} \rangle\rangle r \langle\langle q_i \rangle\rangle$ is provable for all $0 \leq i < m$. We take $p = q_m$ and conclude the inductive case with the proof

$$\begin{array}{c}
\frac{}{\langle\langle q_m \rangle\rangle \text{ r}^* \langle\langle q_m \rangle\rangle} \langle\text{iter0}\rangle \\
\frac{\vdots}{\langle\langle q_m \rangle\rangle \text{ r}^* \langle\langle q_2 \rangle\rangle} \langle\text{unroll}\rangle \quad \frac{(\text{induction})}{\langle\langle q_2 \rangle\rangle \text{ r} \langle\langle q_1 \rangle\rangle} \\
\frac{\langle\langle q_m \rangle\rangle \text{ r}^*; \text{r} \langle\langle q_1 \rangle\rangle}{\langle\langle q_m \rangle\rangle \text{ r}^* \langle\langle q_1 \rangle\rangle} \langle\text{unroll}\rangle \quad \frac{}{\langle\langle q_1 \rangle\rangle \text{ r} \langle\langle q_0 \rangle\rangle} \langle\text{seq}\rangle \quad \frac{(\text{induction})}{\langle\langle q_1 \rangle\rangle \text{ r} \langle\langle q_0 \rangle\rangle} \\
\frac{\langle\langle q_m \rangle\rangle \text{ r}^*; \text{r} \langle\langle q_0 \rangle\rangle}{\langle\langle q_m \rangle\rangle \text{ r}^* \langle\langle q_0 \rangle\rangle} \langle\text{unroll}\rangle \quad \langle\text{seq}\rangle
\end{array}$$

□

Appendix C

LCL_A supplementary materials

This appendix contains technical details of proofs and examples for Chapter 6.

Proof of Theorem 6.2, extensional soundness of LCL_A. First we remark that points (1) and (3) implies point (2):

$$\begin{aligned}
 \alpha(Q) &\leq \alpha(\llbracket r \rrbracket P) && [(1) \text{ and monotonicity of } \alpha] \\
 &\leq \llbracket r \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \\
 &= \alpha(Q) && [(3)]
 \end{aligned}$$

So all the lines are equal, in particular $\alpha(Q) = \alpha(\llbracket r \rrbracket P)$. The proof is then by induction on the derivation tree of $\vdash_A [P] \text{ r } [Q]$, but we only have to prove (1) and (3) because of the observation above.

Case (transfer)

(1) It follows since $\llbracket e \rrbracket P \leq \llbracket e \rrbracket P$.

(3) It follows from the local completeness hypothesis $\mathbb{C}_P^A(\llbracket e \rrbracket)$: $\llbracket e \rrbracket^A \alpha(P) = \alpha(\llbracket e \rrbracket \gamma \alpha(P)) = \alpha(\llbracket e \rrbracket P)$.

Case (relax)

(1) By inductive hypothesis we know $Q' \leq \llbracket r \rrbracket P'$, and together with the other two hypotheses of the rule we have $Q \leq Q' \leq \llbracket r \rrbracket P' \leq \llbracket r \rrbracket P$.

(3) We remark that $\alpha(P) = \alpha(P')$ if and only if $A(P) = A(P')$ by injectivity of γ in a GI, and that $P' \leq P \leq A(P')$ implies $A(P) = A(P')$. Thus the hypotheses of the rule gives $\alpha(P) = \alpha(P')$ and $\alpha(Q) = \alpha(Q')$. Point (3) then follow by the inductive hypothesis $\llbracket r \rrbracket^A \alpha(P') = \alpha(Q')$:

$$\llbracket r \rrbracket^A \alpha(P) = \llbracket r \rrbracket^A \alpha(P') = \alpha(Q') = \alpha(Q)$$

Case (seq)

(1) $Q \leq \llbracket r_2 \rrbracket R \leq \llbracket r_2 \rrbracket (\llbracket r_1 \rrbracket P) = \llbracket r_1; r_2 \rrbracket P$, where the inequalities follow from inductive hypotheses and monotonicity of $\llbracket r_2 \rrbracket$.

(3) We recall that $\llbracket r_1; r_2 \rrbracket^A \leq \llbracket r_2 \rrbracket^A \llbracket r_1 \rrbracket^A$.

$$\begin{aligned}
 \alpha(Q) &\leq \alpha(\llbracket r_1; r_2 \rrbracket P) && [(1) \text{ and monotonicity of } \alpha] \\
 &\leq \llbracket r_1; r_2 \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \\
 &\leq \llbracket r_2 \rrbracket^A \llbracket r_1 \rrbracket^A \alpha(P) && [\text{recalled above}] \\
 &= \llbracket r_2 \rrbracket^A \alpha(R) && [\text{inductive hp}] \\
 &= \alpha(Q) && [\text{inductive hp}]
 \end{aligned}$$

So all the lines are equal, in particular $\llbracket r_1; r_2 \rrbracket^A \alpha(P) = \alpha(Q)$.

Case (join)

(1) By inductive hypotheses, $Q_1 \leq \llbracket r_1 \rrbracket P$ and $Q_2 \leq \llbracket r_2 \rrbracket P$. Hence $Q_1 \vee Q_2 \leq \llbracket r_1 \rrbracket P \vee \llbracket r_2 \rrbracket P = \llbracket r_1 \oplus r_2 \rrbracket P$.

(3) We observe that

$$\begin{aligned} \llbracket r_1 \oplus r_2 \rrbracket^A \alpha(P) &= \alpha(\llbracket r_1 \oplus r_2 \rrbracket \gamma \alpha(P)) \\ &= \alpha(\llbracket r_1 \rrbracket \gamma \alpha(P) \sqcup \llbracket r_2 \rrbracket \gamma \alpha(P)) \\ &= \alpha(\llbracket r_1 \rrbracket \gamma \alpha(P) \sqcup \alpha(\llbracket r_2 \rrbracket \gamma \alpha(P)) \\ &= \llbracket r_1 \rrbracket^A \alpha(P) \sqcup \llbracket r_2 \rrbracket^A \alpha(P) \end{aligned}$$

where we used additivity of α . Recalling that by inductive hypotheses, $\alpha(Q_1) = \llbracket r_1 \rrbracket^A \alpha(P)$ and $\alpha(Q_2) = \llbracket r_2 \rrbracket^A \alpha(P)$, we get

$$\begin{aligned} \alpha(Q_1 \vee Q_2) &= \alpha(Q_1) \sqcup \alpha(Q_2) && [\alpha \text{ is additive}] \\ &= \llbracket r_1 \rrbracket^A \alpha(P) \sqcup \llbracket r_2 \rrbracket^A \alpha(P) && [\text{inductive hypotheses}] \\ &= (\llbracket r_1 \oplus r_2 \rrbracket^A) \alpha(P) && [\text{observation above}] \end{aligned}$$

Case (rec)

(1) First, we show that $\llbracket r^* \rrbracket R \leq \llbracket r^* \rrbracket P$ using the inductive hypothesis $R \leq \llbracket r \rrbracket P$:

$$\begin{aligned} \llbracket r^* \rrbracket R &= \bigsqcup_{n \geq 0} \llbracket r \rrbracket^n R && [\text{definition}] \\ &\leq \bigsqcup_{n \geq 0} \llbracket r \rrbracket^n \llbracket r \rrbracket P && [\text{inductive hp on all operands}] \\ &= \bigsqcup_{n \geq 1} \llbracket r \rrbracket^n P && [\text{renaming the index variable } n] \\ &\leq \bigsqcup_{n \geq 0} \llbracket r \rrbracket^n P && [\text{adding an element to the lub}] \\ &= \llbracket r^* \rrbracket P && [\text{definition}] \end{aligned}$$

Now we show (1):

$$\begin{aligned} Q &\leq \llbracket r^* \rrbracket (P \vee R) && [\text{inductive hp}] \\ &= \llbracket r^* \rrbracket P \vee \llbracket r^* \rrbracket R && [\text{additivity of } \llbracket r^* \rrbracket] \\ &\leq \llbracket r^* \rrbracket P \vee \llbracket r^* \rrbracket P && [\text{observation above}] \\ &= \llbracket r^* \rrbracket P \end{aligned}$$

(3)

$$\begin{aligned} \llbracket r^* \rrbracket^A \alpha(P) &\leq \llbracket r^* \rrbracket^A \alpha(P \vee R) && [\text{monotonicity of } \llbracket r^* \rrbracket^A \alpha] \\ &= \alpha(Q) && [\text{inductive hp}] \\ &\leq \alpha(\llbracket r^* \rrbracket P) && [(1) \text{ and monotonicity of } \alpha] \\ &\leq \llbracket r^* \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r^* \rrbracket^A] \end{aligned}$$

Hence all the lines are equal, and in particular $\llbracket r^* \rrbracket^A \alpha(P) = \alpha(Q)$.

Case (iterate)

(1) We observe that $P = \llbracket r \rrbracket^0 P \leq \llbracket r^* \rrbracket P$. Moreover, by inductive hypothesis $Q \leq \llbracket r \rrbracket P$ so also $Q \leq \llbracket r^* \rrbracket P$. So by definition of lub $P \vee Q \leq \llbracket r^* \rrbracket P$.

(3) First we show by induction on $n \geq 1$ that $\alpha\llbracket r \rrbracket^n \gamma\alpha(P) \leq \llbracket r \rrbracket^A \alpha(P)$. The base case $n = 1$ is trivial recalling that $\llbracket r \rrbracket^A = \alpha\llbracket r \rrbracket \gamma$. So suppose it is true for n and let us show it for $n + 1$:

$$\begin{aligned}
\alpha\llbracket r \rrbracket^{n+1} \gamma\alpha(P) &\leq \alpha\llbracket r \rrbracket \gamma\alpha\llbracket r \rrbracket^n \gamma\alpha(P) && [\gamma\alpha \leq \text{id}] \\
&\leq \llbracket r \rrbracket^A \llbracket r \rrbracket^A \alpha(P) && [\text{inductive hp } \alpha\llbracket r \rrbracket^n \gamma\alpha(P) \leq \llbracket r \rrbracket^A \alpha(P)] \\
&= \llbracket r \rrbracket^A \alpha(Q) && [\text{inductive hp } \vdash_A [P] \text{ } r [Q]] \\
&\leq \llbracket r \rrbracket^A \alpha(P) && [\text{hypothesis of the rule } Q \leq A(P)]
\end{aligned}$$

For the last inequality, we used that in any GC $Q \leq A(P) = \gamma\alpha(P)$ if and only if $\alpha(Q) \leq \alpha(P)$. Now we can prove the following chain of inequalities:

$$\begin{aligned}
\llbracket r^* \rrbracket^A \alpha(P) &= \alpha \bigsqcup_{n \geq 0} \llbracket r \rrbracket^n \gamma\alpha(P) && [\text{definition}] \\
&= \bigsqcup_{n \geq 0} \alpha\llbracket r \rrbracket^n \gamma\alpha(P) && [\alpha \text{ is additive}] \\
&= \alpha\llbracket r \rrbracket^0 \gamma\alpha(P) \sqcup \left(\bigsqcup_{n \geq 1} \alpha\llbracket r \rrbracket^n \gamma\alpha(P) \right) && [\text{splitting the lub}] \\
&\leq \alpha\llbracket r \rrbracket^0 \gamma\alpha(P) \sqcup \left(\bigsqcup_{n \geq 1} \llbracket r \rrbracket^A \alpha(P) \right) && [\text{shown above by induction}] \\
&= \alpha\llbracket r \rrbracket^0 \gamma\alpha(P) \sqcup \llbracket r \rrbracket^A \alpha(P) && [\text{lub of constant terms}] \\
&= \alpha(P) \sqcup \llbracket r \rrbracket^A \alpha(P) && [\llbracket r \rrbracket^0 = \text{id and } \alpha\gamma\alpha = \alpha] \\
&= \alpha(P) \sqcup \alpha(Q) && [\text{inductive hp}] \\
&= \alpha(P \vee Q) && [\alpha \text{ is additive}] \\
&\leq \alpha(\llbracket r^* \rrbracket P) && [(1) \text{ and monotonicity of } \alpha] \\
&\leq \llbracket r^* \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r^* \rrbracket^A]
\end{aligned}$$

Thus all the lines above are equal, and in particular $\llbracket r^* \rrbracket^A \alpha(P) = \alpha(P \vee Q)$. $\square$

This technical lemma is used in the following proofs.

Lemma C.1. *If $A' \preceq A$ then $A = AA' = A'A$*

Proof. Fix a concrete element $c \in C$. Since $A' \preceq A$ we have $c \leq A'(c) \leq A(c)$. Applying A , by monotonicity we get $A(c) \leq AA'(c) \leq AA(c) = A(c)$, where the last equality is idempotency of A . This means $A = AA'$. Now consider $A'A(c)$. Since A is a closure operator $A'A(c) \leq A(A'A(c))$. But we just showed $AA'(A(c)) = A(A(c)) = A(c)$. Lastly, since A' is a closure operator too, $A(c) \leq A'A(c)$. Hence $A(c) \leq A'A(c) \leq A(c)$, so $A(c) = A'A(c)$. $\square$

We point out that, by injectivity of γ , this also means $\alpha\gamma'\alpha' = \alpha$.

Proof of Theorem 6.3, extensional soundness of rule (refine-ext). We recall that the intuitive premise $A\llbracket r \rrbracket^{A'} A(P) = A(Q)$ of the rule formally is $\alpha\gamma'\llbracket r \rrbracket^{A'} \alpha' A(P) = \alpha(Q)$. Since the proof of Theorem 6.2 is by induction, we extend it by just proving the new inductive case. We also remark that we only need to prove points (1) and (3), since they imply point (2).

Case (refine-ext)

(1) It's the same as point (1) of extensional soundness applied to $\vdash_{A'} [P] \text{ r } [Q]$, since this conclusion does not depend on the abstract domain.

(3)

$$\begin{aligned}
\alpha(Q) &\leq \alpha(\llbracket r \rrbracket P) && [(1) \text{ and monotonicity of } \alpha] \\
&\leq \llbracket r \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \\
&= \alpha \llbracket r \rrbracket \gamma \alpha(P) && [\text{definition}] \\
&= \alpha \gamma' \alpha' \llbracket r \rrbracket \gamma' \alpha' \gamma \alpha(P) && [\text{Lemma C.1}] \\
&= \alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) && [\text{definition}] \\
&= \alpha(Q) && [\text{hypothesis of the rule}]
\end{aligned}$$

Hence all the lines are equal; in particular $\llbracket r \rrbracket^A \alpha(P) = \alpha(Q)$. $\square$

Proof of Proposition 6.7, extensional soundness of other refinement rules. We prove that the hypotheses of both rules implies those of (refine-ext). This means than whenever we can apply the former we could also apply the latter, so that Theorem 6.3 ensures extensional soundness.

The first two hypotheses $\vdash_{A'} [P] \text{ r } [Q]$ and $A' \preceq A$ are shared among all three rules, so we only have to show $\alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) = \alpha(Q)$. We recall that, by extensional soundness, $\vdash_{A'} [P] \text{ r } [Q]$ implies (1) $Q \leq \llbracket r \rrbracket P$ and (3) $\llbracket r \rrbracket^{A'} \alpha'(P) = \alpha'(Q)$.

For (refine-int), we observe

$$\begin{aligned}
\alpha(Q) &\leq \alpha(\llbracket r \rrbracket P) && [Q \leq \llbracket r \rrbracket P \text{ and monotonicity of } \alpha] \\
&\leq \llbracket r \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \\
&= \alpha \llbracket r \rrbracket A(P) && [\text{definition}] \\
&= \alpha \gamma' \alpha' \llbracket r \rrbracket A' A(P) && [\text{Lemma C.1}] \\
&= \alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) && [\text{definition}] \\
&\leq \alpha \gamma' \llbracket r \rrbracket_{A'}^{\#} \alpha' A(P) && [\llbracket r \rrbracket^{A'} \leq \llbracket r \rrbracket_{A'}^{\#}] \\
&= \alpha(Q) && [\text{Last hypothesis of the rule}]
\end{aligned}$$

Hence all the lines are equal, and in particular $\alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) = \alpha(Q)$.

For (refine-pre), we observe

$$\begin{aligned}
\alpha(Q) &\leq \alpha(\llbracket r \rrbracket P) && [Q \leq \llbracket r \rrbracket P \text{ and monotonicity of } \alpha] \\
&\leq \llbracket r \rrbracket^A \alpha(P) && [\text{soundness of } \llbracket r \rrbracket^A] \\
&= \alpha \llbracket r \rrbracket A(P) && [\text{definition}] \\
&= \alpha \llbracket r \rrbracket A'(P) && [\text{hp of the rule}] \\
&= \alpha \gamma' \alpha' \llbracket r \rrbracket A'(P) && [\text{Lemma C.1}] \\
&= \alpha \gamma' \llbracket r \rrbracket^{A'} \alpha'(P) && [\text{definition}] \\
&= \alpha \gamma' \alpha'(Q) && [\text{extensional soundness (3)}] \\
&= \alpha(Q) && [\text{Lemma C.1}]
\end{aligned}$$

Hence all the lines are equal, and in particular $\alpha \gamma' \llbracket r \rrbracket^{A'} \alpha' A(P) = \alpha(Q)$. $\square$

Example C.2 (Details of Example 6.9.). The full derivation of the triple $\vdash_{\text{Oct}} [R] \ r_2 \ [Q]$ is shown in Figure C.1, rotated and split to fit the page. We call the command iterated with the Kleene star $r_i \triangleq (\mathbf{x} > 1)?; \mathbf{y} := \mathbf{y} - 1; \mathbf{x} := \mathbf{x} - 1$, and we let $R_2 \triangleq (\mathbf{y} \in \{1; 2; 100\} \wedge \mathbf{x} = \mathbf{y})$. We also used the logical implication $R_2 \implies (\mathbf{y} \in \{1; 99\} \wedge \mathbf{x} = \mathbf{y})$, both explicitly and implicitly in the equivalence $R_2 \vee (\mathbf{y} \in \{1; 99\} \wedge \mathbf{x} = \mathbf{y}) \iff R_2$. ■

Example C.3. Similarly to Example 6.10, we present another triple which is sound but cannot be proved in LCL_A extended with (**refine-pre**). The key difference is that this triple does not rely on divergence, but only on actual (im)precisions in the abstract domain.

Consider the concrete domain $C = \mathcal{P}(\mathbb{Z})$ of integers, the abstract domain Int of intervals, the concrete initial states $P = \{-1, 1\}$ and commands

$$\begin{aligned} r_1 &\triangleq \mathbf{x} != 0? \\ r_2 &\triangleq \mathbf{x} >= 0? \end{aligned}$$

Then, the triple $\vdash_{\text{Int}} [P] \ r_1; r_2 \ [\{1\}]$ is sound but cannot be proved in LCL_A extended with (**refine-pre**).

Following the same line of reasoning in Example 6.10, we observe that all strict subset $P' \subset P$ are such that $\text{Int}(P') \subset P$, and the same property holds for all refinements $A' \preceq \text{Int}$. Again, this means that we cannot apply (**relax**) to change P : to do it, we would need a $P' \subset P$ such that $P \subseteq A'(P')$.

Let $f_1 = \llbracket r_1 \rrbracket$ and $f_2 = \llbracket r_2 \rrbracket$. Observe that in the concrete semantics $f_1(P) = P$ and $f_2(P) = \{1\}$. Inspecting the logic, to prove the triple $\vdash_{\text{Int}} [P] \ r_1; r_2 \ [\{1\}]$ we can only apply three rules: (**relax**), (**refine-pre**) or (**seq**). To apply rule (**relax**) we would need either an under-approximation P' of P with the same abstraction, that does not exist by the observation above, or an over-approximation of Q , that would be unsound since $Q = f(P)$. Hence we cannot apply (**relax**). Suppose to apply (**refine-pre**): any A' used in the rule should satisfy $A' \preceq \text{Int}$ and $A'(P) = \text{Int}(P)$. As we pointed out above, we cannot apply (**relax**) even after the domain refinement. Therefore, the only rule that can be applied is (**seq**). To do that, we must prove two triples $\vdash_{A'} [P] \ r_1 \ [R]$ and $\vdash_{A'} [R] \ r_2 \ [Q]$ for some R .

Irrespective of how we prove the first triple, by soundness (Theorem 6.2) we have $R \subseteq f_1(P) = P$ and $A'(R) = A'(f_1(P)) = A'(P)$. If $R \subset P$ then $A'(R) \subset P \subseteq A'(P)$, which is a contradiction. So $R = P$. Now we should prove a triple $\vdash_{A'} [P] \ r_2 \ [Q]$, but this is impossible since, by soundness, this would imply local completeness of $\llbracket r_2 \rrbracket = f_2$ on P in A' , that does not hold:

$$\begin{aligned} A'f_2(P) &= A'(\{1\}) \\ &\subseteq \text{Int}(\{1\}) = \{1\} \\ &\subset \{0, 1\} = f_2(\text{Int}(P)) = f_2A'(P) \\ &\subseteq A'f_2A'(P) \end{aligned}$$

Observe that, if we add (**refine-int**) to the proof system, we can use it to change the domain to one where we can express P (for instance, the concrete domain $\mathcal{P}(\mathbb{Z})$ or the refinement $\text{Int}_P \triangleq \text{Int} \cup \{P\}$) to prove the triple by applying (**seq**) and then (**transfer**) on both subtrees, as shown in Figure C.2. The two local completeness proof obligations are

$$\frac{\frac{\mathbb{C}_P^{\text{Int}_P}(\llbracket x \neq 0? \rrbracket)}{\vdash_{\text{Int}_P} [P] \ x \neq 0? [P]} \text{ (transfer)} \quad \frac{\mathbb{C}_P^{\text{Int}_P}(\llbracket x \geq 0? \rrbracket)}{\vdash_{\text{Int}_P} [P] \ x \geq 0? [Q]} \text{ (transfer)}}{\vdash_{\text{Int}_P} [P] \ r_1; r_2 [Q] \quad \text{Int}_P \preceq \text{Int} \quad \text{Int}(\llbracket r \rrbracket_{\text{Int}_P}^{\#}(\text{Int}(P))) = \text{Int}(Q)} \text{ (seq)} \\
\vdash_{\text{Int}} [P] \ r_1; r_2 [Q] \text{ (refine-int)}$$

Figure C.2: Derivation of $\vdash_{\text{Int}} [P] \ r [Q]$ for Example C.3.

trivially satisfied because $P \in \text{Int}_P$, and the hypothesis of (refine-int) is verified:

$$\begin{aligned}
\text{Int}(\llbracket r \rrbracket_{\text{Int}_P}^{\#} \text{Int}(P)) &= \text{Int}(\llbracket r_2 \rrbracket_{\text{Int}_P}^{\text{Int}_P} \llbracket r_1 \rrbracket_{\text{Int}_P}^{\text{Int}_P} \text{Int}(P)) \\
&= \text{Int}(\llbracket r_2 \rrbracket_{\text{Int}_P}^{\text{Int}_P} \llbracket x \neq 0?^{\text{Int}_P} \rrbracket P) \\
&= \text{Int}(\llbracket x \geq 0? \rrbracket P) \\
&= \text{Int}([1; 1]) = \text{Int}(Q)
\end{aligned}$$

■

The following lemma is used in the subsequent proof.

Lemma C.4. *Let $r \in \text{Reg}$ be a regular command. If $A \preceq A'$ then $\gamma \llbracket r \rrbracket_A^{\#} \alpha \leq \gamma' \llbracket r \rrbracket_{A'}^{\#} \alpha'$.*

Proof. The proof is by structural induction on r .

Case ($r = e$)

$$\gamma \llbracket r \rrbracket_A^{\#} \alpha = \gamma \alpha(e) \gamma \alpha = A(e) A \leq A'(e) A' = \gamma' \llbracket r \rrbracket_{A'}^{\#} \alpha'.$$

Case ($r_1; r_2$)

$$\begin{aligned}
\gamma \llbracket r_1; r_2 \rrbracket_A^{\#} \alpha &= \gamma \llbracket r_2 \rrbracket_A^{\#} \llbracket r_1 \rrbracket_A^{\#} \alpha && \text{[definition]} \\
&= \gamma \llbracket r_2 \rrbracket_A^{\#} \alpha \gamma \llbracket r_1 \rrbracket_A^{\#} \alpha && [\alpha \gamma = \text{id}_A] \\
&\leq \gamma' \llbracket r_2 \rrbracket_{A'}^{\#} \alpha' \gamma' \llbracket r_1 \rrbracket_{A'}^{\#} \alpha' && \text{[inductive hp]} \\
&= \gamma' \llbracket r_2 \rrbracket_{A'}^{\#} \llbracket r_1 \rrbracket_{A'}^{\#} \alpha' && [\alpha' \gamma' = \text{id}_{A'}] \\
&= \gamma' \llbracket r_1; r_2 \rrbracket_{A'}^{\#} \alpha' && \text{[definition]}
\end{aligned}$$

Case ($r_1 \oplus r_2$)

$$\begin{aligned}
\gamma \llbracket r_1 \oplus r_2 \rrbracket_A^{\#} \alpha &= \gamma(\llbracket r_2 \rrbracket_A^{\#} \alpha \sqcup \llbracket r_1 \rrbracket_A^{\#} \alpha) && \text{[definition]} \\
&= \gamma(\alpha \gamma \llbracket r_2 \rrbracket_A^{\#} \alpha \sqcup \alpha \gamma \llbracket r_1 \rrbracket_A^{\#} \alpha) && [\alpha \gamma = \text{id}_A] \\
&= \gamma \alpha(\gamma \llbracket r_2 \rrbracket_A^{\#} \alpha \sqcup \gamma \llbracket r_1 \rrbracket_A^{\#} \alpha) && [\alpha \text{ is additive}] \\
&\leq A(\gamma' \llbracket r_2 \rrbracket_{A'}^{\#} \alpha' \sqcup \gamma' \llbracket r_1 \rrbracket_{A'}^{\#} \alpha') && \text{[inductive hp]} \\
&\leq A'(\gamma' \llbracket r_2 \rrbracket_{A'}^{\#} \alpha' \sqcup \gamma' \llbracket r_1 \rrbracket_{A'}^{\#} \alpha') && [A \leq A'] \\
&= \gamma'(\alpha' \gamma' \llbracket r_2 \rrbracket_{A'}^{\#} \alpha' \sqcup \alpha' \gamma' \llbracket r_1 \rrbracket_{A'}^{\#} \alpha') && [\alpha' \text{ is additive}] \\
&= \gamma'(\llbracket r_2 \rrbracket_{A'}^{\#} \alpha' \sqcup \llbracket r_1 \rrbracket_{A'}^{\#} \alpha') && [\alpha' \gamma' = \text{id}_{A'}] \\
&= \gamma' \llbracket r_1 \oplus r_2 \rrbracket_{A'}^{\#} \alpha' && \text{[definition]}
\end{aligned}$$

Case (r^*)

First we prove by induction on $n \geq 0$ that $\gamma(\llbracket r \rrbracket_A^{\#})^n \alpha \leq \gamma'(\llbracket r \rrbracket_{A'}^{\#})^n \alpha'$ using the inductive

hypothesis that $\gamma \llbracket r \rrbracket_A^\# \alpha \leq \gamma' \llbracket r \rrbracket_{A'}^\# \alpha'$. For $n = 0$ the inequality to prove reduces to $\gamma \text{id}_A \alpha = A \leq A' = \gamma' \text{id}_{A'} \alpha'$. So suppose it is true for n :

$$\begin{aligned}
 \gamma (\llbracket r \rrbracket_A^\#)^{n+1} \alpha &= \gamma (\llbracket r \rrbracket_A^\#)^n \llbracket r \rrbracket_A^\# \alpha && [\text{definition}] \\
 &= \gamma (\llbracket r \rrbracket_A^\#)^n \alpha \gamma \llbracket r \rrbracket_A^\# \alpha && [\gamma \alpha = \text{id}_A] \\
 &\leq \gamma' (\llbracket r \rrbracket_{A'}^\#)^n \alpha' \gamma' \llbracket r \rrbracket_{A'}^\# \alpha' && [\text{inductive hps}] \\
 &= \gamma' (\llbracket r \rrbracket_{A'}^\#)^{n+1} \alpha' && [\gamma' \alpha' = \text{id}_{A'}]
 \end{aligned}$$

Now we proceed with the proof of the structural inductive statement:

$$\begin{aligned}
 \gamma \llbracket r^* \rrbracket_A^\# \alpha &= \gamma \left(\bigsqcup_{n \geq 0} (\llbracket r \rrbracket_A^\#)^n \alpha \right) && [\text{definition}] \\
 &= \gamma \left(\bigsqcup_{n \geq 0} \alpha \gamma (\llbracket r \rrbracket_A^\#)^n \alpha \right) && [\alpha \gamma = \text{id}_A] \\
 &= \gamma \alpha \left(\bigsqcup_{n \geq 0} \gamma (\llbracket r \rrbracket_A^\#)^n \alpha \right) && [\alpha \text{ is additive}] \\
 &\leq A \left(\bigsqcup_{n \geq 0} \gamma' (\llbracket r \rrbracket_{A'}^\#)^n \alpha' \right) && [\text{statement above}] \\
 &\leq A' \left(\bigsqcup_{n \geq 0} \gamma' (\llbracket r \rrbracket_{A'}^\#)^n \alpha' \right) && [A \leq A'] \\
 &= \gamma' \left(\bigsqcup_{n \geq 0} \alpha' \gamma' (\llbracket r \rrbracket_{A'}^\#)^n \alpha' \right) && [\alpha' \text{ is additive}] \\
 &= \gamma' \left(\bigsqcup_{n \geq 0} (\llbracket r \rrbracket_{A'}^\#)^n \alpha' \right) && [\alpha' \gamma' = \text{id}_{A'}] \\
 &= \gamma' \llbracket r^* \rrbracket_{A'}^\# \alpha' && [\text{definition}]
 \end{aligned}$$

□

Proof of Theorem 6.12, intensional soundness of rule (simplify). Since the proof of Theorem 3.7 in [BGGR21] is by induction, we extend it by just proving the new inductive case.

Case (simplify)

(1) It's the same as point (1) of intensional soundness applied to $\vdash_{A'} [P] \text{ r } [Q]$, since this conclusion doesn't depend on the abstract domain.

(2-3)

$$\begin{aligned}
A'(Q) &= A(Q) && \text{[hypothesis of the rule]} \\
&\leq A(\llbracket r \rrbracket P) && \text{[(1) and monotonicity of } A] \\
&= \gamma\alpha(\llbracket r \rrbracket P) && \text{[definition]} \\
&\leq \gamma\llbracket r \rrbracket_A^\# \alpha(P) && \text{[soundness of } \llbracket r \rrbracket_A^\#] \\
&\leq \gamma'\llbracket r \rrbracket_{A'}^\# \alpha'(P) && \text{[Lemma C.4]} \\
&= \gamma'\alpha'(Q) && \text{[Soundness of } \vdash_{A'} [P] \text{ r } [Q]] \\
&= A'(Q) && \text{[definition]}
\end{aligned}$$

Hence all the lines are equal; in particular $\gamma\alpha(Q) = \gamma\alpha(\llbracket r \rrbracket P)$ and $\gamma\llbracket r \rrbracket_A^\# \alpha(P) = \gamma\alpha(Q)$. Since γ is injective, we get points (2-3) of intensional soundness. $\square$

Proof of Theorem 6.14, intrinsic incompleteness of LCL_A with rule (simplify). The proof follows closely that of intrinsic incompleteness of LCL_A [BGGR21, Theorem 5.12]. The Turing completeness hypothesis allows to define, for any store $c \in \Sigma$, three regular commands $r_{c?}$, $r_{\neg c?}$ and r_c such that, given any input $S \in C = \mathcal{P}(\Sigma)$

$$\begin{aligned}
\llbracket r_{c?} \rrbracket S &= S \cap \{c\} \\
\llbracket r_{\neg c?} \rrbracket S &= S \setminus \{c\} \\
\llbracket r_c \rrbracket S &= \{c\}
\end{aligned}$$

Moreover, by Turing completeness we are able to define a regular command r_w such that, for all $S \in C$ we have $\llbracket r_w \rrbracket S = \emptyset$. As an example, for a single variable x these four commands are

$$\begin{aligned}
r_w &\triangleq \text{while (true) \{ skip \}} \\
r_{c?} &\triangleq \text{if (x == c) then skip else } r_w \\
r_{\neg c?} &\triangleq \text{if (x == c) then } r_w \text{ else skip} \\
r_c &\triangleq x := c
\end{aligned}$$

Since A is not trivial, there exists two set of stores P and R such that $P \subsetneq A(P)$ and $A(R) \subsetneq C$. These imply that there exists two stores $a \in A(P) \setminus P$ and $b \in C \setminus A(R)$. Since by monotonicity of A we get $A(\emptyset) \subseteq A(R)$ and we know $b \notin A(R)$, we conclude that $A(\{b\}) \neq A(\emptyset)$. Moreover, by monotonicity of A we also have $A(\emptyset) \subseteq A(\{b\})$, so that $A(\emptyset) \subsetneq A(\{b\})$. Given such a and b , let us consider the command r defined as

$$r = (r_{a?}; r_b) \oplus (r_{\neg a?}; r_w)$$

for $r_{a?}$, r_b , $r_{\neg a?}$ and r_w as defined above. We now show that we cannot prove the triple $\vdash_A [P] r_1; r_w [\emptyset]$ in LCL_A extended with (simplify) even though it is intensionally sound.

First, we verify soundness: $\llbracket r; r_w \rrbracket P = \llbracket r_w \rrbracket (\llbracket r \rrbracket P) = \emptyset$, so that $\emptyset \subseteq \llbracket r; r_w \rrbracket P$, and

$$\alpha(\llbracket r; r_w \rrbracket P) = \alpha(\emptyset) = \llbracket r_w \rrbracket_A^\# (\llbracket r \rrbracket_A^\# \alpha(P)) = \llbracket r; r_w \rrbracket_A^\# \alpha(P)$$

so the triple is intensionally sound.

However, we can't derive the triple because r is not locally complete on P in A , nor in any $A' \succeq A$ such that $A(\emptyset) = A'(\emptyset)$, that are the only domain we can simplify to using rule (simplify). The local incompleteness is a consequence of

$$\llbracket r \rrbracket A(P) = \llbracket r_{a?}; r_b \rrbracket A(P) \cup \llbracket r_{\neg a?}; r_w \rrbracket A(P) = \llbracket r_b \rrbracket \llbracket r_{a?} \rrbracket A(P) \cup \emptyset = \llbracket r_b \rrbracket \{a\} = \{b\}$$

where we used $a \in A(P)$. Using this we easily get $\neg\mathbb{C}_P^A(\llbracket r \rrbracket)$:

$$A(\llbracket r \rrbracket P) = A(\emptyset) \subsetneq A(\{b\}) = A(\llbracket r \rrbracket A(P))$$

Formally, to derive the triple $\vdash_A [P] \text{ r } r_w [\emptyset]$, we can apply only three rules: (**relax**), (**seq**) and (**simplify**). By the arbitrariness of P we can assume without loss of generality that there is no $P' \subseteq P \subseteq A(P')$, and clearly any $Q' \supseteq \emptyset$ is no longer sound because $\emptyset = \llbracket r; r_w \rrbracket P$. So we can't apply (**relax**). Let us distinguish the other two cases.

If we apply (**seq**), we get

$$\frac{\vdash_A [P] \text{ r } [Q] \quad \vdash_A [Q] \text{ r}_w [\emptyset]}{\vdash_A [P] \text{ r } r_w [\emptyset]} \text{ (seq)}$$

for some intermediate Q . However, any provable triple $\vdash_A [P] \text{ r } [Q]$ would imply by soundness (of LCL_A extended with (**simplify**), Theorem 6.12) local completeness $\mathbb{C}_P^A(\llbracket r \rrbracket)$, that is false. So we can't prove the triple starting with an application of (**seq**)

If we apply (**simplify**), we get

$$\frac{\vdash_A [P] \text{ r } r_w [\emptyset] \quad A' \succeq A \quad A'(\emptyset) = A(\emptyset)}{\vdash_A [P] \text{ r } r_w [\emptyset]} \text{ (simplify)}$$

for some simplification A' of A . Note that we have $A'(\emptyset) \subsetneq A'(\{b\})$:

$$A'(\emptyset) = A(\emptyset) \subsetneq A(\{b\}) \subseteq A'(\{b\})$$

Now we could apply (**relax**) to change P with a P' satisfying $P' \subseteq P \subseteq A'(P')$. However, we have $a \notin P$, hence $a \notin P' \subseteq P$, and $a \in A(P) \subseteq A'(P) = A'(P')$. Now the only rule we can apply is (**seq**), and as before we need a triple $\vdash_{A'} [P'] \text{ r } [Q]$ for some Q , that would imply $\mathbb{C}_{P'}^{A'}(\llbracket r \rrbracket)$. But this is not the case, and can be shown with computations as before using that $a \in A'(P') \setminus P'$ to obtain $\llbracket r \rrbracket P' = \emptyset$ and $\llbracket r \rrbracket A'(P') = \{b\}$, and the already shown $A'(\emptyset) \subsetneq A'(\{b\})$. $\square$

Proof of Theorem 6.18. The proof follows closely that for LCL (Theorem 6.2).

As for LCL_A , points (1) and (3) implies point (2):

$$\begin{aligned} \alpha(P) &\leq \alpha(\llbracket \check{r} \rrbracket Q) && [(1) \text{ and monotonicity of } \alpha] \\ &\leq \llbracket \check{r} \rrbracket_A^\# \alpha(Q) && [\text{soundness of } \llbracket \check{r} \rrbracket_A^\#] \\ &= \alpha(P) && [(3)] \end{aligned}$$

Therefore, we only have to prove (1) and (3).

The proof is by induction on the derivation tree of the provable triple $\vdash_A \langle P \rangle \text{ r } \langle Q \rangle$.

Case (transfer)

For (1), we have trivially $\llbracket \check{c} \rrbracket Q \leq \llbracket \check{c} \rrbracket Q$. For (3), we have $\alpha(\llbracket \check{c} \rrbracket Q) = \alpha(\llbracket \check{c} \rrbracket \gamma \alpha Q) = \llbracket \check{c} \rrbracket_A^\# \alpha(Q)$ by $\mathbb{C}_Q^A(\llbracket \check{c} \rrbracket)$ and definition of $\llbracket \check{c} \rrbracket_A^\#$, respectively (see Figure 6.4).

Case (relax)

For (1) we have by inductive hypothesis $P' \leq \llbracket \check{r} \rrbracket Q'$, and together with the other hypotheses of the rules we get $P \leq P' \leq \llbracket \check{r} \rrbracket Q' \leq \llbracket \check{r} \rrbracket Q$.

For (3), we recall that $P \leq P' \leq A(P)$ implies $\alpha(P) = \alpha(P')$: one inequality holds by monotonicity of α , the other by the adjunctive property of a GC since $A = \gamma \alpha$. Then

$$\begin{aligned} \alpha(P) &= \alpha(P') && [\text{hp of the rule } P \leq P' \leq A(P)] \\ &= \llbracket \check{r} \rrbracket_A^\# \alpha(Q') && [\text{inductive hp (2) on } \vdash_A \langle P' \rangle \text{ r } \langle Q' \rangle] \\ &= \llbracket \check{r} \rrbracket_A^\# \alpha(Q) && [\text{hp of the rule } Q' \leq Q \leq A(Q')] \end{aligned}$$

Case (seq)

For (1) $P \leq \llbracket \overleftarrow{r_1} \rrbracket R \leq \llbracket \overleftarrow{r_1} \rrbracket (\llbracket \overleftarrow{r_2} \rrbracket Q) = \llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket Q$, where the inequalities follow from inductive hypotheses and monotonicity of $\llbracket \overleftarrow{r_1} \rrbracket$.

For (3),

$$\begin{aligned}
\alpha(P) &\leq \alpha(\llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket Q) && [(1) \text{ and monotonicity of } \alpha] \\
&\leq \llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket_A^\# \alpha(Q) && [\text{soundness of } \llbracket \overleftarrow{r} \rrbracket^A] \\
&= \llbracket \overleftarrow{r_1} \rrbracket_A^\# \llbracket \overleftarrow{r_2} \rrbracket_A^\# \alpha(Q) && [\text{definition}] \\
&= \llbracket r_1 \rrbracket_A^\# \alpha(R) && [\text{inductive hp}] \\
&= \alpha(P) && [\text{inductive hp}]
\end{aligned}$$

So all the lines are equal, in particular $\llbracket \overleftarrow{r_1}; \overleftarrow{r_2} \rrbracket_A^\# \alpha(Q) = \alpha(P)$.

Case (join)

For (1), by inductive hypotheses, $P_1 \leq \llbracket r_1 \rrbracket Q$ and $P_2 \leq \llbracket r_2 \rrbracket Q$. Hence, $P_1 \vee P_2 \leq \llbracket r_1 \rrbracket Q \vee \llbracket r_2 \rrbracket Q = \llbracket r_1 \oplus r_2 \rrbracket Q$.

For (3), recalling that $\alpha(P_1) = \llbracket \overleftarrow{r_1} \rrbracket_A^\# \alpha(Q)$ and $\alpha(P_2) = \llbracket \overleftarrow{r_2} \rrbracket_A^\# \alpha(Q)$ by inductive hypotheses, we get

$$\begin{aligned}
\alpha(P_1 \vee P_2) &= \alpha(P_1) \vee \alpha(P_2) && [\alpha \text{ is additive}] \\
&= \llbracket r_1 \rrbracket_A^\# \alpha(Q) \vee \llbracket r_2 \rrbracket_A^\# \alpha(Q) && [\text{inductive hypotheses}] \\
&= \llbracket r_1 \oplus r_2 \rrbracket_A^\# \alpha(Q) && [\text{definition}]
\end{aligned}$$

Case (rec)

We first observe that

$$\begin{aligned}
\llbracket \overleftarrow{r^*} \rrbracket \llbracket \overleftarrow{r} \rrbracket Q &\leq Q \vee \llbracket \overleftarrow{r^*} \rrbracket \llbracket \overleftarrow{r} \rrbracket Q \\
&= \llbracket \overleftarrow{r} \rrbracket^0 Q \vee \bigvee_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n \llbracket \overleftarrow{r} \rrbracket Q && [\text{Lemma 5.1}] \\
&= \bigvee_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n Q \\
&= \llbracket \overleftarrow{r^*} \rrbracket Q && [\text{Lemma 5.1}]
\end{aligned}$$

We can then prove (1) by

$$\begin{aligned}
P &\leq \llbracket \overleftarrow{r^*} \rrbracket (R \vee Q) && [\text{inductive hp (1) on } \vdash_A \langle P \rangle \text{ } r^* \langle R \vee Q \rangle] \\
&\leq \llbracket \overleftarrow{r^*} \rrbracket (\llbracket \overleftarrow{r} \rrbracket Q \vee Q) && [\text{inductive hp (1) on } \vdash_Q \langle R \rangle \text{ } r \langle Q \rangle] \\
&= \llbracket \overleftarrow{r^*} \rrbracket \llbracket \overleftarrow{r} \rrbracket Q \vee \llbracket \overleftarrow{r^*} \rrbracket Q && [\text{additivity of } \llbracket \overleftarrow{\cdot} \rrbracket] \\
&\leq \llbracket \overleftarrow{r^*} \rrbracket Q \vee \llbracket \overleftarrow{r^*} \rrbracket Q && [\text{observation above}] \\
&= \llbracket \overleftarrow{r^*} \rrbracket Q
\end{aligned}$$

For (3)

$$\begin{aligned}
\llbracket \overleftarrow{r^*} \rrbracket_A^\# \alpha(Q) &\leq_A \llbracket \overleftarrow{r^*} \rrbracket_A^\# (Q \vee R) \\
&= \alpha(P) && [\text{inductive hp (2) on } \vdash_A \langle P \rangle \text{ } r^* \langle R \vee Q \rangle] \\
&\leq_A \alpha(\llbracket \overleftarrow{r^*} \rrbracket Q) && [\text{condition (1) shown above}] \\
&\leq_A \llbracket \overleftarrow{r^*} \rrbracket_A^\# \alpha(Q) && [\text{soundness of } \llbracket \overleftarrow{\cdot} \rrbracket_A^\#]
\end{aligned}$$

Case (iterate)

For (1)

$$\begin{aligned}
P \vee Q &\leq \llbracket \overleftarrow{r} \rrbracket Q \vee Q && [\text{inductive hp (1) on } \vdash_A \langle\langle P \rangle\rangle \text{ r } \langle\langle Q \rangle\rangle] \\
&= \llbracket \overleftarrow{r} \rrbracket^1 Q \vee \llbracket \overleftarrow{r} \rrbracket^0 Q \\
&\leq \bigvee_{n \geq 0} \llbracket \overleftarrow{r} \rrbracket^n Q \\
&= \llbracket \overleftarrow{r}^\star \rrbracket Q && [\text{Lemma 5.1}]
\end{aligned}$$

For (3), we first prove by induction that $(\llbracket \overleftarrow{r} \rrbracket_A^\sharp)^n \alpha(Q) \leq_A \alpha(Q)$. Recall that, by the adjunctive property of a Galois connection, $P \leq A(Q)$ iff $\alpha(P) \leq_A \alpha(Q)$. The base case $n = 0$ is trivial because $(\llbracket \overleftarrow{r} \rrbracket_A^\sharp)^0$ is the identity. Suppose it holds for some n : then

$$\begin{aligned}
(\llbracket \overleftarrow{r} \rrbracket_A^\sharp)^{n+1} \alpha(Q) &= \llbracket \overleftarrow{r} \rrbracket_A^\sharp (\llbracket \overleftarrow{r} \rrbracket_A^\sharp)^n \alpha(Q) \\
&\leq_A \llbracket \overleftarrow{r} \rrbracket_A^\sharp \alpha(Q) && [\text{inductive hp}] \\
&= \alpha(P) && [\text{inductive hp (2) on } \vdash_A \langle\langle P \rangle\rangle \text{ r } \langle\langle Q \rangle\rangle] \\
&\leq_A \alpha(Q) && [\text{hp of the rule } P \leq A(Q)]
\end{aligned}$$

From this, we observe that

$$\alpha(Q) = (\llbracket \overleftarrow{r}^\star \rrbracket_A^\sharp)^0 \alpha(Q) \leq_A \bigvee_{n \geq 0} (\llbracket \overleftarrow{r}^\star \rrbracket_A^\sharp)^n \alpha(Q) \leq_A \bigvee_{n \geq 0} \alpha(Q) = \alpha(Q)$$

and therefore $\llbracket \overleftarrow{r}^\star \rrbracket_A^\sharp \alpha(Q) = \alpha(Q)$ by Lemma 5.1.

We then conclude the proof of (2) with

$$\begin{aligned}
\llbracket \overleftarrow{r}^\star \rrbracket_A^\sharp \alpha(Q) &= \alpha(Q) \\
&= \alpha(P) \vee_A \alpha(Q) && [\text{hp of the rule } P \leq A(Q)] \\
&= \alpha(P \vee Q) && [\text{additivity of } \alpha]
\end{aligned}$$

□

Proof sketch of Proposition 6.20. As an example, we sketch the proof for (refine-ext); other rules are analogous. First, analogously to the proof of Theorem 6.18, we prove by induction extensional soundness of CLCL, that is the same as Theorem 6.18 but replacing point (3) with $\alpha(P) = \llbracket \overleftarrow{r} \rrbracket^A \alpha(Q)$. Then, we extend the inductive proof with a new case for (refine-ext). Analogously to Theorem 6.18, we only have to prove points (1) and (3) because together they imply point (2). We report here only two sample cases.

Case (seq)

(1) is the same as Theorem 6.18.

For (3), we recall that $\llbracket \overleftarrow{r}_1; \overleftarrow{r}_2 \rrbracket^A \leq \llbracket \overleftarrow{r}_1 \rrbracket^A \llbracket \overleftarrow{r}_2 \rrbracket^A$.

$$\begin{aligned}
\alpha(P) &\leq \alpha(\llbracket \overleftarrow{r}_1; \overleftarrow{r}_2 \rrbracket Q) && [(1) \text{ and monotonicity of } \alpha] \\
&\leq \llbracket \overleftarrow{r}_1; \overleftarrow{r}_2 \rrbracket^A \alpha(Q) && [\text{soundness of } \llbracket \overleftarrow{r} \rrbracket^A] \\
&\leq \llbracket \overleftarrow{r}_1 \rrbracket^A \llbracket \overleftarrow{r}_2 \rrbracket^A \alpha(Q) && [\text{recalled above}] \\
&= \llbracket \overleftarrow{r}_1 \rrbracket^A \alpha(R) && [\text{inductive hp}] \\
&= \alpha(P) && [\text{inductive hp}]
\end{aligned}$$

[illegible]

Figure C.3: Derivation of the CLCL triple $\vdash_{\text{Oct}} \langle\langle n > 0 \wedge x + n \leq 2000000 \rangle \vee R_{2M} \rangle \mathbf{r}_w^* \langle R_{2M} \rangle$. For brevity, we omit rule name (transfer), we let $\bar{k} \triangleq 2000000$, we recall that $R_{2M} \triangleq (x + n = \bar{k} \wedge n > 0)$, $T_{2M} \triangleq (x + n = \bar{k} \wedge n > 0)$ and we define $S_{2M} \triangleq (x + n \leq \bar{k} \wedge n > 0)$.

So all the lines are equal, in particular $\llbracket \overleftarrow{r_1; r_2} \rrbracket^A \alpha(Q) = \alpha(P)$.

Case (refine-ext)

(1) follows from soundness of $\vdash_{A'} \langle\langle P \rangle\rangle \text{ r } \langle\langle Q \rangle\rangle$.

For (3),

$$\begin{aligned}
 \alpha(P) &\leq \alpha(\llbracket \overleftarrow{r} \rrbracket Q) && [(1) \text{ and monotonicity of } \alpha] \\
 &\leq \llbracket \overleftarrow{r} \rrbracket^A \alpha(Q) && [\text{soundness of } \llbracket \overleftarrow{r} \rrbracket^A] \\
 &= \alpha(\llbracket \overleftarrow{r} \rrbracket \gamma \alpha(Q)) && [\text{definition}] \\
 &= \alpha \gamma' \alpha' \llbracket \overleftarrow{r} \rrbracket \gamma' \alpha' \gamma \alpha(Q) && [\text{Lemma C.1}] \\
 &= \alpha \gamma' \llbracket \overleftarrow{r} \rrbracket^{A'} \alpha' A(Q) && [\text{definition}] \\
 &= \alpha(P) && [\text{hypothesis of the rule}]
 \end{aligned}$$

Hence all the lines are equal; in particular $\llbracket \overleftarrow{r} \rrbracket^A \alpha(Q) = \alpha(P)$. □

Appendix D

AdjointPDR supplementary materials

This appendix contains technical details of proofs and examples for Chapter 7.

Proof of Proposition 7.4. We use the same notation, and follow the same proof schema of showing

- (a) $s_0 \models (Q)$ and
- (b) if $s \models (Q)$ and $s \rightarrow s'$, then $s' \models (Q)$.

Case (I2): $\forall j \in [0, n-2], x_j \leq x_{j+1}$

- (a) In s_0 , since $n = 2$ one needs to check only the case $j = 0$: $x_0 = \perp \leq \top = x_1$.
- (b) If $s \xrightarrow{U} s'$, then $x'_j = x_j \stackrel{(I2)}{\leq} x_{j+1} = x'_{j+1}$ for all $j \in [0, n-2]$. For $j = n-1$, $x'_{n-1} = x_{n-1} \leq \top = x'_n$. Since $n' = n+1$, then $\forall j \in [0, n'-2], x'_j \leq x'_{j+1}$.
If $s \xrightarrow{Co}_z s'$, then $x'_j = x_j \wedge z \stackrel{(I2)}{\leq} x_{j+1} \wedge z \leq x'_{j+1}$ for all $j \in [0, k]$. For all $j \in [i+1, n-2]$, $x'_j = x_j \stackrel{(I2)}{\leq} x_{j+1} = x'_{j+1}$. Thus, $\forall j \in [0, n'-2], x'_j \leq x'_{j+1}$.

Case (P1): $i \leq x_1$

- (a) In s_0 , $x_1 = \top \geq i$.
- (b) If $s \xrightarrow{U} s'$, then $x'_1 = x_1 \stackrel{(P1)}{\geq} i$.
If $s \xrightarrow{Co}_z s'$, since $z \geq i$, then $x'_1 = x_1 \wedge z \stackrel{(P1)}{\geq} i \wedge i = i$.

Case (P2): $x_{n-2} \leq p$

- (a) In s_0 , $n = 2$ and $x_0 = \perp \leq p$.
- (b) If $s \xrightarrow{U} s'$, since (Unfold) is applied only if $x_{n-1} \leq p$ and $n' = n+1$, then $x'_{n'-2} = x_{n-1} \leq p$.
If $s \xrightarrow{Co} s'$, since $n' = n$, then $x'_{n'-2} = x'_{n-2} \leq x_{n-2} \stackrel{(P2)}{\leq} p$.

Case (P3a): $\forall j \in [0, n-2], x_j \leq g(x_{j+1})$

Follows immediately from (P3) and $f \dashv g$.

Case (N1): If $\vec{y} \neq \varepsilon$ then $p \leq y_{n-1}$

The case of (Conflict) is trivial as in the proof of invariant (N2): if (N1) holds for $\vec{y}$ then it obviously holds for its tail $\text{tail}(\vec{y})$ as well.

(a) In s_0 , $\vec{y} = \varepsilon$. Thus (N1) trivially holds.

(b) If $s \xrightarrow{Ca}_z s'$, since $p \leq z$, then $p \leq z = y'_{n-1}$.

If $s \xrightarrow{D} s'$, then $p \stackrel{(N1)}{\leq} y_{n-1} = y'_{n-1}$.

Case (A1): $\forall j \in [0, n-1], (f \vee i)^j(\perp) \leq x_j \leq (g \wedge p)^{n-1-j}(\top)$

- We prove $(f \vee i)^j \perp \leq x_j$ by induction on $j \in [0, n-1]$. For $j = 0$, $(f \vee i)^j \perp = \perp \leq x_j$. For $j \in [1, n-1]$,

$$\begin{aligned}
 (f \vee i)^j \perp &= (f \vee i)((f \vee i)^{j-1} \perp) && [\text{def.}] \\
 &= f((f \vee i)^{j-1} \perp) \vee i && [\text{def.}] \\
 &\leq f(x_{j-1}) \vee i && [\text{induction hypothesis}] \\
 &\leq x_j \vee i && [(P3)] \\
 &= x_j && [(P1) \text{ and } (I2)]
 \end{aligned}$$

- In order to prove $x_j \leq (g \wedge p)^{n-1-j} \top$ for all $j \in [0, n-1]$, it is convenient to prove the equivalent statement $x_{n-1-j} \leq (g \wedge p)^j \top$ for all $j \in [0, n-1]$. For $j = 0$, $x_{n-1} \leq \top = (g \wedge p)^0 \top$. For $j \in [1, n-1]$,

$$\begin{aligned}
 (g \wedge p)^j \top &= (g \wedge p)((g \wedge p)^{j-1} \top) && [\text{def.}] \\
 &= g((g \wedge p)^{j-1} \top) \wedge p && [\text{def.}] \\
 &\geq g(x_{n-1-j+1}) \wedge p && [\text{induction hypothesis}] \\
 &\geq x_{n-1-j} \wedge p && [(P3a)] \\
 &= x_{n-1-j} && [(P2) \text{ and } (I2)]
 \end{aligned}$$

Case (A2): $\forall j \in [1, n-1], x_{j-1} \leq g^{n-1-j}(p)$

For all $l \in \mathbb{N}$, $(g \wedge p)^{l+1} \top = \bigwedge_{j \leq l} g^j(p)$. Thus, using (A1), it holds that for all $j \in [1, n-1]$, $x_{j-1} \leq (g \wedge p)^{n-j} \top = \bigwedge_{j \leq n-j-1} g^j(p) \leq g^{n-j-1}(p)$.

Case (A3): $\forall j \in [k, n-1], g^{n-1-j}(p) \leq y_j$

In order to prove (A3), we prove the equivalent statement $g^j(p) \leq y_{n-1-j}$ for all $j \in [0, n-1-k]$. For $j = 0$, $g^0(p) = p \leq y_{n-1}$. The last inequality holds by (N1). For $j \in [1, n-1-k]$,

$$\begin{aligned}
 g^j(p) &= g(g^{j-1}(p)) && [\text{def.}] \\
 &\geq g(y_{n-1-(j-1)}) && [\text{induction hypothesis}] \\
 &= g(y_{n-j}) && [\text{def.}] \\
 &\geq y_{n-1-j} && [(N2)]
 \end{aligned}$$

□

Proof of Proposition 7.12. We prove this property by showing the exact sequence of steps the algorithm performs with this heuristics. To do so, assume m is the smallest integer such that $f^m(i) \not\leq p$. Note that, if $\text{lfp}(f \vee i) \leq p$, there is no such m ; if that is the case we say $m = +\infty$. Also note that, for all $n < m$ it holds $f^n(i) \leq p$, hence also $f(f \vee i)^n \perp = f \bigvee_{j < n} f^j(i) = \bigvee_{j < n} f^{j+1}(i) \leq p$, and for $n \leq m$ we have $(f \vee i)^n \perp = \bigvee_{j < n} f^j(i) \leq p$.

Intuitively, while $n < m + 2$ the algorithm performs a cycle of (Unfold), then (Candidate), then (Conflict) and then (Unfold) again. When it reaches $n = m + 2$, it enters a sequence of (Decide) that eventually lead to return false. Of course, if $m = +\infty$, this sequence of (Decide) never happens.

To prove this formally, we prove by induction that after initialization and every (Unfold) applied in this sequence, for all $j \in [0, n - 2]$, $x_j = (f \vee i)^j \perp$. We do so by showing this holds for initialization, and that if we assume this to be true after initialization or (Unfold), (a) if $n < m + 2$, the algorithm does exactly (Candidate), then (Conflict), then (Unfold) and the invariant holds again, and (b) if $n = m + 2$, the algorithm does (Candidate) then (Decide) until $k = 1$, then returns false. In doing so, we also show that the invariant holds after every rule, that is exactly the thesis.

For initialization, as $k = n = 2$ and $x_0 = \perp$ the property holds.

For (a), suppose the algorithm just did (Unfold) or initialization. Then, by the invariant, for all $j \in [0, n - 2]$, $x_j = (f \vee i)^j \perp$, and both after initialization and (Unfold), $x_{n-1} = \top$. Applying the algorithm, the sequence of states is then

$$\begin{aligned} & (\perp, (f \vee i) \perp, \dots, (f \vee i)^{n-2} \perp, \top \parallel \varepsilon)_{n,n} \\ & \xrightarrow{C_a} (\perp, (f \vee i) \perp, \dots, (f \vee i)^{n-2} \perp, \top \parallel p)_{n,n-1} \\ & \xrightarrow{C_o} (\perp, (f \vee i) \perp, \dots, (f \vee i)^{n-2} \perp, (f \vee i)^{n-1} \perp \parallel \varepsilon)_{n,n} \\ & \xrightarrow{U} (\perp, (f \vee i) \perp, \dots, (f \vee i)^{n-2} \perp, (f \vee i)^{n-1} \perp, \top \parallel \varepsilon)_{n+1,n+1} \end{aligned}$$

where the choice for (Candidate) is p and the choice for (Conflict) is $(f \vee i)x_{n-2} = (f \vee i)^{n-1} \perp$. The condition to apply (Candidate) is $x_{n-1} = \top \not\leq p$, which follows from $p \neq \top$. The condition to apply (Conflict) is $f(x_{n-2}) = f(f \vee i)^{n-2} \perp \leq p$, which follows from $n - 2 < m$. The condition to apply (Unfold) is $x_{n-1} = (f \vee i)^{n-1} \perp \not\leq p$, which follows from $n - 1 \leq m$. The invariant clearly holds for all three states traversed.

For (b), suppose again the algorithm just did (Unfold) or initialization, so $\vec{x} = \langle \perp, (f \vee i) \perp, \dots, (f \vee i)^{n-2} \perp, \top \rangle$. Recalling that $n = m + 2$, the sequence of states is

$$\begin{aligned} & (\perp, (f \vee i) \perp, \dots, (f \vee i)^m \perp, \top \parallel \varepsilon)_{m+2,m+2} \\ & \xrightarrow{C_a} (\perp, (f \vee i) \perp, \dots, (f \vee i)^m \perp, \top \parallel p)_{m+2,m+1} \\ & \xrightarrow{D} (\perp, (f \vee i) \perp, \dots, (f \vee i)^m \perp, \top \parallel g(p), p)_{m+2,m} \\ & \xrightarrow{D} \dots \\ & \xrightarrow{D} (\perp, (f \vee i) \perp, \dots, (f \vee i)^m \perp, \top \parallel g^m(p), \dots, g(p), p)_{m+2,1} \end{aligned}$$

where the choice for (Candidate) is p and the choice for (Decide) is $g(y_k)$. The condition to apply (Candidate) is again $x_{n-1} = \top \not\leq p$. The condition to apply (Decide) for k is $f(x_{k-1}) \not\leq y_k$, that is $f(f \vee i)^{k-1} \perp \not\leq g^{m+1-k}(p)$. This holds because $f(f \vee i)^{k-1} \perp \leq f^{k-1}(i)$ and, by $f \dashv g$, $f^{k-1}(i) \not\leq g^{m+1-k}(p)$ if and only if $f^{m+1-k} f^{k-1}(i) = f^m(i) \not\leq p$. Lastly, when $k = 1$, we have $i \not\leq g^m(p)$ again by $f \dashv g$, so the algorithm returns false. The invariant clearly holds for all the m states traversed. $\square$

Proof of Proposition 7.13. To prove this property, first we prove by induction that the following invariants hold:

- (a) either $x_1 = (g \wedge p)^{n-1}\top$ or $x_1 = (g \wedge p)^{n-2}\top$,
- (b) if $x_1 = (g \wedge p)^{n-1}\top$, for all $j \in [1, k-1]$, $x_j = (g \wedge p)^{n-j}\top$ and for all $j \in [k, n-1]$, $x_j = (g \wedge p)^{n-1-j}\top$,
- (c) if $x_1 = (g \wedge p)^{n-2}\top$, for all $j \in [1, n-1]$, $x_j = (g \wedge p)^{n-1-j}\top$,
- (d) if $k = 1$ then $x_1 = (g \wedge p)^{n-2}\top$.

Note that by (a) exactly one of the consequences of (b) and (c) hold, and when $k = 1$ (d) prescribes it must be (c). In the rest of the proof, we say that (b) or (c) hold meaning that x_1 is $(g \wedge p)^{n-1}\top$ or $(g \wedge p)^{n-2}\top$ respectively, so that the respective consequence holds, too.

At initialization, $\vec{x} = \perp, \top$ and $n = 2$, so (a) $x_1 = (g \wedge p)^{n-2}\top$, and (c) holds.

After (Unfold), $\vec{x}' = \vec{x}, \top$ and $n' = n + 1$. Since we applied (Unfold), it must be the case that $\vec{y} = \varepsilon$, so $k = n$, and $x_{n-1} \leq p$. By (a) either $x_{n-1} = (g \wedge p)\top = p$, or $x_{n-1} = (g \wedge p)^0\top = \top$. But since $x_{n-1} \leq p$, it can't be the latter. Thus (b) holds before (Unfold). After the rule, (a) holds because $x'_1 = x_1 = (g \wedge p)^{n-1}\top = (g \wedge p)^{n'-2}\top$; (c) holds too because for all $j \in [1, n' - 1] = [1, n]$, $x'_j = x_j = (g \wedge p)^{n-j}\top = (g \wedge p)^{n'-1-j}\top$ (where we used that (b) holds before the rule) and for $j = n'$, $x'_j = \top = (g \wedge p)^0\top$. (d) holds because $k' = n' > 1$.

For (Candidate), since we applied it, it must be the case that $x_{n-1} \not\leq p$. If (b) held before the rule, it would mean that $x_{n-1} = (g \wedge p)\top = p \leq p$, so (c) holds. After the rule, (a) and (c) still hold because $\vec{x}' = \vec{x}$ and $n' = n$. (d) holds because (c) holds.

For (Decide), since we applied it, it must be the case that $f(x_{k-1}) \not\leq y_k = g^{n-1-k}(p)$. This, by $f \dashv g$, is equivalent to $x_{k-1} \not\leq g^{n-k}(p)$. If (b) held before the rule, it would mean that $x_{k-1} = (g \wedge p)^{n-k+1}\top \leq g^{n-k}(p)$, so (c) holds. After the rule, (a) and (c) still hold because $\vec{x}' = \vec{x}$ and $n' = n$. (d) holds because (c) holds.

For (Conflict), let us distinguish two cases.

If $k = 1$, (c) holds before the rule because of (d). The choice in the rule is $z = y_1 = g^{n-2}(p)$, so after (Conflict) $\vec{x}' = \vec{x} \wedge_1 g^{n-2}(p)$ and $k' = k + 1$. (a) holds because $x'_1 = x_1 \wedge g^{n-2}(p) = (g \wedge p)^{n-2}\top \wedge g^{n-2}(p) = (g \wedge p)^{n-1}\top$. (b) holds because for $j \in [1, k' - 1] = [1, 1]$, $x'_1 = (g \wedge p)^{n-1}\top$ and for $j \in [k', n - 1]$, $x'_j = x_j = (g \wedge p)^{n-1-j}\top$. (d) holds because $k' = 2 > 1$.

If $k > 1$, since we applied (Conflict), it must be the case that $f(x_{k-1}) \leq y_k = g^{n-1-k}(p)$. This, by $f \dashv g$, is equivalent to $x_{k-1} \leq g^{n-k}(p)$. If (c) held before the rule, it would mean that $x_{k-1} = (g \wedge p)^{n-k}\top \not\leq g^{n-k}(p)$, so (b) holds. The choice in the rule is $z = y_k = g^{n-1-k}(p)$. After (Conflict), (a) holds because $x'_1 = x_1 \wedge g^{n-1-k}(p) = (g \wedge p)^{n-1}\top \wedge g^{n-1-k}(p) = (g \wedge p)^{n-1}\top$. (b) holds because for $j \in [1, k' - 2] = [1, k - 1]$, $x'_j = x_j \wedge g^{n-1-k}(p) = (g \wedge p)^{n-j}\top \wedge g^{n-1-k}(p) = (g \wedge p)^{n-j}\top$; for $j = k' - 1 = k$, $x'_j = x_j \wedge g^{n-1-k}(p) = (g \wedge p)^{n-1-k}\top \wedge g^{n-1-k}(p) = (g \wedge p)^{n-k}\top$; for $j \in [k', n - 1]$, $x'_j = x_j = (g \wedge p)^{n-1-j}\top$. (d) holds because $k' = k + 1 > 1$.

This concludes the proof of the invariants. To prove the original statement, it is enough to observe that right after (Unfold) (c) holds. $\square$

Lemma D.1. *If $s_0 \rightarrow^* (\vec{x} \parallel \vec{y})_{n,k} \rightarrow^* (\vec{x}' \parallel \vec{y}')_{n',k'}$, then $n' \geq n$ and, for all $j \in [0, n - 1]$, $x_j \geq x'_j$.*

Proof. Follows from proof of Proposition 7.7. $\square$

Proof of Lemma 7.14. Since s and s' carry the same index (n, k) and the algorithm only increases n , then n is never increased in the steps between s and s' . Thus (Unfold) is never executed. On the other hand, k will be increased by (Conflict) and decreased in (Candidate) and (Decide).

We prove the proposition for (Decide), the case of (Candidate) is analogous. Let us fix $s = (\vec{x} \parallel \vec{y})_{n,k}$. The state immediately after s would be $(\vec{x} \parallel z, \vec{y})_{n,k-1}$. Observe that before arriving to the state $(\vec{x}' \parallel \vec{y}')_{n,k}$, the z inserted by the (Decide) right after s should be removed by some (Conflict), as that's the only rule that can remove elements from $\vec{y}$. The state before such (Conflict) will be of the form $(\vec{x}'' \parallel z, \vec{y})_{n,k+1}$ for some positive chain $\vec{x}''$. Now let z'' be the element chosen by such (Conflict). It holds that $z'' \leq y_{k+1} = z$. The state after the (Conflict) will be $(\vec{x}'' \wedge_k z'' \parallel \vec{y})_{n,k}$. In this state and, by Corollary D.1 in any of the following states, the $(k-1)$ -th element of the positive chain is below z . In particular, for $s' = (\vec{x}' \parallel \vec{y}')_{n,k}$, we have that $x'_{k-1} \leq z$. Since $(\vec{x}' \parallel \vec{y}')_{n,k} \xrightarrow{D} z'$, $x'_{k-1} \not\leq z'$. Thus $z' \neq z$. $\square$

Proof of Theorem 7.15. Let us fix an n . Since the possible choices of $z \in h(CaD_{n,k}^h)$ are finitely many, by Lemma 7.14 we can apply (Candidate) or (Decide) only a finite amount of times for every k . Since by invariant (I1), $1 \leq k \leq n$, we only have a finite amount of different values of k , so (Candidate) and (Decide) occur only finitely many times with the same n .

Since both (Candidate) and (Decide) decrease k , (Conflict) increase k and $1 \leq k \leq n$, then also (Conflict) occurs only finitely many times with the same n . Therefore in any infinite computation of **A-PDR_h** (Unfold), which is the only rule that increase n , should occur infinitely many times.

But when $\text{lfp}(f \vee i) \not\leq p$, by (2.3), there is some $j \in \mathbb{N}$ such that $(f \vee i)^j \perp \not\leq p$. Since (Unfold) can be applied only when $(f \vee i)^{n-1} \perp \leq x_{n-1} \leq p$, then it can applied only a finite amount of time. $\square$

Proof of Corollary 7.16. By Proposition 7.10, h maps any reachable state s such that $s \xrightarrow{D}$ into $g^{n-1-k}(p)$ and any reachable state s such that $s \xrightarrow{Ca}$ into p . Thus $h(CaD_{n,k}^h)$ has cardinality 2. By Theorem 7.15, if $\text{lfp}(f \vee i) \not\leq p$, then **AdjointPDR_h** terminates. $\square$

Proof of Corollary 7.26. Using Theorem 7.15, it is enough to prove that, for all indexes (n, k) , the set of all possible choices for (Candidate) and (Decide), denoted by $h(CaD_{n,k}^h)$, is finite. For simplicity, in this proof let $CaD_{n,k}^h = Ca_{n,k}^h \uplus D_{n,k}^h$, where $Ca_{n,k}^h \triangleq \{s \in CaD_{n,k}^h \mid s \xrightarrow{Ca}\}$ is the set of reachable (n, k) -indexed states that trigger (Candidate) and $D_{n,k}^h \triangleq \{s \in CaD_{n,k}^h \mid s \xrightarrow{D}\}$ is the set of reachable (n, k) -indexed states that trigger (Decide). We prove that both images $h(Ca_{n,k}^h)$ and $h(D_{n,k}^h)$ are finite so that also $h(CaD_{n,k}^h)$ is such.

- In (Candidate), any heuristics h defined as in (7.7) always choose $z = p^\perp$. Therefore the image $h(Ca_{n,k}^h)$ has cardinality 1 for all (n, k) .
- In (Decide), when $k = n - 1$, any heuristic h defined as in (7.7) may select any lower set $\{d \mid b_\alpha(d) \in p^\perp\}$ for some function $\alpha: S \rightarrow A$. Since, there are $|S|^{|A|}$ of such functions α , then there are at most $|S|^{|A|}$ of such lower set. Thus the set $h(D_{n,n-1}^h)$ has at most cardinality $|S|^{|A|}$.

When $k = n - 2$, the heuristic h may select any lower set $\{d \mid b_{\alpha_2}(b_{\alpha_1}(d)) \in p^\perp\}$ for any two functions $\alpha_1, \alpha_2: S \rightarrow A$. Thus, the set $h(D_{n,n-2}^h)$ has at most cardinality $(|S|^{|A|})^2$.

One can easily generalise these cases to arbitrary $j \in [1, n]$, and prove with a simple inductive argument that the cardinality of $h(D_{n,n-j}^h)$ is at most $(|S|^{|A|})^j$. Since both S and A are finite, then the set $h(D_{n,k}^h)$ is finite for all indices (n, k) . $\square$

Proof of Proposition 7.28. We need to prove that the choices of **hCoB** and **hCo01** for (Candidate), (Decide) and (Conflict) respect the constraints imposed by **AdjointPDR**[↓].

- For (Candidate), both **hCoB** and **hCo01** take $Z = p^\downarrow$. We need to prove that $x_{n-1} \notin Z$ and $p \in Z$.
 - For the former, recall that the guard of (Candidate) is $x_{n-1} \not\leq p$. Thus $x_{n-1} \notin p^\downarrow = Z$.
 - The second is trivial: $p \in p^\downarrow = Z$.
- For (Decide), both **hCoB** and **hCo01** take $Z = \{d \mid b_\alpha(d) \in Y_k\}$ for an α such that $b_\alpha(x_{k-1}) \notin Y_k$. We need to prove $x_{k-1} \notin Z$ and $b_r^\downarrow(Y_k) \subseteq Z$.
 - Since $b_\alpha(x_{k-1}) \notin Y_k$, then $x_{k-1} \notin \{d \mid b_\alpha(d) \in Y_k\} = Z$.
 - To see that $b_r^\downarrow(Y_k) \subseteq Z$, it is enough to observe that $b_r^\downarrow(Y_k) = \{d \mid b(d) \in Y_k\} = \bigcap_\alpha \{d \mid b_\alpha(d) \in Y_k\} \subseteq \{d \mid b_\alpha(d) \in Y_k\} = Z$.
- For (Conflict), we start with **hCoB** and show later **hCo01**.

Let us consider first the case $\mathcal{Z}_k = \emptyset$, then **hCoB** chooses $z_B = b(x_{k-1})$. The proof is the same as for Proposition 7.6.4.

Let us consider now the case $\mathcal{Z}_k \neq \emptyset$, and let $z_k = \bigwedge \mathcal{Z}_k$, so that

$$z_B \triangleq \begin{cases} z_k(s) & \text{if } r_s \neq 0 \\ b(x_{k-1})(s) & \text{if } r_s = 0 \end{cases}$$

We first prove $z_k \in Y_k$ and $b(x_{k-1} \wedge z_k) \leq z_k$.

- Since $\mathcal{Z}_k \neq \emptyset$, then there should be at least a $d \in \mathcal{Z}_k$. By definition, d is a (convex) generator of Y_k and thus $d \in Y_k$. Since $z_k = \bigwedge \mathcal{Z}_k \leq d$ and since Y_k is, by definition, a lower set, then $z_k \in Y_k$.
- By definition of $\mathcal{Z}_k$, $b(x_{k-1}) \leq d$, for all $d \in \mathcal{Z}_k$. Thus $b(x_{k-1}) \leq \bigwedge \mathcal{Z}_k = z_k$. Therefore $b(x_{k-1} \wedge z_k) \leq b(x_{k-1}) \leq z_k$.

Now let us show that $z_B \in Y_k$ and $b(x_{k-1} \wedge z_B) \leq z_B$.

- Since $Y_k = \{d \in [0, 1]^S \mid \sum_{s \in S} (r_s \cdot d(s)) \leq r\}$ and z_B differs from z_k only when $r_s = 0$, we have $\sum_{s \in S} (r_s \cdot z_B(s)) = \sum_{s \in S} (r_s \cdot z_k(s)) \leq r$, because we already proved that $z_k \in Y_k$.
- We know that $b(x_{k-1}) \leq z_k$. Then, for any $s \in S$ it follows that $b(x_{k-1})(s) \leq z_k(s) = z_B(s)$ if $r_s \neq 0$, and $b(x_{k-1})(s) = z_B(s)$ if $r_s = 0$. Therefore $b(x_{k-1}) \leq z_B$, from which we get $b(x_{k-1} \wedge z_B) \leq b(x_{k-1}) \leq z_B$.

Finally, we focus on **hCo01**: we need to prove $z_{01} \in Y_k$ and $b(x_{k-1} \wedge z_{01}) \leq z_{01}$.

- Since $Y_k = \{d \in [0, 1]^S \mid \sum_{s \in S} (r_s \cdot d(s)) \leq r\}$ and z_{01} differs from z_B only when $r_s = 0$, we have $\sum_{s \in S} (r_s \cdot z_{01}(s)) = \sum_{s \in S} (r_s \cdot z_B(s)) \leq r$, because we already proved that $z_B \in Y_k$.

AdjointPDR ($\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow$)

```

<INITIALISATION>
   $(\vec{X} \parallel \vec{Y})_{n,k} := (\emptyset, L \parallel \varepsilon)_{2,2}$ 
<ITERATION>
  case  $(\vec{X} \parallel \vec{Y})_{n,k}$  of
     $\vec{Y} = \varepsilon$  and  $X_{n-1} \subseteq p^\downarrow$  :                               %(Unfold)
       $(\vec{X} \parallel \vec{Y})_{n,k} := (\vec{X}, L \parallel \varepsilon)_{n+1,n+1}$ 
     $\vec{Y} = \varepsilon$  and  $X_{n-1} \not\subseteq p^\downarrow$  :                               %(Candidate)
      choose  $Z \in L^\downarrow$  such that  $X_{n-1} \not\subseteq Z$  and  $p^\downarrow \subseteq Z$ ;
       $(\vec{X} \parallel \vec{Y})_{n,k} := (\vec{X} \parallel Z)_{n,n-1}$ 
     $\vec{Y} \neq \varepsilon$  and  $b^\downarrow(X_{k-1}) \not\subseteq Y_k$  :                               %(Decide)
      choose  $Z \in L^\downarrow$  such that  $X_{k-1} \not\subseteq Z$  and  $b_r^\downarrow(Y_k) \subseteq Z$ ;
       $(\vec{X} \parallel \vec{Y})_{n,k} := (\vec{X} \parallel Z, \vec{Y})_{n,k-1}$ 
     $\vec{Y} \neq \varepsilon$  and  $b^\downarrow(X_{k-1}) \subseteq Y_k$  :                               %(Conflict)
      choose  $Z \in L^\downarrow$  such that  $Z \subseteq Y_k$  and  $(b^\downarrow \cup \perp^\downarrow)(X_{k-1} \cap Z) \subseteq Z$ ;
       $(\vec{X} \parallel \vec{Y})_{n,k} := (\vec{X} \cap_k Z \parallel \text{tail}(\vec{Y}))_{n,k+1}$ 
  endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. X_{j+1} \subseteq X_j$  then return true %  $\vec{X}$  conclusive
  if  $\perp^\downarrow \not\subseteq Y_1$  then return false %  $\vec{Y}$  conclusive

```

Figure D.1: AdjointPDR algorithm checking $\text{lfp}(b^\downarrow \cup \perp^\downarrow) \subseteq p^\downarrow$.

- We know that $b(x_{k-1}) \leq z_B$. Then, for any $s \in S$ it follows that $b(x_{k-1})(s) \leq \lceil z_B(s) \rceil = z_{01}(s)$ if $r_s = 0$ and $Z_k \neq \emptyset$, and that $b(x_{k-1})(s) \leq z_B(s) = z_{01}(s)$ otherwise. Therefore $b(x_{k-1}) \leq z_{01}$, from which it readily follows $b(x_{k-1} \wedge z_{01}) \leq b(x_{k-1}) \leq z_{01}$.

□

D.1 Proofs of Section 7.4

D.1.1 Proofs of Section 7.4.1: AdjointPDR $^\downarrow$

Proof of Theorem 7.20. The algorithm $\text{AdjointPDR}^\downarrow$ differs from AdjointPDR instantiated with $(\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow)$ for two main reasons: first we restrict the elements of the positive chain to be principals, second we optimize the initial state of the algorithm.

To prove that the properties of AdjointPDR can be extended to $\text{AdjointPDR}^\downarrow$, we first show the instance of AdjointPDR on $(\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow)$ for the lower set domain $(L^\downarrow, \subseteq)$ in Figure D.1. Clearly, as instance of AdjointPDR , this algorithm inherits all its properties about soundness, progression and negative termination. Note that, in (Candidate), the condition $p^\downarrow \subseteq Z$ is equivalent to $p \in Z$, because $Z \in L^\downarrow$. Moreover, we note that the negative termination condition $\perp^\downarrow \not\subseteq Y_1$ amounts to $Y_1 = \emptyset$.

To restrict the elements of the positive chain to be principals we add the condition $\exists z \in L. Z = z^\downarrow$ in rule (Conflict), which is thus modified as follows w.r.t. Figure D.1:

AdjointPDR' ($\perp^\downarrow, b^\downarrow, b_r^\downarrow, p^\downarrow$)

```

<INITIALISATION>
  ( $\vec{X} \parallel \vec{Y}$ )n,k := ( $\emptyset, \top^\downarrow \parallel \varepsilon$ )2,2
<ITERATION>
  case ( $\vec{X} \parallel \vec{Y}$ )n,k of
    %  $\vec{X}$  has the form  $\emptyset, x_1^\downarrow, \dots, x_{n-1}^\downarrow$ 
     $\vec{Y} = \varepsilon$  and  $x_{n-1} \leq p$  :                               %(Unfold)
      ( $\vec{X} \parallel \vec{Y}$ )n,k := ( $\vec{X}, \top^\downarrow \parallel \varepsilon$ )n+1,n+1
     $\vec{Y} = \varepsilon$  and  $x_{n-1} \not\leq p$  :                               %(Candidate)
      choose  $Z \in L^\downarrow$  such that  $x_{n-1} \notin Z$  and  $p \in Z$ ;
      ( $\vec{X} \parallel \vec{Y}$ )n,k := ( $\vec{X} \parallel Z$ )n,n-1
     $\vec{Y} \neq \varepsilon$  and  $b^\downarrow(x_{k-1}^\downarrow) \not\subseteq Y_k$  :               %(Decide)
      choose  $Z \in L^\downarrow$  such that  $x_{k-1} \notin Z$  and  $b_r^\downarrow(Y_k) \subseteq Z$ ;
      ( $\vec{X} \parallel \vec{Y}$ )n,k := ( $\vec{X} \parallel Z, \vec{Y}$ )n,k-1
     $\vec{Y} \neq \varepsilon$  and  $b^\downarrow(x_{k-1}^\downarrow) \subseteq Y_k$  :               %(Conflict)
      choose  $z \in L$  such that  $z \in Y_k$  and  $(b^\downarrow \cup \perp^\downarrow)(x_{k-1}^\downarrow \cap z^\downarrow) \subseteq z^\downarrow$ ;
      ( $\vec{X} \parallel \vec{Y}$ )n,k := ( $\vec{X} \cap_k z^\downarrow \parallel \text{tail}(\vec{Y})$ )n,k+1
  endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. x_{j+1}^\downarrow \subseteq x_j^\downarrow$  then return true %  $\vec{X}$  conclusive
  if  $Y_1 = \emptyset$  then return false %  $\vec{Y}$  conclusive

```

Figure D.2: AdjointPDR algorithm checking $\text{lfp}(b^\downarrow \cup \perp^\downarrow) \subseteq p^\downarrow$, where we restrict the elements of the positive chain to be principals. Note that: in (Unfold) the condition $x_{n-1} \leq p$ is equivalent to $x_{n-1}^\downarrow \subseteq p^\downarrow$; and similarly for their negation in (Candidate), where moreover the condition $x_{n-1}^\downarrow \not\subseteq Z$ is equivalent to $x \notin Z$; same for (Decide); finally in (Conflict) the condition $z \in L$ is equivalent to $z^\downarrow \in L^\downarrow$ and the condition $z \in Y_k$ is equivalent to $z^\downarrow \subseteq Y_k$.

```

 $\vec{Y} \neq \varepsilon$  and  $b^\downarrow(x_{k-1}^\downarrow) \subseteq Y_k$  :                               %(Conflict)
  choose  $Z \in L^\downarrow$  such that  $Z \subseteq Y_k$  and  $(b^\downarrow \cup \perp^\downarrow)(X_{k-1} \cap Z) \subseteq Z$ 
                                     and  $\exists z \in L. Z = z^\downarrow$ ;
  ( $\vec{X} \parallel \vec{Y}$ )n,k := ( $\vec{X} \cap_k Z \parallel \text{tail}(\vec{Y})$ )n,k+1

```

Let us call AdjointPDR' such algorithm. All the executions of AdjointPDR' are also possible in AdjointPDR, thus all the invariants of AdjointPDR (Proposition 7.4) holds for AdjointPDR'. The invariants suffice to prove Theorem 7.5, Proposition 7.7 and Theorem 7.15.

The elements of the positive chain are introduced by (Unfold) and modified by (Conflict). By choosing $Z = z^\downarrow$ in (Conflict) it follows that all the elements of the positive chain are also principals, with the only exception of $X_0 = \emptyset$. Indeed, every new element of the positive chain has that form (in (Unfold) we take $\top_{L^\downarrow} = \top^\downarrow$) and the meet of two principals $x_j^\downarrow$ and $z^\downarrow$ in (Conflict) is itself the principal $(x_j \wedge z)^\downarrow$ generated by the meet of x_j and z .

Regarding the canonical choices of Proposition 7.6,

1. in (Candidate) $Z = p^\downarrow$;

2. in (Decide) $Z = b_r^\downarrow(Y_k)$;
3. in (Conflict) $Z = Y_k$;
4. in (Conflict) $Z = (b^\downarrow \cup \perp^\downarrow)(X_{k-1})$.

we have that choice 3 is not necessarily possible, because we cannot assume that $Y_k = y_k^\downarrow$ for some $y_k \in L$, but 1, 2 and 4 remain valid choices: in fact, 1 and 2 deal with the negative sequence for which we have no restriction; for 4, if $X_{k-1} = x_{k-1}^\downarrow$ for some $x_{k-1} \in L$, then

$$Z = (b^\downarrow \cup \perp^\downarrow)(X_{k-1}) = (b^\downarrow \cup \perp^\downarrow)(x_{k-1}^\downarrow) = b^\downarrow(x_{k-1}^\downarrow) \cup \perp^\downarrow \stackrel{(7.4)}{=} b(x_{k-1})^\downarrow \cup \perp^\downarrow = b(x_{k-1})^\downarrow$$

is still a principal. Since choices 1, 2 and 4 guarantees the existence of a simple heuristic (i.e., the initial one) we also have that Corollary 7.16 about negative termination is valid for **AdjointPDR**'.

We now take advantage of the shape of the positive chain to present the code of **AdjointPDR**' as reported in Figure D.2: we exploit the fact that $\vec{X} = \emptyset, x_1^\downarrow, \dots, x_{n-1}^\downarrow$ to make some simple code transformations described in the caption. Now we note that $b^\downarrow(x_{k-1}^\downarrow) = b(x_{k-1})^\downarrow$, so that the conditions $b^\downarrow(x_{k-1}^\downarrow) \not\subseteq Y_k$ in (Decide) and $b^\downarrow(x_{k-1}^\downarrow) \subseteq Y_k$ in (Conflict) are equivalent to $b(x_{k-1}) \not\subseteq Y_k$ and to $b(x_{k-1}) \in Y_k$, respectively. Moreover,

$$\begin{aligned} (b^\downarrow \cup \perp^\downarrow)(x_{k-1}^\downarrow \cap z^\downarrow) &= (b^\downarrow \cup \perp^\downarrow)((x_{k-1} \wedge z)^\downarrow) = (b \vee \perp)(x_{k-1} \wedge z)^\downarrow \\ &= (b(x_{k-1} \wedge z) \vee \perp)^\downarrow = b(x_{k-1} \wedge z)^\downarrow \end{aligned}$$

so that $(b^\downarrow \cup \perp^\downarrow)(x_{k-1}^\downarrow \cap z^\downarrow) \subseteq z^\downarrow$ is equivalent to $b(x_{k-1} \wedge z)^\downarrow \subseteq z^\downarrow$ and therefore also to $b(x_{k-1} \wedge z) \leq z$.

Then, the only difference between **AdjointPDR**' and **AdjointPDR**[↓] is the initialization condition: **AdjointPDR**[↓] starts from the state reached after the following three steps of **AdjointPDR**':

$$(\emptyset, \top^\downarrow \parallel \varepsilon)_{2,2} \xrightarrow{C_a} (\emptyset, \top^\downarrow \parallel p^\downarrow)_{2,1} \xrightarrow{C_g} (\emptyset, \perp^\downarrow \parallel \varepsilon)_{2,2} \xrightarrow{U} (\emptyset, \perp^\downarrow, \top^\downarrow \parallel \varepsilon)_{3,3}.$$

Since these steps apply the canonical choices for (Candidate) and (Conflict), we conclude that **AdjointPDR**[↓] is sound and enjoys progression and negative termination. $\square$

D.1.2 Proofs of Section 7.4.2: **AdjointPDR**[↓] simulates LT-PDR

Proof of Theorem 7.23. For the scope of this proof, we call $\mathcal{S} = \{(\vec{x} \parallel \vec{Y})_{n,k}\}$ the set of states of **AdjointPDR**[↓], and $\mathcal{S}' = \{(\vec{x}' \parallel \vec{c}')_{n',k'}\}$ that of LT-PDR, where we use non-primed variables for the former and primed for the latter. The function $\mathcal{R}: \mathcal{S}' \rightarrow \mathcal{S}$ is defined for all states $s' = (\vec{x}' \parallel \vec{c}')_{n',k'} \in \mathcal{S}'$, as

$$\mathcal{R}((\vec{x}' \parallel \vec{c}')_{n',k'}) = (\vec{x} \parallel \vec{Y})_{n'+1,k'+1} \in \mathcal{S}$$

where

$$n = n' + 1, \quad k = k' + 1, \quad \vec{x} = \emptyset, \vec{x}' \quad \text{and} \quad \vec{Y} = \text{neg}(\vec{c}').$$

We prove that $\mathcal{R}$ is a simulation [Mil89], that is for all $s', t' \in \mathcal{S}'$, if $s' \rightarrow t'$ then $\mathcal{R}(s') \rightarrow \mathcal{R}(t')$. The pseudo-code of LT-PDR is reported in Figure D.3.

First, we remark that, for any $z, x \in L$, $x \notin \neg(\{z\}^\uparrow)$ if and only if $z \leq x$. Moreover, note that indices $\vec{x}'$ in s' and $\vec{x}$ in $\mathcal{R}(s')$ are off-setted by one: $x_j = x'_{j-1}$. However, as $n = n' + 1$ we have, for instance, $x_{n-1} = x'_{n-1}$ (and analogously for k').

LT-PDR (b, p)

```

<INITIALISATION>
   $(\vec{x} \parallel \vec{c})_{n,k} := (\perp, b(\perp) \parallel \varepsilon)_{2,2}$ 
<ITERATION>
  case  $(\vec{x} \parallel \vec{c})_{n,k}$  of
     $\vec{c} = \varepsilon$  and  $x_{n-1} \leq p$  :                               %(Unfold)
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x}, \top \parallel \varepsilon)_{n+1,n+1}$ 
     $\vec{c} = \varepsilon$  and  $x_{n-1} \not\leq p$  :                               %(Candidate)
      choose  $z \in L$  such that  $z \leq x_{n-1}$  and  $z \not\leq p$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \parallel z)_{n,n-1}$ 
     $\vec{c} \neq \varepsilon$  and  $c_k \leq b(x_{k-1})$  :                       %(Decide)
      choose  $z \in L$  such that  $z \leq x_{k-1}$  and  $c_k \leq b(z)$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \parallel z, \vec{c})_{n,k-1}$ 
     $\vec{c} \neq \varepsilon$  and  $c_k \not\leq b(x_{k-1})$  :                       %(Conflict)
      choose  $z \in L$  such that  $c_k \not\leq z$  and  $b(x_{k-1} \wedge z) \leq z$ ;
       $(\vec{x} \parallel \vec{c})_{n,k} := (\vec{x} \wedge_k z \parallel \text{tail}(\vec{c}))_{n,k+1}$ 
  endcase
<TERMINATION>
  if  $\exists j \in [0, n-2]. x_{j+1} \leq x_j$  then return true %  $\vec{x}$  is conclusive
  if  $k = 1$  then return false %  $\vec{c}$  is conclusive

```

Figure D.3: LT-PDR algorithm checking $\text{lfp}b \leq p$, adapted from [Kor+22].

Consider now a state $s' = (\vec{x}' \parallel \vec{c}')_{n',k'} \in \mathcal{S}'$ and $\mathcal{R}(s') = (\vec{x} \parallel \vec{Y})_{n,k} \in \mathcal{S}$. Suppose that LT-PDR can perform a transition from s' . This must be determined by one of the four rules of LT-PDR, possibly performing some choice of $z \in L$. We show that $\text{AdjointPDR}^\downarrow$ is able to simulate this transition, using the same rule and performing a corresponding choice. We do so by cases on the rule used by LT-PDR.

- If LT-PDR applies rule (Unfold), we have $\vec{c}' = \varepsilon$ and $x'_{n'-1} \leq p$ so that

$$s' = (\vec{x}' \parallel \varepsilon)_{n',n'} \xrightarrow{U} (\vec{x}', \top \parallel \varepsilon)_{n'+1,n'+1} = t'.$$

Then, for $(\vec{x} \parallel \vec{Y})_{n,k} = \mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \varepsilon)_{n'+1,n'+1}$ it holds $\vec{Y} = \varepsilon$ and $x_{n-1} = x'_{n'-1} \leq p$, so $\text{AdjointPDR}^\downarrow$ can apply (Unfold) too and

$$\mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \varepsilon)_{n'+1,n'+1} \xrightarrow{U} (\emptyset, \vec{x}', \top \parallel \varepsilon)_{n'+2,n'+2} = \mathcal{R}(t').$$

- If LT-PDR applies rule (Candidate), we have $\vec{c}' = \varepsilon$ and $x'_{n'-1} \not\leq p$, so that $z \in L$ is chosen such that $z \leq x'_{n'-1}$ and $z \not\leq p$ to derive

$$s' = (\vec{x}' \parallel \varepsilon)_{n',n'} \xrightarrow{C_a} (\vec{x}' \parallel z)_{n',n'-1} = t'.$$

Then, for $(\vec{x} \parallel \vec{Y})_{n,k} = \mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \varepsilon)_{n'+1,n'+1}$ it holds $\vec{Y} = \varepsilon$ and $x_{n-1} = x'_{n'-1} \not\leq p$, so that $\text{AdjointPDR}^\downarrow$ can apply (Candidate) too. Moreover we can choose $Z \triangleq \neg(\{z\}^\uparrow)$, because $z \leq x'_{n'-1}$ implies $x_{n-1} = x'_{n'-1} \notin Z$, and $z \not\leq p$ implies $p \in Z$. By doing so we derive

$$\mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \varepsilon)_{n'+1,n'+1} \xrightarrow{C_a} (\emptyset, \vec{x}' \parallel Z)_{n'+1,n'} = \mathcal{R}(t').$$

- If LT-PDR applies rule (Decide), we have $\vec{c}' \neq \varepsilon$ and $c'_{k'} \leq b(x'_{k'-1})$, so that $z \in L$ is chosen such that $z \leq x'_{k'-1}$ and $c'_{k'} \leq b(z)$ to derive

$$s' = (\vec{x}' \parallel \vec{c}')_{n',k'} \xrightarrow{D}_z (\vec{x}' \parallel z, \vec{c}')_{n',k'-1} = t'.$$

Then, for $(\vec{x}' \parallel \vec{Y})_{n,k} = \mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \text{neg}(c'))_{n'+1,k'+1}$ it holds $\text{neg}(c') \neq \varepsilon$ and $b(x_{k-1}) = b(x'_{k'-1}) \notin \text{neg}(c')_k = \neg(\{c'_{k-1}\}^\uparrow)$, because the latter is implied by $c'_{k'} \leq b(x'_{k'-1})$. Thus $\text{AdjointPDR}^\downarrow$ can apply (Decide) too. Moreover we can choose $Z \triangleq \neg(\{z\}^\uparrow)$. In fact $z \leq x'_{k'-1}$ implies $x_{k-1} = x'_{n'-1} \notin \neg(\{z\}^\uparrow) = Z$. Moreover $b_r^\downarrow(Y_k) \subseteq Z$ if and only if $x \notin Z$ implies $b(x) \notin Y_k$. Because $Z = \neg(\{z\}^\uparrow)$ and $\text{neg}(c')_k = \neg(\{c'_{k'}\}^\uparrow)$, this implication is equivalent to requiring that $z \leq x$ implies $c'_{k'} \leq b(x)$, which is true as $c'_{k'} \leq b(z)$ and b is monotone. With this choice of Z we derive

$$\mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \vec{Y})_{n'+1,k'+1} \xrightarrow{D}_Z (\emptyset, \vec{x}' \parallel Z, \vec{Y})_{n'+1,k'} = \mathcal{R}(t').$$

- If LT-PDR applies rule (Conflict), we have $\vec{c}' \neq \varepsilon$ and $c'_{k'} \not\leq b(x'_{k'-1})$, so that $z \in L$ is chosen such that $c'_{k'} \not\leq z$ and $b(x'_{k'-1} \wedge z) \leq z$ to derive

$$s' = (\vec{x}' \parallel \vec{c}')_{n',k'} \xrightarrow{C_o}_z (\vec{x}' \wedge_{k'} z \parallel \text{tail}(\vec{c}'))_{n',k'+1} = t'.$$

Then, for $(\vec{x}' \parallel \vec{Y})_{n,k} = \mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \text{neg}(c'))_{n'+1,k'+1}$ it holds $\text{neg}(c') \neq \varepsilon$ and $b(x_{k-1}) = b(x'_{k'-1}) \in Y_k = \neg(\{c'_{k-1}\}^\uparrow)$, because the latter is implied by $c'_{k'} \not\leq b(x'_{k'-1})$. Thus $\text{AdjointPDR}^\downarrow$ can apply (Conflict) too. Moreover we can choose the same z as LT-PDR. In fact $c'_{k'} \not\leq z$ implies $z \in \neg(\{c'_{k'}\}^\uparrow) = \neg(\{c'_{k-1}\}^\uparrow) = \text{neg}(c')_k$, and $b(x_{k-1} \wedge z) = b(x'_{k'-1} \wedge z) \leq z$ is also an hypothesis in LT-PDR. With this choice of z we derive

$$\mathcal{R}(s') = (\emptyset, \vec{x}' \parallel \vec{Y})_{n'+1,k'+1} \xrightarrow{C_o}_z (\emptyset, \vec{x}' \wedge_k z \parallel \text{tail}(\vec{Y}))_{n'+1,k'+2} = \mathcal{R}(t').$$

This concludes the proof that $\mathcal{R}$ is a simulation. However, to complete the proof that $\text{AdjointPDR}^\downarrow$ simulates LT-PDR, we have to take care of initial and final states.

We observe that the initial states $s'_0 = (\perp, b(\perp) \parallel \varepsilon)_{2,2} \in \mathcal{S}'$ of LT-PDR and $s_0 = (\emptyset, \perp, \top \parallel \varepsilon)_{3,3} \in \mathcal{S}$ of $\text{AdjointPDR}^\downarrow$ are not related: $\mathcal{R}(s'_0) \neq s_0$. To solve this issue, first observe that if $b(\perp) \not\leq p$, both algorithms return false in a few steps. If instead $b(\perp) \leq p$, $\text{AdjointPDR}^\downarrow$ can reach the state $\mathcal{R}(s'_0)$ from s_0 in just two steps:

$$s_0 = (\emptyset, \perp, \top \parallel \varepsilon)_{3,3} \xrightarrow{C_a} (\emptyset, \perp, \top \parallel p^\downarrow)_{3,2} \xrightarrow{C_o}_{b(\perp)} (\emptyset, \perp, b(\perp) \parallel \varepsilon)_{3,3} = \mathcal{R}(s'_0)$$

Lastly, we discuss the termination conditions of the two algorithms.

When LT-PDR terminates from a state s' returning true, s' satisfies $x'_{j+1} \leq x'_j$ for some j , so also $\text{AdjointPDR}^\downarrow$ terminates from $\mathcal{R}(s')$ returning true.

Instead, when LT-PDR terminates from s' returning false, the condition $k' = 1$ does not imply that $\text{AdjointPDR}^\downarrow$ terminates from $s = \mathcal{R}(s')$. However, the latter algorithm can always apply (Decide) from s : as proved in [Kor+22], the termination condition $k' = 1$ of LT-PDR implies $c'_1 \leq b(\perp)$, which in turn means $b(x_1) = b(x'_0) = b(\perp) \notin Y_2 = \neg(\{c'_1\}^\uparrow)$. Moreover, we can choose $Z \triangleq \emptyset$: for all $x \in L$ we have $c'_1 \leq b(\perp) \leq b(x)$, so $b(x) \notin Y_2$. After this step, we get that $Y_1 = \emptyset$, so $\text{AdjointPDR}^\downarrow$ returns false, too. $\square$

Todo list

Cousot: dual narrowing?	31
Say forward analysis at the beginning	43